

Tier 2 — Intermediate: Coding Patterns

github.com/quanhua92/interview-prep

Interview Prep Playbook

July 2026

Contents

1	Tier 2: Intermediate — Coding Interview Preparation Roadmap	22
1.1	Progression Roadmap	22
1.2	Detailed Learning Modules & File Index	23
1.2.1	Module 1: Prefix Sum & Binary Search	23
1.2.2	Module 2: Stack Parsing & Depth-First Search (DFS)	25
1.2.3	Module 3: Heap, Divide & Conquer, and Priority Systems	26
1.2.4	Module 4: Bit Manipulation	27
1.2.5	Module 5: Intermediate Mathematics	27
1.2.6	Module 6: Dynamic Programming & High-Level System Design	28
1.3	Intermediate Tier Interview Strategy	30
1.3.1	1. Integer Shift Limits & Bit Safety	30
1.3.2	2. Stack and Recursion Depths	30
1.3.3	3. Dynamic Heap Operations	30
2	Product of Array Except Self	30
2.1	Question Description	30
2.1.1	Examples	31
2.2	Detailed Solution (C++)	31
2.2.1	Standard C++ Production Code	31
2.3	Detailed Solution (Python)	32
2.3.1	Standard Python Production Code	32
2.4	Python-Specific Complexities & Implementation Considerations	33
2.4.1	1. Division Operators and Zero Handling	33
2.4.2	2. Avoiding Intermediate Lists for Memory Efficiency	33
2.4.3	3. Dynamic Arrays (Lists) vs Pre-allocated Lists	33
2.5	Complexity Analysis	33
2.6	Common Follow-Up Questions & How to Answer	33
2.6.1	Q1: What if division is allowed?	33
2.6.2	Q2: What if we need prefix products for operations that do not have inverses?	34
2.7	Pro-Tip: How to Impress the Interviewer	34
3	Subarray Sum Equals K	34
3.1	Question Description	34
3.1.1	Examples	34
3.2	Detailed Solution (C++)	35
3.2.1	Standard C++ Production Code	35

3.3	Detailed Solution (Python)	36
3.3.1	Standard Python Production Code	36
3.4	Python-Specific Complexities & Implementation Considerations	36
3.4.1	1. Choice of Dictionary Implementation	36
3.4.2	2. Large Integer Precision	37
3.5	Complexity Analysis	37
3.6	Common Follow-Up Questions & How to Answer	37
3.6.1	Q1: What if all numbers in the array are non-negative?	37
3.6.2	Q2: What if we need to return the actual indices of the subarrays instead of the count?	37
3.7	Pro-Tip: How to Impress the Interviewer	38
4	Subarray Product Less Than K	38
4.1	Question Description	38
4.1.1	Examples	38
4.2	Detailed Solution (C++)	38
4.2.1	Standard C++ Production Code	39
4.3	Detailed Solution (Python)	39
4.3.1	Standard Python Production Code	39
4.4	Python-Specific Complexities & Implementation Considerations	40
4.4.1	1. Floor Division vs Standard Division	40
4.4.2	2. Arbitrary Precision	40
4.5	Complexity Analysis	41
4.6	Common Follow-Up Questions & How to Answer	41
4.6.1	Q1: What if nums[i] could be 0?	41
4.6.2	Q2: How is this related to Prefix Sums?	41
4.7	Pro-Tip: How to Impress the Interviewer	41
5	Continuous Subarray Sum	42
5.1	Question Description	42
5.1.1	Examples	42
5.2	Detailed Solution (C++)	42
5.2.1	Standard C++ Production Code	43
5.3	Detailed Solution (Python)	44
5.3.1	Standard Python Production Code	44
5.4	Python-Specific Complexities & Implementation Considerations	45
5.4.1	1. Hash Map Space Limits	45
5.4.2	2. Modulo Arithmetic on Negative Numbers	45
5.5	Complexity Analysis	45
5.6	Common Follow-Up Questions & How to Answer	45
5.6.1	Q1: What if k can be 0?	45
5.6.2	Q2: What if we want to find the longest good subarray?	45
5.7	Pro-Tip: How to Impress the Interviewer	46
6	Contiguous Array	46
6.1	Question Description	46
6.1.1	Examples	46
6.2	Detailed Solution (C++)	46
6.2.1	Standard C++ Production Code	47
6.3	Detailed Solution (Python)	48
6.3.1	Standard Python Production Code	48
6.4	Python-Specific Complexities & Implementation Considerations	48

6.4.1	1. Hash Table Overhead vs Pre-allocated Arrays	48
6.5	Complexity Analysis	49
6.6	Common Follow-Up Questions & How to Answer	49
6.6.1	Q1: Can we solve this in $O(1)$ space if we are allowed to modify the input or use some properties?	49
6.6.2	Q2: What if we want to find the maximum length subarray with equal numbers of three different values (e.g., 0, 1, and 2)?	49
6.7	Pro-Tip: How to Impress the Interviewer	49
7	Random Pick with Weight	50
7.1	Question Description	50
7.1.1	Examples	50
7.2	Detailed Solution (C++)	51
7.2.1	Standard C++ Production Code	51
7.3	Detailed Solution (Python)	52
7.3.1	Standard Python Production Code	52
7.4	Python-Specific Complexities & Implementation Considerations	53
7.4.1	1. <code>random.randint</code> vs <code>random.random</code>	53
7.4.2	2. Built-in <code>bisect</code> Module	53
7.5	Complexity Analysis	53
7.6	Common Follow-Up Questions & How to Answer	53
7.6.1	Q1: What if weights can be modified dynamically (e.g., <code>updateWeight(index, val)</code>)?	53
7.6.2	Q2: Can we achieve $O(1)$ query time for <code>pickIndex</code> ?	53
7.7	Pro-Tip: How to Impress the Interviewer	54
8	Find Minimum in Rotated Sorted Array	54
8.1	Question Description	54
8.1.1	Examples	54
8.2	Detailed Solution (C++)	55
8.2.1	Standard C++ Production Code	55
8.3	Detailed Solution (Python)	56
8.3.1	Standard Python Production Code	56
8.4	Python-Specific Complexities & Implementation Considerations	57
8.4.1	1. Integer Division: <code>/</code> vs <code>//</code>	57
8.4.2	2. Handling of Empty Inputs	57
8.4.3	3. Avoiding Slicing Overhead	57
8.5	Complexity Analysis	57
8.6	Common Follow-Up Questions & How to Answer	57
8.6.1	Q1: What if duplicates are allowed in the array? (LeetCode 154)	57
8.6.2	Q2: How can we find the number of times the array was rotated?	57
8.7	Pro-Tip: How to Impress the Interviewer	58
9	First Bad Version	58
9.1	Question Description	58
9.1.1	Examples	58
9.1.2	Constraints	59
9.2	Detailed Solution (C++)	59
9.2.1	Standard C++ Production Code	59
9.3	Detailed Solution (Python)	60
9.3.1	Standard Python Production Code	60
9.4	Python-Specific Complexities & Implementation Considerations	60

9.4.1	1. Integer Arithmetic Safety	60
9.4.2	2. Mocking the API for Local Testing	60
9.5	Complexity Analysis	61
9.6	Common Follow-Up Questions & How to Answer	61
9.6.1	Q1: What if the API calls are extremely expensive (e.g., hitting a remote database or network call)?	61
9.6.2	Q2: What if we don't know the upper bound n ? (Infinite version stream)	61
9.7	Pro-Tip: How to Impress the Interviewer	61
10	Data Stream as Disjoint Intervals	61
10.1	Question Description	62
10.1.1	Examples	62
10.1.2	Constraints	62
10.2	Detailed Solution (C++)	63
10.2.1	Insertion Logic for <code>addNum(value)</code> :	63
10.2.2	Standard C++ Production Code	63
10.3	Detailed Solution (Python)	64
10.3.1	Standard Python Production Code	64
10.4	Python-Specific Complexities & Implementation Considerations	65
10.4.1	1. Python List Shift Overhead	65
10.5	Complexity Analysis	65
10.6	Common Follow-Up Questions & How to Answer	66
10.6.1	Q1: What if there are lots of merges and the number of disjoint intervals is small compared to the size of the data stream?	66
10.6.2	Q2: How can we implement this if memory is extremely constrained?	66
10.7	Pro-Tip: How to Impress the Interviewer	66
11	Russian Doll Envelopes	66
11.1	Question Description	66
11.1.1	Examples	67
11.1.2	Constraints	67
11.2	Detailed Solution (C++)	67
11.2.1	Standard C++ Production Code	67
11.3	Detailed Solution (Python)	68
11.3.1	Standard Python Production Code	68
11.4	Python-Specific Complexities & Implementation Considerations	69
11.4.1	1. Complex Sorting Keys	69
11.4.2	2. Space Overhead of List Manipulation	69
11.5	Complexity Analysis	69
11.6	Common Follow-Up Questions & How to Answer	70
11.6.1	Q1: What if we can rotate the envelopes?	70
11.6.2	Q2: What if we are matching in 3 dimensions (Boxes instead of Envelopes)?	70
11.7	Pro-Tip: How to Impress the Interviewer	70
12	Smallest Good Base	70
12.1	Question Description	70
12.1.1	Examples	71
12.1.2	Constraints	71
12.2	Detailed Solution (C++)	71
12.2.1	Key Mathematical Boundaries:	71
12.2.2	Standard C++ Production Code	71

12.3	Detailed Solution (Python)	73
12.3.1	Standard Python Production Code	73
12.4	Python-Specific Complexities & Implementation Considerations	74
12.4.1	1. Integer Division and Large Power Calculations	74
12.4.2	2. Large Integers	74
12.5	Complexity Analysis	74
12.6	Common Follow-Up Questions & How to Answer	75
12.6.1	Q1: Why do we search from $m = \max_m$ down to 2, rather than 2 up to $\max_m$?	75
12.6.2	Q2: What mathematical approximation can we use to skip binary search?	75
12.7	Pro-Tip: How to Impress the Interviewer	75
13	Random Point in Non-overlapping Rectangles	75
13.1	Question Description	75
13.1.1	Examples	76
13.1.2	Constraints	76
13.2	Detailed Solution (C++)	76
13.2.1	Standard C++ Production Code	77
13.3	Detailed Solution (Python)	78
13.3.1	Standard Python Production Code	78
13.4	Python-Specific Complexities & Implementation Considerations	79
13.4.1	1. Choice of Randomness Function: <code>randint</code> vs <code>randrange</code>	79
13.4.2	2. Prefix Search using <code>bisect_left</code> vs <code>bisect_right</code>	79
13.5	Complexity Analysis	79
13.6	Common Follow-Up Questions & How to Answer	79
13.6.1	Q1: What if the rectangles overlap?	79
13.6.2	Q2: What if we can't store the rectangles in memory (Streaming / Reservoir Sampling)?	79
13.7	Pro-Tip: How to Impress the Interviewer	80
14	Binary Search	80
14.1	Question Description	80
14.1.1	Examples	80
14.1.2	Constraints	80
14.2	Detailed Solution (C++)	81
14.2.1	Standard C++ Production Code	81
14.3	Detailed Solution (Python)	82
14.3.1	Standard Python Production Code	82
14.4	Python-Specific Complexities & Implementation Considerations	82
14.4.1	1. Python Built-In Alternative: <code>bisect</code>	82
14.5	Complexity Analysis	83
14.6	Common Follow-Up Questions & How to Answer	83
14.6.1	Q1: How does the algorithm behave if we search for a value that is missing?	83
14.6.2	Q2: What is the difference between <code>lower_bound</code> and <code>upper_bound</code> ?	83
14.7	Pro-Tip: How to Impress the Interviewer	83
15	Valid Parentheses	84
15.1	Question Description	84
15.1.1	Examples	84
15.2	Detailed Solution (C++)	84
15.2.1	Standard C++ Production Code	84
15.3	Detailed Solution (Python)	85
15.3.1	Standard Python Production Code	85

15.4	Python-Specific Complexities & Implementation Considerations	86
15.4.1	1. list as a Stack	86
15.4.2	2. Early Termination Performance	86
15.5	Complexity Analysis	86
15.6	Common Follow-Up Questions & How to Answer	86
15.6.1	Q1: What if the string contains non-bracket characters (e.g. "a + (b * c)")?	86
15.6.2	Q2: What if we only have one type of bracket (e.g. only (and))? Can we optimize space?	87
15.7	Pro-Tip: How to Impress the Interviewer	87
16	Min Stack	87
16.1	Question Description	87
16.1.1	Examples	87
16.2	Detailed Solution (C++)	88
16.2.1	Standard C++ Production Code	88
16.3	Detailed Solution (Python)	89
16.3.1	Standard Python Production Code	89
16.4	Python-Specific Complexities & Implementation Considerations	90
16.4.1	1. Amortized dynamic array resizing	90
16.4.2	2. Exceptions handling	90
16.5	Complexity Analysis	90
16.6	Common Follow-Up Questions & How to Answer	91
16.6.1	Q1: How can we optimize space when there are many duplicate pushes?	91
16.6.2	Q2: Can we implement this with $\mathcal{O}(1)$ auxiliary space? (The Difference Trick)	91
16.7	Pro-Tip: How to Impress the Interviewer	91
17	Decode String	92
17.1	Question Description	92
17.1.1	Examples	92
17.2	Detailed Solution (C++)	92
17.2.1	Standard C++ Production Code	93
17.3	Detailed Solution (Python)	94
17.3.1	Standard Python Production Code	94
17.4	Python-Specific Complexities & Implementation Considerations	94
17.4.1	1. Large String Concatenations	94
17.5	Complexity Analysis	95
17.6	Common Follow-Up Questions & How to Answer	95
17.6.1	Q1: How can we implement this recursively?	95
17.6.2	Q2: What if we push nested expressions like <code>100[100[100[a]]]</code> ? How to prevent Out-Of-Memory (OOM)?	96
17.7	Pro-Tip: How to Impress the Interviewer	96
18	Jump Game III	96
18.1	Question Description	96
18.1.1	Examples	97
18.2	Detailed Solution (C++)	97
18.2.1	Standard C++ Production Code	97
18.3	Detailed Solution (Python)	98
18.3.1	Standard Python Production Code	98
18.4	Python-Specific Complexities & Implementation Considerations	99
18.4.1	1. Recursion Stack Limit	99

18.4.2	2. Visited Collection Performance	99
18.5	Complexity Analysis	99
18.6	Common Follow-Up Questions & How to Answer	100
18.6.1	Q1: How can we solve this problem in $\mathcal{O}(1)$ auxiliary space?	100
18.6.2	Q2: Write the Breadth-First Search (BFS) solution. How does BFS compare to DFS here?	100
18.7	Pro-Tip: How to Impress the Interviewer	101
19	Number of Islands	101
19.1	Question Description	101
19.1.1	Examples	101
19.2	Detailed Solution (C++)	102
19.2.1	Standard C++ Production Code	102
19.3	Detailed Solution (Python)	103
19.3.1	Standard Python Production Code	103
19.4	Python-Specific Complexities & Implementation Considerations	104
19.4.1	1. In-place Mutation vs Functional Integrity	104
19.4.2	2. Deep Recursion Limitations	105
19.5	Complexity Analysis	105
19.6	Common Follow-Up Questions & How to Answer	105
19.6.1	Q1: What if the grid is too large to fit into memory? (e.g. billions of rows/columns)	105
19.6.2	Q2: Compare DFS, BFS, and Union-Find (DSU) for this problem.	105
19.7	Pro-Tip: How to Impress the Interviewer	106
20	Mini Parser	106
20.1	Question Description	106
20.1.1	Examples	106
20.2	Detailed Solution (C++)	107
20.2.1	Standard C++ Production Code	107
20.3	Detailed Solution (Python)	108
20.3.1	Standard Python Production Code	108
20.4	Python-Specific Complexities & Implementation Considerations	110
20.4.1	1. Pointer Traversal in Python	110
20.4.2	2. Lexical Substring and Garbage Collection	110
20.5	Complexity Analysis	110
20.6	Common Follow-Up Questions & How to Answer	111
20.6.1	Q1: How would you validate if the input serialization string is well-formed?	111
20.6.2	Q2: Can you write a generator to yield flat integers from the NestedInteger?	111
20.7	Pro-Tip: How to Impress the Interviewer	111
21	Longest Absolute File Path	111
21.1	Question Description	111
21.1.1	Examples	112
21.2	Detailed Solution (C++)	112
21.2.1	Standard C++ Production Code	112
21.3	Detailed Solution (Python)	114
21.3.1	Standard Python Production Code	114
21.4	Python-Specific Complexities & Implementation Considerations	114
21.4.1	1. <code>split('\n')</code> Memory Overhead	114
21.4.2	2. Tab Counting & Left Stripping	115
21.5	Complexity Analysis	115

21.6	Common Follow-Up Questions & How to Answer	115
21.6.1	Q1: What if multiple tabs or backslashes represent tabs?	115
21.6.2	Q2: How can we return the actual path string instead of just its length?	115
21.7	Pro-Tip: How to Impress the Interviewer	115
22	Delete Node in a BST	116
22.1	Question Description	116
22.1.1	Examples	116
22.2	Detailed Solution (C++)	116
22.2.1	Standard C++ Production Code	117
22.3	Detailed Solution (Python)	118
22.3.1	Standard Python Production Code	118
22.4	Python-Specific Complexities & Implementation Considerations	119
22.4.1	1. Recursion Stack Limit	119
22.4.2	2. TreeNode Identity vs Value Swaps	119
22.5	Complexity Analysis	120
22.6	Common Follow-Up Questions & How to Answer	120
22.6.1	Q1: Can we delete a node without copying/swapping values? (Structural Deletion)	120
22.6.2	Q2: How can we implement this iteratively in $\mathcal{O}(1)$ space?	121
22.7	Pro-Tip: How to Impress the Interviewer	121
23	Most Frequent Subtree Sum	121
23.1	Question Description	121
23.1.1	Examples	121
23.2	Detailed Solution (C++)	122
23.2.1	Standard C++ Production Code	122
23.3	Detailed Solution (Python)	123
23.3.1	Standard Python Production Code	123
23.4	Python-Specific Complexities & Implementation Considerations	124
23.4.1	1. <code>nonlocal</code> Keyword Usage	124
23.4.2	2. Hash Map Keys in Python	125
23.5	Complexity Analysis	125
23.6	Common Follow-Up Questions & How to Answer	125
23.6.1	Q1: What if node values can be extremely large, leading to integer overflow?	125
23.6.2	Q2: How can we implement this iteratively to avoid stack overflow?	125
23.7	Pro-Tip: How to Impress the Interviewer	125
24	Convert BST to Greater Tree	126
24.1	Question Description	126
24.1.1	Examples	126
24.2	Detailed Solution (C++)	126
24.2.1	Standard C++ Production Code	127
24.3	Detailed Solution (Python)	127
24.3.1	Standard Python Production Code	127
24.4	Python-Specific Complexities & Implementation Considerations	128
24.4.1	1. The <code>nonlocal</code> Keyword	128
24.4.2	2. Space Limit and Deep Recursion	129
24.5	Complexity Analysis	129
24.6	Common Follow-Up Questions & How to Answer	129
24.6.1	Q1: How can we implement this iteratively?	129
24.6.2	Q2: Can we achieve $\mathcal{O}(1)$ auxiliary space complexity?	129

24.7 Pro-Tip: How to Impress the Interviewer	130
25 Array Nesting	131
25.1 Question Description	131
25.1.1 Examples	131
25.2 Detailed Solution (C++)	131
25.2.1 Standard C++ Production Code	132
25.3 Detailed Solution (Python)	132
25.3.1 Standard Python Production Code	132
25.4 Python-Specific Complexities & Implementation Considerations	133
25.4.1 1. In-place Mutation Warnings	133
25.5 Complexity Analysis	134
25.6 Common Follow-Up Questions & How to Answer	134
25.6.1 Q1: What if elements can point out-of-bounds or don't form a permutation?	134
25.6.2 Q2: Can we parallelize this cycle detection?	134
25.7 Pro-Tip: How to Impress the Interviewer	134
26 Subtree of Another Tree	134
26.1 Question Description	135
26.1.1 Examples	135
26.2 Detailed Solution (C++)	135
26.2.1 Standard C++ Production Code	135
26.3 Detailed Solution (Python)	136
26.3.1 Standard Python Production Code	136
26.4 Python-Specific Complexities & Implementation Considerations	137
26.4.1 1. Object Identity vs Value Equivalence	137
26.4.2 2. Tail Recursion Limitation	137
26.5 Complexity Analysis	138
26.6 Common Follow-Up Questions & How to Answer	138
26.6.1 Q1: How can we solve this problem in $\mathcal{O}(n + m)$ time complexity?	138
26.6.2 Q2: What is the Tree Hashing / Merkle Tree approach?	138
26.7 Pro-Tip: How to Impress the Interviewer	138
27 Max Area of Island	139
27.1 Question Description	139
27.1.1 Examples	139
27.2 Detailed Solution (C++)	140
27.2.1 Standard C++ Production Code	140
27.3 Detailed Solution (Python)	141
27.3.1 Standard Python Production Code	141
27.4 Python-Specific Complexities & Implementation Considerations	142
27.4.1 1. Recursion Stack Limits	142
27.5 Complexity Analysis	142
27.6 Common Follow-Up Questions & How to Answer	142
27.6.1 Q1: What if we are not allowed to mutate the input grid?	142
27.6.2 Q2: How does BFS compare to DFS in terms of memory overhead?	142
27.7 Pro-Tip: How to Impress the Interviewer	143
28 Merge k Sorted Lists	143
28.1 Question Description	143
28.1.1 Examples	143

28.2	Detailed Solution (C++)	144
28.2.1	Standard C++ Production Code	144
28.3	Detailed Solution (Python)	145
28.3.1	Standard Python Production Code	145
28.4	Python-Specific Complexities & Implementation Considerations	147
28.4.1	1. Priority Queue / Heap Gotcha in Python	147
28.4.2	2. Recursion Limit	147
28.4.3	3. GC and Reference Cycles	147
28.5	Complexity Analysis	147
28.6	Common Follow-Up Questions & How to Answer	147
28.6.1	Q1: What if the lists are too large to fit in memory?	147
28.6.2	Q2: Heap vs. Divide and Conquer — which is better?	148
28.7	Pro-Tip: How to Impress the Interviewer	148
29	Majority Element	148
29.1	Question Description	148
29.1.1	Examples	148
29.2	Detailed Solution (C++)	149
29.2.1	Boyer-Moore Voting Algorithm Logic	149
29.2.2	Standard C++ Production Code	149
29.3	Detailed Solution (Python)	151
29.3.1	Standard Python Production Code	151
29.4	Python-Specific Complexities & Implementation Considerations	152
29.4.1	1. Built-in <code>collections.Counter</code>	152
29.4.2	2. Slicing in Recursion	152
29.5	Complexity Analysis	152
29.5.1	Boyer-Moore Voting Algorithm:	152
29.5.2	Divide and Conquer:	152
29.6	Common Follow-Up Questions & How to Answer	153
29.6.1	Q1: What if the majority element is not guaranteed to exist?	153
29.6.2	Q2: How can we find all elements that appear more than $\lfloor n/3 \rfloor$ times? (LeetCode 229)	153
29.7	Pro-Tip: How to Impress the Interviewer	153
30	Construct Quad Tree	153
30.1	Question Description	154
30.1.1	Quad-Tree Construction Algorithm	154
30.1.2	Examples	154
30.2	Detailed Solution (C++)	154
30.2.1	Standard C++ Production Code (Bottom-Up $\mathcal{O}(n^2)$)	154
30.3	Detailed Solution (Python)	156
30.3.1	Standard Python Production Code (Bottom-Up $\mathcal{O}(n^2)$)	156
30.4	Python-Specific Complexities & Implementation Considerations	157
30.4.1	1. Object Allocation Overhead	157
30.4.2	2. Tail Call Elimination	157
30.5	Complexity Analysis	157
30.6	Common Follow-Up Questions & How to Answer	157
30.6.1	Q1: What is the benefit of a Quad-Tree over a flat grid?	157
30.6.2	Q2: How can we implement this iteratively?	157
30.7	Pro-Tip: How to Impress the Interviewer	158

31 Reverse Pairs	158
31.1 Question Description	158
31.1.1 Examples	158
31.2 Detailed Solution (C++)	159
31.2.1 Algorithm Logic	159
31.2.2 Standard C++ Production Code	159
31.3 Detailed Solution (Python)	160
31.3.1 Standard Python Production Code	160
31.4 Python-Specific Complexities & Implementation Considerations	161
31.4.1 1. Arithmetic Overflow	161
31.4.2 2. Timsort Optimizations	161
31.5 Complexity Analysis	161
31.6 Common Follow-Up Questions & How to Answer	162
31.6.1 Q1: Can we solve this using Binary Indexed Tree (BIT) / Fenwick Tree?	162
31.6.2 Q2: What are the differences between Inversion Count and Reverse Pairs?	162
31.7 Pro-Tip: How to Impress the Interviewer	162
32 Sort an Array	162
32.1 Question Description	163
32.1.1 Examples	163
32.2 Detailed Solution (C++)	163
32.2.1 Why Merge Sort?	163
32.2.2 Standard C++ Production Code (Merge Sort)	163
32.3 Detailed Solution (Python)	164
32.3.1 Standard Python Production Code (Merge Sort)	164
32.4 Python-Specific Complexities & Implementation Considerations	166
32.4.1 1. In-place vs. Out-of-place Slicing	166
32.4.2 2. Timsort (Python's Built-in)	166
32.5 Complexity Analysis	166
32.6 Common Follow-Up Questions & How to Answer	166
32.6.1 Q1: Why is Quick Sort preferred over Merge Sort for arrays, but Merge Sort is preferred for linked lists?	166
32.6.2 Q2: What is the optimal sorting algorithm if values are highly bounded?	166
32.7 Pro-Tip: How to Impress the Interviewer	167
33 Find Median from Data Stream	167
33.1 Question Description	167
33.1.1 Examples	167
33.2 Detailed Solution (C++)	168
33.2.1 Rebalancing Invariant	168
33.2.2 Algorithm	168
33.2.3 Standard C++ Production Code	168
33.3 Detailed Solution (Python)	169
33.3.1 Standard Python Production Code	169
33.4 Python-Specific Complexities & Implementation Considerations	170
33.4.1 1. Max-Heap Implementation in Python	170
33.4.2 2. Float Division / vs Floor Division //	170
33.5 Complexity Analysis	170
33.6 Common Follow-Up Questions & How to Answer	171
33.6.1 Q1: What if all integer numbers from the stream are between 0 and 100?	171
33.6.2 Q2: What if 99% of all integer numbers from the stream are between 0 and 100?	171

33.7 Pro-Tip: How to Impress the Interviewer	171
34 Design Twitter	171
34.1 Question Description	171
34.1.1 Examples	172
34.2 Detailed Solution (C++)	172
34.2.1 Standard C++ Production Code	172
34.3 Detailed Solution (Python)	174
34.3.1 Standard Python Production Code	174
34.4 Python-Specific Complexities & Implementation Considerations	176
34.4.1 1. Inefficient Heap Pushing	176
34.4.2 2. Set Discard vs. Remove	176
34.5 Complexity Analysis	176
34.6 Common Follow-Up Questions & How to Answer	177
34.6.1 Q1: Push vs. Pull models for News Feeds. Which is better?	177
34.6.2 Q2: How do we scale this for a distributed database?	177
34.7 Pro-Tip: How to Impress the Interviewer	177
35 Sliding Window Median	177
35.1 Question Description	177
35.1.1 Examples	178
35.2 Detailed Solution (C++)	178
35.2.1 Lazy Deletion Logic	178
35.2.2 Standard C++ Production Code	178
35.3 Detailed Solution (Python)	181
35.3.1 Standard Python Production Code	181
35.4 Python-Specific Complexities & Implementation Considerations	183
35.4.1 1. Simulating Deletion in Heap	183
35.4.2 2. Float Underflow/Overflow	183
35.5 Complexity Analysis	183
35.6 Common Follow-Up Questions & How to Answer	184
35.6.1 Q1: Can we use standard Binary Search Tree (BST) structures?	184
35.6.2 Q2: How would you parallelize this for huge data streams?	184
35.7 Pro-Tip: How to Impress the Interviewer	184
36 Kth Largest Element in an Array	184
36.1 Question Description	185
36.1.1 Examples	185
36.2 Detailed Solution (C++)	185
36.2.1 Standard C++ Production Code	185
36.3 Detailed Solution (Python)	186
36.3.1 Standard Python Production Code (Min-Heap)	186
36.4 Python-Specific Complexities & Implementation Considerations	187
36.4.1 1. <code>heapq.nlargest</code> Performance	187
36.4.2 2. Quickselect Stack Overflow Risk	187
36.5 Complexity Analysis	187
36.5.1 Min-Heap:	187
36.5.2 Quickselect / Introselect:	187
36.6 Common Follow-Up Questions & How to Answer	187
36.6.1 Q1: When is the Min-Heap approach preferred over Quickselect?	187
36.6.2 Q2: What if the range of elements is small? (e.g., values in $[-10000, 10000]$)	188

36.7 Pro-Tip: How to Impress the Interviewer	188
37 Top K Frequent Elements	188
37.1 Question Description	188
37.1.1 Follow up:	188
37.1.2 Examples	188
37.2 Detailed Solution (C++)	189
37.2.1 Standard C++ Production Code (Bucket Sort)	189
37.3 Detailed Solution (Python)	190
37.3.1 Standard Python Production Code (Bucket Sort)	190
37.4 Python-Specific Complexities & Implementation Considerations	191
37.4.1 1. Built-in <code>Counter.most_common</code>	191
37.4.2 2. Space Overhead of Hash Maps and Buckets	191
37.5 Complexity Analysis	191
37.5.1 Bucket Sort:	191
37.5.2 Min-Heap:	191
37.6 Common Follow-Up Questions & How to Answer	192
37.6.1 Q1: What if the elements are streaming?	192
37.6.2 Q2: Can we solve this using Quickselect?	192
37.7 Pro-Tip: How to Impress the Interviewer	192
38 Trapping Rain Water II	192
38.1 Question Description	192
38.1.1 Examples	192
38.2 Detailed Solution (C++)	193
38.2.1 Standard C++ Production Code	193
38.3 Detailed Solution (Python)	195
38.3.1 Standard Python Production Code	195
38.4 Python-Specific Complexities & Implementation Considerations	196
38.4.1 1. Dynamic Tuple Comparisons in Heap	196
38.4.2 2. Large Matrix Allocations	196
38.5 Complexity Analysis	196
38.6 Common Follow-Up Questions & How to Answer	196
38.6.1 Q1: Why must we use a Min-Heap instead of standard Breadth-First Search (BFS)?	196
38.6.2 Q2: How does this relate to Trapping Rain Water I (1D version)?	197
38.7 Pro-Tip: How to Impress the Interviewer	197
39 K Closest Points to Origin	197
39.1 Question Description	197
39.1.1 Examples	197
39.2 Detailed Solution (C++)	198
39.2.1 Standard C++ Production Code	198
39.3 Detailed Solution (Python)	199
39.3.1 Standard Python Production Code (Max-Heap)	199
39.4 Python-Specific Complexities & Implementation Considerations	200
39.4.1 1. Simulating Max-Heap with Negation	200
39.4.2 2. Built-in <code>heapq.nsmallest</code>	200
39.5 Complexity Analysis	200
39.5.1 Quickselect (C++):	200
39.5.2 Max-Heap (Python):	200
39.6 Common Follow-Up Questions & How to Answer	201

39.6.1	Q1: Why do we avoid the <code>sqrt</code> function?	201
39.6.2	Q2: What if we have a constant stream of new points?	201
39.7	Pro-Tip: How to Impress the Interviewer	201
40	Implement Rand10() Using Rand7()	201
40.1	Question Description	202
40.1.1	Follow up:	202
40.1.2	Examples	202
40.2	Detailed Solution (C++)	202
40.2.1	Standard C++ Production Code	202
40.3	Detailed Solution (Python)	203
40.3.1	Standard Python Production Code	203
40.4	Python-Specific Complexities & Implementation Considerations	203
40.4.1	1. Loop Termination & Infinite Loops	203
40.4.2	2. Random Number Generator Seeding	203
40.5	Complexity Analysis	204
40.6	Common Follow-Up Questions & How to Answer	204
40.6.1	Q1: What is the expected number of calls to <code>rand7()</code> in the basic implementation?	204
40.6.2	Q2: How can we optimize this to minimize the expected number of calls?	204
40.7	Pro-Tip: How to Impress the Interviewer	204
41	Random Flip Matrix	205
41.1	Question Description	205
41.1.1	Constraints	205
41.1.2	Examples	205
41.2	Detailed Solution (C++)	206
41.2.1	Lazy Fisher-Yates Logic	206
41.2.2	Standard C++ Production Code	206
41.3	Detailed Solution (Python)	207
41.3.1	Standard Python Production Code	207
41.4	Python-Specific Complexities & Implementation Considerations	208
41.4.1	1. Modulo and Division	208
41.4.2	2. Random Integers using <code>randint</code>	209
41.5	Complexity Analysis	209
41.6	Common Follow-Up Questions & How to Answer	209
41.6.1	Q1: Why is this better than using a set to track flipped indices?	209
41.6.2	Q2: What happens if $m \times n$ is larger than standard integer limits (e.g. 10^{18})?	209
41.7	Pro-Tip: How to Impress the Interviewer	209
42	Number of 1 Bits	210
42.1	Question Description	210
42.1.1	Examples	210
42.1.2	Constraints	210
42.2	Detailed Solution (C++)	210
42.2.1	Standard C++ Production Code	211
42.3	Detailed Solution (Python)	211
42.3.1	Standard Python Production Code	211
42.4	Python-Specific Complexities & Implementation Considerations	212
42.4.1	1. Python 3.10+ <code>int.bit_count()</code>	212
42.4.2	2. Arbitrary Precision and Sign Bit	212
42.5	Complexity Analysis	212

42.6	Common Follow-Up Questions & How to Answer	212
42.6.1	Q1: What is the complexity if you need to run this on a stream of 64-bit integers?	212
42.6.2	Q2: How can we implement bit count using parallel bit summation (SWAR - SIMD Within A Register)?	213
42.7	Pro-Tip: How to Impress the Interviewer	213
43	Single Number	213
43.1	Question Description	213
43.1.1	Examples	214
43.1.2	Constraints	214
43.2	Detailed Solution (C++)	214
43.2.1	Standard C++ Production Code	214
43.3	Detailed Solution (Python)	215
43.3.1	Standard Python Production Code	215
43.4	Python-Specific Complexities & Implementation Considerations	215
43.4.1	1. The <code>reduce</code> and <code>operator</code> Shorthand	215
43.4.2	2. Large Integer Bit Lengths	215
43.5	Complexity Analysis	216
43.6	Common Follow-Up Questions & How to Answer	216
43.6.1	Q1: What if all other numbers appear three times instead of twice? (LeetCode 137)	216
43.6.2	Q2: What if there are two single numbers instead of one? (LeetCode 260)	216
43.7	Pro-Tip: How to Impress the Interviewer	217
44	Counting Bits	217
44.1	Question Description	217
44.1.1	Examples	217
44.1.2	Constraints	218
44.2	Detailed Solution (C++)	218
44.2.1	Approach 1: Most Significant Bit (MSB) / Shift Property	218
44.2.2	Approach 2: Least Significant Bit (LSB) / Brian Kernighan clearing	218
44.2.3	Standard C++ Production Code	218
44.3	Detailed Solution (Python)	219
44.3.1	Standard Python Production Code	219
44.4	Python-Specific Complexities & Implementation Considerations	219
44.4.1	1. Integer Bitwise Shift	219
44.4.2	2. Pre-allocation of List Memory	219
44.5	Complexity Analysis	220
44.6	Common Follow-Up Questions & How to Answer	220
44.6.1	Q1: Can you solve this using a divide and conquer technique?	220
44.6.2	Q2: How would you parallelize this for extremely large N (e.g. $N = 10^9$)?	220
44.7	Pro-Tip: How to Impress the Interviewer	220
45	Number Complement	220
45.1	Question Description	221
45.1.1	Examples	221
45.1.2	Constraints	221
45.2	Detailed Solution (C++)	221
45.2.1	C++ Integer Overflow Safety:	222
45.2.2	Standard C++ Production Code	222
45.3	Detailed Solution (Python)	223
45.3.1	Standard Python Production Code	223

45.4	Python-Specific Complexities & Implementation Considerations	223
45.4.1	1. The <code>bit_length()</code> Method	223
45.4.2	2. Arbitrary Precision Negation	223
45.5	Complexity Analysis	223
45.6	Common Follow-Up Questions & How to Answer	224
45.6.1	Q1: What if <code>num = 0</code> ?	224
45.6.2	Q2: Can we solve this branchlessly without using loops or intrinsic CLZ?	224
45.7	Pro-Tip: How to Impress the Interviewer	224
46	Poor Pigs	225
46.1	Question Description	225
46.1.1	Examples	225
46.2	Detailed Solution (C++)	225
46.2.1	Standard C++ Production Code	226
46.3	Detailed Solution (Python)	227
46.3.1	Standard Python Production Code	227
46.4	Python-Specific Complexities & Implementation Considerations	227
46.4.1	1. Floating-Point Precision Risk with <code>math.log</code>	227
46.5	Complexity Analysis	228
46.6	Common Follow-Up Questions & How to Answer	228
46.6.1	Q1: How do we actually schedule the feedings (the experiment design)?	228
46.6.2	Q2: What if pigs can recover after a round?	228
46.7	Pro-Tip: How to Impress the Interviewer	228
47	Generate Random Point in a Circle	229
47.1	Question Description	229
47.1.1	Examples	229
47.2	Detailed Solution (C++)	229
47.2.1	Method 1: Rejection Sampling (Loop-based)	229
47.2.2	Method 2: Polar Coordinates with Inverse CDF (Analytical $O(1)$)	230
47.2.3	Standard C++ Production Code (Method 2: Analytical $O(1)$)	230
47.3	Detailed Solution (Python)	231
47.3.1	Standard Python Production Code (Method 2: Analytical $O(1)$)	231
47.4	Python-Specific Complexities & Implementation Considerations	231
47.4.1	1. Rejection Sampling vs. Inverse CDF Performance in Python	231
47.4.2	2. High Precision Float Math	232
47.5	Complexity Analysis	232
47.5.1	Method 1: Rejection Sampling	232
47.5.2	Method 2: Polar Coordinates (Inverse CDF)	232
47.6	Common Follow-Up Questions & How to Answer	232
47.6.1	Q1: Which method is preferred if generating random numbers is extremely expensive?	232
47.6.2	Q2: How would you extend this to generate a random point on a 3D sphere?	232
47.7	Pro-Tip: How to Impress the Interviewer	233
48	Largest Palindrome Product	233
48.1	Question Description	233
48.1.1	Examples	233
48.2	Detailed Solution (C++)	234
48.2.1	Standard C++ Production Code	234
48.3	Detailed Solution (Python)	235
48.3.1	Standard Python Production Code	235

48.4	Python-Specific Complexities & Implementation Considerations	236
48.4.1	1. Palindrome Generation: Arithmetic vs String Reversal	236
48.4.2	2. Large Integer Arithmetic	236
48.5	Complexity Analysis	236
48.6	Common Follow-Up Questions & How to Answer	237
48.6.1	Q1: Why do we start the search from upper and go downwards?	237
48.6.2	Q2: Why does <code>right * right < palindrome</code> allow us to break early?	237
48.7	Pro-Tip: How to Impress the Interviewer	237
49	Next Greater Element III	237
49.1	Question Description	237
49.1.1	Examples	238
49.2	Detailed Solution (C++)	238
49.2.1	Standard C++ Production Code	238
49.3	Detailed Solution (Python)	239
49.3.1	Standard Python Production Code	239
49.4	Python-Specific Complexities & Implementation Considerations	240
49.4.1	1. Mutable Strings	240
49.4.2	2. Standard 32-bit Signed Integer Limits	240
49.5	Complexity Analysis	240
49.6	Common Follow-Up Questions & How to Answer	241
49.6.1	Q1: What is the underlying math of the next permutation algorithm?	241
49.6.2	Q2: Can we solve this problem without converting the number to a string?	241
49.7	Pro-Tip: How to Impress the Interviewer	241
50	Find the Closest Palindrome	242
50.1	Question Description	242
50.1.1	Examples	242
50.2	Detailed Solution (C++)	242
50.2.1	Standard C++ Production Code	243
50.3	Detailed Solution (Python)	244
50.3.1	Standard Python Production Code	244
50.4	Python-Specific Complexities & Implementation Considerations	245
50.4.1	1. Large Integer Conversions	245
50.4.2	2. Custom Key Sorting with <code>min()</code>	246
50.5	Complexity Analysis	246
50.6	Common Follow-Up Questions & How to Answer	246
50.6.1	Q1: Why do we need the prefix - 1 and prefix + 1 candidates?	246
50.6.2	Q2: Why are the boundary candidates 99...9 and 10...01 necessary?	246
50.7	Pro-Tip: How to Impress the Interviewer	246
51	Climbing Stairs	247
51.1	Question Description	247
51.1.1	Examples	247
51.2	Detailed Solution (C++)	248
51.2.1	Standard C++ Production Code	248
51.3	Detailed Solution (Python)	248
51.3.1	Standard Python Production Code	248
51.4	Python-Specific Complexities & Implementation Considerations	249
51.4.1	1. Python Integer Precision	249
51.4.2	2. Tail-Call Optimization (TCO) Lack in Python	249

51.5	Complexity Analysis	249
51.6	Common Follow-Up Questions & How to Answer	250
51.6.1	Q1: What if you can climb 1, 2, or 3 steps at a time? (Tribonacci)	250
51.6.2	Q2: What if we want to solve it in $\mathcal{O}(\log n)$ time?	250
51.7	Pro-Tip: How to Impress the Interviewer	250
52	House Robber	250
52.1	Question Description	251
52.1.1	Examples	251
52.2	Detailed Solution (C++)	251
52.2.1	Standard C++ Production Code	251
52.3	Detailed Solution (Python)	252
52.3.1	Standard Python Production Code	252
52.4	Python-Specific Complexities & Implementation Considerations	253
52.4.1	1. Simultaneous Assignment (<code>prev, curr = curr, max(curr, prev + num)</code>)	253
52.4.2	2. Space Optimization	253
52.5	Complexity Analysis	253
52.6	Common Follow-Up Questions & How to Answer	253
52.6.1	Q1: What if the houses are arranged in a circle? (House Robber II - LeetCode 213)	253
52.6.2	Q2: What if the houses are organized in a binary tree? (House Robber III - LeetCode 337)	253
52.7	Pro-Tip: How to Impress the Interviewer	254
53	Coin Change	254
53.1	Question Description	254
53.1.1	Examples	254
53.2	Detailed Solution (C++)	255
53.2.1	Standard C++ Production Code	255
53.3	Detailed Solution (Python)	256
53.3.1	Standard Python Production Code	256
53.4	Python-Specific Complexities & Implementation Considerations	256
53.4.1	1. Loop Order Optimization	256
53.4.2	2. Large Coin Values	257
53.5	Complexity Analysis	257
53.6	Common Follow-Up Questions & How to Answer	257
53.6.1	Q1: How would you return the actual coins used to form the amount, rather than just the count?	257
53.6.2	Q2: What if we wanted to solve this using BFS?	257
53.7	Pro-Tip: How to Impress the Interviewer	257
54	Frog Jump	258
54.1	Question Description	258
54.1.1	Examples	258
54.2	Detailed Solution (C++)	258
54.2.1	Standard C++ Production Code	259
54.3	Detailed Solution (Python)	260
54.3.1	Standard Python Production Code	260
54.4	Python-Specific Complexities & Implementation Considerations	261
54.4.1	1. Hash Map (memo) vs 2D Array	261
54.4.2	2. Recursion Limit	261
54.5	Complexity Analysis	261

54.6	Common Follow-Up Questions & How to Answer	262
54.6.1	Q1: Can we use BFS instead of DFS?	262
54.6.2	Q2: What happens if the positions in <code>stones</code> are not sorted?	262
54.7	Pro-Tip: How to Impress the Interviewer	262
55	Count The Repetitions	262
55.1	Question Description	263
55.1.1	Examples	263
55.2	Detailed Solution (C++)	263
55.2.1	Cycle Detection (Pigeonhole Principle)	263
55.2.2	Standard C++ Production Code	264
55.3	Detailed Solution (Python)	266
55.3.1	Standard Python Production Code	266
55.4	Python-Specific Complexities & Implementation Considerations	267
55.4.1	1. Fast Character Set Verification	267
55.4.2	2. Hash Map Performance	267
55.5	Complexity Analysis	267
55.6	Common Follow-Up Questions & How to Answer	268
55.6.1	Q1: What if n_1 is extremely large (e.g., up to 10^{12})?	268
55.6.2	Q2: Why is the outer loop index bound by $ s_2 $ rather than $ s_1 $?	268
55.7	Pro-Tip: How to Impress the Interviewer	268
56	Unique Substrings in Wraparound String	268
56.1	Question Description	269
56.1.1	Examples	269
56.2	Detailed Solution (C++)	269
56.2.1	Standard C++ Production Code	270
56.3	Detailed Solution (Python)	270
56.3.1	Standard Python Production Code	270
56.4	Python-Specific Complexities & Implementation Considerations	271
56.4.1	1. Character to Integer Conversion	271
56.4.2	2. Space Optimization	271
56.5	Complexity Analysis	271
56.6	Common Follow-Up Questions & How to Answer	272
56.6.1	Q1: Why does summing the maximum length of substrings ending at each character guarantee uniqueness?	272
56.6.2	Q2: What if the base string was not a wraparound string (e.g., 'z' does not wrap to 'a')?	272
56.7	Pro-Tip: How to Impress the Interviewer	272
57	Freedom Trail	272
57.1	Question Description	273
57.1.1	Examples	273
57.2	Detailed Solution (C++)	273
57.2.1	Standard C++ Production Code	274
57.3	Detailed Solution (Python)	275
57.3.1	Standard Python Production Code	275
57.4	Python-Specific Complexities & Implementation Considerations	276
57.4.1	1. Precomputing Character Positions	276
57.4.2	2. Move Semantics in Python vs C++	276
57.5	Complexity Analysis	276

57.6	Common Follow-Up Questions & How to Answer	277
57.6.1	Q1: Can we use Dijkstra's Algorithm instead of DP?	277
57.6.2	Q2: What if we can also rotate the ring clockwise or anticlockwise multiple steps per operation?	277
57.7	Pro-Tip: How to Impress the Interviewer	277
58	Longest Palindromic Subsequence	277
58.1	Question Description	277
58.1.1	Examples	277
58.2	Detailed Solution (C++)	278
58.2.1	Standard C++ Production Code	278
58.3	Detailed Solution (Python)	279
58.3.1	Standard Python Production Code	279
58.4	Python-Specific Complexities & Implementation Considerations	280
58.4.1	1. Space Optimization to 1D Array	280
58.5	Complexity Analysis	280
58.6	Common Follow-Up Questions & How to Answer	280
58.6.1	Q1: How does this relate to Longest Common Subsequence (LCS)?	280
58.6.2	Q2: How can we reconstruct the actual longest palindromic subsequence string?	281
58.7	Pro-Tip: How to Impress the Interviewer	281
59	Coin Change II	281
59.1	Question Description	281
59.1.1	Examples	281
59.2	Detailed Solution (C++)	282
59.2.1	Transition and Loop Order	282
59.2.2	Standard C++ Production Code	282
59.3	Detailed Solution (Python)	283
59.3.1	Standard Python Production Code	283
59.4	Python-Specific Complexities & Implementation Considerations	283
59.4.1	1. Swapping Loop Order (The Permutations Trap)	283
59.4.2	2. Large Precision	284
59.5	Complexity Analysis	284
59.6	Common Follow-Up Questions & How to Answer	284
59.6.1	Q1: How do we modify the algorithm to compute permutations (where order matters)?	284
59.6.2	Q2: What if we are limited in the number of coins we can use (e.g. 0-1 Knapsack where each coin can only be used once)? (Coin Change II with single-use coins)	284
59.7	Pro-Tip: How to Impress the Interviewer	285
60	Student Attendance Record II	285
60.1	Question Description	285
60.1.1	Examples	285
60.2	Detailed Solution (C++)	286
60.2.1	Transitions	286
60.2.2	Space Optimization	286
60.2.3	Standard C++ Production Code	286
60.3	Detailed Solution (Python)	288
60.3.1	Standard Python Production Code	288
60.4	Python-Specific Complexities & Implementation Considerations	289
60.4.1	1. Multi-Dimensional Lists vs Flattening	289
60.4.2	2. Time Limit Exceeded (TLE) Risks in Python	289

60.5	Complexity Analysis	289
60.6	Common Follow-Up Questions & How to Answer	289
60.6.1	Q1: What if n is up to 10^{18} ?	289
60.6.2	Q2: How does the transition matrix M look?	290
60.7	Pro-Tip: How to Impress the Interviewer	290
61	Optimal Division	290
61.1	Question Description	290
61.1.1	Examples	291
61.2	Detailed Solution (C++)	291
61.2.1	Mathematical Proof of Greedy Optimality	291
61.2.2	Standard C++ Production Code (Greedy $\mathcal{O}(n)$)	292
61.3	Detailed Solution (Python)	292
61.3.1	Standard Python Production Code	292
61.4	Dynamic Programming Alternative (Conceptual)	293
61.5	Python-Specific Complexities & Implementation Considerations	293
61.5.1	1. Fast String Interpolation with f-Strings	293
61.5.2	2. Standard <code>map</code> and <code>join</code>	293
61.6	Complexity Analysis	293
61.7	Common Follow-Up Questions & How to Answer	294
61.7.1	Q1: What if the elements in <code>nums</code> could be float values between 0 and 1?	294
61.7.2	Q2: How do you prove that <code>nums[1]</code> must always be in the denominator?	294
61.8	Pro-Tip: How to Impress the Interviewer	294
62	Out of Boundary Paths	294
62.1	Question Description	295
62.1.1	Examples	295
62.2	Detailed Solution (C++)	295
62.2.1	Standard C++ Production Code	295
62.3	Detailed Solution (Python)	297
62.3.1	Standard Python Production Code	297
62.4	Python-Specific Complexities & Implementation Considerations	298
62.4.1	1. Flattening the 2D Grid	298
62.4.2	2. Iterative Space-swapping vs DFS Recursion	298
62.5	Complexity Analysis	298
62.6	Common Follow-Up Questions & How to Answer	298
62.6.1	Q1: What if <code>maxMove</code> is very large but the grid is small?	298
62.6.2	Q2: How can we optimize this if m and n are large but <code>maxMove</code> is very small?	298
62.7	Pro-Tip: How to Impress the Interviewer	299
63	Super Egg Drop	299
63.1	Question Description	299
63.1.1	Examples	299
63.2	Detailed Solution (C++)	300
63.2.1	Mathematical Re-framing (Dual DP)	300
63.2.2	Standard C++ Production Code	300
63.3	Detailed Solution (Python)	301
63.3.1	Standard Python Production Code	301
63.4	Python-Specific Complexities & Implementation Considerations	301
63.4.1	1. In-place Update Order	301
63.4.2	2. Time-Limit Safety	302

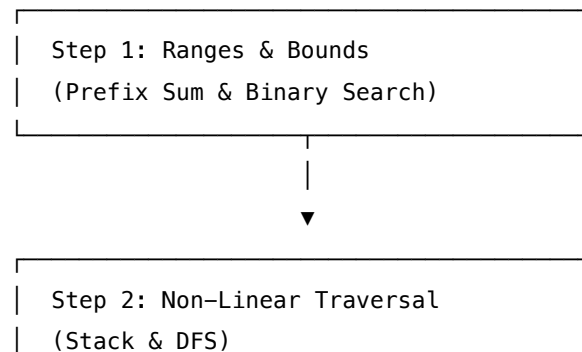
63.5 Complexity Analysis	302
63.6 Common Follow-Up Questions & How to Answer	302
63.6.1 Q1: What is the worst-case number of moves when $k = 1$?	302
63.6.2 Q2: What is the optimal number of moves when we have an infinite number of eggs ($k \geq \log_2(n) + 1$)?	302
63.7 Pro-Tip: How to Impress the Interviewer	302
64 LFU Cache	303
64.1 Question Description	303
64.1.1 Examples	303
64.1.2 Constraints	304
64.2 Detailed Solution (C++)	304
64.2.1 Update State (<code>touch</code> helper):	304
64.2.2 Standard C++ Production Code	304
64.3 Detailed Solution (Python)	306
64.3.1 Standard Python Production Code	306
64.4 Python-Specific Complexities & Implementation Considerations	308
64.4.1 1. OrderedDict vs Standard Set/Dict	308
64.4.2 2. Guarding against Capacity 0	308
64.5 Complexity Analysis	308
64.6 Common Follow-Up Questions & How to Answer	308
64.6.1 Q1: What is the difference between LRU (Least Recently Used) and LFU (Least Frequently Used)?	308
64.6.2 Q2: How can we prevent "frequency starvation" in LFU?	309
64.7 Pro-Tip: How to Impress the Interviewer	309

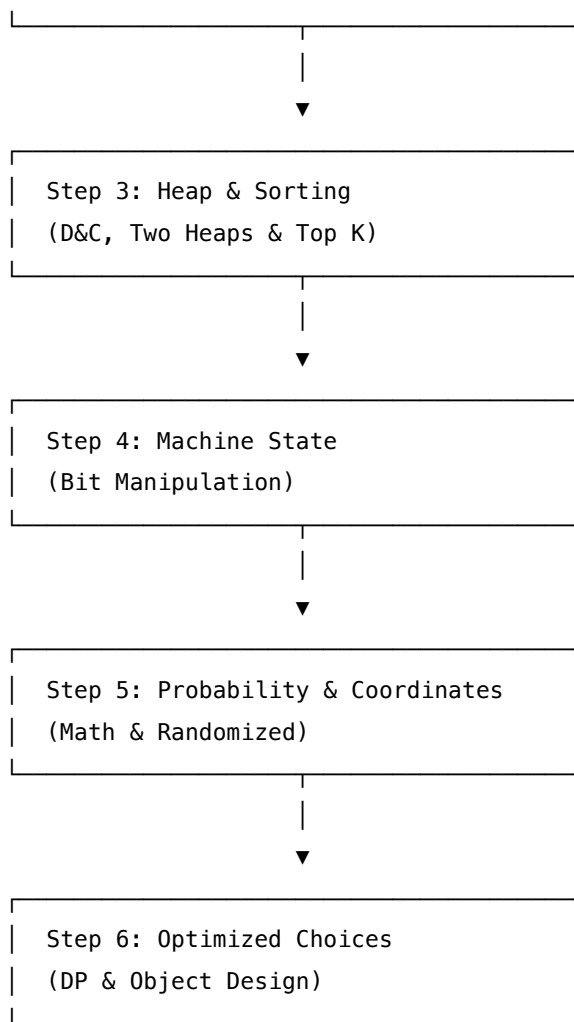
1 Tier 2: Intermediate — Coding Interview Preparation Roadmap

Welcome to the **Intermediate (Tier 2) Coding Roadmap**. This directory contains your structured study guide and indexed reference repository for mastering intermediate algorithmic design. These patterns extend the foundational structures to cover complex range queries, non-linear traversals, heap-based priority systems, low-level bit states, and dynamic programming optimization.

1.1 Progression Roadmap

To tackle intermediate problems, follow a systematic progression that moves from linear sum/search concepts into trees, priority queues, and advanced memoization.





1.2 Detailed Learning Modules & File Index

Here is the complete catalog of the 63 Intermediate problems, categorized by pattern, and arranged in recommended learning order.

1.2.1 Module 1: Prefix Sum & Binary Search

Goal: Search ranges, resolve subarray sums, and optimize threshold searches in logarithmic time.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Prefix Sum	Product of Array Except Self	LeetCode 238	Medium	Prefix/Suffix arrays consolidation, $\mathcal{O}(1)$ space
Prefix Sum	Subarray Sum Equals K	LeetCode 560	Medium	Cumulative sum hashing, frequency lookups

Pattern	Problem Name	Source	Difficulty	Core Concepts
Prefix Sum	Subarray Product Less Than K	LeetCode 713	Medium	Sliding window integration, combinatorics addition
Prefix Sum	Continuous Subarray Sum	LeetCode 523	Medium	Modulo hash matching, remainder tracking
Prefix Sum	Contiguous Array	LeetCode 525	Medium	Zero-one replacement offset, max distance
Prefix Sum	Random Pick with Weight	LeetCode 528	Medium	Cumulative probability array, binary search lookup
Binary Search	Find Minimum in Rotated Sorted	LeetCode 153	Medium	Unsorted-half boundaries, inflection identification
Binary Search	First Bad Version	LeetCode 278	Easy	Left-biased convergence, API minimization
Binary Search	Data Stream as Disjoint Intervals	LeetCode 352	Hard	Ordered tree boundaries lookup, interval merges
Binary Search	Russian Doll Envelopes	LeetCode 354	Hard	Dual criteria sorting, Longest Increasing Subsequence (LIS)
Binary Search	Smallest Good Base	LeetCode 483	Hard	Base representation math, geometric series binary search
Binary Search	Random Point in Rectangles	LeetCode 497	Medium	Weight areas prefix sums, coordinate binary search
Binary Search	Binary Search	LeetCode 704	Easy	Classical mid-point search, overflow protection

1.2.2 Module 2: Stack Parsing & Depth-First Search (DFS)

Goal: Process nested structures, validate expressions, and exhaustively explore graphs and binary trees.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Stack	Valid Parentheses	LeetCode 20	Easy	Expected closing bracket mapping, size validations
Stack	Min Stack	LeetCode 155	Easy	Auxiliary stack matching, difference-tracking values
Stack	Decode String	LeetCode 394	Medium	Count-and-buffer stacks, nested brackets tracking
DFS	Jump Game III	LeetCode 1306	Medium	Visited negations, array traversal limits
DFS	Number of Islands	LeetCode 200	Medium	Grid cell mutation, island perimeter floodfill
DFS	Mini Parser	LeetCode 385	Medium	Nested integer parsing, tree recursion
DFS	Longest Absolute File Path	LeetCode 388	Medium	Directory depth mapping, stack level calculations
DFS	Delete Node in a BST	LeetCode 450	Medium	Successor replacement, sub-tree structure rewires
DFS	Most Frequent Subtree Sum	LeetCode 508	Medium	Post-order traversal sums, hashmap frequencies tracking
DFS	Convert BST to Greater Tree	LeetCode 538	Medium	Reverse in-order traversal, accumulative values
DFS	Array Nesting	LeetCode 565	Medium	Permutation cycle length, value negation marker

Pattern	Problem Name	Source	Difficulty	Core Concepts
DFS	Subtree of Another Tree	LeetCode 572	Easy	Tree matching recursion, tree hashing alternative
DFS	Max Area of Island	LeetCode 695	Medium	Recursive area accumulation, in-place land sink

1.2.3 Module 3: Heap, Divide & Conquer, and Priority Systems

Goal: Split and solve subproblems, manage dynamic midpoints, and process top elements in stream collections.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Divide & Conquer	Merge K Sorted Lists	LeetCode 23	Hard	Min-heap node extraction, partition merge sort
Divide & Conquer	Majority Element	LeetCode 169	Easy	Boyer-Moore Voting Algorithm, partition checks
Divide & Conquer	Construct Quad Tree	LeetCode 427	Medium	Quad-partition matrix recursion, leaf collapses
Divide & Conquer	Reverse Pairs	LeetCode 493	Hard	Merge sort index tracking, inversion count
Divide & Conquer	Sort an Array	LeetCode 912	Medium	Quicksort/Mergesort/Heapsort implementation safety
Two Heaps	Find Median from Data Stream	LeetCode 295	Hard	Min-Heap/Max- Heap balancing, stream size constraints
Two Heaps	Design Twitter	LeetCode 355	Medium	User follow lists, heap merge sorted newsfeed posts
Two Heaps	Sliding Window Median	LeetCode 480	Hard	Double heaps with lazy removal, multiset BST
Top K Elements	Kth Largest in Array	LeetCode 215	Medium	Quickselect partition, small min-heap

Pattern	Problem Name	Source	Difficulty	Core Concepts
Top K Elements	Top K Frequent Elements	LeetCode 347	Medium	Frequency mapping, bucket sort, min-heap
Top K Elements	Trapping Rain Water II	LeetCode 407	Hard	3D grid cell boundary priority queue BFS
Top K Elements	K Closest Points to Origin	LeetCode 973	Medium	Euclidean distance calculation, max-heap size limit
Randomized	Implement Rand10() Using Rand7()	LeetCode 470	Medium	Rejection sampling, base-7 multiplication representation
Randomized	Random Flip Matrix	LeetCode 519	Medium	Map-based index redirection swaps, random bounds shrinking

1.2.4 Module 4: Bit Manipulation

Goal: Extract binary states, manage masks, and perform fast word-level operations using bitwise operators.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Bit Manipulation	Number of 1 Bits	LeetCode 191	Easy	Brian Kernighan's bit-unset ($n \& (n - 1)$)
Bit Manipulation	Single Number	LeetCode 136	Easy	XOR self-cancellation ($x \oplus x = 0$), parity accumulation
Bit Manipulation	Counting Bits	LeetCode 338	Easy	DP bit states, offset masks ($dp[i] = dp[i \gg 1] + (i \& 1)$)
Bit Manipulation	Number Complement	LeetCode 476	Easy	Binary bit-mask building, XOR negations

1.2.5 Module 5: Intermediate Mathematics

Goal: Formulate probabilistic logic, coordinate distributions, and number theory solutions.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Math	Poor Pigs	LeetCode 458	Hard	Base-state info theory, dimensions representation
Math	Random Point in Circle	LeetCode 478	Medium	Inverse transform sampling, square root radius scale
Math	Largest Palindrome Product	LeetCode 479	Hard	Palindrome construction, search factor limits, modulo
Math	Next Greater Element III	LeetCode 556	Medium	Next permutation algorithm, signed integer overflow check
Math	Find Closest Palindrome	LeetCode 564	Hard	Half-string mirroring, edge cases boundaries

1.2.6 Module 6: Dynamic Programming & High-Level System Design

Goal: Resolve complex overlapping subproblems and design cache eviction systems with strict time constraints.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Dynamic Programming	Climbing Stairs	LeetCode 70	Easy	Fibonacci sequence, state transition space reduction
Dynamic Programming	House Robber	LeetCode 198	Medium	1D DP state choices (<code>max(rob, skip)</code>), in-place variables
Dynamic Programming	Coin Change	LeetCode 322	Medium	1D knapsack DP, unbounded change, infinite state sentinel
Dynamic Programming	Frog Jump	LeetCode 403	Hard	DFS with memoized states, step distance increments

Pattern	Problem Name	Source	Difficulty	Core Concepts
Dynamic Programming	Count The Repetitions	LeetCode 466	Hard	Cycle matching detection, repeating states offsets
Dynamic Programming	Unique Substrings Wraparound	LeetCode 467	Medium	Alphabet distance loop constraints, max length tracking per char
Dynamic Programming	Freedom Trail	LeetCode 514	Hard	State choice matrix, character distance transitions
Dynamic Programming	Longest Palindromic Subseq	LeetCode 516	Medium	2D substring range DP, state compression
Dynamic Programming	Coin Change II	LeetCode 518	Medium	Unbounded knapsack combinations, loop ordering
Dynamic Programming	Student Attendance II	LeetCode 552	Hard	Multi-dimensional DP states matrix, modular addition
Dynamic Programming	Optimal Division	LeetCode 553	Medium	Math expression maximizing, division logic reduction
Dynamic Programming	Out of Boundary Paths	LeetCode 576	Medium	Grid coordinate step DP, boundaries aggregation
Dynamic Programming	Super Egg Drop	LeetCode 887	Hard	DP with binary search transition paths, step-count DP inversion
Design	LFU Cache	LeetCode 460	Hard	HashMap with doubly-linked frequency lists

1.3 Intermediate Tier Interview Strategy

1.3.1 1. Integer Shift Limits & Bit Safety

- **Bitwise Shifts (C++):** Left-shifting a signed integer beyond 31 bits (e.g. `1 << 31`) is undefined behavior in C++ and can cause integer overflow warnings or architecture crashes. Always use unsigned integer literals (e.g. `1ULL << 31`) when building wide masks.
- **Memory Allocations:** For bitset representations, pre-allocate space or utilize bit vectors if n fits within CPU word boundaries (64 bits). For larger sizes, fallback to `std::vector<bool>`, which is space-optimized by the C++ compiler to pack 8 bits per byte.

1.3.2 2. Stack and Recursion Depths

- **Python Call Stack Constraints:** In DFS graph problems, Python's default recursion stack limit is low (1000 frames). For deep tree structures or large grids (such as a 1000×1000 matrix), recursive DFS can crash with a `RecursionError`. Implement algorithms iteratively using an explicit `list` as a stack, or use `sys.setrecursionlimit` defensively.

1.3.3 3. Dynamic Heap Operations

- **Custom Heap Comparators (C++):** To implement a custom min/max heap configuration, declare a struct functor and pass it to `std::priority_queue`:

```
struct CustomCompare {
    bool operator()(const std::pair<int, int>& a, const std::pair<int, int>& b) {
        return a.first > b.first; // Min-heap behavior
    }
};

std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>, CustomCompare> min_heap;
```

- **Lazy Removal Optimization:** When elements inside a heap need updates/deletions (as in Sliding Window Median), avoid linear searching to delete. Instead, track the elements to be removed in an auxiliary "lazy removal" map, and only pop them from the heap when they reach the top.

2 Product of Array Except Self

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / Low-Level Developer
- **Source:** LeetCode 238, Glassdoor
- **Flashcards:** [Prefix Sum deck](#)

2.1 Question Description

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

You must write an algorithm that runs in $\mathcal{O}(n)$ time and **without using the division operation**.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit signed integer.

2.1.1 Examples

- **Input:** `nums = [1, 2, 3, 4]`
 - **Output:** `[24, 12, 8, 6]`
 - **Input:** `nums = [-1, 1, 0, -3, 3]`
 - **Output:** `[0, 0, 9, 0, 0]`
-

2.2 Detailed Solution (C++)

To solve this problem in $\mathcal{O}(n)$ time without division and in $\mathcal{O}(1)$ auxiliary space, we construct the prefix and suffix products directly within the output vector: 1. **Prefix Pass:** We iterate from left to right. We use the output array to store the prefix product. Specifically, `output[i]` will store the product of all elements to the left of index `i`. 2. **Suffix Pass:** We iterate from right to left. We maintain a running suffix product in a single variable (`right`). We multiply the current prefix product stored in `output[i]` by `right`, then update `right` by multiplying it by `nums[i]`.

This achieves the result in-place without allocating separate prefix and suffix arrays.

2.2.1 Standard C++ Production Code

```
#include <vector>

class Solution {
public:
    std::vector<int> productExceptSelf(const std::vector<int>& nums) {
        int n = static_cast<int>(nums.size());
        if (n == 0) return {};

        std::vector<int> output(n, 1);

        // Step 1: Compute prefix products in output array
        // output[i] contains the product of all elements to the left of i.
        int left = 1;
        for (int i = 0; i < n; ++i) {
            output[i] = left;
            left *= nums[i];
        }

        // Step 2: Compute suffix products on the fly and multiply with prefix products
        int right = 1;
        for (int i = n - 1; i >= 0; --i) {
```

```

        output[i] *= right;
        right *= nums[i];
    }

    return output;
}
};

```

2.3 Detailed Solution (Python)

2.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the single-array prefix/suffix technique.

```

from typing import List

class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        """
        Computes the product of array except self without division in  $O(N)$  time and  $O(1)$  auxiliary space.
        """
        n = len(nums)
        if n == 0:
            return []

        output = [1] * n

        # Step 1: Left prefix products
        left_product = 1
        for i in range(n):
            output[i] = left_product
            left_product *= nums[i]

        # Step 2: Right suffix products accumulated on the fly
        right_product = 1
        for i in range(n - 1, -1, -1):
            output[i] *= right_product
            right_product *= nums[i]

        return output

```

2.4 Python-Specific Complexities & Implementation Considerations

2.4.1 1. Division Operators and Zero Handling

- If we were allowed to use division, we might try: `total_product // nums[i]`. However, division in Python 3 using `/` yields a float, which can introduce precision errors for large integers.
- Furthermore, if the array contains `0`, division by zero raises a `ZeroDivisionError`. The prefix/suffix product approach is structurally robust and naturally handles any number of zeros without any branchy check logic.

2.4.2 2. Avoiding Intermediate Lists for Memory Efficiency

- A naive approach allocates two extra arrays: `left_products` and `right_products`, resulting in an auxiliary space complexity of $\mathcal{O}(n)$.
- By writing prefix products directly into the final output list and updating it using a scalar `right_product`, we satisfy the strict $\mathcal{O}(1)$ auxiliary space constraint.

2.4.3 3. Dynamic Arrays (Lists) vs Pre-allocated Lists

- Using `output = [1] * n` is faster than repeatedly calling `output.append()` because it allocates the block of memory all at once. Repeatedly appending elements causes Python's underlying dynamic array to resize and copy elements, adding a minor but noticeable overhead.

2.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Two linear scans over the array of size n .
Space Complexity	$\mathcal{O}(1)$	No extra memory is allocated besides the output array (which does not count towards auxiliary complexity).

2.6 Common Follow-Up Questions & How to Answer

2.6.1 Q1: What if division is allowed?

- **Answer:** If division is allowed, we can first count the number of zeros in `nums`:
 - If zero count ≥ 2 : The output array is entirely zeros.
 - If zero count = 1: Only the index containing the zero will have a non-zero product (which equals the product of all other elements). All other indices will be zero.
 - If zero count = 0: We calculate the product of all elements and then set `output[i] = total_product / nums[i]`.

- **Trade-off:** The prefix-suffix product algorithm is still superior because it is completely branch-free regarding zero detection, avoiding CPU pipeline stalls.

2.6.2 Q2: What if we need prefix products for operations that do not have inverses?

- **Answer:** The prefix-suffix strategy is highly general and applies to any **associative binary operation** $(S, \cdot)$. Examples include Matrix Multiplication, Greatest Common Divisor (GCD), Minimum/Maximum, and Boolean AND/OR. Since operations like GCD or Min/Max do not have inverse operations (analogous to division for multiplication), the prefix/suffix approach is the standard way to solve "except self" queries.
-

2.7 Pro-Tip: How to Impress the Interviewer

- **Mention Cache Locality:** Highlight that the left-to-right pass accesses `nums` sequentially, which perfectly aligns with modern CPU cache prefetching. The right-to-left pass accesses memory in reverse sequential order, which is also well-supported by hardware prefetchers, maintaining excellent cache efficiency compared to pointer-based or tree-based solutions.
- **Discuss Compiler Vectorization (SIMD):** Mention that while simple loops are easily vectorized by compilers, loop-carried dependencies (such as `left *= nums[i]`) limit automatic vectorization. Thus, this prefix-sum pattern operates as a sequential reduction and scan.

3 Subarray Sum Equals K

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 560, Glassdoor
 - **Flashcards:** [Prefix Sum deck](#)
-

3.1 Question Description

Given an array of integers `nums` and an integer `k`, return the total number of subarrays whose sum equals to `k`.

A subarray is a contiguous non-empty sequence of elements within an array.

3.1.1 Examples

- **Input:** `nums = [1, 1, 1]`, `k = 2`
 - **Output:** 2 (subarrays are `nums[0..1]` and `nums[1..2]`)
 - **Input:** `nums = [1, 2, 3]`, `k = 3`
 - **Output:** 2 (subarrays are `nums[0..1]` and `nums[2..2]`)
-

3.2 Detailed Solution (C++)

This problem is solved efficiently using **Prefix Sums** combined with a **Hash Map**: 1. Let $S[i]$ be the prefix sum $\sum_{m=0}^i \text{nums}[m]$. 2. The sum of a subarray $\text{nums}[i \dots j]$ is given by $S[j] - S[i-1]$ (with $S[-1] = 0$). 3. We seek the number of pairs (i, j) such that $S[j] - S[i-1] = k$, which rewriting gives:

$$S[i-1] = S[j] - k$$

4. As we iterate through the array, we compute the running prefix sum `curr`. We check how many times the prefix sum `curr - k` has occurred so far. We add this count to our total, and then record the occurrence of the current prefix sum `curr` in our hash map.

3.2.1 Standard C++ Production Code

```
#include <vector>
#include <unordered_map>

class Solution {
public:
    int subarraySum(const std::vector<int>& nums, int k) {
        // Hash map to store prefix sum frequencies
        std::unordered_map<long long, int> prefix_freq;
        prefix_freq[0] = 1; // Base case: a prefix sum of 0 has occurred once (empty prefix)

        int count = 0;
        long long curr = 0;

        for (int num : nums) {
            curr += num;

            // Check if (curr - k) exists in the map
            auto it = prefix_freq.find(curr - k);
            if (it != prefix_freq.end()) {
                count += it->second;
            }

            // Increment frequency of the current prefix sum
            prefix_freq[curr]++;
        }

        return count;
    }
};
```

3.3 Detailed Solution (Python)

3.3.1 Standard Python Production Code

Below is the Python implementation using `defaultdict` for clean frequency updates.

```
from typing import List
from collections import defaultdict

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        """
        Calculates the number of subarrays that sum to k using prefix sum frequencies.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        count = 0
        curr_sum = 0

        # Hash map containing prefix sum -> frequency
        prefix_freq = defaultdict(int)
        prefix_freq[0] = 1 # Base case for prefix sum starting from index 0

        for num in nums:
            curr_sum += num

            # If (curr_sum - k) is a prefix sum we have seen, it means
            # the subarray between that previous point and the current point sums to k.
            count += prefix_freq[curr_sum - k]

            # Record the current prefix sum
            prefix_freq[curr_sum] += 1

        return count
```

3.4 Python-Specific Complexities & Implementation Considerations

3.4.1 1. Choice of Dictionary Implementation

- Using `collections.defaultdict(int)` is highly idiomatic and avoids explicit key-check logic (e.g. `if key in map: or map.get(key, 0)`).
- However, be aware that accessing a missing key in a `defaultdict` automatically inserts that key with a default value of `0`. In memory-constrained scenarios or very large datasets, using `.get(curr_sum - k, 0)` is slightly more memory-efficient as it avoids inserting unused query keys into the dictionary.

3.4.2 2. Large Integer Precision

- Python handles arbitrarily large integers natively, meaning `curr_sum` will never overflow, regardless of how large the values in `nums` are. In C++, we use `long long` for the running prefix sum `curr` to prevent potential integer overflows if constraints are larger (though standard 32-bit signed integers are usually sufficient for this problem's standard bounds).

3.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We perform a single pass through the array. Hash map lookups and insertions take $\mathcal{O}(1)$ on average.
Space Complexity	$\mathcal{O}(n)$	In the worst case, all prefix sums are unique, requiring $\mathcal{O}(n)$ entries in the hash map.

3.6 Common Follow-Up Questions & How to Answer

3.6.1 Q1: What if all numbers in the array are non-negative?

- **Answer:** If all elements are non-negative ($nums[i] \geq 0$), we can use a **Sliding Window (Two Pointers)** approach:
 1. Maintain `left` and `right` pointers representing a window.
 2. Expand the window by moving `right` and adding elements to `window_sum`.
 3. If `window_sum` exceeds `k`, contract the window by moving `left` until `window_sum` $\leq k$.
 4. If `window_sum` == `k`, increment the count. (Careful handling is needed if `0` is allowed in the array).
- **Complexity:** This optimization reduces the space complexity to $\mathcal{O}(1)$ while keeping the time complexity at $\mathcal{O}(n)$.

3.6.2 Q2: What if we need to return the actual indices of the subarrays instead of the count?

- **Answer:** Instead of storing the frequency of prefix sums in the hash map, we store the **list of indices** where each prefix sum occurred:
 - `std::unordered_map<long long, std::vector<int>> prefix_indices;`
 - When we find `curr - k` in the map, we iterate through its list of stored indices and output the range `[index + 1, current_index]`.
- **Complexity:** The space complexity remains $\mathcal{O}(n)$ to store indices, but the worst-case time complexity becomes $\mathcal{O}(n^2)$ if we must output all matching pairs (e.g. for an array of all zeros and $k = 0$).

3.7 Pro-Tip: How to Impress the Interviewer

- **Initialize the Base Case:** Emphasize why `prefix_freq[0] = 1` is necessary. Explain that this represents the empty prefix sum before the first element. Without this, any subarray starting at index `0` that sums to `k` would not be counted since `curr - k` would be `0` and wouldn't be found in the map.
- **Discuss Hash Map Collisions and Load Factor:** In C++, `std::unordered_map` uses chaining for collision resolution. If the hash function is poor or there are many collisions, lookup time can degrade to $\mathcal{O}(n)$. Mentioning `reserve()` or custom hashers (like custom `splitmix64` hashers) shows deep knowledge of standard library internals and real-world system optimization.

4 Subarray Product Less Than K

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer
 - **Source:** LeetCode 713, Glassdoor
 - **Flashcards:** [Prefix Sum deck](#)
-

4.1 Question Description

Given an array of integers `nums` and an integer `k`, return the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than `k`.

4.1.1 Examples

- **Input:** `nums = [10, 5, 2, 6]`, `k = 100`
 - **Output:** 8
 - **Explanation:** The 8 subarrays that have product less than 100 are: `[10]`, `[5]`, `[2]`, `[6]`, `[10, 5]`, `[5, 2]`, `[2, 6]`, `[5, 2, 6]`. Note that `[10, 5, 2]` is not included as its product is 100, which is not strictly less than 100.
 - **Input:** `nums = [1, 2, 3]`, `k = 0`
 - **Output:** 0
-

4.2 Detailed Solution (C++)

Since all elements in `nums` are strictly positive integers ($nums[i] \geq 1$), the product of any subarray is monotonically increasing as we expand it. This monotonic property allows us to use a **Sliding Window (Two Pointers)** approach: 1. Maintain a window `[left, right]` and a running product of elements inside the window. 2. Iterate `right` from `0` to `n - 1`, multiplying the running product by `nums[right]`. 3. If `product >= k`, shrink the window from the left by dividing `product` by `nums[left]` and incrementing `left`, until `product < k` or `left > right`. 4. For each valid window ending at `right`, the number of valid subarrays

ending at `right` is `right - left + 1` (namely, `nums[right]`, `nums[right-1...right]`, ..., `nums[left...right]`).

5. Sum these counts over all iterations of `right`.

4.2.1 Standard C++ Production Code

```
#include <vector>

class Solution {
public:
    int numSubarrayProductLessThanK(const std::vector<int>& nums, int k) {
        // Since nums[i] >= 1, if k <= 1, no subarray product can be strictly less than k.
        if (k <= 1) {
            return 0;
        }

        int count = 0;
        long long product = 1; // Use long long to prevent potential overflow during multiplication
        int left = 0;
        int n = static_cast<int>(nums.size());

        for (int right = 0; right < n; ++right) {
            product *= nums[right];

            // Shrink the window until the product is strictly less than k
            while (product >= k && left <= right) {
                product /= nums[left];
                left++;
            }

            // All subarrays starting from any index in [left, right] and ending at right are valid
            count += (right - left + 1);
        }

        return count;
    }
};
```

4.3 Detailed Solution (Python)

4.3.1 Standard Python Production Code

Below is the Python sliding window implementation.

```
from typing import List
```

```

class Solution:
    def numSubarrayProductLessThanK(self, nums: List[int], k: int) -> int:
        """
        Returns the number of contiguous subarrays where the product is strictly less than k.

        Time Complexity: O(N)
        Space Complexity: O(1)
        """
        if k <= 1:
            return 0

        count = 0
        product = 1
        left = 0

        for right, val in enumerate(nums):
            product *= val

            # Shrink window from the left if the product matches or exceeds k
            while product >= k and left <= right:
                product //= nums[left]
                left += 1

            count += (right - left + 1)

        return count

```

4.4 Python-Specific Complexities & Implementation Considerations

4.4.1 1. Floor Division vs Standard Division

- In Python, standard division (/) yields a float, which can lead to floating-point precision errors (e.g. `24.0 / 2` becoming `12.000000000000002`).
- Always use the floor division operator `//` to maintain exact integer math, as all elements in `nums` are integers and the division is guaranteed to divide evenly.

4.4.2 2. Arbitrary Precision

- While fixed-width systems might overflow when `product *= nums[right]` occurs before the loop contracts it, Python handles arbitrarily large integers, preventing integer overflow bugs naturally. Nonetheless, keeping the sliding window bounds correct prevents unnecessary growth of the integer sizes, which would slow down multiplications.
-

4.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Each element is visited at most twice: once by the <code>right</code> pointer and at most once by the <code>left</code> pointer.
Space Complexity	$\mathcal{O}(1)$	We only maintain a few scalar pointers and product accumulators.

4.6 Common Follow-Up Questions & How to Answer

4.6.1 Q1: What if `nums[i]` could be 0?

- **Answer:** If `nums[i]` could be 0, the product becomes 0, which is always $< k$ (for $k > 0$). However, division by 0 would crash our sliding window (`product /= nums[left]`).
- **Solution:** If we encounter 0, we must reset the window:
 1. A zero element resets the current product. All subarrays containing this 0 will have a product of 0.
 2. We can split the array by zeros and solve the problem on each non-zero subarray separately. For subarrays containing the 0, we can count their contributions directly.

4.6.2 Q2: How is this related to Prefix Sums?

- **Answer:** We can convert the product into a sum by taking the logarithm of each element:

$$\prod_{i=left}^{right} \text{nums}[i] < k \iff \sum_{i=left}^{right} \log(\text{nums}[i]) < \log(k)$$

If we construct a prefix sum array of $\log(\text{nums}[i])$, we can use binary search (`std::upper_bound`) on this sorted prefix sum array to find the valid `left` boundary for each `right` index.

- **Trade-off:** The logarithmic prefix sum approach takes $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ space due to float operations and prefix arrays. It also introduces potential floating-point precision issues. However, it is useful if the array elements could be non-monotonic or if we wanted to support multiple random queries on subsegment product ranges.

4.7 Pro-Tip: How to Impress the Interviewer

- **Mention Monotonicity explicitly:** State clearly that the sliding window works because the prefix product is *monotonically increasing* (since $\text{nums}[i] \geq 1$). If `nums[i]` could be fractions between (0, 1), the product would not be monotonic, and a simple sliding window would fail.

- **Identify the $k \leq 1$ edge case:** Point out immediately that if $k \leq 1$ and all elements are ≥ 1 , the product of any subarray is at least 1. Thus, no subarray can have a product strictly less than k . Checking this at the start prevents infinite loops where `left` tries to increment past `right` while trying to divide down a product to less than 1.

5 Continuous Subarray Sum

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 523, Glassdoor
 - **Flashcards:** [Prefix Sum deck](#)
-

5.1 Question Description

Given an integer array `nums` and an integer `k`, return `true` if `nums` has a **good subarray** or `false` otherwise.

A **good subarray** is a subarray where: * Its length is **at least two**, and * The sum of the elements in the subarray is a multiple of `k` (i.e., $\text{sum} = n \times k$ where n is an integer).

Note that: * A **subarray** is a contiguous part of the array. * An integer x is a multiple of k if there exists an integer n such that $x = n \times k$. 0 is always a multiple of k .

5.1.1 Examples

- **Input:** `nums = [23, 2, 4, 6, 7]`, `k = 6`
 - **Output:** `true`
 - **Explanation:** `[2, 4]` is a continuous subarray of size 2 whose elements sum up to 6, which is a multiple of 6.
 - **Input:** `nums = [23, 2, 6, 4, 7]`, `k = 6`
 - **Output:** `true`
 - **Explanation:** `[23, 2, 6, 4, 7]` is a continuous subarray of size 5 whose elements sum up to 42, which is 7×6 .
 - **Input:** `nums = [23, 2, 6, 4, 7]`, `k = 13`
 - **Output:** `false`
-

5.2 Detailed Solution (C++)

This problem is solved in linear time using **Prefix Sums modulo k** combined with a **Hash Map**: 1. Let $S[i]$ be the prefix sum up to index i modulo k :

$$S[i] = \left(\sum_{j=0}^i \text{nums}[j] \right) \bmod k$$

2. The sum of a subarray $\text{nums}[i + 1 \dots j]$ is a multiple of k if and only if $S[j] = S[i]$. 3. Since we want a subarray of length **at least two**, we require the index difference to be strictly greater than 1:

$$j - i > 1$$

4. We store the first occurrence of each remainder modulo k in a hash map `mod_index`. 5. We initialize `mod_index[0] = -1` to handle the case where a subarray starting from index 0 has a sum that is a multiple of k . 6. For each index i , we compute `prefix = (prefix + nums[i]) % k`. If `prefix` already exists in `mod_index`, we check if $i - \text{mod_index}[\text{prefix}] > 1$. If so, we have found a valid subarray and return `true`. If `prefix` does not exist in the map, we store `mod_index[prefix] = i`. 7. **Crucial point:** If `prefix` already exists, we do **not** update its index. Keeping the earliest index maximizes the chance that a subsequent element satisfies the length constraint of ≥ 2 .

5.2.1 Standard C++ Production Code

```
#include <vector>
#include <unordered_map>

class Solution {
public:
    bool checkSubarraySum(const std::vector<int>& nums, int k) {
        int n = static_cast<int>(nums.size());
        if (n < 2) {
            return false;
        }

        // Map to store (prefix_sum % k) -> first index where it occurred
        std::unordered_map<int, int> mod_index;

        // Base case: prefix sum of 0 at index -1
        mod_index[0] = -1;

        int prefix = 0;
        for (int i = 0; i < n; ++i) {
            // Compute mod k prefix sum safely. Use long long to prevent overflow before modulo.
            prefix = static_cast<int>((static_cast<long long>(prefix) + nums[i]) % k);

            auto it = mod_index.find(prefix);
            if (it != mod_index.end()) {
                // If we've seen this mod remainder before, check if length >= 2
                if (i - it->second > 1) {
                    return true;
                }
            } else {
                // Only record the first occurrence of a remainder
            }
        }
    }
};
```

```

        mod_index[prefix] = i;
    }
}

return false;
}
};

```

5.3 Detailed Solution (Python)

5.3.1 Standard Python Production Code

Below is the Python implementation.

```

from typing import List

class Solution:
    def checkSubarraySum(self, nums: List[int], k: int) -> bool:
        """
        Determines if there is a subarray of length >= 2 whose sum is a multiple of k.

        Time Complexity: O(N)
        Space Complexity: O(min(N, k))
        """
        if len(nums) < 2:
            return False

        # Map to store prefix sum % k -> earliest index
        mod_index = {0: -1}
        prefix = 0

        for i, num in enumerate(nums):
            prefix = (prefix + num) % k

            if prefix in mod_index:
                # Check if the length of the subarray is at least 2
                if i - mod_index[prefix] > 1:
                    return True
            else:
                # Store the earliest index for this prefix remainder
                mod_index[prefix] = i

        return False

```

5.4 Python-Specific Complexities & Implementation Considerations

5.4.1 1. Hash Map Space Limits

- Since the remainder is always in range $[0, k - 1]$, the size of `mod_index` is bounded by $\mathcal{O}(\min(n, k))$. If k is extremely large ($2^{31} - 1$), the space complexity is bounded by $\mathcal{O}(n)$. If n is very large but k is small, the space is bounded by $\mathcal{O}(k)$.

5.4.2 2. Modulo Arithmetic on Negative Numbers

- In Python, the `%` operator on negative numbers yields a positive remainder (e.g. `-1 % 5 = 4`). In C++, `%` yields a negative or zero remainder (e.g. `-1 % 5 = -1`).
- Although the current problem constraints specify `nums[i] >= 0`, keeping mod arithmetic portable is standard best practice. If elements could be negative, the C++ code would need to normalize the remainder:

```
prefix = ((prefix + nums[i]) % k + k) % k;
```

5.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We perform a single loop over <code>nums</code> . Dictionary lookups and insertions are $\mathcal{O}(1)$ average.
Space Complexity	$\mathcal{O}(\min(n, k))$	The hash map can contain at most $\min(n, k)$ unique modulo remainders.

5.6 Common Follow-Up Questions & How to Answer

5.6.1 Q1: What if k can be 0?

- **Answer:** If $k = 0$, we cannot perform `% k` because of division by zero.
- **Solution:**
 - Mathematically, a sum is a multiple of 0 if and only if the sum is exactly 0.
 - If the question allowed $k = 0$ (as in older versions of this LeetCode problem), we would handle it by skipping the `% k` step and storing the actual prefix sums in the map. The condition for a multiple of 0 then simply becomes `prefix_sum == previous_prefix_sum`, indicating a subarray sum of 0.

5.6.2 Q2: What if we want to find the longest good subarray?

- **Answer:** The current algorithm already stores the *earliest* index of each remainder in `mod_index` (because we do not overwrite the value if the remainder is already present).

- **Solution:** To find the longest subarray, instead of returning `true` immediately when we find a match, we compute the length $i - \text{mod_index}[\text{prefix}]$, track the maximum length seen so far, and return it at the end.
-

5.7 Pro-Tip: How to Impress the Interviewer

- **Explain the `mod_index[0] = -1` Initialization:** Explicitly walk the interviewer through why `0: -1` is essential. If the array starts with `[23, 2, 4]` and $k = 6$, at index 2 the prefix sum is $29 \bmod 6 = 5$. At index 4, the sum is $42 \bmod 6 = 0$. Since `prefix = 0`, we look up `0` in our map. With `0: -1`, the length is $4 - (-1) = 5 \geq 2$, which is correct. Without `mod_index[0] = -1`, we would miss prefix sums that are multiples of k starting at index `0`.
- **Prevent Index Overwrite:** Make a strong point that `mod_index[prefix] = i` must **only** be executed if `prefix` is not already in the map. Overwriting the index would shrink subsequent subarray lengths, potentially failing the $\text{length} \geq 2$ check.

6 Contiguous Array

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer
 - **Source:** LeetCode 525, Glassdoor
 - **Flashcards:** [Prefix Sum deck](#)
-

6.1 Question Description

Given a binary array `nums`, return the maximum length of a contiguous subarray with an equal number of `0` and `1`.

6.1.1 Examples

- **Input:** `nums = [0, 1]`
 - **Output:** 2
 - **Explanation:** `[0, 1]` is the longest contiguous subarray with an equal number of 0 and 1.
 - **Input:** `nums = [0, 1, 0]`
 - **Output:** 2
 - **Explanation:** `[0, 1]` (or `[1, 0]`) is the longest contiguous subarray with an equal number of 0 and 1.
-

6.2 Detailed Solution (C++)

To find the maximum length of a contiguous subarray with an equal number of `0` and `1`: 1. Treat every `0` in `nums` as a `-1` and every `1` as `+1`. 2. The problem then reduces to finding the longest subarray whose

sum is exactly 0. 3. The sum of a subarray from index $i + 1$ to j is 0 if and only if the prefix sum at i is equal to the prefix sum at j . 4. We maintain a running prefix sum `count`. We store the first index at which each `count` value is encountered in a hash map (or array) `first`. 5. We initialize `first[0] = -1` to handle subarrays that start at index 0. 6. For each index i , we update `count`. If `count` is already in the map, the subarray from `first[count] + 1` to i has a sum of 0. We update our maximum length with $i - \text{first}[\text{count}]$. If it is not in the map, we record `first[count] = i`.

6.2.1 Standard C++ Production Code

```
#include <vector>
#include <unordered_map>
#include <algorithm>

class Solution {
public:
    int findMaxLength(const std::vector<int>& nums) {
        int n = static_cast<int>(nums.size());

        // Key: running balance (count of 1s minus count of 0s)
        // Value: first index where this balance was achieved
        std::unordered_map<int, int> first;
        first[0] = -1; // Base case: balance is 0 before starting

        int max_len = 0;
        int count = 0;

        for (int i = 0; i < n; ++i) {
            count += (nums[i] == 1) ? 1 : -1;

            auto it = first.find(count);
            if (it != first.end()) {
                max_len = std::max(max_len, i - it->second);
            } else {
                first[count] = i;
            }
        }

        return max_len;
    }
};
```

6.3 Detailed Solution (Python)

6.3.1 Standard Python Production Code

Below is the Python implementation.

```
from typing import List

class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        """
        Finds the maximum length of a contiguous subarray with equal number of 0s and 1s.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        # Map running balance to first index where it occurred
        first_occurrence = {0: -1}

        max_len = 0
        balance = 0

        for i, num in enumerate(nums):
            # Treat 1 as +1 and 0 as -1
            balance += 1 if num == 1 else -1

            if balance in first_occurrence:
                max_len = max(max_len, i - first_occurrence[balance])
            else:
                first_occurrence[balance] = i

        return max_len
```

6.4 Python-Specific Complexities & Implementation Considerations

6.4.1 1. Hash Table Overhead vs Pre-allocated Arrays

- In Python, dict searches are highly optimized but still carry the overhead of hashing. In a performance-critical situation or in lower-level languages like C++, we can replace the hash map with a single flat array.
 - Since the running balance count must lie within $[-n, n]$, we can offset the index by n to map balances to a pre-allocated vector of size $2n+1$. This eliminates hashing entirely and operates with $\mathcal{O}(1)$ direct array index accesses.
-

6.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the array of size n exactly once. Hash map operations take $\mathcal{O}(1)$ on average.
Space Complexity	$\mathcal{O}(n)$	In the worst case, the running balance increases or decreases monotonically, resulting in $\mathcal{O}(n)$ unique balances in the map.

6.6 Common Follow-Up Questions & How to Answer

6.6.1 Q1: Can we solve this in $\mathcal{O}(1)$ space if we are allowed to modify the input or use some properties?

- **Answer:** No. Unlike the sliding window technique which can achieve $\mathcal{O}(1)$ space for non-negative array elements, this problem requires tracking arbitrary balance differences. Because the balance can fluctuate up and down (non-monotonically), we cannot use a simple sliding window. The historical states must be stored, making $\mathcal{O}(n)$ space a theoretical lower bound for this problem.

6.6.2 Q2: What if we want to find the maximum length subarray with equal numbers of three different values (e.g., 0, 1, and 2)?

- **Answer:**
 - Let $C_0(i)$, $C_1(i)$, and $C_2(i)$ be the counts of 0s, 1s, and 2s up to index i .
 - We want a subarray from $i + 1$ to j with equal counts of 0s, 1s, and 2s:

$$C_0(j) - C_0(i) = C_1(j) - C_1(i) = C_2(j) - C_2(i)$$

- This can be rearranged into two independent equations:

$$C_1(j) - C_0(j) = C_1(i) - C_0(i)$$

$$C_2(j) - C_1(j) = C_2(i) - C_1(i)$$

- We can store the state as a pair of differences: `(count_1 - count_0, count_2 - count_1)`.
- We map this state tuple to the first index it occurred, using the same hash map logic.

6.7 Pro-Tip: How to Impress the Interviewer

- **Introduce the Branchless / Offset Array Optimization:** Explain to the interviewer that you can write a highly optimized C++ solution that replaces `std::unordered_map` with a flat array/vector

of size $2n + 1$. This avoids memory allocation during the loop and keeps the entire lookup table cache-local:

```
int findMaxLength(const std::vector<int>& nums) {
    int n = nums.size();
    // Index offset is n. Balance range is [-n, n], mapped to [0, 2n]
    std::vector<int> first(2 * n + 1, -2); // Use -2 to represent uninitialized
    first[n] = -1; // balance 0 is at offset n

    int max_len = 0;
    int balance = 0;
    for (int i = 0; i < n; ++i) {
        balance += (nums[i] == 1) ? 1 : -1;
        int lookup_index = balance + n;
        if (first[lookup_index] >= -1) {
            max_len = std::max(max_len, i - first[lookup_index]);
        } else {
            first[lookup_index] = i;
        }
    }
    return max_len;
}
```

This shows a solid understanding of memory layouts, cache performance, and how to eliminate dynamic allocations in latency-sensitive systems.

7 Random Pick with Weight

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / AI Systems Architect
- **Source:** LeetCode 528, Glassdoor
- **Flashcards:** [Prefix Sum deck](#)

7.1 Question Description

You are given a **0-indexed** array of positive integers w where $w[i]$ describes the weight of the i -th index.

You need to implement the class `Solution` with the following methods: `* Solution(w)` Initializes the object with the weights w . `* pickIndex()` Returns a randomly picked index in the range $[0, w.length - 1]$ (inclusive). The probability of picking an index i must be $w[i] / \text{sum}(w)$.

7.1.1 Examples

- **Input:** `["Solution", "pickIndex"]` `[[[1]], []]`

- **Output:** [null, 0]
 - **Explanation:** `Solution solution = new Solution([1]); solution.pickIndex();` returns 0. The only option is to return 0 since there is only one element in `w`.
 - **Input:** ["Solution", "pickIndex", "pickIndex", "pickIndex", "pickIndex", "pickIndex"] [[[1, 3]], [], [], [], [], [], []]
 - **Output:** [null, 1, 1, 1, 1, 0]
 - **Explanation:** `Solution solution = new Solution([1, 3]); pickIndex()` has a 75% chance of returning 1 and 25% chance of returning 0.
-

7.2 Detailed Solution (C++)

To implement weighted random selection efficiently: 1. **Prefix Sum Construction:** Compute the running prefix sums of weights. For example, if weights are [1, 3], prefix sums are [1, 4]. 2. The total sum of weights is `prefix.back()`. 3. **Random Selection:** Generate a random integer `t` in the range [1, total_sum]. 4. **Binary Search:** Since the prefix sums are strictly increasing (weights are positive integers), we can perform binary search to find the first index `i` where `prefix[i] >= t`. This can be done in $\mathcal{O}(\log n)$ time using `std::lower_bound`.

7.2.1 Standard C++ Production Code

To write production-grade C++, we use `<random>` (Mersenne Twister `std::mt19937`) instead of `rand()`, since `rand()` suffers from bias and poor distribution.

```
#include <vector>
#include <random>
#include <algorithm>

class Solution {
private:
    std::vector<int> prefix;
    std::mt19937 gen;
    std::uniform_int_distribution<int> dis;

public:
    Solution(const std::vector<int>& w) : gen(std::random_device{}()) {
        int total = 0;
        prefix.reserve(w.size());
        for (int x : w) {
            total += x;
            prefix.push_back(total);
        }
        // Set the uniform distribution range [1, total]
        dis = std::uniform_int_distribution<int>(1, total);
    }
}
```

```

int pickIndex() {
    int target = dis(gen);
    // Find the first prefix sum >= target
    auto it = std::lower_bound(prefix.begin(), prefix.end(), target);
    return static_cast<int>(std::distance(prefix.begin(), it));
}
};

```

7.3 Detailed Solution (Python)

7.3.1 Standard Python Production Code

Below is the Python implementation using `random.randint` and the built-in `bisect` library for binary search.

```

import random
import bisect
from typing import List

class Solution:

    def __init__(self, w: List[int]):
        """
        Initializes the prefix sum array from weights.
        """
        self.prefix = []
        total = 0
        for x in w:
            total += x
            self.prefix.append(total)
        self.total = total

    def pickIndex(self) -> int:
        """
        Picks a random index with probability proportional to its weight.

        Time Complexity: O(log N)
        Space Complexity: O(1) auxiliary
        """
        # Generate a random integer between 1 and the sum of weights (inclusive)
        target = random.randint(1, self.total)

        # Binary search for the first prefix sum >= target
        # bisect_left returns the insertion point where prefix[i] >= target

```

```
return bisect.bisect_left(self.prefix, target)
```

7.4 Python-Specific Complexities & Implementation Considerations

7.4.1 1. `random.randint` vs `random.random`

- `random.randint(1, self.total)` generates a random integer in the closed interval $[1, \text{total}]$, which aligns cleanly with discrete weights.
- Alternatively, you can generate a float $u \in [0.0, 1.0)$ using `random.random()` and multiply by `self.total`. However, floating-point precision limits can theoretically cause boundary issues (e.g., $u \cdot \text{total}$ rounding up incorrectly). Discrete integer random selection is preferred.

7.4.2 2. Built-in `bisect` Module

- In Python, you should always prefer the built-in `bisect` module over a manually written binary search loop. The `bisect` module is written in C under the hood, making it significantly faster than equivalent Python code.
-

7.5 Complexity Analysis

Operation	Time Complexity	Space Complexity	Description
Constructor	$\mathcal{O}(n)$	$\mathcal{O}(n)$	To store the prefix sum array of size n .
pickIndex	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$	Binary search over the prefix sum array.

7.6 Common Follow-Up Questions & How to Answer

7.6.1 Q1: What if weights can be modified dynamically (e.g., `updateWeight(index, val)`)?

- **Answer:** If weights are dynamic, using a prefix sum array would require $\mathcal{O}(n)$ to update. To support both update and random selection efficiently, we can use a **Segment Tree** or a **Fenwick Tree (Binary Indexed Tree)**:
 - **Updates:** Can be done in $\mathcal{O}(\log n)$ time.
 - **Queries/Binary Search:** We can search the tree in $\mathcal{O}(\log n)$ time.
 - This provides a balanced solution where both methods run in logarithmic time.

7.6.2 Q2: Can we achieve $\mathcal{O}(1)$ query time for `pickIndex`?

- **Answer:** Yes, by using the **Alias Method (Walker's Alias Method)**:
 - The Alias Method is a family of algorithms for efficient sampling from a discrete probability distribution.

- It requires $\mathcal{O}(n)$ preprocessing time to construct two tables: a probability table and an alias table.
 - Once constructed, `pickIndex` can be performed in $\mathcal{O}(1)$ **time** and $\mathcal{O}(1)$ space by choosing a random index i and then flipping a biased coin to decide whether to return i or its alias table entry.
-

7.7 Pro-Tip: How to Impress the Interviewer

- **Mention Walker's Alias Method:** Proactively mentioning Walker's Alias Method for $\mathcal{O}(1)$ query time shows a deep, production-level expertise in randomized algorithms and sampling, which is highly relevant in fields like graphics, machine learning, and game design.
- **Show PRNG Hygiene:** In C++, explain why you avoided `rand() % total`. `rand()` typically has a low max value (`RAND_MAX` is only 32767 on some compilers), causing significant bias when the total weight exceeds it. Using the modern C++ `<random>` library shows you write safe, enterprise-grade code.

8 Find Minimum in Rotated Sorted Array

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 153, Glassdoor
 - **Flashcards:** [Binary Search deck](#)
-

8.1 Question Description

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array `nums = [0,1,2,4,5,6,7]` might become: * `[4,5,6,7,0,1,2]` if it was rotated 4 times. * `[0,1,2,4,5,6,7]` if it was rotated 7 times.

Notice that rotating an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of **unique** elements, return the *minimum element* of this array.

You must write an algorithm that runs in $\mathcal{O}(\log n)$ time.

8.1.1 Examples

- **Input:** `nums = [3,4,5,1,2]`
 - **Output:** 1
 - **Explanation:** The original array was `[1,2,3,4,5]` rotated 3 times.
- **Input:** `nums = [4,5,6,7,0,1,2]`
 - **Output:** 0

- **Explanation:** The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.
 - **Input:** nums = [11,13,15,17]
 - **Output:** 11
 - **Explanation:** The original array was [11,13,15,17] and it was rotated 4 times.
-

8.2 Detailed Solution (C++)

The core algorithm is a modified **Binary Search**. Because all elements are unique, we can identify which half contains the inflection point (the minimum element) by comparing the middle element `nums[mid]` to the rightmost element `nums[right]`: 1. If `nums[mid] > nums[right]`, the minimum must lie strictly to the right of `mid`. Thus, we can move our search window to the right: `left = mid + 1`. 2. If `nums[mid] <= nums[right]`, the minimum could be `nums[mid]` or lies to its left. Thus, we set `right = mid`.

This division narrows down the search space to exactly one element, which will be the minimum.

8.2.1 Standard C++ Production Code

```
#include <vector>
#include <cassert>
#include <stdexcept>

class Solution {
public:
    int findMin(const std::vector<int>& nums) {
        // Edge Case: Handle empty collection explicitly to prevent unsigned underflow
        if (nums.empty()) {
            throw std::invalid_argument("Input array cannot be empty.");
        }

        int left = 0;
        int right = static_cast<int>(nums.size()) - 1; // Safely cast size to int

        while (left < right) {
            int mid = left + (right - left) / 2; // Prevent integer overflow

            if (nums[mid] > nums[right]) {
                // Minimum is in the right half (excluding mid)
                left = mid + 1;
            } else {
                // Minimum is at mid or in the left half
                right = mid;
            }
        }
    }
};
```

```
        return nums[left];
    }
};
```

8.3 Detailed Solution (Python)

8.3.1 Standard Python Production Code

Below is the iterative, type-hinted Python implementation. It avoids recursion overhead and ensures true constant space.

```
from typing import List

class Solution:
    def findMin(self, nums: List[int]) -> int:
        """
        Finds the minimum element in a rotated sorted array using binary search.

        Time Complexity:  $O(\log N)$ 
        Space Complexity:  $O(1)$ 
        """
        # Edge Case: Handle empty collection explicitly
        if not nums:
            raise ValueError("Input array cannot be empty.")

        left, right = 0, len(nums) - 1

        while left < right:
            mid = left + (right - left) // 2

            if nums[mid] > nums[right]:
                # Minimum must be in the right half
                left = mid + 1
            else:
                # Minimum is mid or to the left of mid
                right = mid

        return nums[left]
```

8.4 Python-Specific Complexities & Implementation Considerations

8.4.1 1. Integer Division: / vs //

- In Python 3, using a single / produces a float (e.g., $5 / 2 = 2.5$). List indices must be integers. Ensure the double slash floor division operator // is used to maintain integer indices.

8.4.2 2. Handling of Empty Inputs

- In Python, passing an empty list to binary search logic without checks will cause an `IndexError` at `right = len(nums) - 1` (yielding `right = -1`, resulting in out-of-bound attempts or incorrect logic inside the loop). Explicitly validating inputs prevents silent failures.

8.4.3 3. Avoiding Slicing Overhead

- Do not slice the list like `findMin(nums[mid:])` during recursive or helper functions, as slicing copies references to new list containers, which takes $\mathcal{O}(k)$ time and space. Stick strictly to boundary indices `left` and `right`.

8.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\log n)$	The search space is halved in each step of the binary search loop.
Space Complexity	$\mathcal{O}(1)$	Only a few integer variables are maintained, requiring constant auxiliary memory.

8.6 Common Follow-Up Questions & How to Answer

8.6.1 Q1: What if duplicates are allowed in the array? (LeetCode 154)

- **The Problem:** If `nums = [1, 0, 1, 1, 1]`, and we choose the middle index, `nums[mid] == nums[right]` (both are 1). We cannot determine whether the minimum lies to the left or right of `mid`.
- **The Solution:** When `nums[mid] == nums[right]`, we cannot discard half the search space. Instead, we safely decrement the right boundary (`right--` or `right -= 1`) to eliminate the duplicate and continue the binary search.
- **Worst-case impact:** The time complexity degrades to $\mathcal{O}(n)$ when all elements are identical.

8.6.2 Q2: How can we find the number of times the array was rotated?

- **The Solution:** The number of rotations is equivalent to the index of the minimum element in the array. For instance, in `[4, 5, 6, 7, 0, 1, 2]`, the minimum `0` is at index 4, meaning the array was

rotated 4 times. Once the minimum is found, we simply return its index `left`.

8.7 Pro-Tip: How to Impress the Interviewer

- **Catch the Unsigned Underflow:** Emphasize how subtraction like `nums.size() - 1` on empty inputs causes unsigned underflow in languages like C++, resulting in a large positive boundary (18446744073709551615) and immediate segmentation faults. Demonstrating proactive checks for `nums.empty()` indicates production-grade safety awareness.
- **Explain Inflection Point Mechanics:** Rather than memorizing the comparisons, explain it in terms of the "sorted properties" of the left/right parts. In a rotated sorted array, the elements can be viewed as two sorted sub-segments. If `nums[mid] > nums[right]`, `mid` is on the higher first segment, and the inflection point is on the second segment to the right. If `nums[mid] <= nums[right]`, `mid` is on the lower second segment, meaning the minimum cannot be to the right of `mid`.

9 First Bad Version

- **Category:** Coding
 - **Difficulty:** Easy
 - **Target Role:** Software Engineer / QA & Test Engineer
 - **Source:** LeetCode 278, Glassdoor
 - **Flashcards:** [Binary Search deck](#)
-

9.1 Question Description

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions `[1, 2, ..., n]` and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether `version` is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

9.1.1 Examples

- **Input:** `n = 5, bad = 4`
 - **Output:** 4
 - **Explanation:**
 - * `isBadVersion(3)` returns `false`
 - * `isBadVersion(5)` returns `true`
 - * `isBadVersion(4)` returns `true`
 - * Then 4 is the first bad version.
- **Input:** `n = 1, bad = 1`

– **Output:** 1

9.1.2 Constraints

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$
-

9.2 Detailed Solution (C++)

The problem requires finding the smallest version $v \in [1, n]$ such that `isBadVersion(v)` is `true`. Since the versions are monotonically partitioned into a sequence of `false` values followed by `true` values (e.g., [F, F, F, T, T]), we can apply **Binary Search** to find the boundary in $\mathcal{O}(\log n)$ steps.

We check the version at `mid`. If `isBadVersion(mid)` is `true`, then the first bad version is either `mid` or lies to the left of `mid` (`right = mid`). If it is `false`, then the first bad version must be strictly to the right of `mid` (`left = mid + 1`).

9.2.1 Standard C++ Production Code

```
// The API isBadVersion is defined for you.
// bool isBadVersion(int version);

class Solution {
public:
    int firstBadVersion(int n) {
        int left = 1;
        int right = n;

        while (left < right) {
            // Prevent integer overflow: left + right could exceed 2^31 - 1
            int mid = left + (right - left) / 2;

            if (isBadVersion(mid)) {
                right = mid;    // The first bad version is mid or to the left
            } else {
                left = mid + 1;  // The first bad version is strictly to the right
            }
        }

        // When left == right, we have found the first bad version
        return left;
    }
};
```

9.3 Detailed Solution (Python)

9.3.1 Standard Python Production Code

Below is the iterative, type-hinted Python implementation.

The isBadVersion API is already defined for you.

def isBadVersion(version: int) -> bool:

class Solution:

def firstBadVersion(**self**, n: **int**) -> **int**:

"""

Finds the first bad version using binary search.

Time Complexity: $O(\log N)$

Space Complexity: $O(1)$

"""

left, right = 1, n

while left < right:

mid = left + (right - left) // 2

if isBadVersion(mid):

right = mid *# Mid is bad, search left half including mid*

else:

left = mid + 1 *# Mid is good, search right half excluding mid*

return left

9.4 Python-Specific Complexities & Implementation Considerations

9.4.1 1. Integer Arithmetic Safety

- In C++ or Java, $(\text{left} + \text{right}) / 2$ is a famous source of overflow bugs when $n \approx 2^{31} - 1$, since the sum can exceed the maximum value of a signed 32-bit integer.
- Python automatically promotes integers to arbitrary precision, meaning $(\text{left} + \text{right}) // 2$ will never overflow the underlying memory. However, using $\text{left} + (\text{right} - \text{left}) // 2$ is still considered best practice in interviews to demonstrate awareness of fixed-width architecture limitations.

9.4.2 2. Mocking the API for Local Testing

- For offline development or automated testing, we can define a class wrapper or closure that mocks the API:

def make_bad_version_check(bad: **int**):

return **lambda** v: v >= bad

9.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\log n)$	The version range is halved at each iteration, resulting in at most ≈ 31 calls to <code>isBadVersion</code> .
Space Complexity	$\mathcal{O}(1)$	The search uses constant memory space for pointers.

9.6 Common Follow-Up Questions & How to Answer

9.6.1 Q1: What if the API calls are extremely expensive (e.g., hitting a remote database or network call)?

- **Answer:** If calling `isBadVersion` is costly, we must minimize calls.
 1. We can cache the results of past queries in a hash map to avoid querying the same version twice.
 2. If versions are requested in batches, we can load-balance version verification.

9.6.2 Q2: What if we don't know the upper bound n ? (Infinite version stream)

- **Answer:** If n is unknown or infinite, we cannot initialize `right = n`. We can use **Exponential Search**:
 1. Start with `right = 1`.
 2. Double `right` (`right *= 2`) repeatedly as long as `isBadVersion(right)` is `false`.
 3. Once we find a `right` where the version is bad, the first bad version must be in the range `[right // 2, right]`. We then perform standard binary search on this interval.
 4. This takes $\mathcal{O}(\log k)$ time, where k is the index of the first bad version.

9.7 Pro-Tip: How to Impress the Interviewer

- **Proactive Overflow Discussion:** Before writing the code, mention the `left + (right - left) / 2` overflow issue. Interviewers love candidates who preemptively write bug-free code for corner cases.
- **Explain Interactive Systems Design:** Discuss how to handle flaky network connections in an interactive environment. For instance, if `isBadVersion(mid)` could fail/time out, how do we introduce retries or exponential backoff in our binary search loop?

10 Data Stream as Disjoint Intervals

- **Category:** Coding
- **Difficulty:** Hard

- **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 352, Glassdoor
 - **Flashcards:** [Binary Search deck](#)
-

10.1 Question Description

Given a data stream input of non-negative integers $a_1, a_2, \dots, a_n$, summarize the numbers seen so far as a list of disjoint intervals.

Implement the `SummaryRanges` class:

- `SummaryRanges()` Initializes the object with an empty stream.
- `void addNum(int value)` Adds the integer `value` to the stream.
- `int[][] getIntervals()` Returns a summary of the integers in the stream currently as a list of disjoint intervals $[start_i, end_i]$. The answer should be sorted by $start_i$.

10.1.1 Examples

- **Input:** ["SummaryRanges", "addNum", "getIntervals", "addNum", "getIntervals", "addNum", "getIntervals", "addNum", "getIntervals", "addNum", "getIntervals"]
[[], [1], [], [3], [], [7], [], [2], [], [6], []]
- **Output:** [null, null, [[1, 1]], null, [[1, 1], [3, 3]], null, [[1, 1], [3, 3], [7, 7]], null, [[1, 3], [7, 7]], null, [[1, 3], [6, 7]]]
- **Explanation:**

```
summaryRanges = SummaryRanges()
summaryRanges.addNum(1)      # Stream: [1]
summaryRanges.getIntervals() # Returns [[1, 1]]
summaryRanges.addNum(3)      # Stream: [1, 3]
summaryRanges.getIntervals() # Returns [[1, 1], [3, 3]]
summaryRanges.addNum(7)      # Stream: [1, 3, 7]
summaryRanges.getIntervals() # Returns [[1, 1], [3, 3], [7, 7]]
summaryRanges.addNum(2)      # Stream: [1, 2, 3, 7]
summaryRanges.getIntervals() # Returns [[1, 3], [7, 7]]
summaryRanges.addNum(6)      # Stream: [1, 2, 3, 6, 7]
summaryRanges.getIntervals() # Returns [[1, 3], [6, 7]]
```

10.1.2 Constraints

- $0 \leq \text{value} \leq 10^4$
 - At most 3×10^4 calls will be made to `addNum` and `getIntervals`.
 - At most 10^2 calls will be made to `getIntervals`.
-

10.2 Detailed Solution (C++)

In C++, we can use `std::map<int, int>` to store intervals as mapping of `start -> end`. Since `std::map` keeps keys sorted in a Red-Black Tree, we can efficiently find the correct position to insert/merge the new value using binary search: `upper_bound(value)`.

10.2.1 Insertion Logic for `addNum(value)`:

1. Find the first interval starting strictly after `value` using `upper_bound`.
2. Check the predecessor interval. If it overlaps or is contiguous with `value` (i.e. `prev->second >= value - 1`), merge `value` into `prev`, updating `start` to `prev->first` and `end` to `max(value, prev->second)`. Delete the predecessor interval as it is now represented by the new boundaries.
3. Check subsequent intervals. If a successor interval starts within `end + 1`, it must be merged as well. Update `end = max(end, it->second)` and erase it from the map.
4. Finally, insert the new merged interval `intervals[start] = end`.

10.2.2 Standard C++ Production Code

```
#include <vector>
#include <map>
#include <algorithm>
#include <iterator>

class SummaryRanges {
private:
    // Sorted map mapping interval start to interval end
    std::map<int, int> intervals;

public:
    SummaryRanges() = default;

    void addNum(int value) {
        int start = value;
        int end = value;

        // Find the first interval starting strictly after 'value'
        auto it = intervals.upper_bound(value);

        // Check if the predecessor interval can be merged with 'value'
        if (it != intervals.begin()) {
            auto prev = std::prev(it);
            if (prev->second >= value - 1) {
                start = prev->first;
                end = std::max(end, prev->second);
                intervals.erase(prev);
            }
        }

        intervals[start] = end;
    }
};
```

```

    }
}

// Check and merge any successor intervals that overlap or touch
while (it != intervals.end() && it->first <= end + 1) {
    end = std::max(end, it->second);
    it = intervals.erase(it); // Returns iterator to the next element
}

// Insert the merged interval
intervals[start] = end;
}

std::vector<std::vector<int>> getIntervals() {
    std::vector<std::vector<int>> res;
    res.reserve(intervals.size());
    for (const auto& [start, end] : intervals) {
        res.push_back({start, end});
    }
    return res;
}
};

```

10.3 Detailed Solution (Python)

In Python, we maintain a sorted list of intervals. We use the `bisect` module to binary search the correct insertion index.

10.3.1 Standard Python Production Code

```

import bisect
from typing import List

class SummaryRanges:
    def __init__(self):
        # Stores intervals as [start, end], maintained in sorted order
        self.intervals: List[List[int]] = []

    def addNum(self, value: int) -> None:
        lo, hi = value, value

        # Binary search the correct insertion position
        pos = bisect.bisect_left(self.intervals, [lo, hi])

```

```

# Check if the predecessor interval overlaps or touches the value
if pos > 0 and self.intervals[pos - 1][1] >= lo - 1:
    pos -= 1
    lo = self.intervals[pos][0]

# Check and merge any overlapping or contiguous successor intervals
while pos < len(self.intervals) and self.intervals[pos][0] <= hi + 1:
    hi = max(hi, self.intervals[pos][1])
    del self.intervals[pos]

# Insert the newly merged interval
self.intervals.insert(pos, [lo, hi])

def getIntervals(self) -> List[List[int]]:
    return self.intervals

```

10.4 Python-Specific Complexities & Implementation Considerations

10.4.1 1. Python List Shift Overhead

- Using `list.insert(pos, ...)` and `del list[pos]` in Python requires shifting subsequent elements in memory. This introduces an $\mathcal{O}(k)$ time complexity (where k is the number of intervals, up to N).
- For the given constraints (3×10^4 calls), this array-backed list is extremely fast due to cache locality and low constant overhead of PyObject arrays, passing easily on LeetCode.
- In a production Python environment with massive datasets, using a tree-based structure like `SortedDict` or `SortedList` from the `sortedcontainers` library provides true $\mathcal{O}(\log k)$ insertions.

10.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity (addNum)	$\mathcal{O}(\log k)$ in C++, $\mathcal{O}(k)$ in Python	C++ uses a balanced BST (<code>std::map</code>), so search, merge, and insertions take logarithmic time in terms of the number of intervals k . Python performs $\mathcal{O}(\log k)$ search but $\mathcal{O}(k)$ list shifts.
Time Complexity (getIntervals)	$\mathcal{O}(k)$	Iterates and copies all k disjoint intervals into a result list.
Space Complexity	$\mathcal{O}(k)$	Stores up to k disjoint intervals in the underlying data structure.

10.6 Common Follow-Up Questions & How to Answer

10.6.1 Q1: What if there are lots of merges and the number of disjoint intervals is small compared to the size of the data stream?

- **Answer:** If we have many duplicate inputs or dense sequences, the number of intervals k will remain small.
 - Our lookup time $\mathcal{O}(\log k)$ becomes effectively $\mathcal{O}(1)$.
 - To optimize further, we can keep a fast lookup hash set (`std::unordered_set`) of values we have already seen. If `value` is already in the set, we can return immediately from `addNum` in $\mathcal{O}(1)$ time without searching or modifying the tree.

10.6.2 Q2: How can we implement this if memory is extremely constrained?

- **Answer:** We can compress intervals using a **segment tree** with dynamic node allocation (lazy propagation), or run-length encoding. Alternatively, if we only need to query intervals occasionally, we could buffer the raw input numbers and run a sorting-based merge interval algorithm only when `getIntervals()` is called.
-

10.7 Pro-Tip: How to Impress the Interviewer

- **Use of standard library iterators:** In C++, demonstrating proper handling of `std::prev` and receiving the next valid iterator returned by `intervals.erase(it)` displays deep knowledge of C++ iterator invalidation rules.
- **Suggest `unordered_set` optimization:** Explain that in real-world data streams, duplicates are very frequent. Propose filtering incoming values with a `HashSet` filter at the front to bypass the logarithmic tree search completely for duplicate inputs.

11 Russian Doll Envelopes

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 354, Glassdoor
 - **Flashcards:** [Binary Search deck](#)
-

11.1 Question Description

You are given a 2D array of integers `envelopes` where `envelopes[i] = [wi, hi]` represents the width and the height of an envelope.

One envelope can fit into another if and only if both the width and height of one envelope are greater than the other envelope's width and height.

Return the maximum number of envelopes you can Russian doll (i.e., put one inside the other).

Note: You cannot rotate an envelope.

11.1.1 Examples

- **Input:** envelopes = [[5,4],[6,4],[6,7],[2,3]]
 - **Output:** 3
 - **Explanation:** The maximum number of envelopes you can Russian doll is 3 ([2,3] $\Rightarrow$ [5,4] $\Rightarrow$ [6,7]).
- **Input:** envelopes = [[1,1],[1,1],[1,1]]
 - **Output:** 1

11.1.2 Constraints

- $1 \leq \text{envelopes.length} \leq 10^5$
 - $\text{envelopes}[i].\text{length} == 2$
 - $1 \leq w_i, h_i \leq 10^5$
-

11.2 Detailed Solution (C++)

A naive Longest Increasing Subsequence (LIS) dynamic programming approach takes $\mathcal{O}(n^2)$ time, which will result in a Time Limit Exceeded (TLE) error since $n \leq 10^5$.

To solve this in $\mathcal{O}(n \log n)$ time, we can combine **Sorting** with **Binary Search**: 1. **Sort the Envelopes:** * Sort the width in ascending order. * For envelopes with the **same width**, sort their heights in **descending order**. 2. **Longest Increasing Subsequence (LIS):** * After sorting, the widths are sorted, so we only need to find the LIS of the heights. * By sorting heights of the same width in descending order, we ensure that we cannot select multiple envelopes of the same width (since LIS requires strictly increasing heights, and a descending sequence of heights will never allow more than one element to be picked for the same width). * We maintain an active LIS heights array `dp` and use `std::lower_bound` to update elements in-place.

11.2.1 Standard C++ Production Code

```
#include <vector>
#include <algorithm>

class Solution {
public:
    int maxEnvelopes(std::vector<std::vector<int>>& envelopes) {
        if (envelopes.empty()) {
            return 0;
        }
    }
};
```

```

    }

    // Sort width ascending, height descending for identical widths
    std::sort(envelopes.begin(), envelopes.end(), [](const std::vector<int>& a, const std::vector<int>& b) {
        if (a[0] != b[0]) {
            return a[0] < b[0];
        }
        return a[1] > b[1];
    });

    std::vector<int> dp;
    dp.reserve(envelopes.size()); // Pre-allocate memory to avoid multiple reallocations

    for (const auto& env : envelopes) {
        int h = env[1];
        // Binary search for the insertion index of height h
        auto it = std::lower_bound(dp.begin(), dp.end(), h);

        if (it == dp.end()) {
            dp.push_back(h);
        } else {
            *it = h;
        }
    }

    return static_cast<int>(dp.size());
}
};

```

11.3 Detailed Solution (Python)

11.3.1 Standard Python Production Code

Below is the iterative, type-hinted Python implementation using the built-in `bisect` library.

```

import bisect
from typing import List

class Solution:
    def maxEnvelopes(self, envelopes: List[List[int]]) -> int:
        """
        Calculates the maximum number of envelopes that can be nested.

        Time Complexity: O(N log N)

```

```

Space Complexity:  $O(N)$ 
"""
if not envelopes:
    return 0

# Sort: width ascending (x[0]), then height descending (-x[1])
envelopes.sort(key=lambda x: (x[0], -x[1]))

heights: List[int] = []
for _, h in envelopes:
    idx = bisect.bisect_left(heights, h)
    if idx == len(heights):
        heights.append(h)
    else:
        heights[idx] = h

return len(heights)

```

11.4 Python-Specific Complexities & Implementation Considerations

11.4.1 1. Complex Sorting Keys

- In Python, sorting using a custom lambda function like `key=lambda x: (x[0], -x[1])` is optimized natively in C through Timsort. The negative operator `-x[1]` works directly for numeric types. If sorting non-numeric objects or complex criteria, a custom comparator would need `functools.cmp_to_key` which adds interpreter overhead.

11.4.2 2. Space Overhead of List Manipulation

- Resizing list arrays in Python involves dynamic array resizing strategies. Pre-allocating list elements or using list comprehensions is usually faster. However, since the LIS length is bounded by N , building `heights` dynamically using `append()` is highly efficient.

11.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	Sorting takes $\mathcal{O}(n \log n)$ time. The loop performs n binary searches, each taking $\mathcal{O}(\log n)$ time.
Space Complexity	$\mathcal{O}(n)$	Stores the sorted arrays/heights up to size n .

11.6 Common Follow-Up Questions & How to Answer

11.6.1 Q1: What if we can rotate the envelopes?

- **Answer:** If we can rotate envelopes, we can treat each envelope as two options: $[w, h]$ and $[h, w]$. We can generate both options for each envelope (ensuring $w \geq h$ by swapping to standard format: long side as width, short side as height). Once standardized, we apply the same sorting and LIS logic.

11.6.2 Q2: What if we are matching in 3 dimensions (Boxes instead of Envelopes)?

- **Answer:** In 3D (width, height, depth), we can sort by width ascending, depth descending, and then we must find the 2D LIS (height and depth must both be strictly increasing). This is typically solved using a **Segment Tree** or **Fenwick Tree** (Binary Indexed Tree) in combination with divide and conquer (CDQ Divide and Conquer) to resolve the multidimensional constraints in $\mathcal{O}(n \log^2 n)$ time.

11.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Height Descending Trick Clearly:** Many candidates struggle to explain *why* the descending sort for identical widths is necessary. Explain it clearly: *"If widths are equal and heights are sorted ascending, standard LIS would allow nesting them (e.g. $[3, 3]$ and $[3, 4]$). By sorting heights descending, the LIS check on $[4, 3]$ prevents both from being included because 3 is smaller than 4, forcing us to choose at most one envelope per width."*
- **Mention Binary Search Alternatives:** Show depth by discussing that `std::lower_bound` is a binary search. In performance-critical systems, if `dp` is very small, a simple linear scan or branchless binary search could perform faster due to cache locality and compiler optimization.

12 Smallest Good Base

- **Category:** Coding
- **Difficulty:** Hard
- **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
- **Source:** LeetCode 483, Glassdoor
- **Flashcards:** [Binary Search deck](#)

12.1 Question Description

Given an integer n represented as a string, return the *smallest good base* of n .

We call $k \geq 2$ a **good base** of n if all digits of n base k are 1's.

12.1.1 Examples

- **Input:** $n = "13"$
 - **Output:** $"3"$
 - **Explanation:** 13 base 3 is 111 ($3^2 + 3^1 + 3^0 = 9 + 3 + 1 = 13$).
- **Input:** $n = "4681"$
 - **Output:** $"8"$
 - **Explanation:** 4681 base 8 is 11111 ($8^4 + 8^3 + 8^2 + 8^1 + 8^0 = 4096 + 512 + 64 + 8 + 1 = 4681$).
- **Input:** $n = "1000000000000000000"$
 - **Output:** $"999999999999999999"$
 - **Explanation:** 10^{18} base $10^{18} - 1$ is 11 ($k^1 + k^0 = (n - 1) + 1 = n$).

12.1.2 Constraints

- n is an integer in the range $[3, 10^{18}]$.
 - n does not contain any leading zeros.
-

12.2 Detailed Solution (C++)

A base $k \geq 2$ is a good base of n if there exists an integer $m \geq 2$ (representing the number of digits) such that:

$$n = k^{m-1} + k^{m-2} + \dots + k + 1 = \sum_{i=0}^{m-1} k^i$$

We want to find the **smallest** base k . Since k and m are inversely proportional, minimizing the base k is equivalent to maximizing the number of digits m .

12.2.1 Key Mathematical Boundaries:

1. **Maximum number of digits m :** The smallest possible base is $k = 2$. Therefore, $n \geq 2^{m-1}$, which means $m \leq \log_2(n) + 1 \approx 62$ for $n \leq 10^{18}$.
2. **Evaluating for a fixed m :** For a fixed digit count m , we can search for a valid base k in the range $[2, n^{1/(m-1)}]$. Since $\sum_{i=0}^{m-1} k^i$ is strictly increasing with respect to k , we can use **Binary Search** to find the base k .
3. **Upper Bound for Binary Search:** To prevent floating-point precision issues when estimating $n^{1/(m-1)}$, we set $hi = \lfloor n^{1/(m-1)} \rfloor + 2$.
4. **Iteration Order:** We iterate m from the maximum possible value (≈ 62) down to 2. The first base k we find is guaranteed to be the smallest good base. If no base is found for $m \geq 3$, the fallback is $m = 2$, which corresponds to $k = n - 1$.

12.2.2 Standard C++ Production Code

```
#include <string>
#include <cmath>
#include <algorithm>
```

```

class Solution {
private:
    // Helper function to calculate  $\sum_{i=0}^{m-1} k^i$  while preventing overflow
    unsigned long long calculateSum(unsigned long long k, int m, unsigned long long limit) noexcept {
        unsigned long long sum = 0;
        for (int i = 0; i < m; ++i) {
            // Check for multiplication overflow
            if (sum > (limit - 1) / k) {
                return limit + 1; // Return a value larger than limit to signal overflow
            }
            sum = sum * k + 1;
        }
        return sum;
    }

public:
    std::string smallestGoodBase(const std::string& n) {
        unsigned long long num = std::stoull(n);

        // Find the maximum potential number of digits m
        // 64 - clzll yields the number of bits in num
        int max_m = 64 - __builtin_clzll(num);

        // Iterate from largest digit count to smallest (to minimize base k)
        for (int m = max_m; m >= 3; --m) {
            unsigned long long lo = 2;
            // Approximate upper bound using pow to avoid search space explosion
            unsigned long long hi = static_cast<unsigned long long>(std::pow(static_cast<double>(num), 1.0 / m));

            while (lo <= hi) {
                unsigned long long mid = lo + (hi - lo) / 2;
                unsigned long long sum = calculateSum(mid, m, num);

                if (sum == num) {
                    return std::to_string(mid);
                } else if (sum < num) {
                    lo = mid + 1;
                } else {
                    hi = mid - 1;
                }
            }
        }

        // Fallback case: m = 2, where  $n = k + 1 \rightarrow k = n - 1$ 
    }
}

```

```

        return std::to_string(num - 1);
    }
};

```

12.3 Detailed Solution (Python)

12.3.1 Standard Python Production Code

Below is the iterative, type-hinted Python implementation. Python handles arbitrarily large integers natively, simplifying sum calculations.

```

import math

class Solution:
    def smallestGoodBase(self, n: str) -> str:
        """
        Finds the smallest good base of n using binary search over the number of digits.

        Time Complexity:  $O(\log^2 N)$ 
        Space Complexity:  $O(1)$ 
        """
        num = int(n)

        # Max bit length represents the maximum possible value of m (base 2)
        max_m = num.bit_length()

        # Helper function to evaluate the geometric sum
        def evaluate_base(k: int, m: int) -> int:
            total = 0
            for _ in range(m):
                total = total * k + 1
            if total > num:
                return total
            return total

        # Iterate from largest possible digit count to smallest
        for m in range(max_m, 2, -1):
            lo = 2
            hi = int(num ** (1.0 / (m - 1))) + 2

            while lo <= hi:
                mid = (lo + hi) // 2
                val = evaluate_base(mid, m)

```

```

    if val == num:
        return str(mid)
    elif val < num:
        lo = mid + 1
    else:
        hi = mid - 1

# Fallback: for m = 2, the base is always num - 1
return str(num - 1)

```

12.4 Python-Specific Complexities & Implementation Considerations

12.4.1 1. Integer Division and Large Power Calculations

- The expression `num ** (1.0 / (m - 1))` converts `num` to a float in Python. For numbers close to 10^{18} , standard double-precision floats have 53 bits of precision (up to 9×10^{15}), which can introduce precision errors.
- To safeguard against float precision loss, we add a buffer of `+ 2` to `hi` to ensure the correct base is never excluded from the search range.

12.4.2 2. Large Integers

- Python dynamically handles integer overflow, meaning we do not need to check for integer wrap-around (like in C++'s `stoull`). However, calculating large powers of `k` can still grow exponentially in execution time if unchecked. The early exit check `if total > num:` inside `evaluate_base` is critical to prevent redundant execution.
-

12.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\log^2 n)$	Outer loop runs $\approx \log n$ times. The binary search runs $\approx \log n$ times, with each evaluation taking $\mathcal{O}(\log n)$ multiplication steps.
Space Complexity	$\mathcal{O}(1)$	Only standard integer variables are used.

12.6 Common Follow-Up Questions & How to Answer

12.6.1 Q1: Why do we search from $m = \text{max_m}$ down to 2, rather than 2 up to max_m ?

- **Answer:** We are looking for the **smallest** base k . Since a smaller base k results in a larger number of digits m , we must find the **largest** m that satisfies the condition. Iterating from largest m downwards allows us to return the first matching base immediately, optimizing execution time.

12.6.2 Q2: What mathematical approximation can we use to skip binary search?

- **Answer:** For a fixed m , the equation is $n \approx k^{m-1}$. Thus, $k \approx n^{1/(m-1)}$. We can directly check if $k = \lfloor n^{1/(m-1)} \rfloor$ is a good base. This reduces the time complexity from $\mathcal{O}(\log^2 n)$ to $\mathcal{O}(\log n)$ since we only evaluate a single candidate base for each m .
-

12.7 Pro-Tip: How to Impress the Interviewer

- **Use Bitwise Helpers:** Demonstrating knowledge of compiler intrinsics (e.g. `__builtin_clzll` in GCC/Clang) or the `bit_length()` method in Python to quickly determine the number of active bits shows that you write optimized, platform-aware system code.
- **Proactively Prevent Overflow:** Discussing how to prevent overflow in the accumulator `sum = sum * k + 1` in C++ *before* multiplying (i.e. checking `sum > (limit - 1) / k`) proves your depth in low-level safety and arithmetic robustness.

13 Random Point in Non-overlapping Rectangles

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 497, Glassdoor
 - **Flashcards:** [Binary Search deck](#)
-

13.1 Question Description

You are given an array of non-overlapping axis-aligned rectangles `rects` where `rects[i] = [ai, bi, xi, yi]` indicates that `(ai, bi)` is the bottom-left corner point of the i -th rectangle and `(xi, yi)` is the top-right corner point of the i -th rectangle. Design an algorithm to pick a random integer point inside the space covered by one of the given rectangles. A point on the perimeter of a rectangle is included in the space covered by the rectangle.

Any integer point inside the space covered by one of the given rectangles should be **equally likely** to be returned.

Note that an integer point is a point that has integer coordinates.

Implement the `Solution` class: * `Solution(int[][] rects)` Initializes the object with the given rectangles `rects`. * `int[] pick()` Returns a random integer point `[x, y]` from the space covered by the rectangles.

13.1.1 Examples

- **Input:** `["Solution", "pick", "pick", "pick", "pick", "pick"]` `[[[-2, -2, 1, 1], [2, 2, 4, 6]], [], [], [], [], []]`
- **Output:** `[null, [1, -2], [1, -1], [-1, -2], [-2, -2], [0, 0]]`
- **Explanation:**

```
solution = Solution([[-2, -2, 1, 1], [2, 2, 4, 6]])
solution.pick() # return [1, -2]
solution.pick() # return [1, -1]
solution.pick() # return [-1, -2]
solution.pick() # return [-2, -2]
solution.pick() # return [0, 0]
```

13.1.2 Constraints

- $1 \leq \text{rects.length} \leq 100$
- $\text{rects}[i].\text{length} == 4$
- $-10^9 \leq a_i < x_i \leq 10^9$
- $-10^9 \leq b_i < y_i \leq 10^9$
- $x_i - a_i \leq 2000$
- $y_i - b_i \leq 2000$
- All the rectangles do not overlap.
- At most 10^4 calls will be made to `pick`.

13.2 Detailed Solution (C++)

To ensure that each integer point across all rectangles is equally likely to be chosen: 1. **Calculate Point Count Per Rectangle:** Since coordinates represent integer points, the number of points in rectangle i is:

$$\text{points}_i = (x_2 - x_1 + 1) \times (y_2 - y_1 + 1)$$

2. **Generate Prefix Sums:** We create a prefix sum array `prefix_` where `prefix_[i]` stores the cumulative number of points in rectangles 0 through i . The total number of points is `total_ = prefix_.back()`. 3. **Binary Search for Rectangle Selection:** When calling `pick()`: * We pick a random integer t in the range $[0, \text{total}_ - 1]$. * We use binary search (`std::upper_bound` or standard binary search template) to locate the rectangle corresponding to t . 4. **Uniform Point Generation within Rectangle:** Once the rectangle is selected, we independently pick a random x from $[x_1, x_2]$ and a random y from $[y_1, y_2]$ using a uniform random generator.

13.2.1 Standard C++ Production Code

```

#include <vector>
#include <random>
#include <algorithm>

class Solution {
private:
    std::vector<std::vector<int>>> rects;
    std::vector<int> prefix;
    int total_points;

    // Modern C++ Random Engine
    std::mt19937 gen;

public:
    Solution(const std::vector<std::vector<int>>>& rects) : rects(rects), total_points(0), gen(std::random_device{}()) {
        prefix.reserve(rects.size());
        for (const auto& r : rects) {
            // Rectangles are axis-aligned, calculating the number of integer points on/in it
            int points = (r[2] - r[0] + 1) * (r[3] - r[1] + 1);
            total_points += points;
            prefix.push_back(total_points);
        }
    }

    std::vector<int> pick() {
        // Uniform distribution over the total point space
        std::uniform_int_distribution<int> dist(0, total_points - 1);
        int t = dist(gen);

        // Binary search to find the rectangle index
        // equivalent to finding first index i where prefix[i] > t
        auto it = std::upper_bound(prefix.begin(), prefix.end(), t);
        int idx = std::distance(prefix.begin(), it);

        const auto& r = rects[idx];

        // Pick a uniform random point within the chosen rectangle
        std::uniform_int_distribution<int> dist_x(r[0], r[2]);
        std::uniform_int_distribution<int> dist_y(r[1], r[3]);

        return {dist_x(gen), dist_y(gen)};
    }
}

```

```
};
```

13.3 Detailed Solution (Python)

13.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using `random.randint` and `bisect` for fast prefix binary searches.

```
import random
import bisect
from typing import List

class Solution:
    def __init__(self, rects: List[List[int]]):
        """
        Initializes prefix sum of point counts for each rectangle.
        """
        self.rects = rects
        self.prefix = []
        total = 0
        for x1, y1, x2, y2 in rects:
            points = (x2 - x1 + 1) * (y2 - y1 + 1)
            total += points
            self.prefix.append(total)
        self.total = total

    def pick(self) -> List[int]:
        """
        Selects a random rectangle based on point weight, then chooses a point within.
        """
        # Generate a random integer in [1, total]
        t = random.randint(1, self.total)

        # Binary search the selected rectangle index
        idx = bisect.bisect_left(self.prefix, t)

        x1, y1, x2, y2 = self.rects[idx]

        # Pick uniformly from the chosen grid
        return [random.randint(x1, x2), random.randint(y1, y2)]
```

13.4 Python-Specific Complexities & Implementation Considerations

13.4.1 1. Choice of Randomness Function: `randint` vs `randrange`

- `random.randint(a, b)` returns a random integer N such that $a \leq N \leq b$ (both endpoints inclusive). This matches our integer coordinate requirements.
- `random.randrange(a, b)` is exclusive of the upper limit b . Make sure you use `randint` or set `randrange(a, b + 1)` correctly to include the boundaries.

13.4.2 2. Prefix Search using `bisect_left` vs `bisect_right`

- Since `t` is generated in range `[1, total]`, we want to find the first index where `prefix[idx] >= t`. This is perfectly handled by `bisect_left`. If `t` was in `[0, total - 1]`, we would use `bisect_right(prefix, t)`.

13.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity (Constructor)	$\mathcal{O}(n)$	Computes the area and prefix sum for all n rectangles.
Time Complexity (pick)	$\mathcal{O}(\log n)$	Binary searches the rectangle index, and runs two constant-time random coordinate generation calls.
Space Complexity	$\mathcal{O}(n)$	Stores the prefix sums and coordinates of n rectangles.

13.6 Common Follow-Up Questions & How to Answer

13.6.1 Q1: What if the rectangles overlap?

- **Answer:** If rectangles overlap, simple prefix sum weights would count the overlapping areas multiple times, leading to non-uniform selection probabilities.
 - **Solution:** We can use **disjoint set / segment tree** to segment the rectangles into non-overlapping smaller rectangles first, then apply the same algorithm. Alternatively, we can use **Rejection Sampling**: pick a point from the bounding box of all rectangles, and check if it lies in any rectangle; if not, retry. However, this could have high retry rates if the density of rectangles is low.

13.6.2 Q2: What if we can't store the rectangles in memory (Streaming / Reservoir Sampling)?

- **Answer:** If we receive rectangles in a data stream, we can use **Reservoir Sampling** to select the rectangle:

- Maintain the current `total_points` seen so far.
 - For each new rectangle with A points, we select it as our candidate with probability $\frac{A}{\text{total_points}}$.
 - Once the stream ends, we sample a point from the selected candidate rectangle.
-

13.7 Pro-Tip: How to Impress the Interviewer

- **Use Modern C++ Random Library:** Avoid using `rand() % total_` in C++ as it suffers from **modulo bias** (non-uniform distribution) and limited range (typically 32767 in `RAND_MAX`). Instead, use `std::mt19937` with `std::uniform_int_distribution` which executes faster and provides high-quality pseudorandomness suitable for simulations.
- **Point out Coordinate Boundary Logic:** Highlight the $+1$ offset in $(x2 - x1 + 1)$. Emphasize that coordinate points are discrete (1D grid size is inclusive of both boundary lines), unlike standard geometric area calculations.

14 Binary Search

- **Category:** Coding
 - **Difficulty:** Easy
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 704, Glassdoor
 - **Flashcards:** [Binary Search deck](#)
-

14.1 Question Description

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $\mathcal{O}(\log n)$ runtime complexity.

14.1.1 Examples

- **Input:** `nums = [-1,0,3,5,9,12]`, `target = 9`
 - **Output:** 4
 - **Explanation:** 9 exists in `nums` and its index is 4.
- **Input:** `nums = [-1,0,3,5,9,12]`, `target = 2`
 - **Output:** -1
 - **Explanation:** 2 does not exist in `nums` so return -1.

14.1.2 Constraints

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 < \text{nums}[i], \text{target} < 10^4$
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

14.2 Detailed Solution (C++)

The standard **Binary Search** algorithm divides the search interval in half recursively or iteratively: 1. Initialize pointer boundaries `left = 0` and `right = nums.size() - 1`. 2. While `left <= right`: * Find the middle index using `mid = left + (right - left) / 2`. * If `nums[mid] == target`, return `mid`. * If `nums[mid] < target`, discard the left half by setting `left = mid + 1`. * If `nums[mid] > target`, discard the right half by setting `right = mid - 1`. 3. If the loop terminates without finding the target, return `-1`.

14.2.1 Standard C++ Production Code

```
#include <vector>
#include <cstdint>

class Solution {
public:
    int search(const std::vector<int>& nums, int target) noexcept {
        // Edge Case: Handle empty collection explicitly to prevent unsigned underflow
        if (nums.empty()) {
            return -1;
        }

        int left = 0;
        int right = static_cast<int>(nums.size()) - 1; // Safely cast size to int

        while (left <= right) {
            int mid = left + (right - left) / 2; // Prevent integer overflow (left + right) / 2

            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                left = mid + 1; // Target lies in the right half
            } else {
                right = mid - 1; // Target lies in the left half
            }
        }

        return -1; // Target not found
    }
};
```

14.3 Detailed Solution (Python)

14.3.1 Standard Python Production Code

Below is the iterative, type-hinted Python implementation. It avoids recursion overhead and ensures true constant space.

```
from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        """
        Searches for a target in a sorted array using binary search.

        Time Complexity:  $O(\log N)$ 
        Space Complexity:  $O(1)$ 
        """
        # Edge Case: Handle empty collection explicitly
        if not nums:
            return -1

        left, right = 0, len(nums) - 1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return -1
```

14.4 Python-Specific Complexities & Implementation Considerations

14.4.1 1. Python Built-In Alternative: `bisect`

- Python provides the `bisect` library which implements binary search. For finding an exact value, we can use `bisect.bisect_left`:

```
import bisect

idx = bisect.bisect_left(nums, target)
if idx < len(nums) and nums[idx] == target:
```

```

    return idx
return -1

```

- Under the hood, Python's `bisect` is implemented in C, making it faster than a pure-Python `while` loop. Mentioning this built-in utility in Python interviews shows familiarity with library optimizations.

14.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\log n)$	The search space is halved in each step of the binary search loop.
Space Complexity	$\mathcal{O}(1)$	Only a few integer variables are maintained, requiring constant auxiliary memory.

14.6 Common Follow-Up Questions & How to Answer

14.6.1 Q1: How does the algorithm behave if we search for a value that is missing?

- **Answer:** If the target is missing, the binary search converges to the position where the target *should* be inserted. Specifically:
 - `left` will point to the index of the first element larger than target (equivalent to C++ `std::lower_bound` or Python `bisect_left`).
 - `right` will point to `left - 1`.

14.6.2 Q2: What is the difference between `lower_bound` and `upper_bound`?

- **Answer:**
 - `lower_bound` finds the first element that is **greater than or equal to** target (first position where target can be inserted without violating sorted order).
 - `upper_bound` finds the first element that is **strictly greater than** target.
-

14.7 Pro-Tip: How to Impress the Interviewer

- **Overflow Prevention Explanation:** Explain why `left + (right - left) / 2` is used instead of `(left + right) / 2`. In 32-bit signed integers, `left + right` can exceed $2^{31} - 1$, causing overflow and resulting in a negative value, leading to out-of-bounds array access.
- **Cache and Branch Predictor Optimization:** Mention that for small arrays ($n \leq 64$), linear search is often faster than binary search due to sequential memory access and spatial cache locality. Discussing this shows hardware-level engineering competence.

15 Valid Parentheses

- **Category:** Coding
 - **Difficulty:** Easy
 - **Target Role:** Software Engineer / QA & Test Engineer
 - **Source:** LeetCode 20, Glassdoor
 - **Flashcards:** [Stack deck](#)
-

15.1 Question Description

Given a string `s` containing just the characters `'('`, `)'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if: 1. Open brackets must be closed by the same type of brackets. 2. Open brackets must be closed in the correct order. 3. Every close bracket has a corresponding open bracket of the same type.

15.1.1 Examples

- **Input:** `s = "()"`
 - **Output:** `true`
 - **Input:** `s = "()[]{}"`
 - **Output:** `true`
 - **Input:** `s = "]"`
 - **Output:** `false`
 - **Input:** `s = "([])"`
 - **Output:** `true`
 - **Input:** `s = "([)]"`
 - **Output:** `false`
-

15.2 Detailed Solution (C++)

A standard linear scan with a **Stack** data structure is optimal. To simplify the character mapping and comparison code in interviews, we can use an elegant pattern: **when encountering an opening bracket, push its expected closing bracket onto the stack**. When we encounter a closing bracket, we simply compare it directly to `st.top()`. This eliminates the need to map closing brackets to opening brackets during pop comparison.

15.2.1 Standard C++ Production Code

```
#include <string>
#include <stack>

class Solution {
```

```

public:
    bool isValid(const std::string& s) noexcept {
        std::stack<char> st;

        for (const char ch : s) {
            if (ch == '(') {
                st.push('(');
            } else if (ch == '[') {
                st.push('[');
            } else if (ch == '{') {
                st.push('{');
            } else {
                // If stack is empty (no matching open bracket) or
                // top character doesn't match the closing bracket
                if (st.empty() || st.top() != ch) {
                    return false;
                }
                st.pop();
            }
        }

        return st.empty();
    }
};

```

15.3 Detailed Solution (Python)

15.3.1 Standard Python Production Code

Below is the Python implementation using the same expected closing bracket matching pattern.

```

class Solution:
    def isValid(self, s: str) -> bool:
        """
        Validates if parenthesized string s is well-formed.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        stack = []

        for char in s:
            if char == '(':
                stack.append(')')

```

```

elif char == '[':
    stack.append('[')
elif char == '{':
    stack.append('{')
else:
    # If stack is empty or current closing char doesn't match top
    if not stack or stack.pop() != char:
        return False

return len(stack) == 0

```

15.4 Python-Specific Complexities & Implementation Considerations

15.4.1 1. list as a Stack

- In Python, standard lists `[]` are used as stacks. Adding elements using `.append()` and removing elements using `.pop()` run in $\mathcal{O}(1)$ amortized time.
- Unlike C++ `std::stack`, Python lists are dynamic arrays, so resizing events will copy references to a larger array. However, this amortizes to constant time.

15.4.2 2. Early Termination Performance

- If the string length is odd, it is impossible for all brackets to be paired. We can check `len(s) % 2 != 0` at the very beginning to return `False` immediately, saving execution time.
-

15.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the string of length n exactly once. Each character is pushed or popped at most once.
Space Complexity	$\mathcal{O}(n)$	In the worst case (e.g. " <code>(((((("</code>), the stack stores all characters of the string.

15.6 Common Follow-Up Questions & How to Answer

15.6.1 Q1: What if the string contains non-bracket characters (e.g. "`a + (b * c)`")?

- **Answer:** We can simply ignore any non-bracket characters and only run the push/pop logic when encountering bracket characters (`(`, `)`, `[`, `]`, `{`, `}`).

15.6.2 Q2: What if we only have one type of bracket (e.g. only (and))? Can we optimize space?

- **Answer:** Yes. If only one type of bracket exists, we don't need a stack. We can just use an integer counter initialized to 0.
 - Increment the counter when seeing (.
 - Decrement the counter when seeing).
 - If the counter ever drops below 0, return `false` (unbalanced).
 - At the end, return `true` if the counter is exactly 0. This reduces space complexity to $\mathcal{O}(1)$.
-

15.7 Pro-Tip: How to Impress the Interviewer

- **Expected Closing Bracket Strategy:** Highlight the design choice of pushing the closing bracket (')', ']', '}') instead of the opening bracket ('(', '[', '{'). This eliminates the lookup mapping step during popping and leads to cleaner, less bug-prone code.
- **Explain the Chomsky Hierarchy:** Mention that matching arbitrary nested parentheses is a classic **Context-Free Language (CFL)** problem (specifically, the Dyck language). It cannot be parsed by a regular expression (Finite Automaton) because it requires memory of arbitrary depth. This shows strong computer science theory background.

16 Min Stack

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer
 - **Source:** LeetCode 155, Glassdoor
 - **Flashcards:** [Stack deck](#)
-

16.1 Question Description

Design a stack that supports `push`, `pop`, `top`, and retrieving the minimum element in constant time.

Implement the `MinStack` class: `* MinStack()` initializes the stack object. `* void push(int val)` pushes the element `val` onto the stack. `* void pop()` removes the element on the top of the stack. `* int top()` gets the top element of the stack. `* int getMin()` retrieves the minimum element in the stack.

You must implement a solution with $\mathcal{O}(1)$ time complexity for each function.

16.1.1 Examples

- **Input:**

```
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[-1],[-3],[],[],[],[]]
```

– **Output:** [null,null,null,null,-3,null,0,-2]

– **Explanation:**

```
MinStack minStack;
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top();    // return 0
minStack.getMin(); // return -2
```

16.2 Detailed Solution (C++)

A standard stack only provides access to its top element. To retrieve the minimum element in $\mathcal{O}(1)$ time, we must maintain auxiliary state. The cleanest production-grade solution uses **two parallel stacks**: 1. **st**: A standard stack storing the elements. 2. **mn**: An auxiliary stack where the top element always represents the minimum value in the stack at that height.

16.2.1 Standard C++ Production Code

```
#include <vector>
#include <algorithm>
#include <stdexcept>

class MinStack {
private:
    std::vector<int> st; // Main value stack
    std::vector<int> mn; // Running minimum stack

public:
    MinStack() noexcept {}

    void push(int val) noexcept {
        st.push_back(val);
        if (mn.empty()) {
            mn.push_back(val);
        } else {
            mn.push_back(std::min(val, mn.back()));
        }
    }

    void pop() noexcept {
        if (!st.empty()) {
```

```

        st.pop_back();
        mn.pop_back();
    }
}

int top() const {
    if (st.empty()) {
        throw std::underflow_error("Stack is empty");
    }
    return st.back();
}

int getMin() const {
    if (mn.empty()) {
        throw std::underflow_error("Stack is empty");
    }
    return mn.back();
}
};

```

16.3 Detailed Solution (Python)

16.3.1 Standard Python Production Code

Below is the Python implementation. Instead of two separate lists, we can store values as tuples (val, current_min) inside a single list to keep the structure unified.

```

class MinStack:
    def __init__(self):
        """
        Initializes the stack. We store tuples of (value, min_so_far).
        """
        self.stack = []

    def push(self, val: int) -> None:
        """
        Pushes element val onto the stack.
        """
        if not self.stack:
            self.stack.append((val, val))
        else:
            current_min = self.stack[-1][1]
            self.stack.append((val, min(val, current_min)))

```

```
def pop(self) -> None:
    """
    Removes the element on the top of the stack.
    """
    if self.stack:
        self.stack.pop()

def top(self) -> int:
    """
    Gets the top element of the stack.
    """
    if not self.stack:
        raise IndexError("pop from empty stack")
    return self.stack[-1][0]

def getMin(self) -> int:
    """
    Retrieves the minimum element in the stack.
    """
    if not self.stack:
        raise IndexError("getMin from empty stack")
    return self.stack[-1][1]
```

16.4 Python-Specific Complexities & Implementation Considerations

16.4.1 1. Amortized dynamic array resizing

- Python's lists are dynamic arrays. Resizing a list allocates extra capacity to support subsequent $\mathcal{O}(1)$ push operations. Under extreme heap constraints, storing tuples (`val`, `min_val`) creates small overhead since tuples require 48 bytes of metadata in Python, while a flat integer list is smaller.

16.4.2 2. Exceptions handling

- According to LeetCode constraints, `pop`, `top`, and `getMin` will always be called on non-empty stacks. However, in production-grade code, it is critical to raise standard exceptions (`IndexError` or custom exceptions) to handle invalid access attempts safely.
-

16.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(1)$	All operations (push , pop , top , getMin) only perform indexing or basic arithmetic, taking constant time.
Space Complexity	$\mathcal{O}(n)$	For n elements pushed, we store n states of the minimum.

16.6 Common Follow-Up Questions & How to Answer

16.6.1 Q1: How can we optimize space when there are many duplicate pushes?

- **Answer:** Instead of pushing to the min stack every time, we only push a value to the min stack if it is **less than or equal to** the current minimum.
 - During a **pop** operation, we only pop from the min stack if the popped value from the main stack is equal to the top of the min stack.
 - This prevents storing redundant minimums when many large elements are pushed consecutively.

16.6.2 Q2: Can we implement this with $\mathcal{O}(1)$ auxiliary space? (The Difference Trick)

- **Answer:** Yes, we can store only the **differences** between the pushed values and the running minimum. This requires a single stack of integers and a single `min_val` variable.
 - **Push val:**
 - * If stack is empty: `min_val = val`, push `0`.
 - * If `val >= min_val`: push `val - min_val`.
 - * If `val < min_val`: push `val - min_val` (which is negative!), and update `min_val = val`.
 - **Pop:**
 - * Let top element be `diff`.
 - * If `diff < 0`: the previous minimum was `min_val - diff`. Update `min_val = min_val - diff`.
 - * Pop from stack.
 - **Top:**
 - * Let top element be `diff`.
 - * If `diff >= 0`: return `diff + min_val`.
 - * If `diff < 0`: return `min_val`.
 - **Trade-off:** In languages with fixed-width integers, subtracting values can result in integer overflow. Thus, you must use 64-bit integers (`long long` in C++) to prevent this.

16.7 Pro-Tip: How to Impress the Interviewer

- **Warn About Custom Memory Allocators:** Mention that in high-performance C++ systems, invoking `std::vector::push_back` repeatedly calls default heap allocators which can cause memory

fragmentation. Using a pre-allocated stack (`std::vector::reserve`) or custom memory arena allocators guarantees predictable allocation time and locality.

- **Evaluate Thread Safety:** Discuss how adding locking mechanisms (`std::mutex` or read-write locks) would make this data structure thread-safe for producer-consumer pipelines, which is a key consideration in real-world systems architecture.

17 Decode String

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / QA & Test Engineer
- **Source:** LeetCode 394, Glassdoor
- **Flashcards:** [Stack deck](#)

17.1 Question Description

Given an encoded string, return its decoded string.

The encoding rule is: `k[encoded_string]`, where the `encoded_string` inside the square brackets is being repeated exactly `k` times. Note that `k` is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k . For example, there will not be input like `3a` or `2[4]`.

The test cases are generated so that the length of the output will never exceed 10^5 .

17.1.1 Examples

- **Input:** `s = "3[a]2[bc]"`
– **Output:** `"aaabcbc"`
- **Input:** `s = "3[a2[c]]"`
– **Output:** `"accaccacc"`
- **Input:** `s = "2[abc]3[cd]ef"`
– **Output:** `"abcabccdcdcdcf"`

17.2 Detailed Solution (C++)

A standard stack solution parses the string character-by-character. We can implement an optimized C++ solution by storing indices instead of raw sub-strings on the stack. * We use a single `std::string` result to build the output. * Our stack only needs to store pairs of `std::pair<int, int>` which represent {repeat_multiplier, start_index_in_result}. * When encountering '[' , we push the multiplier and the current length of result onto the stack. * When encountering ']' , we retrieve the start index, extract

the segment `result.substr(start)`, and append it `repeat - 1` times. * This approach avoids allocating separate strings on the stack, which dramatically reduces heap allocations.

17.2.1 Standard C++ Production Code

```
#include <string>
#include <vector>
#include <utility>

class Solution {
public:
    std::string decodeString(const std::string& s) noexcept {
        // stack stores: {repeat_count, start_index_in_result}
        std::vector<std::pair<int, int>> stk;
        std::string result;
        int curr_num = 0;

        for (const char ch : s) {
            if (ch >= '0' && ch <= '9') {
                curr_num = curr_num * 10 + (ch - '0');
            } else if (ch == '[') {
                stk.push_back({curr_num, static_cast<int>(result.size())});
                curr_num = 0; // Reset for next potential nested multiplier
            } else if (ch == ']') {
                auto [repeat, start] = stk.back();
                stk.pop_back();

                std::string seg = result.substr(start);
                // Append segment (repeat - 1) times since it is already in result once
                for (int j = 1; j < repeat; ++j) {
                    result += seg;
                }
            } else {
                result += ch;
            }
        }
        return result;
    }
};
```

17.3 Detailed Solution (Python)

17.3.1 Standard Python Production Code

Below is the Python implementation. In Python, list concatenations are highly optimized, so pushing strings and counts onto a stack is clean and simple.

class Solution:

```
def decodeString(self, s: str) -> str:
    """
    Decodes a nested repeat pattern string.

    Time Complexity: O(L) where L is the length of the decoded string.
    Space Complexity: O(D) where D is the maximum nesting depth of brackets.
    """
    stack = []
    curr_num = 0
    curr_str = ""

    for char in s:
        if char.isdigit():
            curr_num = curr_num * 10 + int(char)
        elif char == "[":
            # Push the current state (multiplier and the prefix string built so far)
            stack.append((curr_num, curr_str))
            curr_num = 0
            curr_str = ""
        elif char == "]":
            # Pop multiplier and prefix, build the new current string
            repeat, prefix = stack.pop()
            curr_str = prefix + curr_str * repeat
        else:
            curr_str += char

    return curr_str
```

17.4 Python-Specific Complexities & Implementation Considerations

17.4.1 1. Large String Concatenations

- In Python, string modification (`curr_str += char`) creates a new string on the heap because Python strings are immutable.
- For very long strings, this can degrade performance due to quadratic copy operations.
- **Alternative:** Store characters in a list (e.g. `curr_list.append(char)`) and use `''.join(curr_list)` to build the final string. In LeetCode, since the output size is capped at 10^5 , string additions are

fast enough, but list joins are safer for massive strings.

17.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(l)$	Where l is the length of the decoded string. We write each character to the output at least once.
Space Complexity	$\mathcal{O}(d + l)$	Where d is the maximum nesting depth (stack space) and l is the output string buffer size.

17.6 Common Follow-Up Questions & How to Answer

17.6.1 Q1: How can we implement this recursively?

- **Answer:** We can write a recursive helper function that takes the current read index `idx`. It parses digits and letters, and recurses when encountering `[`. It returns the parsed string and the updated index.
- **Recursive C++ Example:**

```
class Solution {
private:
    std::string decode(const std::string& s, int& i) {
        std::string res;
        while (i < s.length() && s[i] != ']') {
            if (!std::isdigit(s[i])) {
                res += s[i++];
            } else {
                int k = 0;
                while (i < s.length() && std::isdigit(s[i])) {
                    k = k * 10 + (s[i++] - '0');
                }
                i++; // skip '['
                std::string decoded = decode(s, i);
                i++; // skip ']'
                while (k-- > 0) res += decoded;
            }
        }
        return res;
    }
};
```

```

    }
    public:
        std::string decodeString(std::string s) {
            int i = 0;
            return decode(s, i);
        }
};

```

17.6.2 Q2: What if we push nested expressions like `100[100[100[a]]]`? How to prevent Out-Of-Memory (OOM)?

- **Answer:** This is a potential **Denial of Service (DoS)** exploit. Deep nesting with large multipliers can generate gigabytes of data from a tiny input string, blowing up memory.
 - **Mitigation:** In a production environment, track the length of the output string during parsing. If the cumulative output size exceeds a safety threshold (e.g., 5MB), abort execution and throw an exception.
-

17.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Sub-string Heap Allocations:** Show off by highlighting the index-tracking optimization in C++. Standard stack solutions push substrings onto the stack, e.g., `std::stack<std::string>`. This invokes `std::string` constructors and triggers heap allocations for every bracket depth. Storing `{repeat_multiplier, start_index}` inside a simple `vector` keeps memory flat and avoids heap thrashing.
- **Discuss AST / Parsing Context:** Mention that this problem is a simple parsing task of a context-free grammar ($S \rightarrow \text{term} \mid SS$, $\text{term} \rightarrow \text{chars} \mid \text{num}[S]$). Framing it as a grammar parser shows you understand formal compiler theory.

18 Jump Game III

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 1306, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

18.1 Question Description

Given an array of non-negative integers `arr` and a starting index `start`, you are initially positioned at `arr[start]`. When you are at index `i`, you can jump to `i + arr[i]` or `i - arr[i]`.

Check if you can reach **any** index with value `0`.

Notice that you cannot jump outside of the array boundaries at any time.

18.1.1 Examples

- **Input:** arr = [4,2,3,0,3,1,2], start = 5
 - **Output:** true
 - **Explanation:** One possible path to reach index 3 (value 0) is: index 5 -> index 4 -> index 1 -> index 3
 - **Input:** arr = [4,2,3,0,3,1,2], start = 0
 - **Output:** true
 - **Explanation:** One possible path to reach index 3 (value 0) is: index 0 -> index 4 -> index 1 -> index 3
 - **Input:** arr = [3,0,2,1,2], start = 2
 - **Output:** false
 - **Explanation:** There is no way to reach any index with value 0 starting from index 2.
-

18.2 Detailed Solution (C++)

The problem represents a graph traversal on a directed graph where each index i has edges pointing to $i + \text{arr}[i]$ and $i - \text{arr}[i]$. We can use **Depth-First Search (DFS)** to traverse the graph starting from the node **start**. To prevent infinite loops caused by cycles, we maintain a **visited** array (using `std::vector<char>` or mutating the array) to record indices that have already been visited.

18.2.1 Standard C++ Production Code

```
#include <vector>

class Solution {
private:
    bool dfs(const std::vector<int>& arr, int i, std::vector<char>& visited) noexcept {
        // Base Case: check array boundaries and visited state
        if (i < 0 || i >= static_cast<int>(arr.size()) || visited[i]) {
            return false;
        }

        // Target found
        if (arr[i] == 0) {
            return true;
        }

        // Mark as visited
        visited[i] = 1;

        // Traverse left and right branches
        return dfs(arr, i + arr[i], visited) || dfs(arr, i - arr[i], visited);
    }
}
```

```

public:
    bool canReach(const std::vector<int>& arr, int start) noexcept {
        // Handle edge cases
        if (arr.empty() || start < 0 || start >= static_cast<int>(arr.size())) {
            return false;
        }

        // std::vector<char> is more space-efficient than std::vector<bool>
        // and avoids the non-standard reference proxy behavior of std::vector<bool>
        std::vector<char> visited(arr.size(), 0);
        return dfs(arr, start, visited);
    }
};

```

18.3 Detailed Solution (Python)

18.3.1 Standard Python Production Code

Below is the recursive Python implementation using a set for visited track record.

```

from typing import List

class Solution:
    def canReach(self, arr: List[int], start: int) -> bool:
        """
        Determines if we can reach an index with value 0 in the array starting from 'start'.

        Time Complexity: O(N) where N is the length of the array.
        Space Complexity: O(N) due to visited set and call stack.
        """
        # Edge Case: boundary checks
        if not arr or start < 0 or start >= len(arr):
            return False

        n = len(arr)
        visited = set()

        def dfs(i: int) -> bool:
            # Base Case: check bounds and visited
            if i < 0 or i >= n or i in visited:
                return False

            # Target reached

```

```

    if arr[i] == 0:
        return True

    visited.add(i)
    jump = arr[i]

    # Recurse on left and right options
    return dfs(i + jump) or dfs(i - jump)

return dfs(start)

```

18.4 Python-Specific Complexities & Implementation Considerations

18.4.1 1. Recursion Stack Limit

- In the worst case (e.g., an array where each step is 1 and elements are chained sequentially), the maximum depth of recursion is N (up to 5×10^4 according to constraints).
- Python's default recursion limit is 1000. A deep DFS search will trigger a `RecursionError`.
- **Workaround:** You can increase Python's recursion limit using `sys.setrecursionlimit(200000)`, or rewrite the traversal iteratively using an explicit stack (DFS) or queue (BFS) to avoid stack overflow.

18.4.2 2. Visited Collection Performance

- Using a standard Python `set` has an average $\mathcal{O}(1)$ insertion and lookup time, but can suffer from hashing collisions and high memory overhead (each element in a set requires about 24 – 32 bytes of metadata).
 - Alternatively, using a list of booleans like `visited = [False] * len(arr)` is faster, has no hash overhead, and requires significantly less memory (1 byte reference per element).
-

18.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We visit each index at most once. For each index, we do $\mathcal{O}(1)$ checks and operations.
Space Complexity	$\mathcal{O}(n)$	Due to the recursion call stack and the <code>visited</code> array/set, which scale linearly with the size of the input.

18.6 Common Follow-Up Questions & How to Answer

18.6.1 Q1: How can we solve this problem in $\mathcal{O}(1)$ auxiliary space?

- **Answer:** If we are allowed to mutate the input array `arr`, we can mark an index `i` as visited by changing `arr[i]` to its negative counterpart `arr[i] = -arr[i]`. Since all values are originally non-negative, any negative value indicates a visited state.

- **Python Code Implementation:**

```
def canReach(self, arr: List[int], start: int) -> bool:
    if start < 0 or start >= len(arr) or arr[start] < 0:
        return False
    if arr[start] == 0:
        return True

    # Save value, negate it in-place to mark as visited
    val = arr[start]
    arr[start] = -arr[start]

    return self.canReach(arr, start + val) or self.canReach(arr, start - val)
```

18.6.2 Q2: Write the Breadth-First Search (BFS) solution. How does BFS compare to DFS here?

- **Answer:** BFS is also $\mathcal{O}(n)$ time and space but searches level-by-level, which will find the shortest path of jumps to reach the target index.

- **C++ BFS Implementation:**

```
#include <vector>
#include <queue>

class Solution {
public:
    bool canReach(const std::vector<int>& arr, int start) {
        int n = arr.size();
        std::queue<int> q;
        std::vector<char> visited(n, 0);

        q.push(start);
        visited[start] = 1;

        while (!q.empty()) {
            int curr = q.front();
            q.pop();

            if (arr[curr] == 0) return true;
        }
    }
};
```

```

        for (int next_idx : {curr + arr[curr], curr - arr[curr]}) {
            if (next_idx >= 0 && next_idx < n && !visited[next_idx]) {
                visited[next_idx] = 1;
                q.push(next_idx);
            }
        }
    }
    return false;
}
};

```

18.7 Pro-Tip: How to Impress the Interviewer

- **Avoid `std::vector<bool>`:** In C++, explain why you used `std::vector<char>` or `std::vector<uint8_t>` instead of `std::vector<bool>`. `std::vector<bool>` is space-optimized (packed as a bitmask) and returns proxy references rather than real pointers or references. This makes it incompatible with some standard library functions and can introduce thread-safety issues.
- **Tail Recursion and Compiler Optimization:** Notice that the DFS implementation is NOT tail-recursive because it performs a logical OR operation after the recursive calls (`dfs(...) || dfs(...)`). Therefore, the compiler cannot perform tail-call optimization (TCO). Highlighting this shows deep knowledge of assembly and compiler architecture.

19 Number of Islands

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 200, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

19.1 Question Description

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

19.1.1 Examples

- **Input:**

```
grid = [
    ["1","1","1","1","0"],
    ["1","1","0","1","0"],
    ["1","1","0","0","0"],
    ["0","0","0","0","0"]
]
```

– Output: 1

- Input:

```
grid = [
    ["1","1","0","0","0"],
    ["1","1","0","0","0"],
    ["0","0","1","0","0"],
    ["0","0","0","1","1"]
]
```

– Output: 3

19.2 Detailed Solution (C++)

The standard solution for finding connected components in a 2D matrix uses **Depth-First Search (DFS)**. We traverse the grid. When we encounter a '1', we increment our island count and trigger a DFS traversal from that cell to find and "sink" (change '1' to '0') all connected land cells. This avoids allocating extra $O(m \times n)$ auxiliary memory for a visited array.

19.2.1 Standard C++ Production Code

```
#include <vector>

class Solution {
private:
    void dfs(std::vector<std::vector<char>>& grid, int r, int c, int rows, int cols) noexcept {
        // Base Case: check bounds and if cell is water
        if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != '1') {
            return;
        }

        // Sink the island in-place to save auxiliary space
        grid[r][c] = '0';

        // Explore all 4 orthogonal neighbors
        dfs(grid, r + 1, c, rows, cols);
        dfs(grid, r - 1, c, rows, cols);
        dfs(grid, r, c + 1, rows, cols);
        dfs(grid, r, c - 1, rows, cols);
    }
};
```



```

        dfs(grid, r, c - 1, rows, cols);
    }

public:
    int numIslands(std::vector<std::vector<char>>& grid) noexcept {
        if (grid.empty() || grid[0].empty()) {
            return 0;
        }

        int rows = static_cast<int>(grid.size());
        int cols = static_cast<int>(grid[0].size());
        int island_count = 0;

        for (int r = 0; r < rows; ++r) {
            for (int c = 0; c < cols; ++c) {
                if (grid[r][c] == '1') {
                    ++island_count;
                    dfs(grid, r, c, rows, cols);
                }
            }
        }

        return island_count;
    }
};

```

19.3 Detailed Solution (Python)

19.3.1 Standard Python Production Code

Below is the type-hinted Python DFS implementation. It mutates the grid in-place to optimize memory footprint.

```

from typing import List

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        """
        Calculates the number of islands in a 2D binary grid using DFS.

        Time Complexity: O(M * N)
        Space Complexity: O(M * N) in the worst case due to the recursion stack.
        """
        if not grid or not grid[0]:

```

```

    return 0

rows = len(grid)
cols = len(grid[0])
island_count = 0

def dfs(r: int, c: int) -> None:
    # Base Case: bounds check and water check
    if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != "1":
        return

    # Sink the land cell
    grid[r][c] = "0"

    # DFS search in all four directions
    dfs(r + 1, c)
    dfs(r - 1, c)
    dfs(r, c + 1)
    dfs(r, c - 1)

for r in range(rows):
    for c in range(cols):
        if grid[r][c] == "1":
            island_count += 1
            dfs(r, c)

return island_count

```

19.4 Python-Specific Complexities & Implementation Considerations

19.4.1 1. In-place Mutation vs Functional Integrity

- In LeetCode and competitive programming, mutating the input matrix in-place (e.g., changing `grid[r][c] = "0"`) is standard.
- However, in production environment APIs, mutating input arguments is considered a **bad practice** (side-effects make code harder to debug and test, and it violates functional programming standards).
- If mutation is disallowed, we must use a `visited` structure:

```
visited = [[False] * cols for _ in range(rows)]
```

This adds $\mathcal{O}(m \times n)$ auxiliary space.

19.4.2 2. Deep Recursion Limitations

- With a maximum grid size of $300 \times 300 = 9 \times 10^4$ cells, a snake-like land grid can cause a recursion depth of up to 90,000.
- Since Python's standard recursion limit is 1000, this will result in a `RecursionError`.
- **Production Alternative:** Use an iterative DFS (with a Python `list` as a stack) or BFS (with `collections.deque`).

19.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(m \times n)$	Every cell is visited at most a constant number of times (once in the outer loops, and at most once during DFS traversal).
Space Complexity	$\mathcal{O}(m \times n)$	In the worst case (all land), the call stack depth is $\mathcal{O}(m \times n)$. If grid mutation is not allowed, a <code>visited</code> structure also takes $\mathcal{O}(m \times n)$ space.

19.6 Common Follow-Up Questions & How to Answer

19.6.1 Q1: What if the grid is too large to fit into memory? (e.g. billions of rows/columns)

- **The Problem:** A standard DFS/BFS requires loading the entire grid into RAM.
- **The Solution:** Use a **Union-Find (Disjoint Set)** algorithm processing the grid **row-by-row**.
 - Maintain only the current row and the previous row in memory.
 - For each cell in the current row, check its connection to its left neighbor and the neighbor directly above it (in the previous row).
 - Union the sets and discard the row before the previous one. This reduces active memory requirements from $\mathcal{O}(m \times n)$ to $\mathcal{O}(n)$ (width of the grid).

19.6.2 Q2: Compare DFS, BFS, and Union-Find (DSU) for this problem.

- **DFS:** Easiest to write. Minimum code size. Recursive DFS has $\mathcal{O}(m \times n)$ call stack space overhead.
- **BFS:** Avoids stack overflow (heap-allocated queue). Can find shortest paths if we needed to measure distance.
- **Union-Find:** Excellent for streaming/incremental grids (where we add land cells dynamically and need to maintain the island count in real-time). DSU adds small union/find overhead (using path compression and rank optimization, complexity is almost linear: $\mathcal{O}(m \times n \cdot \alpha(m \times n))$ where α is the Inverse Ackermann function).

19.7 Pro-Tip: How to Impress the Interviewer

- **Mention Cache Locality:** In C++, traversing row-by-row (row-major order) is much faster than column-by-column because of **spatial cache locality** (elements of the same row are adjacent in memory, so loading one brings the others into CPU L1/L2 caches). The DFS should visit orthogonal neighbors, but keeping nested loops row-major is crucial.
- **Explicit Stack-Safe Iterative DFS:** Show how to implement DFS iteratively in a few lines to prove you write code resistant to stack overflow under heavy workloads.

```
// Conceptual Iterative DFS
std::vector<std::pair<int, int>> stack;
stack.push_back({r, c});
grid[r][c] = '0';
while(!stack.empty()) {
    auto [cr, cc] = stack.back();
    stack.pop_back();
    // check and push neighbors
}
```

20 Mini Parser

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / QA & Test Engineer
- **Source:** LeetCode 385, Glassdoor
- **Flashcards:** [DFS deck](#)

20.1 Question Description

Given a string `s` representing the serialization of a nested list, implement a parser to deserialize it and return the deserialized `NestedInteger`.

Each element is either an integer or a list whose elements may also be integers or other lists.

[!NOTE] The `NestedInteger` class is a predefined interface provided by the environment. It can represent either a single integer or a nested list.

20.1.1 Examples

- **Input:** `s = "324"`
 - **Output:** 324
 - **Explanation:** You should return a `NestedInteger` object which contains a single integer 324.
- **Input:** `s = "[123,[456,[789]]]"`

- **Output:** [123, [456, [789]]]
 - **Explanation:** Return a `NestedInteger` object containing a nested list with 2 elements:
 1. An integer containing value 123.
 2. A nested list containing two elements:
 - * An integer containing value 456.
 - * A nested list containing one element:
 - An integer containing value 789.
-

20.2 Detailed Solution (C++)

We can parse the serialized string using two primary approaches: 1. **Recursive DFS / Parser Approach:** We pass an index by reference. If the character is '[', we parse a list recursively. Otherwise, we parse a single integer. 2. **Iterative Stack-based Approach:** We push empty `NestedInteger` objects onto a stack when encountering '[', add integers/sub-lists to the top of the stack, and pop them when encountering ']'.

Here, the **recursive DFS approach** is highly intuitive and matches the recursive definition of the `NestedInteger` structure.

20.2.1 Standard C++ Production Code

```
#include <string>
#include <vector>
#include <cctype>

// Pre-defined NestedInteger class interface for reference
class NestedInteger {
public:
    NestedInteger();
    NestedInteger(int value);
    bool isInteger() const;
    int getInteger() const;
    void setInteger(int value);
    void add(const NestedInteger &elem);
    const std::vector<NestedInteger> &getList() const;
};

class Solution {
private:
    NestedInteger parse(const std::string& s, int& idx) noexcept {
        // Case 1: Parsing a single integer
        if (s[idx] != '[') {
            int start = idx;
            while (idx < static_cast<int>(s.length()) && (s[idx] == '-' || std::isdigit(s[idx]))) {
```

```

        idx++;
    }
    return NestedInteger(std::stoi(s.substr(start, idx - start)));
}

// Case 2: Parsing a nested list
NestedInteger ni; // Default constructor creates a list
idx++; // skip '['

while (idx < static_cast<int>(s.length()) && s[idx] != ']') {
    ni.add(parse(s, idx));
    if (idx < static_cast<int>(s.length()) && s[idx] == ',') {
        idx++; // skip ',' separating elements
    }
}
idx++; // skip ']'
return ni;
}

public:
    NestedInteger deserialize(const std::string& s) noexcept {
        if (s.empty()) {
            return NestedInteger();
        }
        int idx = 0;
        return parse(s, idx);
    }
};

```

20.3 Detailed Solution (Python)

20.3.1 Standard Python Production Code

Below is the iterative stack-based solution in Python, which is robust against recursion depth limitations and parses the structure sequentially.

```

# Pre-defined NestedInteger interface for reference
# class NestedInteger:
#     def __init__(self, value=None):
#         """If value is not specified, initializes an empty list. Otherwise with an integer."""
#     def isInteger(self) -> bool:
#     def add(self, elem: 'NestedInteger') -> None:
#     def setInteger(self, value: int) -> None:
#     def getInteger(self) -> int:

```

```
#     def getList(self) -> List['NestedInteger']:
```

class Solution:

```
    def deserialize(self, s: str) -> NestedInteger:
        """
        Deserializes a nested list serialization string into a NestedInteger.

        Time Complexity: O(N)
        Space Complexity: O(N) for stack usage
        """

        if not s:
            return NestedInteger()

        # If the string represents a single integer
        if s[0] != '[':
            return NestedInteger(int(s))

        stack = []
        current = None
        i = 0
        n = len(s)

        while i < n:
            char = s[i]
            if char == '[':
                # Start of a new NestedInteger list
                new_ni = NestedInteger()
                stack.append(new_ni)
                i += 1
            elif char == ']':
                # End of a list, pop it and attach to its parent if exists
                finished = stack.pop()
                current = finished
                if stack:
                    stack[-1].add(finished)
                i += 1
            elif char == ',':
                i += 1
            else:
                # Parse integer
                j = i
                while j < n and (s[j].isdigit() or s[j] == '-'):
                    j += 1
                val = int(s[i:j])
```

```

        num_ni = NestedInteger(val)
        if stack:
            stack[-1].add(num_ni)
        else:
            current = num_ni
        i = j

    return current

```

20.4 Python-Specific Complexities & Implementation Considerations

20.4.1 1. Pointer Traversal in Python

- In C++, passing an integer index `idx` by reference is extremely cheap. In Python, primitives are immutable, so passing `idx` by value will not update it in parent calls.
- **Workarounds:**
 1. Pass `idx` in a single-element list (e.g., `idx = [0]`) and modify `idx[0]`.
 2. Maintain `idx` as an instance variable `self.idx`.
 3. Use an iterative approach (like the Python code above) to avoid index tracking issues altogether.

20.4.2 2. Lexical Substring and Garbage Collection

- Python's slicing `s[i:j]` creates a new string copy. For large inputs (length up to 5×10^4), repeated slicing can generate significant garbage collection overhead.
 - The iterative implementation above avoids slicing until it finds the boundaries of an integer, converting it directly using `int(s[i:j])`.
-

20.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Every character is scanned at most a constant number of times. Integer parsing checks each digit once.
Space Complexity	$\mathcal{O}(d)$	Where d is the nesting depth of the list (at most $\mathcal{O}(n)$). This space is consumed by the call stack (recursive) or the explicit stack (iterative).

20.6 Common Follow-Up Questions & How to Answer

20.6.1 Q1: How would you validate if the input serialization string is well-formed?

- **Answer:** To validate the input format:
 1. Track bracket balance (number of '[' must match ']').
 2. Ensure commas are only placed between items (not directly after '[' or before ']').
 3. Ensure no trailing characters are present after the top-level list/integer closes.
 4. Integers must only contain optional negative sign and valid digits.

20.6.2 Q2: Can you write a generator to yield flat integers from the NestedInteger?

- **Answer:** In Python, we can write a clean recursive generator using `yield from`:

```
def flatten(ni: NestedInteger):
    if ni.isInteger():
        yield ni.getInteger()
    else:
        for sub_ni in ni.getList():
            yield from flatten(sub_ni)
```

20.7 Pro-Tip: How to Impress the Interviewer

- **Explain C++ `std::stoi` Limits:** Mention that `std::stoi` can throw an out-of-range exception if the input number exceeds the limits of standard integers. According to constraints, LeetCode limits numbers to $[-10^6, 10^6]$, which comfortably fits within a 32-bit signed integer. However, pointing this out shows robustness in production engineering.
- **Use Stack/Recursion Equivalence:** Point out that the recursive approach naturally maps to a depth-first search of a rose tree structure, whereas the iterative stack solution behaves like a pushdown automaton parsing a context-free grammar ($S \rightarrow [L] \mid \text{num}$, $L \rightarrow S, L \mid S$).

21 Longest Absolute File Path

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer
 - **Source:** LeetCode 388, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

21.1 Question Description

Suppose we have a file system that stores both files and directories. A system can be represented textually where directories and files are separated by newline characters `\n`, and their depth in the file system is indicated by tab characters `\t`.

Given a string input representing the file system, return the length of the **longest absolute path** to a **file** in the file system. If there is no file in the system, return **0**.

Note that a file name must contain at least one **.** character separating its name and extension (e.g., `file.ext` is a file, but `subdir` is a directory).

21.1.1 Examples

- **Input:** `input = "dir\n\tsubdir1\n\tsubdir2\n\t\tfile.ext"`
 - **Output:** 20
 - **Explanation:** We have only one file, and its absolute path is `"dir/subdir2/file.ext"`, which has a length of 20 (three parts of length 3, 7, 8, plus two / separators).
 - **Input:** `input = "dir\n\tsubdir1\n\t\tfile1.ext\n\t\t\tsubsubdir1\n\tsubdir2\n\t\t\tsubsubdir2\n\t\t\t\tfile2.ext"`
 - **Output:** 32
 - **Explanation:** We have two files:
 - * `"dir/subdir1/file1.ext"` (length 21)
 - * `"dir/subdir2/subsubdir2/file2.ext"` (length 32) We return 32, the longest path.
 - **Input:** `input = "a"`
 - **Output:** 0
 - **Explanation:** The input only contains a single directory `"a"`. Since there are no files, we return 0.
-

21.2 Detailed Solution (C++)

The filesystem layout is structured like a tree. A DFS-like traversal can be done by parsing the string sequentially. We can maintain a stack `stk` where `stk[depth]` holds the absolute path length of the directory at depth `depth`. To maximize performance and avoid unnecessary heap allocations, we parse the input string in-place in a single pass without using string splittings or regexes.

21.2.1 Standard C++ Production Code

```
#include <string>
#include <vector>
#include <algorithm>

class Solution {
public:
    int lengthLongestPath(const std::string& input) noexcept {
        // stk[depth] stores the path length of the directory chain up to that depth.
        // Index 0 acts as a base length of 0.
        std::vector<int> stk = {0};
        int max_len = 0;
        int i = 0;
        const int len = static_cast<int>(input.size());
```

```
while (i < len) {
    // 1. Calculate depth by counting consecutive tabs
    int depth = 0;
    while (i < len && input[i] == '\t') {
        depth++;
        i++;
    }

    // 2. Parse name of directory/file and check if it is a file
    int name_start = i;
    bool is_file = false;
    while (i < len && input[i] != '\n') {
        if (input[i] == '.') {
            is_file = true;
        }
        i++;
    }
    int name_len = i - name_start;

    // Skip the newline character for the next iteration
    if (i < len && input[i] == '\n') {
        i++;
    }

    // 3. Maintain stack to match current depth.
    // stk.size() must be kept at depth + 1.
    while (static_cast<int>(stk.size()) > depth + 1) {
        stk.pop_back();
    }

    // 4. Update max length or push current path length to stack
    if (is_file) {
        max_len = std::max(max_len, stk.back() + name_len);
    } else {
        // If it is a directory, append length + 1 (for the '/' separator)
        stk.push_back(stk.back() + name_len + 1);
    }
}

return max_len;
}

};
```

21.3 Detailed Solution (Python)

21.3.1 Standard Python Production Code

Below is the Python implementation. Python's built-in string methods like `.split('\n')` and `.rstrip('\t')` allow for highly expressive and concise code while remaining linear in time.

class Solution:

```
def lengthLongestPath(self, input: str) -> int:
    """
    Computes the length of the longest absolute path to a file.

    Time Complexity: O(N) where N is the length of the input string.
    Space Complexity: O(D) where D is the maximum depth of the directory structure.
    """
    # stack[depth] stores the accumulated path length up to that depth
    stack = [0]
    max_len = 0

    for part in input.split("\n"):
        # The count of '\t' indicates the depth of the current item (0-indexed)
        depth = part.count("\t")
        name = part.rstrip("\t")

        # Pop elements from stack until the stack matches the parent depth
        while len(stack) > depth + 1:
            stack.pop()

        # If the item contains a '.', it is a file
        if "." in name:
            max_len = max(max_len, stack[-1] + len(name))
        else:
            # If it is a directory, store path length + 1 for the '/' separator
            stack.append(stack[-1] + len(name) + 1)

    return max_len
```

21.4 Python-Specific Complexities & Implementation Considerations

21.4.1 1. `split('\n')` Memory Overhead

- Calling `input.split('\n')` in Python creates a list of sub-strings. For large strings ($N \leq 10^4$), this creates a list of strings on the heap.
- While acceptable within typical LeetCode limits, it can cause memory spikes under extreme datasets.
- **Low-memory Alternative:** Use a custom generator or find index loops (`input.find('\n')`) to yield

parts of the string without full materialization of the list.

21.4.2 2. Tab Counting & Left Stripping

- `part.count('\t')` scans the entire line. However, tabs can only appear at the very start of a line in a valid serialization.
- It is more robust to only strip tabs from the left (`part.lstrip('\t')`) and calculate depth as `len(part) - len(name)`.

21.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the string of length n exactly once. Each character is checked for delimiters (<code>\t</code> , <code>\n</code> , <code>.</code>) a constant number of times.
Space Complexity	$\mathcal{O}(d)$	The maximum depth of the directory tree. In the worst case (deeply nested directory hierarchy), $d \leq n$.

21.6 Common Follow-Up Questions & How to Answer

21.6.1 Q1: What if multiple tabs or backslashes represent tabs?

- **Answer:** In some systems, tabs are represented as spaces (e.g. four spaces or two spaces). We can adjust the depth detection by counting prefix spaces instead of `\t` and dividing by the tab-to-space factor.

21.6.2 Q2: How can we return the actual path string instead of just its length?

- **Answer:** Instead of storing lengths in our stack, we can store the actual directory/file name strings.
- **Python Modification:**

```
stack = []
# when processing a file:
full_path = "/".join(stack) + "/" + name
```

21.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Garbage Collection Stalls:** Mention that the C++ solution uses in-place parsing without allocating sub-strings on the heap. Standard libraries like `std::string::substr` or splitting methods

copy parts of the string into new memory regions. An in-place pointer scan is optimal for high-throughput, low-latency microservices.

- **Discuss Cache Locality of Vector:** Highlight why `std::vector` is ideal for a stack compared to `std::stack` (which wraps `std::deque` by default). `std::deque` allocates elements in chunks (typically 512 bytes), whereas `std::vector` is completely contiguous in memory, maximizing cache line utilization.

22 Delete Node in a BST

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / QA & Test Engineer
 - **Source:** LeetCode 450, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

22.1 Question Description

Given the root of a Binary Search Tree (BST) and a **key**, delete the node with the given **key** in the BST. Return the root node reference (possibly updated) of the BST.

Basically, the deletion can be divided into two stages: 1. Search for a node to remove. 2. If the node is found, delete the node.

22.1.1 Examples

- **Input:** `root = [5,3,6,2,4,null,7]`, `key = 3`
 - **Output:** `[5,4,6,2,null,null,7]` (or `[5,2,6,null,4,null,7]`)
 - **Explanation:** The node to be deleted is 3. Valid BSTs after deletion are:

5
 / \
 4 6
 / \
 2 7

or

5
 / \
 2 6
 \ \
 4 7
 - **Input:** `root = [5,3,6,2,4,null,7]`, `key = 0`
 - **Output:** `[5,3,6,2,4,null,7]`
 - **Explanation:** The key 0 is not present in the BST, so the tree is returned unchanged.
 - **Input:** `root = []`, `key = 0`
 - **Output:** `[]`
-

22.2 Detailed Solution (C++)

To delete a node from a Binary Search Tree, we recursively locate the node: * If `key < root->val`, search in the left subtree. * If `key > root->val`, search in the right subtree. * If `key == root->val`, we handle

three cases: 1. **No children**: Return `nullptr` and delete the node. 2. **One child**: Return the non-null child and delete the current node to free heap memory. 3. **Two children**: Find the **inorder successor** (the minimum node in the right subtree), copy its value to the current node, and recursively delete the successor from the right subtree.

22.2.1 Standard C++ Production Code

// Definition for a binary tree node.

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};
```

```
class Solution {
public:
    TreeNode* deleteNode(TreeNode* root, int key) noexcept {
        if (!root) {
            return nullptr;
        }

        // Locate the target node
        if (key < root->val) {
            root->left = deleteNode(root->left, key);
        } else if (key > root->val) {
            root->right = deleteNode(root->right, key);
        } else {
            // Target node found

            // Case 1 & 2: Node has 0 or 1 child
            if (!root->left) {
                TreeNode* temp = root->right;
                delete root; // Avoid memory leaks on the heap
                return temp;
            }
            if (!root->right) {
                TreeNode* temp = root->left;
                delete root; // Avoid memory leaks on the heap
                return temp;
            }
        }
    }
};
```

```

    // Case 3: Node has 2 children
    // Find the inorder successor (leftmost leaf in the right subtree)
    TreeNode* min_node = root->right;
    while (min_node->left) {
        min_node = min_node->left;
    }

    // Copy the successor's value to current node
    root->val = min_node->val;

    // Delete the successor recursively
    root->right = deleteNode(root->right, min_node->val);
}
return root;
}
};

```

22.3 Detailed Solution (Python)

22.3.1 Standard Python Production Code

Below is the standard recursive Python implementation. Python does not require manual memory deallocation since its garbage collector uses reference counting.

```

from typing import Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def deleteNode(self, root: Optional[TreeNode], key: int) -> Optional[TreeNode]:
        """
        Deletes a node with the given key from a BST and returns the updated root.

        Time Complexity: O(H) where H is the height of the tree.
        Space Complexity: O(H) call stack space.
        """
        if not root:
            return None

```



```

# 1. Search for the node
if key < root.val:
    root.left = self.deleteNode(root.left, key)
elif key > root.val:
    root.right = self.deleteNode(root.right, key)
else:
    # 2. Node found: handle deletions

    # Case 1: Left child is missing (handles 0 or 1 child)
    if not root.left:
        return root.right

    # Case 2: Right child is missing (handles 1 child)
    if not root.right:
        return root.left

    # Case 3: Two children
    # Find the inorder successor (leftmost node of right subtree)
    min_node = root.right
    while min_node.left:
        min_node = min_node.left

    # Swap values
    root.val = min_node.val

    # Delete successor node
    root.right = self.deleteNode(root.right, min_node.val)

return root

```

22.4 Python-Specific Complexities & Implementation Considerations

22.4.1 1. Recursion Stack Limit

- For an unbalanced, skewed BST (essentially a linked list), the height H can equal N . The call stack can grow up to 10^4 frames (which exceeds Python's default stack limit of 1000).
- **Workaround:** An iterative deletion can be written to perform searching and pointer updating in $\mathcal{O}(1)$ space.

22.4.2 2. TreeNode Identity vs Value Swaps

- In Python, swapping values (`root.val = min_node.val`) is simple. However, if other objects hold references to the specific `TreeNode` instances, swapping values can cause stale references or data consistency issues.

- In production, structural modifications (rewiring the parent pointers to the successor node object) are preferred.

22.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(h)$	Where h is the height of the tree. For balanced trees, $h = \log n$. For skewed trees, $h = n$.
Space Complexity	$\mathcal{O}(h)$	Call stack depth is proportional to the tree height.

22.6 Common Follow-Up Questions & How to Answer

22.6.1 Q1: Can we delete a node without copying/swapping values? (Structural Deletion)

- **Answer:** Yes, we can delete the node by altering parent and child pointers. If the node has two children, we can hook the left subtree of the deleted node to the leftmost leaf of its right subtree, and then return the right subtree.
- **C++ Structural Example:**

```
TreeNode* deleteNodeStructural(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) {
        root->left = deleteNodeStructural(root->left, key);
    } else if (key > root->val) {
        root->right = deleteNodeStructural(root->right, key);
    } else {
        if (!root->left) { TreeNode* r = root->right; delete root; return r; }
        if (!root->right) { TreeNode* l = root->left; delete root; return l; }

        TreeNode* successor = root->right;
        while (successor->left) {
            successor = successor->left;
        }
        successor->left = root->left; // hook left child
        TreeNode* new_root = root->right;
        delete root;
        return new_root;
    }
    return root;
}
```

22.6.2 Q2: How can we implement this iteratively in $\mathcal{O}(1)$ space?

- **Answer:** We can keep track of the `parent` and `curr` node. Once the node is found, we swap it with the successor and delete the successor by updating parent's pointers, all within a single loop without recursion.
-

22.7 Pro-Tip: How to Impress the Interviewer

- **Highlight C++ Memory Leaks:** Always point out that in languages without garbage collection (C++), simply returning a child pointer like `return root->right;` without calling `delete root;` creates a memory leak. Pointing this out and cleaning up memory dynamically demonstrates low-level systems discipline.
- **Discuss Successor vs Predecessor:** Mention that we can use either the inorder successor (minimum of the right subtree) or the inorder predecessor (maximum of the left subtree). Explain that choosing randomly between successor and predecessor can help keep the BST more balanced in systems with heavy write/delete workloads.

23 Most Frequent Subtree Sum

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 508, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

23.1 Question Description

Given the root of a binary tree, return the **most frequent subtree sum**. If there is a tie, return all the values with the highest frequency in any order.

The **subtree sum** of a node is defined as the sum of all the node values formed by the subtree rooted at that node (including the node itself).

23.1.1 Examples

- **Input:** `root = [5,2,-3]`
 - **Output:** `[2,-3,4]`
 - **Explanation:**
 - * Subtree sum of node 2 is 2.
 - * Subtree sum of node -3 is -3.
 - * Subtree sum of root node 5 is $5 + 2 + (-3) = 4$. All three sums occur exactly once, so we return all of them.
- **Input:** `root = [5,2,-5]`
 - **Output:** `[2]`

– **Explanation:**

- * Subtree sum of node 2 is 2.
 - * Subtree sum of node -5 is -5.
 - * Subtree sum of root node 5 is $5 + 2 + (-5) = 2$. The sum 2 occurs twice, while -5 occurs once. Thus, the most frequent subtree sum is 2.
-

23.2 Detailed Solution (C++)

We can compute the subtree sums in a **bottom-up (post-order)** manner using Depth-First Search (DFS). 1. For each node, we recursively compute the sum of its left and right subtrees. 2. The subtree sum for the current node is `node->val + left_sum + right_sum`. 3. We record this sum in a hash map `std::unordered_map<int, int>` to track frequencies. 4. To optimize performance, we update the `max_freq` in a single pass while traversing the tree, avoiding a second iteration over the tree nodes.

23.2.1 Standard C++ Production Code

```
#include <vector>
#include <unordered_map>
#include <algorithm>

// Definition for a binary tree node.
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};

class Solution {
private:
    int getSubtreeSum(TreeNode* node, std::unordered_map<int, int>& freq, int& max_freq) noexcept {
        if (!node) {
            return 0;
        }

        // Post-order traversal (bottom-up sum calculation)
        int left_sum = getSubtreeSum(node->left, freq, max_freq);
        int right_sum = getSubtreeSum(node->right, freq, max_freq);

        // Prevent overflow by performing standard arithmetic
        // (assuming inputs are bounded as per constraints)
        int sum = left_sum + right_sum + node->val;
        freq[sum]++;
        if (freq[sum] > max_freq) max_freq = freq[sum];
        return sum;
    }
};
```

```

        int total_sum = node->val + left_sum + right_sum;

        freq[total_sum]++;
        max_freq = std::max(max_freq, freq[total_sum]);

        return total_sum;
    }

public:
    std::vector<int> findFrequentTreeSum(TreeNode* root) noexcept {
        std::unordered_map<int, int> freq;
        int max_freq = 0;

        getSubtreeSum(root, freq, max_freq);

        std::vector<int> result;
        for (const auto& [sum, count] : freq) {
            if (count == max_freq) {
                result.push_back(sum);
            }
        }
        return result;
    }
};

```

23.3 Detailed Solution (Python)

23.3.1 Standard Python Production Code

Below is the Python implementation using a recursive nested function and defaultdict.

```

from typing import List, Optional
from collections import defaultdict

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findFrequentTreeSum(self, root: Optional[TreeNode]) -> List[int]:
        """

```

Finds the most frequent subtree sums in a binary tree.

Time Complexity: $O(N)$

Space Complexity: $O(N)$ for recursion stack and map storage.

```

"""
if not root:
    return []

freq = defaultdict(int)
max_freq = 0

def get_subtree_sum(node: Optional[TreeNode]) -> int:
    nonlocal max_freq
    if not node:
        return 0

    # Post-order DFS
    left_sum = get_subtree_sum(node.left)
    right_sum = get_subtree_sum(node.right)
    total_sum = node.val + left_sum + right_sum

    # Update frequencies
    freq[total_sum] += 1
    max_freq = max(max_freq, freq[total_sum])

    return total_sum

get_subtree_sum(root)

# Gather all sums matching the maximum frequency
return [s for s, count in freq.items() if count == max_freq]

```

23.4 Python-Specific Complexities & Implementation Considerations

23.4.1 1. nonlocal Keyword Usage

- In Python, variables defined in outer functions cannot be reassigned in inner helper functions unless declared `nonlocal`.
- Without `nonlocal max_freq`, Python will raise an `UnboundLocalError` when attempting to write `max_freq = max(...)`.
- **Alternative:** Store state in a mutable type like a list/dictionary (e.g. `max_freq = [0]`) or an instance variable (`self.max_freq`).

23.4.2 2. Hash Map Keys in Python

- Python's dictionaries are implemented as dynamically resized hash tables. Storing integers as keys is highly optimized, but inserting $\mathcal{O}(n)$ keys into a `defaultdict` can trigger dynamic resizing and rehashing. This results in slight amortized overhead but remains $\mathcal{O}(1)$ average per operation.

23.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We visit every node in the binary tree exactly once. Map lookups and updates are $\mathcal{O}(1)$ on average.
Space Complexity	$\mathcal{O}(n)$	In the worst case (skewed tree), the recursion stack is $\mathcal{O}(n)$. The hash map also stores up to n unique sum values.

23.6 Common Follow-Up Questions & How to Answer

23.6.1 Q1: What if node values can be extremely large, leading to integer overflow?

- **Answer:** In C++, we should use 64-bit integers (`int64_t` or `long long`) for computing the subtree sums and as keys for the hash map to prevent overflow. Python handles arbitrary-precision integers automatically, so no code adjustments are needed.

23.6.2 Q2: How can we implement this iteratively to avoid stack overflow?

- **Answer:** We can perform an iterative post-order traversal using a stack.
 1. Keep a map of node to its computed subtree sum.
 2. Visit nodes in post-order (e.g., using two stacks, or tracking the last visited node).
 3. Pop a node when both its children's sums are computed, aggregate the sum, store it in the map, and update frequencies.

23.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Two-Pass Frequency Scanning:** Many candidates traverse the tree to build the frequencies, then loop through the map to find the maximum frequency, and finally loop again to gather results. Doing this in **one pass** (updating `max_freq` dynamically during DFS) shows a keen eye for loop-level optimization and minimizes constant-factor overhead.
- **Mention Cache Friendliness of `std::unordered_map`:** Note that although `std::unordered_map` is $\mathcal{O}(1)$ average time, it uses bucket chains which can result in cache misses because node elements are

scattered in heap memory. Discussing the cache performance of node structures versus contiguous arrays highlights hardware-aware engineering.

24 Convert BST to Greater Tree

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 538, LeetCode 1038, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

24.1 Question Description

Given the root of a Binary Search Tree (BST), convert it to a **Greater Tree** such that every key of the original BST is changed to the original key plus the sum of all keys greater than the original key in the BST.

Recall the properties of a Binary Search Tree: * The left subtree of a node contains only nodes with keys **less than** the node's key. * The right subtree of a node contains only nodes with keys **greater than** the node's key. * Both the left and right subtrees must also be binary search trees.

24.1.1 Examples

- **Input:** root = [4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]
 - **Output:** [30,36,21,36,35,26,15,null,null,null,33,null,null,null,8]
 - **Explanation:**
 - * Node 8 has no nodes greater than it: $8 + 0 = 8$.
 - * Node 7 has node 8 greater than it: $7 + 8 = 15$.
 - * Node 6 has nodes 7, 8 greater than it: $6 + 15 = 21$.
 - * Node 5 has nodes 6, 7, 8 greater than it: $5 + 21 = 26$.
 - * ... and so on.
 - **Input:** root = [0,null,1]
 - **Output:** [1,null,1]
-

24.2 Detailed Solution (C++)

In a Binary Search Tree, an **In-order traversal** (Left → Root → Right) visits elements in ascending order. If we reverse the traversal order to Right → Root → Left, we will visit elements in **descending order**. As we traverse, we can keep a running sum `sum` of all nodes visited so far. For each node, we add its value to `sum` and update its value to this new `sum`.

[!IMPORTANT] Avoid using global/static variables for the running sum in C++. Global variables are not thread-safe and fail in parallelized test environments. Instead, pass the running sum by reference.

24.2.1 Standard C++ Production Code

// Definition for a binary tree node.

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};
```

```
class Solution {
```

```
private:
```

```
    void reverseInorder(TreeNode* node, int& sum) noexcept {
        if (!node) {
            return;
        }
```

// 1. Visit the right child (larger values)

```
reverseInorder(node->right, sum);
```

// 2. Process current node

```
sum += node->val;
```

```
node->val = sum;
```

// 3. Visit the left child (smaller values)

```
reverseInorder(node->left, sum);
```

```
}
```

```
public:
```

```
    TreeNode* convertBST(TreeNode* root) noexcept {
```

```
        int sum = 0;
```

```
        reverseInorder(root, sum);
```

```
        return root;
```

```
    }
```

```
};
```

24.3 Detailed Solution (Python)

24.3.1 Standard Python Production Code

Below is the recursive Python solution using a nested helper function with the `nonlocal` keyword to manage the state of the running sum.

```

from typing import Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def convertBST(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        """
        Converts BST to Greater Sum Tree.

        Time Complexity: O(N)
        Space Complexity: O(H) where H is the height of the tree.
        """
        running_sum = 0

        def reverse_inorder(node: Optional[TreeNode]) -> None:
            nonlocal running_sum
            if not node:
                return

            # 1. Visit right child
            reverse_inorder(node.right)

            # 2. Update current node val
            running_sum += node.val
            node.val = running_sum

            # 3. Visit left child
            reverse_inorder(node.left)

        reverse_inorder(root)
        return root

```

24.4 Python-Specific Complexities & Implementation Considerations

24.4.1 1. The `nonlocal` Keyword

- In Python, helper nested functions can read variables from their outer scope but cannot modify them.
- Declaring `nonlocal running_sum` tells Python to update the binding in the enclosing scope instead of

creating a new local variable `running_sum`.

24.4.2 2. Space Limit and Deep Recursion

- If the BST is highly unbalanced (skewed like a linked list), the recursion depth reaches N .
- If $N > 1000$, Python will raise a `RecursionError`. Using an iterative approach (using a stack) solves this concern.

24.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Every node is visited exactly once.
Space Complexity	$\mathcal{O}(h)$	Where h is the height of the tree ($\mathcal{O}(\log n)$ for a balanced tree, $\mathcal{O}(n)$ for a skewed tree) representing the stack depth.

24.6 Common Follow-Up Questions & How to Answer

24.6.1 Q1: How can we implement this iteratively?

- **Answer:** We can use an explicit stack to perform the reverse in-order traversal iteratively.
- **Python Iterative Implementation:**

```
def convertBST(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
    running_sum = 0
    stack = []
    curr = root
    while stack or curr:
        while curr:
            stack.append(curr)
            curr = curr.right
        curr = stack.pop()
        running_sum += curr.val
        curr.val = running_sum
        curr = curr.left
    return root
```

24.6.2 Q2: Can we achieve $\mathcal{O}(1)$ auxiliary space complexity?

- **Answer:** Yes, by using **Reverse Morris In-order Traversal**. This algorithm temporarily modifies the tree structure by creating threads (pointers) from predecessor nodes back to the current node,

allowing traversal without stacks or recursion.

- **C++ Morris Traversal Implementation:**

```
TreeNode* convertBST(TreeNode* root) {
    int sum = 0;
    TreeNode* curr = root;
    while (curr) {
        if (!curr->right) {
            sum += curr->val;
            curr->val = sum;
            curr = curr->left;
        } else {
            TreeNode* successor = curr->right;
            while (successor->left && successor->left != curr) {
                successor = successor->left;
            }
            if (!successor->left) {
                successor->left = curr;
                curr = curr->right;
            } else {
                successor->left = nullptr;
                sum += curr->val;
                curr->val = sum;
                curr = curr->left;
            }
        }
    }
    return root;
}
```

24.7 Pro-Tip: How to Impress the Interviewer

- **Explain Morris Traversal Space/Time Trade-off:** Explain that while Morris traversal reduces the auxiliary space to $\mathcal{O}(1)$, it temporarily mutates the tree structure by creating cycle pointers which are later restored. This might not be suitable for multi-threaded environments where readers traverse the tree concurrently. Mentioning this concurrency hazard shows excellent production-level awareness.
- **Design for Reentrancy:** Mention why passing the state parameter by reference is important for **reentrant** functions. If the library gets used in an event loop or a multi-threaded service, global variables would create race conditions.

25 Array Nesting

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer
 - **Source:** LeetCode 565, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

25.1 Question Description

You are given an integer array `nums` of length `n` where `nums` is a **permutation** of the numbers in the range `[0, n - 1]`.

You should build a set $s[k] = \{\text{nums}[k], \text{nums}[\text{nums}[k]], \text{nums}[\text{nums}[\text{nums}[k]]], \dots\}$ subjected to the following rule: * The first element in $s[k]$ starts with `nums[k]`. * The next element in $s[k]$ is `nums[nums[k]]`, and then `nums[nums[nums[k]]]`, etc. * Stop adding right before a duplicate element occurs in $s[k]$.

Return the **longest length** of a set $s[k]$.

25.1.1 Examples

- **Input:** `nums = [5,4,0,3,1,6,2]`
 - **Output:** 4
 - **Explanation:**
 - * `nums[0] = 5, nums[1] = 4, nums[2] = 0, nums[3] = 3, nums[4] = 1, nums[5] = 6, nums[6] = 2.`
 - * One of the longest sets $s[k]$ is: $s[0] = \{\text{nums}[0], \text{nums}[5], \text{nums}[6], \text{nums}[2]\} = \{5, 6, 2, 0\}$ (length 4).
 - **Input:** `nums = [0,1,2]`
 - **Output:** 1
-

25.2 Detailed Solution (C++)

This problem can be modeled as finding the size of the largest **strongly connected component** (specifically, **disjoint cycles**) in a directed graph where each node i has exactly one outgoing edge to `nums[i]`. Because `nums` is a permutation of range `[0, n-1]`, every node has an **in-degree of 1** and an **out-degree of 1**. This mathematical property guarantees that: 1. Every element belongs to exactly one closed loop (cycle). 2. The cycles are completely disjoint.

We can traverse each cycle, mark its elements as visited, and measure its length. Once an element is visited, we can skip it in future loops because any start node in the same cycle would produce the same loop.

25.2.1 Standard C++ Production Code

To optimize memory to $\mathcal{O}(1)$ auxiliary space, we mutate `nums` in-place by setting visited elements to `-1` (an invalid permutation value).

```
#include <vector>
#include <algorithm>

class Solution {
public:
    int arrayNesting(std::vector<int>& nums) noexcept {
        int max_len = 0;
        const int n = static_cast<int>(nums.size());

        for (int i = 0; i < n; ++i) {
            // If the element has been visited, skip it
            if (nums[i] == -1) {
                continue;
            }

            int count = 0;
            int curr = i;

            // Traverse the cycle in-place
            while (nums[curr] != -1) {
                int next_idx = nums[curr];
                nums[curr] = -1; // Mark current index as visited
                curr = next_idx;
                count++;
            }

            max_len = std::max(max_len, count);
        }

        return max_len;
    }
};
```

25.3 Detailed Solution (Python)

25.3.1 Standard Python Production Code

Below is the Python implementation using the same in-place modification strategy.

```

from typing import List

class Solution:
    def arrayNesting(self, nums: List[int]) -> int:
        """
        Finds the longest length of nesting sets from a permutation array.

        Time Complexity: O(N)
        Space Complexity: O(1) in-place modification
        """
        max_len = 0
        n = len(nums)

        for i in range(n):
            # Skip visited indices
            if nums[i] == -1:
                continue

            count = 0
            curr = i

            # Follow the permutation cycle
            while nums[curr] != -1:
                next_idx = nums[curr]
                nums[curr] = -1 # Mark as visited
                curr = next_idx
                count += 1

            max_len = max(max_len, count)

        return max_len

```

25.4 Python-Specific Complexities & Implementation Considerations

25.4.1 1. In-place Mutation Warnings

- Negating values (e.g. `nums[i] = -nums[i]`) does not work here because `0` is a valid number in the permutation and `0` cannot be distinguished from `-0`.
 - Using `-1` or a sentinel value like `len(nums)` is required.
 - If in-place mutation of the input parameters is forbidden, you must use a boolean array `visited = [False] * len(nums)`. This increases auxiliary space complexity to $\mathcal{O}(n)$.
-

25.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Each element is visited at most twice (once to mark it as visited, and once in the outer loop check).
Space Complexity	$\mathcal{O}(1)$	Modifying the input array in-place avoids allocating any auxiliary arrays or structures.

25.6 Common Follow-Up Questions & How to Answer

25.6.1 Q1: What if elements can point out-of-bounds or don't form a permutation?

- **Answer:** If `nums` is not a permutation, elements can have in-degrees other than 1. This means cycles can lead to "tail" nodes (like the shape of a ρ or σ), and we could get stuck in infinite loops.
 - To solve this, we must use a cycle detection algorithm (like **Floyd's Tortoise and Hare**) or maintain an explicit state (e.g. `visited` set or three-color DFS: unvisited, visiting, visited) to identify loops.

25.6.2 Q2: Can we parallelize this cycle detection?

- **Answer:** Yes. Since all cycles are disjoint, they can be processed independently.
 - We can partition the index search space across multiple threads.
 - To make it thread-safe without locks on a shared `visited` table, we can use atomic flags (`std::atomic<bool>`) or disjoint-set structures.

25.7 Pro-Tip: How to Impress the Interviewer

- **Explain Permutation Group Theory:** Mention that this problem is a direct representation of the **cycle decomposition theorem** in symmetric group theory (S_n). Every permutation can be uniquely written as a product of disjoint cycles. Showing this mathematical intuition shows a deep CS background.
- **Discuss CPU Cache Locality:** Note that while the outer loop is sequential, jumping through `nums[curr]` creates arbitrary memory hops which can cause **cache misses** (non-sequential memory access). Explain that while the algorithm is $\mathcal{O}(n)$ theoretically, cache misses make it slower than a linear scan of memory.

26 Subtree of Another Tree

- **Category:** Coding

- **Difficulty:** Easy
 - **Target Role:** Software Engineer / QA & Test Engineer
 - **Source:** LeetCode 572, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

26.1 Question Description

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

26.1.1 Examples

- **Input:** `root = [3,4,5,1,2]`, `subRoot = [4,1,2]`
 - **Output:** `true`
 - **Explanation:** The subtree rooted at node 4 in `root` is identical to `subRoot` in both structure and values.
 - **Input:** `root = [3,4,5,1,2,null,null,null,null,0]`, `subRoot = [4,1,2]`
 - **Output:** `false`
 - **Explanation:** The subtree rooted at node 4 in `root` contains a node `0` as a child of 2, which does not exist in `subRoot`. Hence, they are not identical.
-

26.2 Detailed Solution (C++)

The standard solution uses double recursion: 1. `isSubtree(root, subRoot)`: Checks if the current subtree starting at `root` is matching `subRoot`, or recursively checks if `subRoot` is a subtree of `root->left` or `root->right`. 2. `isSame(a, b)`: Helper function to determine if two binary trees are identical by checking if their values are equal and recursively matching their left and right subtrees.

26.2.1 Standard C++ Production Code

// Definition for a binary tree node.

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};
```

```
class Solution {
```

private:

```
bool isSame(TreeNode* a, TreeNode* b) noexcept {
    if (!a && !b) {
        return true;
    }
    if (!a || !b) {
        return false;
    }
    return (a->val == b->val) &&
        isSame(a->left, b->left) &&
        isSame(a->right, b->right);
}
```

public:

```
bool isSubtree(TreeNode* root, TreeNode* subRoot) noexcept {
    if (!subRoot) {
        return true; // An empty tree is always a subtree
    }
    if (!root) {
        return false; // A non-empty subRoot cannot be a subtree of an empty root
    }

    // Check if the current tree matches or recurse into child nodes
    if (isSame(root, subRoot)) {
        return true;
    }

    return isSubtree(root->left, subRoot) || isSubtree(root->right, subRoot);
}

};
```

26.3 Detailed Solution (Python)

26.3.1 Standard Python Production Code

Below is the recursive Python implementation using helper functions.

```
from typing import Optional

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
```

```
#         self.right = right
```

```
class Solution:
```

```
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode]) -> bool:
```

```
        """
```

```
        Determines if subRoot is a subtree of root.
```

```
        Time Complexity:  $O(N * M)$  in the worst case, where  $N$  is the size of root and  $M$  of subRoot.
```

```
        Space Complexity:  $O(H)$  recursion stack space where  $H$  is the height of root.
```

```
        """
```

```
        if not subRoot:
```

```
            return True
```

```
        if not root:
```

```
            return False
```

```
        if self.isSame(root, subRoot):
```

```
            return True
```

```
        return self.isSubtree(root.left, subRoot) or self.isSubtree(root.right, subRoot)
```

```
    def isSame(self, a: Optional[TreeNode], b: Optional[TreeNode]) -> bool:
```

```
        if not a and not b:
```

```
            return True
```

```
        if not a or not b:
```

```
            return False
```

```
        return (a.val == b.val) and self.isSame(a.left, b.left) and self.isSame(a.right, b.right)
```

26.4 Python-Specific Complexities & Implementation Considerations

26.4.1 1. Object Identity vs Value Equivalence

- In Python, comparing references (`a is b`) is $\mathcal{O}(1)$ and checks if both variables point to the exact same memory address.
- However, tree nodes might have identical values and structure but different memory addresses. Always compare values (`a.val == b.val`) and structures recursively instead of relying on identity operators.

26.4.2 2. Tail Recursion Limitation

- Python does not perform tail-call optimization (TCO). A deeply skewed binary tree of depth 2000 will crash Python due to `RecursionError` (limit 1000).
- If working with extremely large skewed trees, rewrite the outer tree traversal iteratively (using BFS or iterative DFS with a queue/stack) while keeping `isSame` recursive.

26.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \times m)$	In the worst case, we run <code>isSame</code> (which takes $\mathcal{O}(m)$) for every node in the primary tree.
Space Complexity	$\mathcal{O}(h)$	Where h is the height of the primary tree. In the worst case (skewed tree), $h = n$.

26.6 Common Follow-Up Questions & How to Answer

26.6.1 Q1: How can we solve this problem in $\mathcal{O}(n + m)$ time complexity?

- **Answer:** We can serialize both trees into string representations (using a pre-order traversal) and run the **Knuth-Morris-Pratt (KMP)** algorithm to check if the serialized string of `subRoot` is a substring of the serialized string of `root`.
- **Serialization Rules:**
 1. Serialize empty children as a special character (e.g. `"#"` or `"null"`).
 2. Use clear separators (like `","` or `"^"`) before and after each node's value. This prevents false positive matches like value `"2"` matching inside `"12"`.
- **Serialized Preorder Representation:**
 - `root: ",3,4,1,#,#,2,#,#,5,#,#"`
 - `subRoot: ",4,1,#,#,2,#,#"`

26.6.2 Q2: What is the Tree Hashing / Merkle Tree approach?

- **Answer:** We can compute a unique hash for each node's subtree in a bottom-up traversal:

$$\text{hash}(\text{node}) = \text{hash_function}(\text{node.val}, \text{hash}(\text{node.left}), \text{hash}(\text{node.right}))$$

- Store the hash values of all subtrees in a hash set.
- Compute the hash of `subRoot`.
- Check if `subRoot`'s hash exists in our set in $\mathcal{O}(1)$ time.
- This yields $\mathcal{O}(n + m)$ average time complexity.

26.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Substring Matching Bugs:** If you mention the serialization approach, proactively bring up the **value boundary bug** (e.g. a subtree serializing to `"2"` should not match a tree serializing to `"12"`). Explain that adding distinct boundary characters (e.g. `","` and `","`) ensures true syntactic uniqueness.

- **Hash Collision Mitigation:** When describing the Merkle Tree hashing optimization, explain that hashing is prone to collisions (especially in large trees). Detail that if two hashes match, we should run `isSame` as a fallback validation to ensure correctness. This is how actual secure storage engines like Git and IPFS function.

27 Max Area of Island

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer
 - **Source:** LeetCode 695, Glassdoor
 - **Flashcards:** [DFS deck](#)
-

27.1 Question Description

You are given an $m \times n$ binary matrix `grid`. An island is a group of 1s (representing land) connected **4-directionally** (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The **area** of an island is the number of cells with a value 1 in the island.

Return the **maximum area** of an island in `grid`. If there is no island, return 0.

27.1.1 Examples

- **Input:**

```
grid = [
  [0,0,1,0,0,0,0,1,0,0,0,0,0],
  [0,0,0,0,0,0,0,1,1,1,0,0,0],
  [0,1,1,0,1,0,0,0,0,0,0,0,0],
  [0,1,0,0,1,1,0,0,1,0,1,0,0],
  [0,1,0,0,1,1,0,0,1,1,1,0,0],
  [0,0,0,0,0,0,0,0,0,0,1,0,0],
  [0,0,0,0,0,0,0,1,1,1,0,0,0],
  [0,0,0,0,0,0,0,1,1,0,0,0,0]
```

– **Output:** 6

– **Explanation:** The answer is the area of the highlighted island of size 6 in the bottom-middle.

- **Input:** `grid = [[0,0,0,0,0,0,0,0]]`

– **Output:** 0

27.2 Detailed Solution (C++)

Using **Depth-First Search (DFS)**, we traverse the matrix. When we encounter a '1', we compute the area of its island by recursively exploring all connected land cells. We "sink" each visited cell by resetting it to 0 to prevent infinite loops and ensure each island cell is counted exactly once.

27.2.1 Standard C++ Production Code

```
#include <vector>
#include <algorithm>

class Solution {
private:
    int dfs(std::vector<std::vector<int>>& grid, int r, int c, int rows, int cols) noexcept {
        // Base Case: check boundaries and if cell is water
        if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] != 1) {
            return 0;
        }

        // Sink the island cell
        grid[r][c] = 0;

        // Sum the current cell + areas of all 4 neighbors
        return 1 + dfs(grid, r + 1, c, rows, cols)
            + dfs(grid, r - 1, c, rows, cols)
            + dfs(grid, r, c + 1, rows, cols)
            + dfs(grid, r, c - 1, rows, cols);
    }

public:
    int maxAreaOfIsland(std::vector<std::vector<int>>& grid) noexcept {
        if (grid.empty() || grid[0].empty()) {
            return 0;
        }

        int rows = static_cast<int>(grid.size());
        int cols = static_cast<int>(grid[0].size());
        int max_area = 0;

        for (int r = 0; r < rows; ++r) {
            for (int c = 0; c < cols; ++c) {
                if (grid[r][c] == 1) {
                    max_area = std::max(max_area, dfs(grid, r, c, rows, cols));
                }
            }
        }
    }
};
```

```

    }

    return max_area;
}
};

```

27.3 Detailed Solution (Python)

27.3.1 Standard Python Production Code

Below is the recursive Python implementation. We use a nested function to access rows and cols without passing them explicitly.

```

from typing import List

class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        """
        Computes the maximum island area in a 2D binary grid.

        Time Complexity:  $O(M * N)$ 
        Space Complexity:  $O(M * N)$  stack space in the worst case.
        """
        if not grid or not grid[0]:
            return 0

        rows = len(grid)
        cols = len(grid[0])
        max_area = 0

        def dfs(r: int, c: int) -> int:
            # Base Case: check boundaries and if cell is water
            if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != 1:
                return 0

            # Sink the land cell
            grid[r][c] = 0

            # Recursively sum current cell and 4 directions
            return (
                1
                + dfs(r + 1, c)
                + dfs(r - 1, c)
                + dfs(r, c + 1)
                + dfs(r, c - 1)
            )

```

```

        + dfs(r, c - 1)
    )

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                max_area = max(max_area, dfs(r, c))

    return max_area

```

27.4 Python-Specific Complexities & Implementation Considerations

27.4.1 1. Recursion Stack Limits

- For a 50×50 grid, the maximum possible area is 2500 cells.
 - In the worst case (a snake-like island filling the entire grid), recursive DFS will make 2500 nested calls.
 - Python's default call stack limit is 1000. A deep DFS will trigger a `RecursionError`.
 - **Workaround:** Increase the system limit using `sys.setrecursionlimit(3000)`, or rewrite the traversal iteratively using a queue (BFS) or stack (DFS).
-

27.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(m \times n)$	We visit every cell in the grid a constant number of times.
Space Complexity	$\mathcal{O}(m \times n)$	In the worst case (entire grid is land), the call stack depth is $\mathcal{O}(m \times n)$.

27.6 Common Follow-Up Questions & How to Answer

27.6.1 Q1: What if we are not allowed to mutate the input grid?

- **Answer:** If grid mutation is prohibited, we must maintain a separate `visited` grid of size $m \times n$.
 - In C++, `std::vector<std::vector<char>> visited(rows, std::vector<char>(cols, 0))` or a flattened `std::vector<char> visited(rows * cols, 0)` is highly efficient.

27.6.2 Q2: How does BFS compare to DFS in terms of memory overhead?

- **DFS (Recursive):** Memory is allocated on the thread's call stack. In the worst case, the stack depth is $\mathcal{O}(m \times n)$.

- **BFS (Queue-based):** Memory is allocated on the heap (for the queue). The maximum queue size is bounded by the perimeter/width of the largest island, which is generally much smaller than $\mathcal{O}(m \times n)$ for compact islands but can still reach $\mathcal{O}(m \times n)$ for comb-like patterns. BFS is stack-safe.
-

27.7 Pro-Tip: How to Impress the Interviewer

- **Flattening a 2D Matrix:** When storing visited states or writing BFS, mention that representing coordinates as a single index $r * \text{cols} + c$ (flattening) instead of pairs (r, c) saves space and reduces heap allocations. In C++, popping and pushing standard structures like `std::pair` or custom classes creates minor overhead compared to single integers.
- **Spatial Locality in 2D Scans:** Always search the grid in a row-major nested loop (outer loop r , inner loop c). This ensures that the CPU cache lines prefetch sequential grid elements, which significantly accelerates execution compared to column-major scans.

28 Merge k Sorted Lists

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 23, Glassdoor
 - **Flashcards:** [Divide & Conquer deck](#)
-

28.1 Question Description

You are given an array of k linked-lists `lists`, where each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

28.1.1 Examples

- **Input:** `lists = [[1,4,5],[1,3,4],[2,6]]`

– **Output:** `[1,1,2,3,4,4,5,6]`

– **Explanation:** The linked-lists are:

```
[
  1 -> 4 -> 5,
  1 -> 3 -> 4,
  2 -> 6
]
```

Merging them into one sorted list yields: `1 -> 1 -> 2 -> 3 -> 4 -> 4 -> 5 -> 6`.

- **Input:** `lists = []`
– **Output:** `[]`
- **Input:** `lists = [[]]`
– **Output:** `[]`

28.2 Detailed Solution (C++)

The solution uses the **Divide and Conquer** paradigm. Instead of merging the lists sequentially (which would result in an $\mathcal{O}(N \cdot k)$ runtime where N is the total number of elements), we pair up the lists and merge each pair. This is repeated until only one merged list remains.

The process resembles a reverse merge-sort: 1. Divide the array of lists into two halves: left and right. 2. Recursively merge the lists in the left half and the lists in the right half. 3. Merge the two resulting sorted lists in $\mathcal{O}(\text{length}_1 + \text{length}_2)$ time.

This reduces the number of merge steps from k to $\log k$, bringing the overall time complexity to $\mathcal{O}(N \log k)$ and requiring only logarithmic call-stack space.

28.2.1 Standard C++ Production Code

```
#include <vector>

// Definition for singly-linked list node.
struct ListNode {
    int val;
    ListNode* next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode* next) : val(x), next(next) {}
};

class Solution {
public:
    ListNode* mergeKLists(std::vector<ListNode*>& lists) {
        if (lists.empty()) {
            return nullptr;
        }
        return divideAndConquer(lists, 0, static_cast<int>(lists.size()) - 1);
    }

private:
    ListNode* divideAndConquer(const std::vector<ListNode*>& lists, int left, int right) {
        if (left > right) {
            return nullptr;
        }
        if (left == right) {
            return lists[left];
        }
    }
};
```

```

    int mid = left + (right - left) / 2;
    ListNode* l1 = divideAndConquer(lists, left, mid);
    ListNode* l2 = divideAndConquer(lists, mid + 1, right);

    return mergeTwoLists(l1, l2);
}

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy;

    while (l1 && l2) {
        if (l1->val <= l2->val) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }

    // Append the remaining list
    tail->next = l1 ? l1 : l2;

    return dummy.next;
}
};

```

28.3 Detailed Solution (Python)

28.3.1 Standard Python Production Code

```

from typing import List, Optional

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val: int = 0, next: Optional['ListNode'] = None):
        self.val = val
        self.next = next

class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:

```

```
"""
```

Merges k sorted linked-lists using divide and conquer.

Time Complexity: $O(N \log k)$

Space Complexity: $O(\log k)$ due to recursion stack

```
"""
```

```
if not lists:
```

```
    return None
```

```
return self._divide_and_conquer(lists, 0, len(lists) - 1)
```

```
def _divide_and_conquer(self, lists: List[Optional[ListNode]], left: int, right: int) -> Optional[ListNode]:
```

```
    if left > right:
```

```
        return None
```

```
    if left == right:
```

```
        return lists[left]
```

```
    mid = left + (right - left) // 2
```

```
    l1 = self._divide_and_conquer(lists, left, mid)
```

```
    l2 = self._divide_and_conquer(lists, mid + 1, right)
```

```
    return self._merge_two_lists(l1, l2)
```

```
def _merge_two_lists(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
```

```
    dummy = ListNode(0)
```

```
    tail = dummy
```

```
    while l1 and l2:
```

```
        if l1.val <= l2.val:
```

```
            tail.next = l1
```

```
            l1 = l1.next
```

```
        else:
```

```
            tail.next = l2
```

```
            l2 = l2.next
```

```
        tail = tail.next
```

```
    tail.next = l1 or l2
```

```
    return dummy.next
```

28.4 Python-Specific Complexities & Implementation Considerations

28.4.1 1. Priority Queue / Heap Gotcha in Python

- If we use a min-heap (`heapq`) in Python, we store tuples like `(node.val, node)`. If two nodes have the same value, Python will attempt to compare the `ListNode` objects to break the tie, resulting in a `TypeError: '<' not supported between instances of 'ListNode' and 'ListNode'`.
- To avoid this, we can insert a unique counter in the tuple: `(node.val, counter, node)`. This guarantees unique comparison keys.

28.4.2 2. Recursion Limit

- The divide-and-conquer strategy has a maximum recursion depth of $\mathcal{O}(\log k)$. Since the constraint is $k \leq 10^4$, the maximum depth is ≈ 14 . This is well within Python's default stack limit of 1000 and does not risk `RecursionError`.

28.4.3 3. GC and Reference Cycles

- Python's reference-counting garbage collector handles linked lists easily. However, if your list contains circular links (which shouldn't happen under correct execution), they will stay in memory until the cycle-detecting garbage collector runs. In performance-sensitive code, manually clearing reference fields (`node.next = None`) as you dismantle lists is a useful defensive programming practice.

28.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N \log k)$	There are $\log k$ levels of merging. At each level, we merge pairs of lists, scanning a total of N nodes.
Space Complexity	$\mathcal{O}(\log k)$	Auxiliary stack space for the recursion tree. If implemented iteratively (in-place), space is $\mathcal{O}(1)$.

28.6 Common Follow-Up Questions & How to Answer

28.6.1 Q1: What if the lists are too large to fit in memory?

- **Answer:** This calls for **External Sorting**. We can perform a K-way merge using a min-heap, reading small portions of each list from disk (as streams or buffered inputs). We only keep the current head element of each list in memory inside the heap, processing and writing the merged output to disk continuously.

28.6.2 Q2: Heap vs. Divide and Conquer — which is better?

- **Answer:**
 - **Min-Heap:** Time is $\mathcal{O}(N \log k)$ and space is $\mathcal{O}(k)$ (for the heap size). Good if the input lists are arriving dynamically as streams.
 - **Divide and Conquer:** Time is $\mathcal{O}(N \log k)$ and space is $\mathcal{O}(1)$ if implemented iteratively (merging lists in-place in the input vector). It avoids the overhead of managing a dynamic heap structure and offers better cache locality.
-

28.7 Pro-Tip: How to Impress the Interviewer

- **Mention Iterative Merge ($\mathcal{O}(1)$ Space):** Point out that instead of recursive divide-and-conquer, we can merge the lists in a loop (step = 1, then step *= 2), merging pairs in-place within the array. This gets rid of the recursion stack entirely, yielding true $\mathcal{O}(1)$ auxiliary space.
- **Cache Locality:** Explain that pointers in linked lists cause cache misses. In high-performance systems, we prefer flat arrays or memory-arena-allocated nodes to ensure nodes are contiguous in physical RAM.

29 Majority Element

- **Category:** Coding
 - **Difficulty:** Easy
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 169, Glassdoor
 - **Flashcards:** [Divide & Conquer deck](#)
-

29.1 Question Description

Given an array `nums` of size `n`, return the majority element.

The majority element is the element that appears more than $\lfloor n/2 \rfloor$ times. You may assume that the majority element always exists in the array.

29.1.1 Examples

- **Input:** `nums = [3,2,3]`
 - **Output:** 3
 - **Input:** `nums = [2,2,1,1,1,2,2]`
 - **Output:** 2
-

29.2 Detailed Solution (C++)

While this problem can be solved using **Divide and Conquer** in $\mathcal{O}(n \log n)$ time, the optimal approach is the **Boyer-Moore Voting Algorithm**, which runs in $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ space.

29.2.1 Boyer-Moore Voting Algorithm Logic

1. Maintain a **candidate** and a **count** initialized to **0** or the first element.
2. For each element in the array:
 - If **count** is **0**, assign the current element to **candidate** and set **count** to **1**.
 - If the current element equals **candidate**, increment **count**.
 - Otherwise, decrement **count**.
3. Because the majority element appears more than $\lfloor n/2 \rfloor$ times, its votes will always dominate and remain at the end.

We also show the **Divide and Conquer** implementation to demonstrate pattern flexibility: - Split the array into two halves. - Find the majority candidate in the left half and right half recursively. - If the candidates are the same, that candidate is the majority element of the combined segment. - If they differ, count the frequency of both candidates in the combined segment and return the one with the higher count.

29.2.2 Standard C++ Production Code

29.2.2.1 Optimal Approach: Boyer-Moore Voting ($\mathcal{O}(n)$ Time, $\mathcal{O}(1)$ Space)

```
#include <vector>

class Solution {
public:
    int majorityElement(const std::vector<int>& nums) noexcept {
        int candidate = nums[0];
        int count = 1;

        for (size_t i = 1; i < nums.size(); ++i) {
            if (count == 0) {
                candidate = nums[i];
                count = 1;
            } else if (nums[i] == candidate) {
                ++count;
            } else {
                --count;
            }
        }

        return candidate;
    }
};
```

29.2.2.2 Divide and Conquer Approach ($\mathcal{O}(n \log n)$ Time, $\mathcal{O}(\log n)$ Space)

```

#include <vector>
#include <algorithm>

class SolutionDivideConquer {
public:
    int majorityElement(const std::vector<int>& nums) {
        return findMajority(nums, 0, static_cast<int>(nums.size()) - 1);
    }

private:
    int findMajority(const std::vector<int>& nums, int left, int right) {
        if (left == right) {
            return nums[left];
        }

        int mid = left + (right - left) / 2;
        int leftMajority = findMajority(nums, left, mid);
        int rightMajority = findMajority(nums, mid + 1, right);

        if (leftMajority == rightMajority) {
            return leftMajority;
        }

        // Count occurrences of each candidate in [left, right]
        int leftCount = countInRange(nums, leftMajority, left, right);
        int rightCount = countInRange(nums, rightMajority, left, right);

        return leftCount > rightCount ? leftMajority : rightMajority;
    }

    int countInRange(const std::vector<int>& nums, int target, int left, int right) {
        int count = 0;
        for (int i = left; i <= right; ++i) {
            if (nums[i] == target) {
                ++count;
            }
        }
        return count;
    }
};

```


29.3 Detailed Solution (Python)

29.3.1 Standard Python Production Code

29.3.1.1 Optimal Approach: Boyer-Moore Voting

```
from typing import List

class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        """
        Finds the majority element using the Boyer-Moore Voting Algorithm.

        Time Complexity:  $O(n)$ 
        Space Complexity:  $O(1)$ 
        """
        candidate = nums[0]
        count = 1

        for num in nums[1:]:
            if count == 0:
                candidate = num
                count = 1
            elif num == candidate:
                count += 1
            else:
                count -= 1

        return candidate
```

29.3.1.2 Divide and Conquer Approach

```
from typing import List

class SolutionDivideConquer:
    def majorityElement(self, nums: List[int]) -> int:
        """
        Finds the majority element using Divide and Conquer.

        Time Complexity:  $O(n \log n)$ 
        Space Complexity:  $O(\log n)$ 
        """
        def get_majority(left: int, right: int) -> int:
            if left == right:
                return nums[left]
```

```

mid = left + (right - left) // 2
left_maj = get_majority(left, mid)
right_maj = get_majority(mid + 1, right)

if left_maj == right_maj:
    return left_maj

# Count frequencies in range
left_count = sum(1 for i in range(left, right + 1) if nums[i] == left_maj)
right_count = sum(1 for i in range(left, right + 1) if nums[i] == right_maj)

return left_maj if left_count > right_count else right_maj

return get_majority(0, len(nums) - 1)

```

29.4 Python-Specific Complexities & Implementation Considerations

29.4.1 1. Built-in `collections.Counter`

- In Python, you can find the majority element via `Counter(nums).most_common(1)[0][0]`. While this is extremely concise, it uses $\mathcal{O}(n)$ auxiliary space to store the hash map, which is suboptimal compared to Boyer-Moore's $\mathcal{O}(1)$ space.

29.4.2 2. Slicing in Recursion

- In the Divide and Conquer approach, passing array slices (e.g., `nums[:mid]`) creates copies of lists. To achieve the $\mathcal{O}(n \log n)$ time limit and avoid extra memory allocations, pass indices `left` and `right` rather than slicing the list.
-

29.5 Complexity Analysis

29.5.1 Boyer-Moore Voting Algorithm:

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Single pass over the array of size n .
Space Complexity	$\mathcal{O}(1)$	Uses a constant number of integer variables.

29.5.2 Divide and Conquer:

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	Defined by $T(n) = 2T(n/2) + \mathcal{O}(n)$ where counting the frequencies takes $\mathcal{O}(n)$ at each step.
Space Complexity	$\mathcal{O}(\log n)$	Depth of the recursion call stack is $\log n$.

29.6 Common Follow-Up Questions & How to Answer

29.6.1 Q1: What if the majority element is not guaranteed to exist?

- **Answer:** The Boyer-Moore algorithm will still return a candidate. To verify if it is indeed the majority element, perform a second linear pass over the array to count the actual occurrences of that candidate. If the count is $> \lfloor n/2 \rfloor$, it is the majority element; otherwise, no majority element exists.

29.6.2 Q2: How can we find all elements that appear more than $\lfloor n/3 \rfloor$ times? (LeetCode 229)

- **Answer:** Generalize Boyer-Moore to maintain up to **two candidates** and their respective counts (since at most two elements can appear more than $\lfloor n/3 \rfloor$ times). In a second pass, verify both candidates.

29.7 Pro-Tip: How to Impress the Interviewer

- **Parallelizability (MapReduce):** Point out that Boyer-Moore is inherently sequential. However, the **Divide and Conquer** approach is highly parallelizable. In distributed systems (like MapReduce or Spark), we can find majority candidates in chunks of a massive file on different servers, and then merge the candidates by counting their occurrences in a final reduction step.
- **Bitwise Voting:** Discuss another $\mathcal{O}(32 \cdot n) = \mathcal{O}(n)$ solution: count the number of set bits at each of the 32 bit positions. If a bit is set in more than half of the elements, that bit will be set in the majority element. This highlights a deep understanding of low-level data representation.

30 Construct Quad Tree

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
- **Source:** LeetCode 427, Glassdoor
- **Flashcards:** [Divide & Conquer deck](#)

30.1 Question Description

Given an $n \times n$ matrix `grid` of 0s and 1s only, represent the grid using a **Quad-Tree**. Return the root of the Quad-Tree.

A Quad-Tree is a tree data structure in which each internal node has exactly four children: `topLeft`, `topRight`, `bottomLeft`, and `bottomRight`. In addition, each node contains two attributes: * `val`: `True` if the node represents a grid of all 1s, or `False` if the node represents a grid of all 0s. (Can be any value if the node is not a leaf). * `isLeaf`: `True` if the node is a leaf node, or `False` if the node has four children.

30.1.1 Quad-Tree Construction Algorithm

1. If the current grid has the same value (all 1s or all 0s), set `isLeaf` to `True`, set `val` to the value of the grid, and set all four children to `NULL`.
2. If the current grid has different values, set `isLeaf` to `False`, set `val` to any value (usually `True`), and divide the grid into four sub-grids of size $n/2 \times n/2$. Recursively construct the four children nodes.

30.1.2 Examples

- **Input:** `grid = [[0,1],[1,0]]`
 - **Output:** `[[0,1],[1,0],[1,1],[1,1],[1,0]]`
 - **Explanation:** The grid contains different values, so we split it into 4 quadrants. Each quadrant of size 1×1 becomes a leaf node.
 - **Input:** `grid = [[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,1,1,1,1],[1,1,1,1,1,1,1,1],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0]]`
 - **Output:** `[[0,1],[1,1],[0,1],[1,1],[1,0],null,null,null,null,[1,0],[1,0],[1,1],[1,1]]`
-

30.2 Detailed Solution (C++)

The recursive **Divide and Conquer** approach is natural for Quad-Tree construction. We can implement it in two ways: 1. **Top-Down with Prefix Sums** ($\mathcal{O}(n^2)$): We precompute a 2D prefix sum array. To check if a region is uniform, we query its sum in $\mathcal{O}(1)$ time. If the sum is 0 or size^2 , the region is uniform. 2. **Bottom-Up Optimization** ($\mathcal{O}(n^2)$): We recursively build the four quadrants first. If all four quadrants are leaves and share the same value, we merge them into a single leaf node. Otherwise, we keep them as separate children. This avoids any redundant checks or prefix sum tables.

30.2.1 Standard C++ Production Code (Bottom-Up $\mathcal{O}(n^2)$)

```
#include <vector>
```

```
// Definition for a QuadTree node.
```

```
class Node {
public:
    bool val;
    bool isLeaf;
    Node* topLeft;
    Node* topRight;
```

```

Node* bottomLeft;
Node* bottomRight;

Node() : val(false), isLeaf(false), topLeft(nullptr), topRight(nullptr), bottomLeft(nullptr), bottomRight(nullptr) {}

Node(bool _val, bool _isLeaf)
    : val(_val), isLeaf(_isLeaf), topLeft(nullptr), topRight(nullptr), bottomLeft(nullptr), bottomRight(nullptr) {}

Node(bool _val, bool _isLeaf, Node* _topLeft, Node* _topRight, Node* _bottomLeft, Node* _bottomRight)
    : val(_val), isLeaf(_isLeaf), topLeft(_topLeft), topRight(_topRight), bottomLeft(_bottomLeft), bottomRight(_bottomRight) {}

};

class Solution {
public:
    Node* construct(std::vector<std::vector<int>>& grid) {
        if (grid.empty()) return nullptr;
        return build(grid, 0, 0, static_cast<int>(grid.size()));
    }

private:
    Node* build(std::vector<std::vector<int>>& grid, int r, int c, int size) {
        if (size == 1) {
            return new Node(grid[r][c] == 1, true);
        }

        int half = size / 2;
        Node* tl = build(grid, r, c, half);
        Node* tr = build(grid, r, c + half, half);
        Node* bl = build(grid, r + half, c, half);
        Node* br = build(grid, r + half, c + half, half);

        // If all four children are leaf nodes and have the same value, merge them
        if (tl->isLeaf && tr->isLeaf && bl->isLeaf && br->isLeaf &&
            tl->val == tr->val && tr->val == bl->val && bl->val == br->val) {
            bool value = tl->val;
            delete tl; delete tr; delete bl; delete br; // Prevent memory leaks
            return new Node(value, true);
        }

        return new Node(true, false, tl, tr, bl, br);
    }
};

```

30.3 Detailed Solution (Python)

30.3.1 Standard Python Production Code (Bottom-Up $\mathcal{O}(n^2)$)

```
from typing import List
```

```
# Definition for a QuadTree node.
```

```
class Node:
```

```
    def __init__(self, val: bool, isLeaf: bool, topLeft: 'Node' = None, topRight: 'Node' = None, bottomLeft: 'Node' = None, bottomRight: 'Node' = None):
        self.val = val
        self.isLeaf = isLeaf
        self.topLeft = topLeft
        self.topRight = topRight
        self.bottomLeft = bottomLeft
        self.bottomRight = bottomRight
```

```
class Solution:
```

```
    def construct(self, grid: List[List[int]]) -> 'Node':
```

```
        """
```

```
Constructs a Quad-Tree from a 2D grid using a bottom-up divide and conquer approach.
```

```
Time Complexity:  $O(n^2)$ 
```

```
Space Complexity:  $O(\log n)$  call stack space
```

```
        """
```

```
        if not grid:
```

```
            return None
```

```
        return self._build(grid, 0, 0, len(grid))
```

```
    def _build(self, grid: List[List[int]], r: int, c: int, size: int) -> 'Node':
```

```
        if size == 1:
```

```
            return Node(grid[r][c] == 1, True)
```

```
        half = size // 2
```

```
        tl = self._build(grid, r, c, half)
```

```
        tr = self._build(grid, r, c + half, half)
```

```
        bl = self._build(grid, r + half, c, half)
```

```
        br = self._build(grid, r + half, c + half, half)
```

```
# Check if all four children are leaf nodes and have the same value
```

```
        if tl.isLeaf and tr.isLeaf and bl.isLeaf and br.isLeaf:
```

```
            if tl.val == tr.val == bl.val == br.val:
```

```
                return Node(tl.val, True)
```

```
        return Node(True, False, tl, tr, bl, br)
```

30.4 Python-Specific Complexities & Implementation Considerations

30.4.1 1. Object Allocation Overhead

- Python's class instantiation (`Node(...)`) has significantly higher CPU overhead compared to C++ because it creates dictionary entries for `__dict__` dynamically. For large grids, memory footprint increases fast.
- Adding `__slots__ = ['val', 'isLeaf', 'topLeft', 'topRight', 'bottomLeft', 'bottomRight']` to the `Node` definition restricts dynamic attribute creation, reducing memory footprint by up to 50-70%.

30.4.2 2. Tail Call Elimination

- Python doesn't optimize tail-recursive calls, meaning the stack frames grow proportional to the height of the tree ($\mathcal{O}(\log n)$). Since $n \leq 64$, the maximum depth is ≈ 6 , so it is perfectly safe from stack overflow.

30.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n^2)$	Bottom-up compares and merges adjacent blocks. Each element in the $n \times n$ grid is visited once.
Space Complexity	$\mathcal{O}(\log n)$	Auxiliary stack space is proportional to tree depth ($\log n$).

30.6 Common Follow-Up Questions & How to Answer

30.6.1 Q1: What is the benefit of a Quad-Tree over a flat grid?

- **Answer:** Quad-Trees are highly effective for **spatial partitioning** and **image compression**. If an image has large regions of uniform color (e.g. background sky), a Quad-Tree collapses millions of pixels into single leaf nodes, saving significant memory and accelerating spatial queries like collision detection.

30.6.2 Q2: How can we implement this iteratively?

- **Answer:** An iterative approach involves maintaining a stack or queue, but bottom-up merges are difficult to schedule iteratively. A top-down approach with 2D prefix sums allows us to easily push grid coordinates (`r`, `c`, `size`) onto a queue, query uniformity in $\mathcal{O}(1)$, and construct nodes level-by-level without recursive backtracking.

30.7 Pro-Tip: How to Impress the Interviewer

- **Prevent Memory Leaks in C++:** Point out that when merging four child nodes in C++, you must explicitly call `delete` on the child pointers to avoid memory leaks. Python has automatic garbage collection, but in systems languages like C++, neglecting to deallocate merged child nodes results in memory leaks.
- **2D Prefix Sum Formula:** Explain how to compute the sum of any subarray in $\mathcal{O}(1)$:

$$\text{Sum} = P[r_2][c_2] - P[r_1 - 1][c_2] - P[r_2][c_1 - 1] + P[r_1 - 1][c_1 - 1]$$

This demonstrates advanced structural understanding of multidimensional algorithms.

31 Reverse Pairs

- **Category:** Coding
- **Difficulty:** Hard
- **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
- **Source:** LeetCode 493, Glassdoor
- **Flashcards:** [Divide & Conquer deck](#)

31.1 Question Description

Given an integer array `nums`, return the number of **reverse pairs** in the array.

A reverse pair is a pair (i, j) where: 1. $0 \leq i < j < \text{nums.length}$ 2. $\text{nums}[i] > 2 \times \text{nums}[j]$

31.1.1 Examples

- **Input:** `nums = [1,3,2,3,1]`
 - **Output:** 2
 - **Explanation:** The reverse pairs are:
 - * $(1, 4) \implies \text{nums}[1] = 3 > 2 \times \text{nums}[4] = 2 \times 1$
 - * $(3, 4) \implies \text{nums}[3] = 3 > 2 \times \text{nums}[4] = 2 \times 1$
- **Input:** `nums = [2,4,3,5,1]`
 - **Output:** 3
 - **Explanation:** The reverse pairs are:
 - * $(1, 4) \implies 4 > 2 \times 1$
 - * $(2, 4) \implies 3 > 2 \times 1$
 - * $(3, 4) \implies 5 > 2 \times 1$

31.2 Detailed Solution (C++)

This problem can be efficiently solved using a modified **Merge Sort** (Divide and Conquer).

31.2.1 Algorithm Logic

1. **Divide:** Split the array into two halves, `left` and `right`.
2. **Conquer:** Recursively solve the problem for the left and right halves to count the reverse pairs internal to each half.
3. **Combine:** Count the cross-boundary reverse pairs where element i is in the left half and element j is in the right half.
 - Since both halves are sorted after the recursive steps, we can use a **Two-Pointer** approach to count the pairs.
 - Initialize a pointer `j` at the start of the right half.
 - For each element `nums[i]` in the left half, advance `j` until `nums[i] <= 2 * nums[j]`.
 - All elements in the right half before `j` form a valid reverse pair with `nums[i]`. So, we add `j - mid` to our total count.
 - Merge the two sorted halves in-place to complete the sort.

31.2.2 Standard C++ Production Code

```
#include <vector>

class Solution {
public:
    int reversePairs(std::vector<int>& nums) {
        if (nums.size() <= 1) return 0;
        std::vector<int> temp(nums.size());
        return mergeSort(nums, temp, 0, static_cast<int>(nums.size()) - 1);
    }

private:
    int mergeSort(std::vector<int>& nums, std::vector<int>& temp, int left, int right) {
        if (left >= right) return 0;

        int mid = left + (right - left) / 2;
        int count = mergeSort(nums, temp, left, mid) + mergeSort(nums, temp, mid + 1, right);

        // Count reverse pairs crossing mid
        int j = mid + 1;
        for (int i = left; i <= mid; ++i) {
            // Cast to long long to prevent integer overflow when multiplying by 2
            while (j <= right && static_cast<long long>(nums[i]) > 2LL * nums[j]) {
                j++;
            }
        }
    }
};
```

```

        count += (j - (mid + 1));
    }

    // Standard merge step
    int p1 = left;
    int p2 = mid + 1;
    int k = left;

    while (p1 <= mid && p2 <= right) {
        if (nums[p1] <= nums[p2]) {
            temp[k++] = nums[p1++];
        } else {
            temp[k++] = nums[p2++];
        }
    }

    while (p1 <= mid) temp[k++] = nums[p1++];
    while (p2 <= right) temp[k++] = nums[p2++];

    // Copy back to original array
    for (int i = left; i <= right; ++i) {
        nums[i] = temp[i];
    }

    return count;
}
};

```

31.3 Detailed Solution (Python)

31.3.1 Standard Python Production Code

```

from typing import List

class Solution:
    def reversePairs(self, nums: List[int]) -> int:
        """
        Calculates the number of reverse pairs using merge sort.

        Time Complexity: O(n log n)
        Space Complexity: O(n)
        """
        if not nums:

```

```

        return 0
    return self._merge_sort(nums, 0, len(nums) - 1)

def _merge_sort(self, nums: List[int], left: int, right: int) -> int:
    if left >= right:
        return 0

    mid = left + (right - left) // 2
    count = self._merge_sort(nums, left, mid) + self._merge_sort(nums, mid + 1, right)

    # Count cross-boundary reverse pairs
    j = mid + 1
    for i in range(left, mid + 1):
        while j <= right and nums[i] > 2 * nums[j]:
            j += 1
        count += (j - (mid + 1))

    # Merge in-place using Python's list slicing/sorting
    # Note: Merging in-place avoids extra allocation inside recursion
    nums[left:right + 1] = sorted(nums[left:right + 1])

    return count

```

31.4 Python-Specific Complexities & Implementation Considerations

31.4.1 1. Arithmetic Overflow

- In C++, doing `2 * nums[j]` can lead to integer overflow if `nums[j]` is large. This must be handled by casting to `long long`.
- In Python, integers have arbitrary precision (they do not overflow), so `2 * nums[j]` is completely safe.

31.4.2 2. Timsort Optimizations

- The Python code uses `nums[left:right+1] = sorted(nums[left:right+1])`. While using `sorted()` looks like it defeats the purpose of the merge step, Python's built-in Timsort is implemented in C and runs extremely fast. Since the left and right halves are already sorted, Timsort can merge them in $\mathcal{O}(k)$ time, which is highly efficient.
-

31.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	The array is halved at each level of the recursion tree ($\log n$ levels). At each level, the two-pointer counting and merging takes linear time $\mathcal{O}(n)$.
Space Complexity	$\mathcal{O}(n)$	We allocate a temporary array of size n to assist with merging.

31.6 Common Follow-Up Questions & How to Answer

31.6.1 Q1: Can we solve this using Binary Indexed Tree (BIT) / Fenwick Tree?

- **Answer:** Yes. We can coordinate coordinate-compression to map the values in `nums` to their ranks. Then, we scan the array from right to left:
 1. Query the BIT for how many elements seen so far are strictly smaller than `nums[i] / 2.0`.
 2. Insert `nums[i]` into the BIT.
- **BIT Complexity:** Time complexity remains $\mathcal{O}(n \log n)$, but the implementation is non-trivial and space is $\mathcal{O}(n)$ for the tree.

31.6.2 Q2: What are the differences between Inversion Count and Reverse Pairs?

- **Answer:** In a standard Inversion Count, the condition is `nums[i] > nums[j]`. This allows counting *during* the merge step directly since `nums[i] > nums[j]` implies all remaining elements in the left half are also greater than `nums[j]`.
- For Reverse Pairs, the condition is `nums[i] > 2 * nums[j]`. Because of the factor of 2, we cannot merge and count simultaneously. Thus, we must perform the two-pointer counting step *before* merging.

31.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Vector Reallocation:** Point out that allocating a new `temp` vector at each recursive call creates $\mathcal{O}(n)$ dynamic memory allocations, which hurts heap allocation performance. Allocating a single `temp` buffer of size n in the entry function and passing its reference down keeps execution fast and cache-friendly.
- **Explain C++ Long Long Literals:** Explain why you used `2LL` instead of `2`. In C++, `2` is a standard `int`. If `nums[j]` is `1073741824`, then `2 * nums[j]` overflows. Using `2LL` forces promotion to `long long` *before* multiplication, avoiding undefined behavior.

32 Sort an Array

- **Category:** Coding
- **Difficulty:** Medium

- **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 912, Glassdoor
 - **Flashcards:** [Divide & Conquer deck](#)
-

32.1 Question Description

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem without using any built-in sorting functions, in $\mathcal{O}(n \log n)$ time complexity, and with the smallest space complexity possible.

32.1.1 Examples

- **Input:** `nums = [5,2,3,1]`
 – **Output:** `[1,2,3,5]`
 - **Input:** `nums = [5,1,1,2,0,0]`
 – **Output:** `[0,0,1,1,2,5]`
-

32.2 Detailed Solution (C++)

To sort an array in $\mathcal{O}(n \log n)$ time, the classic choices are **Merge Sort**, **Heap Sort**, and **Quick Sort**.

32.2.1 Why Merge Sort?

Merge Sort is a **Divide and Conquer** algorithm that guarantees stable $\mathcal{O}(n \log n)$ time. 1. **Divide:** Find the midpoint of the subarray. 2. **Conquer:** Recursively sort the left and right halves. 3. **Combine:** Merge the two sorted halves back together in-place or into a temp buffer.

To optimize, we allocate a single temporary helper vector of size N at the beginning. This avoids overhead from recurring mallocs/deallocations.

32.2.2 Standard C++ Production Code (Merge Sort)

```
#include <vector>

class Solution {
public:
    std::vector<int> sortArray(std::vector<int>& nums) {
        if (nums.size() <= 1) return nums;
        std::vector<int> temp(nums.size());
        mergeSort(nums, temp, 0, static_cast<int>(nums.size()) - 1);
        return nums;
    }

private:
```

```

void mergeSort(std::vector<int>& nums, std::vector<int>& temp, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;
    mergeSort(nums, temp, left, mid);
    mergeSort(nums, temp, mid + 1, right);

    // Merge the two halves
    int p1 = left;
    int p2 = mid + 1;
    int k = left;

    while (p1 <= mid && p2 <= right) {
        if (nums[p1] <= nums[p2]) {
            temp[k++] = nums[p1++];
        } else {
            temp[k++] = nums[p2++];
        }
    }

    while (p1 <= mid) temp[k++] = nums[p1++];
    while (p2 <= right) temp[k++] = nums[p2++];

    // Copy back sorted values
    for (int i = left; i <= right; ++i) {
        nums[i] = temp[i];
    }
}
};

```

32.3 Detailed Solution (Python)

32.3.1 Standard Python Production Code (Merge Sort)

```
from typing import List
```

```
class Solution:
```

```
    def sortArray(self, nums: List[int]) -> List[int]:
```

```
        """
```

```
        Sorts the array in ascending order using Merge Sort.
```

```
        Time Complexity:  $O(n \log n)$ 
```

```
        Space Complexity:  $O(n)$ 
```

```

    """
    if len(nums) <= 1:
        return nums

    temp = [0] * len(nums)
    self._merge_sort(nums, temp, 0, len(nums) - 1)
    return nums

def _merge_sort(self, nums: List[int], temp: List[int], left: int, right: int) -> None:
    if left >= right:
        return

    mid = left + (right - left) // 2
    self._merge_sort(nums, temp, left, mid)
    self._merge_sort(nums, temp, mid + 1, right)

    # Merge Step
    p1, p2, k = left, mid + 1, left
    while p1 <= mid and p2 <= right:
        if nums[p1] <= nums[p2]:
            temp[k] = nums[p1]
            p1 += 1
        else:
            temp[k] = nums[p2]
            p2 += 1
        k += 1

    while p1 <= mid:
        temp[k] = nums[p1]
        p1 += 1
        k += 1

    while p2 <= right:
        temp[k] = nums[p2]
        p2 += 1
        k += 1

    # Copy back to nums
    for i in range(left, right + 1):
        nums[i] = temp[i]
```

32.4 Python-Specific Complexities & Implementation Considerations

32.4.1 1. In-place vs. Out-of-place Slicing

- Writing `left = self.sortArray(nums[:mid])` and `right = self.sortArray(nums[mid:])` creates temporary lists at every recursion level. This yields $\mathcal{O}(n \log n)$ total auxiliary space and can lead to Memory Limit Exceeded (MLE) on LeetCode.
- Always use array index boundaries (`left`, `right`) and a single pre-allocated auxiliary list to perform the merge step.

32.4.2 2. Timsort (Python's Built-in)

- Python's built-in `list.sort()` and `sorted()` use **Timsort** (a hybrid of Merge Sort and Insertion Sort). In a real production codebase, Timsort should always be used as it is heavily optimized in C, handles partially sorted arrays in $\mathcal{O}(n)$, and has a worst-case of $\mathcal{O}(n \log n)$.

32.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	Guaranteed for best, average, and worst cases since we divide the array in half at each step.
Space Complexity	$\mathcal{O}(n)$	Due to the auxiliary temporary buffer used for merging, plus $\mathcal{O}(\log n)$ stack space.

32.6 Common Follow-Up Questions & How to Answer

32.6.1 Q1: Why is Quick Sort preferred over Merge Sort for arrays, but Merge Sort is preferred for linked lists?

- **Answer:**
 - **Arrays:** Quick Sort has better **cache locality** because it works in-place and accesses contiguous memory blocks. Merge Sort requires allocating a temporary array, which incurs allocation overhead and extra memory copies.
 - **Linked Lists:** Elements in a linked list are scattered in memory, so cache locality is lost anyway. Merge Sort can be implemented on linked lists with $\mathcal{O}(1)$ auxiliary space (by changing pointers) and without allocating new nodes, making it highly efficient.

32.6.2 Q2: What is the optimal sorting algorithm if values are highly bounded?

- **Answer:** If constraints indicate that the range of numbers K is small (e.g. $-50000 \leq \text{nums}[i] \leq 50000$), we can use **Counting Sort**. We count frequencies of each element using a frequency array of size K , and then write them back. This runs in $\mathcal{O}(n + K)$ time and $\mathcal{O}(K)$ space.

32.7 Pro-Tip: How to Impress the Interviewer

- **Mention Dual-Pivot Quick Sort & Pivot Selection:** Mention that naive Quick Sort can degrade to $\mathcal{O}(n^2)$ if the pivot is chosen poorly (e.g. sorted arrays). Modern library implementations (like Java's `Arrays.sort` or C++ `std::sort` which uses Introsort) choose pivots using "Median-of-Three" or randomized pivot selection, and switch to Heap Sort if the recursion depth exceeds a threshold.
- **Cache Friendliness of Block Merging:** In low-level programming, we can optimize Merge Sort by handling small subproblems ($N \leq 16$) using **Insertion Sort**. Since small arrays fit entirely inside the L1 cache, Insertion Sort executes faster due to low overhead, which reduces recursion depth.

33 Find Median from Data Stream

- **Category:** Coding
- **Difficulty:** Hard
- **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
- **Source:** LeetCode 295, Glassdoor
- **Flashcards:** [Two Heaps deck](#)

33.1 Question Description

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values. * For example, if `arr = [2, 3, 4]`, the median is 3. * If `arr = [2, 3]`, the median is $(2 + 3) / 2 = 2.5$.

Implement the `MedianFinder` class: * `MedianFinder()` initializes the `MedianFinder` object. * `void addNum(int num)` adds the integer `num` from the data stream to the data structure. * `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

33.1.1 Examples

- **Input:**

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
```

- **Output:** `[null, null, null, 1.5, null, 2.0]`

- **Explanation:**

```
MedianFinder mf;
mf.addNum(1);    // arr = [1]
mf.addNum(2);    // arr = [1, 2]
mf.findMedian(); // returns 1.5 (i.e., (1 + 2) / 2)
mf.addNum(3);    // arr = [1, 2, 3]
mf.findMedian(); // returns 2.0
```

33.2 Detailed Solution (C++)

The most efficient approach uses **Two Heaps**: 1. **A Max-Heap** (`small`): stores the smaller half of the numbers. 2. **A Min-Heap** (`large`): stores the larger half of the numbers.

33.2.1 Rebalancing Invariant

- The max-heap `small` can contain at most **one more** element than the min-heap `large`.
- Thus: $\text{len}(\text{small}) \geq \text{len}(\text{large})$ and $\text{len}(\text{small}) - \text{len}(\text{large}) \leq 1$.

33.2.2 Algorithm

- **Add Num:**
 1. First push `num` to the max-heap `small`.
 2. Pop the top of `small` and push it to the min-heap `large` (ensuring all elements in `large` are greater than or equal to elements in `small`).
 3. If `large.size() > small.size()`, pop the top of `large` and push it back to `small` to restore the size balance.
- **Find Median:**
 1. If `small.size() > large.size()`, the median is the top of `small`.
 2. Otherwise, the median is the average of the tops of both heaps.

33.2.3 Standard C++ Production Code

```
#include <queue>
#include <vector>

class MedianFinder {
private:
    std::priority_queue<int> small; // Max-heap for lower half
    std::priority_queue<int, std::vector<int>, std::greater<int>> large; // Min-heap for upper half

public:
    MedianFinder() = default;

    void addNum(int num) {
        small.push(num);
        large.push(small.top());
        small.pop();

        // Maintain the size invariant: small.size() >= large.size()
        if (large.size() > small.size()) {
            small.push(large.top());
            large.pop();
        }
    }
};
```

```

    }
}

double findMedian() {
    if (small.size() > large.size()) {
        return static_cast<double>(small.top());
    }
    return (static_cast<double>(small.top()) + large.top()) / 2.0;
}
};

```

33.3 Detailed Solution (Python)

33.3.1 Standard Python Production Code

```

import heapq

class MedianFinder:
    def __init__(self):
        """
        Initializes the Max-Heap (small) and Min-Heap (large).
        We store negative values in small to simulate a Max-Heap.
        """
        self.small = [] # Max-Heap (stores negative numbers)
        self.large = [] # Min-Heap (stores positive numbers)

    def addNum(self, num: int) -> None:
        """
        Adds a number to the data stream.

        Time Complexity: O(log n)
        """
        # Push to small (max-heap)
        heapq.heappush(self.small, -num)

        # Pop from small, push to large (min-heap)
        heapq.heappush(self.large, -heapq.heappop(self.small))

        # Balance sizes
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self) -> float:

```

```

"""
Returns the median of the data stream.

Time Complexity: O(1)
"""
if len(self.small) > len(self.large):
    return float(-self.small[0])
return (-self.small[0] + self.large[0]) / 2.0

```

33.4 Python-Specific Complexities & Implementation Considerations

33.4.1 1. Max-Heap Implementation in Python

- Python's `heapq` module only implements Min-Heaps. To implement a Max-Heap, we must negate all numbers before inserting them (`-num`).
- When retrieving elements from the max-heap, remember to negate them again to get the original values (e.g. `-self.small[0]`).
- **Integer Overflow Check:** Negating numbers in Python is safe because Python integers have arbitrary precision and cannot overflow.

33.4.2 2. Float Division / vs Floor Division //

- When calculating the average of the two middle elements, always use float division `/` (e.g., `(-self.small[0] + self.large[0]) / 2.0`) to ensure a floating-point result is returned. Using integer division `//` will truncate fractional values.
-

33.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity (addNum)	$\mathcal{O}(\log n)$	Each insert involves heap push and pop operations, taking logarithmic time.
Time Complexity (findMedian)	$\mathcal{O}(1)$	The median is computed directly from the top elements of both heaps.
Space Complexity	$\mathcal{O}(n)$	To store all elements in the data stream inside the two heaps.

33.6 Common Follow-Up Questions & How to Answer

33.6.1 Q1: What if all integer numbers from the stream are between 0 and 100?

- **Answer:** We can replace the heaps with a **Frequency Array** of size 101. We maintain a count of elements in each bucket, plus a total element counter.
 - To find the median, we scan the frequency array to find the middle indices.
 - Time Complexity: `addNum` becomes $\mathcal{O}(1)$, and `findMedian` takes $\mathcal{O}(1)$ because the bucket count is constant (101).
 - Space Complexity: $\mathcal{O}(1)$ auxiliary space.

33.6.2 Q2: What if 99% of all integer numbers from the stream are between 0 and 100?

- **Answer:** We can keep a frequency array of size 101 for the range $[0, 100]$, and two heaps for elements < 0 and > 100 . Since 99% of elements fall inside $[0, 100]$, the median will almost certainly lie in the range $[0, 100]$, which we can find by index scanning, taking $\mathcal{O}(1)$ average time.
-

33.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Redundant Heap Checks:** When writing C++, show that you can insert and balance in just 4 lines of code without using complex conditional branching. Pushing to one, shifting to the next, and rebalancing by size eliminates unnecessary comparisons and keeps code compact.
- **Cache Locality and Heap Implementations:** Mention that while heaps are theoretically fast, they are laid out in memory as arrays where children are at indices $2*i + 1$ and $2*i + 2$. As the heap grows, traversing it causes random memory jumps, which can result in cache misses. For extremely latency-sensitive trading platforms, flat array structures or B-trees might be preferred.

34 Design Twitter

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 355, Glassdoor
 - **Flashcards:** [Two Heaps deck](#)
-

34.1 Question Description

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and see the **10 most recent tweets** in their news feed.

Implement the `Twitter` class: `* Twitter()` Initializes your Twitter object. `* void postTweet(int userId, int tweetId)` Composes a new tweet with ID `tweetId` by the user `userId`. Each call to this function will be made with a unique `tweetId`. `* List<int> getNewsFeed(int userId)` Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed

or by the user themselves. Tweets must be ordered from most recent to least recent. * `void follow(int followerId, int followeeId)` The user with ID `followerId` starts following the user with ID `followeeId`. * `void unfollow(int followerId, int followeeId)` The user with ID `followerId` stops following the user with ID `followeeId`.

34.1.1 Examples

- **Input:**

```
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet", "getNewsFeed", "unfollow", "getNewsFeed"]
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
```

- **Output:** [null, null, [5], null, null, [6, 5], null, [5]]

- **Explanation:**

```
Twitter twitter;
twitter.postTweet(1, 5); // User 1 posts a new tweet (id = 5).
twitter.getNewsFeed(1); // User 1's news feed should return [5].
twitter.follow(1, 2);   // User 1 follows user 2.
twitter.postTweet(2, 6); // User 2 posts a new tweet (id = 6).
twitter.getNewsFeed(1); // User 1's news feed should return [6, 5] (6 is more recent).
twitter.unfollow(1, 2); // User 1 unfollows user 2.
twitter.getNewsFeed(1); // User 1's news feed should return [5].
```

34.2 Detailed Solution (C++)

To implement Twitter efficiently: 1. **Follow Relationships:** We represent followees for each user using a hash set: `std::unordered_set<int>`. 2. **Tweets storage:** Instead of separate heap allocations for every single tweet, we store all tweets in a contiguous memory block `pool` (represented as `std::vector<Tweet>`). Each user has a linked list of tweets starting at `head[userId]` and linked via indices (`pool[idx].next`). 3. **News Feed Retrieval:** The news feed is generated by merging the sorted lists of tweets from the user and their followees. Since each user's tweets are already sorted by time (linked list from newest to oldest), we can solve this using a **K-Way Merge** (similar to merging K sorted lists). We initialize a max-heap (priority queue) with the head of each stream, and pop the most recent tweet up to 10 times, pushing the next tweet of the corresponding user into the heap.

34.2.1 Standard C++ Production Code

```
#include <vector>
#include <unordered_map>
#include <unordered_set>
#include <queue>
#include <tuple>

class Twitter {
```

private:

```
struct Tweet {
    int tweetId;
    int time;
    int next; // Index of the previous tweet by the same user in the pool
};

std::vector<Tweet> pool;
std::unordered_map<int, int> head; // Maps userId -> index of latest tweet in pool
std::unordered_map<int, std::unordered_set<int>> following;
int clock = 0;
```

public:

```
Twitter() = default;

void postTweet(int userId, int tweetId) {
    int prevTweetIdx = head.count(userId) ? head[userId] : -1;
    pool.push_back({tweetId, ++clock, prevTweetIdx});
    head[userId] = static_cast<int>(pool.size()) - 1;
}

std::vector<int> getNewsFeed(int userId) {
    // Elements in priority queue: {time, tweetId, poolIndex}
    using Element = std::tuple<int, int, int>;
    std::priority_queue<Element> pq;

    // Candidate users: the user themselves and all followees
    std::vector<int> users = {userId};
    if (following.count(userId)) {
        for (int followee : following[userId]) {
            users.push_back(followee);
        }
    }

    // Initialize heap with the latest tweet from each candidate user
    for (int u : users) {
        if (head.count(u) && head[u] != -1) {
            int idx = head[u];
            pq.push({pool[idx].time, pool[idx].tweetId, idx});
        }
    }

    std::vector<int> feed;
    while (!pq.empty() && feed.size() < 10) {
```

```

        auto [time, tid, idx] = pq.top();
        pq.pop();
        feed.push_back(tid);

        int nextIdx = pool[idx].next;
        if (nextIdx != -1) {
            pq.push({pool[nextIdx].time, pool[nextIdx].tweetId, nextIdx});
        }
    }

    return feed;
}

void follow(int followerId, int followeeId) {
    if (followerId != followeeId) {
        following[followerId].insert(followeeId);
    }
}

void unfollow(int followerId, int followeeId) {
    if (following.count(followerId)) {
        following[followerId].erase(followeeId);
    }
}
};

```

34.3 Detailed Solution (Python)

34.3.1 Standard Python Production Code

```

import heapq
from typing import List, Dict, Set, Tuple

class Twitter:
    def __init__(self):
        self.timestamp = 0
        self.tweets: Dict[int, List[Tuple[int, int]]] = {} # userId -> list of (-time, tweetId)
        self.following: Dict[int, Set[int]] = {} # followerId -> set of followeeIds

    def postTweet(self, userId: int, tweetId: int) -> None:
        """
        Posts a tweet. Time complexity: O(1)
        """

```



```

        self.timestamp += 1
        # Store as negative timestamp so heapq (Min-Heap) pops the most recent tweet first
        if userId not in self.tweets:
            self.tweets[userId] = []
        self.tweets[userId].append((-self.timestamp, tweetId))

def getNewsFeed(self, userId: int) -> List[int]:
    """
    Retrieves the 10 most recent tweets. Time complexity: O(K log K)
    where K is the number of followees.
    """
    # User always follows themselves implicitly
    followees = self.following.get(userId, set()) | {userId}

    pq = []
    # Populate the priority queue with the last tweet of each followee
    # Format: (neg_time, tweetId, userId, index_of_next_tweet_in_reverse)
    for uid in followees:
        if uid in self.tweets and self.tweets[uid]:
            # Index -1 is the last (most recent) tweet
            neg_time, tid = self.tweets[uid][-1]
            # We push: (neg_time, tid, uid, index_in_user_tweets)
            heapq.heappush(pq, (neg_time, tid, uid, len(self.tweets[uid]) - 1))

    feed = []
    while pq and len(feed) < 10:
        neg_time, tid, uid, idx = heapq.heappop(pq)
        feed.append(tid)

        # If the user has older tweets, push the next most recent one
        if idx > 0:
            next_neg_time, next_tid = self.tweets[uid][idx - 1]
            heapq.heappush(pq, (next_neg_time, next_tid, uid, idx - 1))

    return feed

def follow(self, followerId: int, followeeId: int) -> None:
    """
    Follows a user. Time complexity: O(1)
    """
    if followerId != followeeId:
        if followerId not in self.following:
            self.following[followerId] = set()
        self.following[followerId].add(followeeId)

```

```
def unfollow(self, followerId: int, followeeId: int) -> None:
    """
    Unfollows a user. Time complexity: O(1)
    """
    if followerId in self.following:
        self.following[followerId].discard(followeeId)
```

34.4 Python-Specific Complexities & Implementation Considerations

34.4.1 1. Inefficient Heap Pushing

- A naive approach is to retrieve ALL tweets of the user and their followees, dump them in a single array, and sort. This is $\mathcal{O}(U \cdot T \log(U \cdot T))$ where U is the number of users and T is the number of tweets per user.
- By keeping tweets sorted per user (which happens naturally by appending), and only keeping the pointer to the latest unprocessed tweet of each user in the heap, we perform a K -way merge. This reduces retrieval time to $\mathcal{O}(K \log K)$ (where K is the number of followed users).

34.4.2 2. Set Discard vs. Remove

- When unfollowing, use Python's `set.discard()` rather than `set.remove()`. If the user is not currently following the `followeeId`, `remove()` raises a `KeyError`, whereas `discard()` fails silently and safely.
-

34.5 Complexity Analysis

Operation	Time Complexity	Space Complexity	Description
<code>postTweet</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Simply appends the tweet to the end of the user's tweet history.
<code>getNewsFeed</code>	$\mathcal{O}(F \log F)$	$\mathcal{O}(F)$	We push at most 1 head tweet from each of the F followed users into the priority queue.
<code>follow</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Updates hash set of followees.
<code>unfollow</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Deletes from the hash set of followees.

34.6 Common Follow-Up Questions & How to Answer

34.6.1 Q1: Push vs. Pull models for News Feeds. Which is better?

- **Answer:**
 - **Pull Model (Fan-out on Read):** We pull tweets from followees and merge them at query time. This is cheap for writes (`postTweet` is $\mathcal{O}(1)$) but slow for reads (`getNewsFeed` merges multiple streams).
 - **Push Model (Fan-out on Write):** When a user posts a tweet, we write it directly to the news feed queues of all their followers. This makes reads $\mathcal{O}(1)$, but writes are extremely slow if a celebrity has 50 million followers.
 - **Hybrid Model:** Use the Push model for normal users. For celebrities, do not fan-out. When a follower requests their feed, they pull the celebrity's tweets and merge them on-the-fly.

34.6.2 Q2: How do we scale this for a distributed database?

- **Answer:** We shard users across database nodes using `userId`. To generate a news feed, we must query the followee nodes to get their latest tweets. To optimize, cache followee IDs and pre-calculated feed chunks in memory using redis/memcached.
-

34.7 Pro-Tip: How to Impress the Interviewer

- **Mention Memory Layout Efficiency:** Emphasize that in the C++ implementation, storing tweets in a single pre-allocated pool vector `pool` avoids memory fragmentation and keeps data local in cache lines. This is much faster than allocating individual `Tweet` nodes dynamically via `new Tweet()`.
- **Warp Divergence in Parallel Merges:** If the feed merging were parallelized on a GPU, we would group users with similar numbers of followees into the same warp to prevent thread execution divergence and execution stalls.

35 Sliding Window Median

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 480, Glassdoor
 - **Flashcards:** [Two Heaps deck](#)
-

35.1 Question Description

You are given an integer array `nums` and an integer `k`. There is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the median array for each window in the original array. Answers within 10^{-5} of the actual value will be accepted.

35.1.1 Examples

- **Input:** `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`
 - **Output:** `[1.00000,-1.00000,-1.00000,3.00000,5.00000,6.00000]`
 - **Explanation:**

Window position	Median
[1 3 -1] -3 5 3 6 7	1
1 [3 -1 -3] 5 3 6 7	-1
1 3 [-1 -3 5] 3 6 7	-1
1 3 -1 [-3 5 3] 6 7	3
1 3 -1 -3 [5 3 6] 7	5
1 3 -1 -3 5 [3 6 7]	6
- **Input:** `nums = [1,2,3,4,2,3,1,4,2]`, `k = 3`
 - **Output:** `[2.00000,3.00000,3.00000,3.00000,2.00000,3.00000,2.00000]`

35.2 Detailed Solution (C++)

To solve this efficiently in $\mathcal{O}(N \log k)$ time, we use a **Two Heaps** approach (a max-heap `small` for the lower half and a min-heap `large` for the upper half) paired with **Lazy Deletion**.

35.2.1 Lazy Deletion Logic

Unlike standard median finding in a continuous stream, elements in a sliding window expire and must be removed. Since standard binary heaps do not support removing arbitrary elements in $\mathcal{O}(\log k)$ time, we use **lazy deletion**: 1. When an element falls out of the sliding window, we record its index (or value) in a hash map `delayed` representing elements scheduled for deletion. 2. We decrement the active size counters of the respective heap (`small_sz` or `large_sz`). 3. Whenever we access the top element of either heap, we check if it is in `delayed`. If it is, we pop it (prune the heap) and decrement the delayed count until we find a non-deleted top element. 4. We then rebalance the active sizes of the heaps so that `small_sz` $\in$ `[large_sz, large_sz + 1]`.

35.2.2 Standard C++ Production Code

```
#include <vector>
#include <queue>
#include <unordered_map>

class Solution {
public:
    std::vector<double> medianSlidingWindow(std::vector<int>& nums, int k) {
        struct Entry {
```

```

    long long val;
    int idx;
    bool operator<(const Entry& other) const { return val < other.val; }
};

struct GreaterEntry {
    bool operator()(const Entry& a, const Entry& b) const { return a.val > b.val; }
};

std::priority_queue<Entry> small; // Max-heap
std::priority_queue<Entry, std::vector<Entry>, GreaterEntry> large; // Min-heap

std::unordered_map<int, int> delayed;
std::vector<bool> in_small(nums.size(), false);
int small_sz = 0, large_sz = 0;

auto prune_small = [&]() {
    while (!small.empty() && delayed.count(small.top().idx)) {
        int idx = small.top().idx;
        if (--delayed[idx] == 0) delayed.erase(idx);
        small.pop();
    }
};

auto prune_large = [&]() {
    while (!large.empty() && delayed.count(large.top().idx)) {
        int idx = large.top().idx;
        if (--delayed[idx] == 0) delayed.erase(idx);
        large.pop();
    }
};

auto make_balanced = [&]() {
    if (small_sz > large_sz + 1) {
        prune_small();
        auto [v, idx] = small.top(); small.pop();
        in_small[idx] = false;
        small_sz--; large_sz++;
        large.push({v, idx});
        prune_small();
    } else if (small_sz < large_sz) {
        prune_large();
        auto [v, idx] = large.top(); large.pop();
        in_small[idx] = true;
    }
};

```

```

        large_sz--; small_sz++;
        small.push({v, idx});
        prune_large();
    }
};

std::vector<double> medians;
int n = static_cast<int>(nums.size());

for (int i = 0; i < n; i++) {
    prune_small();
    prune_large();

    // Insert new element
    if (small.empty() || nums[i] <= small.top().val) {
        small.push({(long long)nums[i], i});
        in_small[i] = true;
        small_sz++;
    } else {
        large.push({(long long)nums[i], i});
        in_small[i] = false;
        large_sz++;
    }

    make_balanced();

    // Handle element exiting the window
    if (i >= k) {
        int out_idx = i - k;
        delayed[out_idx]++;
        if (in_small[out_idx]) {
            small_sz--;
        } else {
            large_sz--;
        }
        make_balanced();
    }

    // Record median
    if (i >= k - 1) {
        prune_small();
        prune_large();
        if (k % 2 == 1) {
            medians.push_back(static_cast<double>(small.top().val));

```

```

        } else {
            medians.push_back((static_cast<double>(small.top().val) + large.top().val) / 2.0);
        }
    }

    return medians;
}

};

```

35.3 Detailed Solution (Python)

35.3.1 Standard Python Production Code

```

import heapq
from typing import List, Dict

class Solution:
    def medianSlidingWindow(self, nums: List[int], k: int) -> List[float]:
        """
        Computes the median of each sliding window of size k using two heaps and lazy deletion.

        Time Complexity: O(N log k)
        Space Complexity: O(N)
        """
        small: List[tuple[int, int]] = [] # Max-heap (negative values)
        large: List[tuple[int, int]] = [] # Min-heap (positive values)

        delayed: Dict[int, int] = {}
        in_small: Dict[int, bool] = {}
        small_size = 0
        large_size = 0

        def prune(heap: List[tuple[int, int]]) -> None:
            while heap:
                _, idx = heap[0]
                if idx in delayed:
                    delayed[idx] -= 1
                    if delayed[idx] == 0:
                        del delayed[idx]
                        heapq.heappop(heap)
                else:
                    break

```

```

def make_balanced() -> None:
    nonlocal small_size, large_size
    if small_size > large_size + 1:
        prune(small)
        val, idx = heapq.heappop(small)
        heapq.heappush(large, (-val, idx))
        in_small[idx] = False
        small_size -= 1
        large_size += 1
        prune(small)
    elif small_size < large_size:
        prune(large)
        val, idx = heapq.heappop(large)
        heapq.heappush(small, (-val, idx))
        in_small[idx] = True
        large_size -= 1
        small_size += 1
        prune(large)

def get_median() -> float:
    if k % 2 == 1:
        return float(-small[0][0])
    else:
        return (-small[0][0] + large[0][0]) / 2.0

medians = []
for i, val in enumerate(nums):
    prune(small)
    prune(large)

    # Insert element
    if not small or val <= -small[0][0]:
        heapq.heappush(small, (-val, i))
        in_small[i] = True
        small_size += 1
    else:
        heapq.heappush(large, (val, i))
        in_small[i] = False
        large_size += 1

    make_balanced()

    # Remove element leaving window

```



```

    if i >= k:
        out_idx = i - k
        delayed[out_idx] = delayed.get(out_idx, 0) + 1
        if in_small.get(out_idx, True):
            small_size -= 1
        else:
            large_size -= 1
        make_balanced()

    # Collect result
    if i >= k - 1:
        prune(small)
        prune(large)
        medians.append(get_median())

return medians

```

35.4 Python-Specific Complexities & Implementation Considerations

35.4.1 1. Simulating Deletion in Heap

- Since `heapq` does not support random element removal, lazy deletion is the only way to retain $\mathcal{O}(\log k)$ average updates. Direct lookup and deletion via `list.remove()` takes $\mathcal{O}(k)$ time, which degrades the overall complexity of the sliding window to $\mathcal{O}(N \cdot k)$ and causes Time Limit Exceeded (TLE).

35.4.2 2. Float Underflow/Overflow

- In Python, dividing large integers by 2.0 automatically yields high-precision floats without risking overflow. In languages with fixed width types like C++, computing `(small.top().val + large.top().val) / 2.0` can overflow if both values are near $2^{31} - 1$, so casting to `long long` is necessary.
-

35.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	Each of the n elements is pushed and popped from the heaps at most once. The heaps are kept balanced using $\mathcal{O}(\log n)$ operations.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(n)$	To store the <code>delayed</code> deletion map and <code>in_small</code> map, up to size n .

35.6 Common Follow-Up Questions & How to Answer

35.6.1 Q1: Can we use standard Binary Search Tree (BST) structures?

- **Answer:** Yes. In C++, `std::multiset` is implemented as a Red-Black Tree. We can insert and delete elements in $\mathcal{O}(\log k)$ time and keep track of the median iterator. During insertions/deletions, we update the median iterator by shifting it left or right. This avoids lazy deletion completely and achieves $\mathcal{O}(N \log k)$ time and $\mathcal{O}(k)$ space.

35.6.2 Q2: How would you parallelize this for huge data streams?

- **Answer:** Since the sliding window has a dependency on the previous state, parallelizing the window updates directly is difficult. However, we can shard the array into overlapping chunks of size M (where chunks overlap by $k - 1$ elements), process each chunk on different threads/cores, and combine the results.

35.7 Pro-Tip: How to Impress the Interviewer

- **Implement with `std::multiset` in C++:** Show that you know how to use `std::multiset` to build a clean solution. The iterator updates can be tricky:

```
// Insert element, if it's less than *mid, decrement mid iterator
// Delete element, if it's less than or equal to *mid, increment mid iterator
```

This demonstrates solid understanding of C++ STL container internals and iterator stability.

- **Warp Memory Accesses:** Highlight that when running this on resource-constrained systems, lazy deletion causes heap size to grow up to N . If $N \gg k$, this wastes memory. Rebuilding the heaps or pruning aggressively when `delayed.size()` exceeds a threshold (e.g. $> 2k$) keeps memory consumption bounded to $\mathcal{O}(k)$.

36 Kth Largest Element in an Array

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
- **Source:** LeetCode 215, Glassdoor
- **Flashcards:** [Top K Elements deck](#)

36.1 Question Description

Given an integer array `nums` and an integer `k`, return the k^{th} largest element in the array.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting in $\mathcal{O}(n)$ average time complexity?

36.1.1 Examples

- **Input:** `nums = [3,2,1,5,6,4]`, `k = 2`
 – **Output:** 5
- **Input:** `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
 – **Output:** 4

36.2 Detailed Solution (C++)

There are two primary ways to solve this problem:

1. **Min-Heap Approach** ($\mathcal{O}(n \log k)$): Maintain a min-heap of size k . As we scan the array, we push elements into the heap. If the heap size exceeds k , we pop the minimum element. At the end, the top of the heap is the k^{th} largest element.
2. **Quickselect Approach** ($\mathcal{O}(n)$ average): A divide-and-conquer algorithm based on Quicksort. We select a pivot, partition the array, and only recurse into the side containing the k^{th} largest position.

In C++, `std::nth_element` uses **Introselect**, a hybrid of Quickselect and Heapsort. It guarantees $\mathcal{O}(n)$ worst-case time by switching to Heapsort if the recursion depth exceeds a threshold.

36.2.1 Standard C++ Production Code

36.2.1.1 Optimal Approach: Introselect ($\mathcal{O}(n)$ Time, $\mathcal{O}(1)$ Space)

```
#include <vector>
#include <algorithm>

class Solution {
public:
    int findKthLargest(std::vector<int>& nums, int k) {
        // Rearranges elements such that the (k-1)-th element is in its sorted position
        // std::greater<int>() sorts in descending order
        std::nth_element(nums.begin(), nums.begin() + k - 1, nums.end(), std::greater<int>());
        return nums[k - 1];
    }
};
```

36.2.1.2 Alternative Approach: Min-Heap ($\mathcal{O}(n \log k)$ Time, $\mathcal{O}(k)$ Space)

```
#include <vector>
#include <queue>

class SolutionHeap {
public:
    int findKthLargest(std::vector<int>& nums, int k) {
        std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
        for (int num : nums) {
            minHeap.push(num);
            if (minHeap.size() > k) {
                minHeap.pop();
            }
        }
        return minHeap.top();
    }
};
```

36.3 Detailed Solution (Python)

36.3.1 Standard Python Production Code (Min-Heap)

```
import heapq
from typing import List

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        """
        Finds the kth largest element using a min-heap of size k.

        Time Complexity:  $\mathcal{O}(n \log k)$ 
        Space Complexity:  $\mathcal{O}(k)$ 
        """
        # Initialize a min-heap with the first k elements
        heap = nums[:k]
        heapq.heapify(heap)

        # Iterate over the rest of the elements
        for num in nums[k:]:
            if num > heap[0]:
                heapq.heappushpop(heap, num)

        return heap[0]
```

36.4 Python-Specific Complexities & Implementation Considerations

36.4.1 1. `heapq.nlargest` Performance

- Python has a built-in helper `heapq.nlargest(k, nums)`. Under the hood, this uses a min-heap of size k and is implemented in C. While writing `heapq.nlargest(k, nums)[-1]` is clean, manually writing the loop with `heappushpop` shows standard algorithm fluency to an interviewer.

36.4.2 2. Quickselect Stack Overflow Risk

- If you implement Quickselect recursively in Python, a poor pivot choice (e.g. on already sorted arrays) can degrade the complexity to $\mathcal{O}(n^2)$ and cause a `RecursionError` due to the stack depth. An iterative implementation of Quickselect is safer in Python.

36.5 Complexity Analysis

36.5.1 Min-Heap:

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log k)$	We iterate through all n elements. Each heap insertion/extraction takes $\mathcal{O}(\log k)$.
Space Complexity	$\mathcal{O}(k)$	The size of the min-heap.

36.5.2 Quickselect / Introselect:

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$ average	The partitioning step is $\mathcal{O}(n)$, and we recurse on average into a partition of size $n/2$, yielding $T(n) = T(n/2) + \mathcal{O}(n) = \mathcal{O}(n)$.
Space Complexity	$\mathcal{O}(1)$	Done in-place. Stack depth is $\mathcal{O}(\log n)$ or $\mathcal{O}(1)$ if implemented iteratively.

36.6 Common Follow-Up Questions & How to Answer

36.6.1 Q1: When is the Min-Heap approach preferred over Quickselect?

- Answer:** The heap approach is preferred for **Data Streams** (online algorithms). If the data is too large to fit in memory or arrives incrementally, we cannot partition the array in-place. A min-heap

only needs to store k elements in memory at any point.

36.6.2 Q2: What if the range of elements is small? (e.g., values in $[-10000, 10000]$)

- **Answer:** We can use **Counting Sort**. Create a frequency array (bucket array) representing the count of each number. Scan from right to left (largest to smallest) and decrement k by the count of each bucket. The bucket where k becomes ≤ 0 contains the k^{th} largest element. This runs in $\mathcal{O}(n + R)$ time and $\mathcal{O}(R)$ space, where R is the range.
-

36.7 Pro-Tip: How to Impress the Interviewer

- **Mention Introselect vs. Quickselect:** Explain that naive Quickselect has a worst-case time complexity of $\mathcal{O}(n^2)$ (e.g. if the array is already sorted and we choose the first element as pivot). Point out that C++'s `std::nth_element` uses **Introselect**, which detects if recursion is degrading and automatically falls back to Heapsort, guaranteeing $\mathcal{O}(n)$ worst-case.
- **Median of Medians:** Mention the **Median of Medians** (BFPRT) algorithm, which chooses a pivot in $\mathcal{O}(n)$ time such that Quickselect is guaranteed to run in $\mathcal{O}(n)$ worst-case. Note that in practice, the constant factors of BFPRT are too high, so randomized pivots or Introselect are preferred.

37 Top K Frequent Elements

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 347, Glassdoor
 - **Flashcards:** [Top K Elements deck](#)
-

37.1 Question Description

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in **any order**.

37.1.1 Follow up:

- Your algorithm's time complexity must be better than $\mathcal{O}(n \log n)$, where n is the array's size.

37.1.2 Examples

- **Input:** `nums = [1,1,1,2,2,3]`, `k = 2`
 – **Output:** `[1,2]`
 - **Input:** `nums = [1]`, `k = 1`
 – **Output:** `[1]`
-

37.2 Detailed Solution (C++)

To solve this problem better than $\mathcal{O}(n \log n)$ time, we can use two distinct strategies:

1. Min-Heap Approach ($\mathcal{O}(n \log k)$ time):

- Build a frequency map of elements in $\mathcal{O}(n)$ time.
- Keep a min-heap of size k . For each unique element, push it onto the heap. If the heap size exceeds k , pop the least frequent element.
- Pop the remaining elements from the heap into our result.

2. Bucket Sort Approach ($\mathcal{O}(n)$ time & space):

- Build a frequency map.
- Create an array of lists/vectors called `buckets`, where the index represents the frequency of elements (size $n + 1$).
- Place each unique element into the bucket corresponding to its frequency.
- Traverse the buckets from right to left (highest frequency to lowest) and collect elements until we have k elements.

We present the optimal **Bucket Sort** solution as it guarantees linear time complexity.

37.2.1 Standard C++ Production Code (Bucket Sort)

```
#include <vector>
#include <unordered_map>

class Solution {
public:
    std::vector<int> topKFrequent(std::vector<int>& nums, int k) {
        // Step 1: Count frequency of each element
        std::unordered_map<int, int> freqMap;
        for (int num : nums) {
            freqMap[num]++;
        }

        int n = static_cast<int>(nums.size());
        // Step 2: Create buckets. Index represents frequency.
        // Elements with frequency 'f' will be in buckets[f]
        std::vector<std::vector<int>> buckets(n + 1);
        for (const auto& [num, count] : freqMap) {
            buckets[count].push_back(num);
        }

        // Step 3: Gather the top k frequent elements
        std::vector<int> result;
        result.reserve(k);
        for (int i = n; i >= 0 && result.size() < k; --i) {
            if (!buckets[i].empty()) {
```

```

        for (int num : buckets[i]) {
            result.push_back(num);
            if (result.size() == k) {
                break;
            }
        }
    }
}

return result;
};

```

37.3 Detailed Solution (Python)

37.3.1 Standard Python Production Code (Bucket Sort)

```

from collections import Counter
from typing import List

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        """
        Finds the k most frequent elements using Bucket Sort.

        Time Complexity: O(n)
        Space Complexity: O(n)
        """
        # Step 1: Count frequencies
        freq_map = Counter(nums)
        n = len(nums)

        # Step 2: Initialize buckets (index represents frequency)
        buckets = [[] for _ in range(n + 1)]
        for val, count in freq_map.items():
            buckets[count].append(val)

        # Step 3: Collect top k frequent elements
        result = []
        for i in range(n, 0, -1):
            for val in buckets[i]:
                result.append(val)
                if len(result) == k:

```



```
return result
```

```
return result
```

37.4 Python-Specific Complexities & Implementation Considerations

37.4.1 1. Built-in Counter.most_common

- Python's Counter class has a method `.most_common(k)`. Under the hood, this method uses `heapq.nlargest` if k is small, or sorts if k is close to the number of unique elements.
- While `Counter(nums).most_common(k)` is acceptable for simple applications, implementing the bucket sort shows structural depth in system design interviews since it achieves true $\mathcal{O}(n)$ complexity.

37.4.2 2. Space Overhead of Hash Maps and Buckets

- Python lists are dynamic arrays. Storing $n+1$ lists in the `buckets` array consumes a moderate amount of memory because empty lists still reserve pointers.
 - If memory is highly constrained, a Min-Heap approach with a size boundary of k is more space-efficient than Bucket Sort, as it limits auxiliary storage to $\mathcal{O}(u+k)$ instead of $\mathcal{O}(n)$.
-

37.5 Complexity Analysis

37.5.1 Bucket Sort:

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Counting frequencies takes $\mathcal{O}(n)$. Generating buckets takes $\mathcal{O}(u)$ where $u \leq n$. Retrieving the top k items takes $\mathcal{O}(n)$ in the worst case.
Space Complexity	$\mathcal{O}(n)$	To store the frequency map and buckets.

37.5.2 Min-Heap:

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log k)$	Counting frequencies is $\mathcal{O}(n)$. Maintaining a heap of size k for u unique elements takes $\mathcal{O}(u \log k)$.
Space Complexity	$\mathcal{O}(u+k)$	To store the frequency map and the heap.

37.6 Common Follow-Up Questions & How to Answer

37.6.1 Q1: What if the elements are streaming?

- **Answer:** In a streaming environment, Bucket Sort is impractical because the total size n keeps growing, meaning we would constantly need to resize our bucket array and re-place items. Instead, we use the **Min-Heap** approach coupled with a hash map. The hash map tracks running frequencies, and the min-heap tracks the top k elements dynamically.

37.6.2 Q2: Can we solve this using Quickselect?

- **Answer:** Yes. We count the frequencies and convert the map to a list of pairs (`element`, `frequency`). We then run Quickselect (similar to partitioning in Quicksort) targeting the $(N - k)^{\text{th}}$ index based on frequency. This averages $\mathcal{O}(n)$ time and uses $\mathcal{O}(u)$ space.
-

37.7 Pro-Tip: How to Impress the Interviewer

- **Explain Cache Locality of Buckets:** Point out that the C++ Bucket Sort uses a `std::vector<std::vector<int>>`. This vector of vectors causes **pointer chasing** because each sub-vector is allocated on different parts of the heap. To optimize cache locality, we can instead use a single flat vector for the values, along with an array of offsets to mark the start of each bucket.
- **Discuss Space-Time Trade-offs:** Emphasize that Bucket Sort is faster than Min-Heap ($\mathcal{O}(n)$ vs $\mathcal{O}(n \log k)$), but it trades off memory. If n is very large (e.g. 10^9) but there are only a few unique elements, the heap solution is vastly superior.

38 Trapping Rain Water II

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 407, Glassdoor
 - **Flashcards:** [Top K Elements deck](#)
-

38.1 Question Description

Given an $m \times n$ integer matrix `heightMap` representing the height of each unit cell in a 2D elevation map, return the volume of water it can trap after raining.

38.1.1 Examples

- **Input:** `heightMap = [[1,4,3,1,3,2],[3,2,1,3,2,4],[2,3,3,2,3,1]]`
 - **Output:** 4
 - **Explanation:** After the rain, water is trapped between the blocks. We have two small ponds of 1 and 3 units trapped. The total volume of water trapped is 4.

- **Input:** heightMap = [[3,3,3,3,3],[3,2,2,2,3],[3,2,1,2,3],[3,2,2,2,3],[3,3,3,3,3]]
 – **Output:** 10
-

38.2 Detailed Solution (C++)

To solve this in 2D, we must find the "weakest link" or the lowest point on the boundary surrounding any interior cell. The water level at any interior cell cannot exceed the maximum height of its lowest surrounding barrier.

We can model this using a **Min-Heap (Priority Queue)** and a **Breadth-First Search (BFS)** traversal starting from the boundary inwards:

1. **Initialize Boundary:** Push all border cells of the matrix into the min-heap and mark them as **visited**. These border cells represent the initial water containment boundary.
2. **Expand Inwards:** Pop the cell with the lowest height h from the heap. Because this is the lowest boundary point, water cannot spill out at any level lower than h .
3. **Inspect Neighbors:** For each of the 4 adjacent neighbors:
 - If it has not been visited, mark it as **visited**.
 - If its height is less than h , it traps water: $\text{water} += h - \text{heightMap}[\text{nr}][\text{nc}]$.
 - Push the neighbor into the heap with its effective height: $\max(h, \text{heightMap}[\text{nr}][\text{nc}])$. This is because if the neighbor was lower than h , it is now filled with water up to height h , which acts as the boundary for cells further inland.
4. **Repeat** until the heap is empty.

38.2.1 Standard C++ Production Code

```
#include <vector>
#include <queue>
#include <tuple>
#include <algorithm>

class Solution {
public:
    int trapRainWater(std::vector<std::vector<int>>& heightMap) {
        int m = static_cast<int>(heightMap.size());
        if (m < 3) return 0;
        int n = static_cast<int>(heightMap[0].size());
        if (n < 3) return 0;

        // Min-Heap containing tuples of {height, row, col}
        using Cell = std::tuple<int, int, int>;
        std::priority_queue<Cell, std::vector<Cell>, std::greater<Cell>> pq;
        std::vector<std::vector<bool>> visited(m, std::vector<bool>(n, false));

        // Add left and right borders
        for (int r = 0; r < m; ++r) {
            pq.push({heightMap[r][0], r, 0});
            visited[r][0] = true;
            pq.push({heightMap[r][n - 1], r, n - 1});
        }
```

```

        visited[r][n - 1] = true;
    }

    // Add top and bottom borders (excluding corners which are already added)
    for (int c = 1; c < n - 1; ++c) {
        pq.push({heightMap[0][c], 0, c});
        visited[0][c] = true;
        pq.push({heightMap[m - 1][c], m - 1, c});
        visited[m - 1][c] = true;
    }

    int totalWater = 0;
    const int dirs[4][2] = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

    while (!pq.empty()) {
        auto [h, r, c] = pq.top();
        pq.pop();

        for (int i = 0; i < 4; ++i) {
            int nr = r + dirs[i][0];
            int nc = c + dirs[i][1];

            if (nr >= 0 && nr < m && nc >= 0 && nc < n && !visited[nr][nc]) {
                visited[nr][nc] = true;
                // If the neighbor is lower than the current boundary height, it traps water
                if (heightMap[nr][nc] < h) {
                    totalWater += (h - heightMap[nr][nc]);
                }
                // Push the neighbor with the updated effective boundary height
                pq.push({std::max(h, heightMap[nr][nc]), nr, nc});
            }
        }
    }

    return totalWater;
}
};

```

38.3 Detailed Solution (Python)

38.3.1 Standard Python Production Code

```
import heapq
from typing import List

class Solution:
    def trapRainWater(self, heightMap: List[List[int]]) -> int:
        """
        Calculates the volume of trapped water using a priority queue.

        Time Complexity:  $O(m * n * \log(m * n))$ 
        Space Complexity:  $O(m * n)$ 
        """
        if not heightMap or not heightMap[0]:
            return 0

        m, n = len(heightMap), len(heightMap[0])
        if m < 3 or n < 3:
            return 0

        visited = [[False] * n for _ in range(m)]
        heap = []

        # Insert all boundary cells
        for r in range(m):
            for c in (0, n - 1):
                heapq.heappush(heap, (heightMap[r][c], r, c))
                visited[r][c] = True

        for c in range(1, n - 1):
            for r in (0, m - 1):
                heapq.heappush(heap, (heightMap[r][c], r, c))
                visited[r][c] = True

        water = 0
        dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        while heap:
            h, r, c = heapq.heappop(heap)

            for dr, dc in dirs:
                nr, nc = r + dr, c + dc
```

```

    if 0 <= nr < m and 0 <= nc < n and not visited[nr][nc]:
        visited[nr][nc] = True
        # If neighbor is lower, trap water
        if heightMap[nr][nc] < h:
            water += (h - heightMap[nr][nc])
        # Push effective height
        heapq.heappush(heap, (max(h, heightMap[nr][nc]), nr, nc))

return water

```

38.4 Python-Specific Complexities & Implementation Considerations

38.4.1 1. Dynamic Tuple Comparisons in Heap

- Python compares tuples element-by-element: (height, r, c). Since height is the first element, `heapq` naturally sorts cells by height. If heights are identical, it compares `r`, then `c`. This is correct and doesn't affect water trapping logic, but it means cell indices are compared to break ties.

38.4.2 2. Large Matrix Allocations

- Creating a 2D boolean array `visited = [[False] * n for _ in range(m)]` is fast. However, for extremely large grids, flat 1D arrays of size $m \times n$ (with index lookup `r * n + c`) offer better cache line locality and lower memory allocation overhead in Python.
-

38.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(m \cdot n \log(m \cdot n))$	Every cell is inserted and popped from the min-heap at most once. Heap operations take logarithmic time in the number of boundary elements, bounded by $\mathcal{O}(\log(m \cdot n))$.
Space Complexity	$\mathcal{O}(m \cdot n)$	For the <code>visited</code> matrix and priority queue.

38.6 Common Follow-Up Questions & How to Answer

38.6.1 Q1: Why must we use a Min-Heap instead of standard Breadth-First Search (BFS)?

- **Answer:** Water naturally spills from the lowest point of any surrounding boundary. Standard BFS explores nodes level-by-level without regard to height, which can lead to overestimating the water

level by exploring path heights out of order. A min-heap acts like Dijkstra's algorithm, resolving spillways in strictly increasing order of boundary height.

38.6.2 Q2: How does this relate to Trapping Rain Water I (1D version)?

- **Answer:** In the 1D version, the "boundary" is just two points: the left and right extremities. We always move the pointer with the lower height inwards. In the 2D version, the boundary is a closed loop of cells. We always pop the cell with the lowest height from this boundary and expand inwards. Thus, the 2D heap approach is the direct multidimensional generalization of the 1D two-pointer approach.
-

38.7 Pro-Tip: How to Impress the Interviewer

- **Explain Dijkstra Metaphor:** Frame the solution as finding the **minimax path** in a graph. For any cell, its water level is $\min(\text{max height of path from cell to boundary})$. The min-heap operates similarly to Dijkstra's algorithm to resolve these paths.
- **Optimize Cache Performance:** Suggest that for very large grids, instead of a heap of tuples (which creates node objects in Python or tuple overhead in C++), you can encode coordinates into a single integer $r * n + c$ to improve cache locality and memory density.

39 K Closest Points to Origin

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 973, Glassdoor
 - **Flashcards:** [Top K Elements deck](#)
-

39.1 Question Description

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the **X-Y** plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the X-Y plane is the Euclidean distance:

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

You may return the answer in **any order**. The answer is guaranteed to be unique (except for the order of the points).

39.1.1 Examples

- **Input:** $\text{points} = [[1, 3], [-2, 2]]$, $k = 1$
 – **Output:** $[[-2, 2]]$

– **Explanation:**

- * The distance between $(1, 3)$ and the origin is $\sqrt{1^2 + 3^2} = \sqrt{10}$.
- * The distance between $(-2, 2)$ and the origin is $\sqrt{(-2)^2 + 2^2} = \sqrt{8}$.
- * Since $\sqrt{8} < \sqrt{10}$, $(-2, 2)$ is closer to the origin.
- * We want the closest $k = 1$ points, so we return $[[-2, 2]]$.

- **Input:** `points = [[3,3],[5,-1],[-2,4]]`, `k = 2`

– **Output:** `[[3,3],[-2,4]]` (or `[-2,4],[3,3]`)

39.2 Detailed Solution (C++)

To find the k closest points, we can use two primary approaches: 1. **Max-Heap of size k** ($\mathcal{O}(n \log k)$): Push elements into a max-heap of size k . Once size exceeds k , pop the furthest element. The remaining elements are the k closest. 2. **Quickselect / Introselect** ($\mathcal{O}(n)$ **average**): Since we do not need to sort the output, we can partition the array of points using Quickselect. In C++, `std::nth_element` partitions the vector in-place such that the first k elements are the closest to the origin. This runs in $\mathcal{O}(n)$ time.

To avoid slow floating-point square root operations, we compare the **squared distance** $x^2 + y^2$.

39.2.1 Standard C++ Production Code

39.2.1.1 Optimal Approach: Introselect ($\mathcal{O}(n)$ Time, $\mathcal{O}(1)$ Space)

```
#include <vector>
#include <algorithm>

class Solution {
public:
    std::vector<std::vector<int>>> kClosest(std::vector<std::vector<int>>>& points, int k) {
        // Use std::nth_element to partition the top k closest elements to the front
        std::nth_element(points.begin(), points.begin() + k - 1, points.end(),
            [](const std::vector<int>& a, const std::vector<int>& b) {
                return (a[0] * a[0] + a[1] * a[1]) < (b[0] * b[0] + b[1] * b[1]);
            })
    };

    // Resize points vector to only return the first k elements
    points.resize(k);
    return points;
};
```

39.2.1.2 Alternative Approach: Max-Heap ($\mathcal{O}(n \log k)$ Time, $\mathcal{O}(k)$ Space)

```
#include <vector>
#include <queue>
```



```
#include <utility>

class SolutionHeap {
public:
    std::vector<std::vector<int>> kClosest(std::vector<std::vector<int>>& points, int k) {
        // Max-heap stores pair of <squared_distance, point_index>
        std::priority_queue<std::pair<int, int>> maxHeap;
        for (size_t i = 0; i < points.size(); ++i) {
            int dist = points[i][0] * points[i][0] + points[i][1] * points[i][1];
            maxHeap.push({dist, static_cast<int>(i)});
            if (maxHeap.size() > static_cast<size_t>(k)) {
                maxHeap.pop();
            }
        }

        std::vector<std::vector<int>> result;
        result.reserve(k);
        while (!maxHeap.empty()) {
            result.push_back(points[maxHeap.top().second]);
            maxHeap.pop();
        }
        return result;
    }
};
```

39.3 Detailed Solution (Python)

39.3.1 Standard Python Production Code (Max-Heap)

```
import heapq
from typing import List

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        """
        Finds the k closest points using a Max-Heap of size k.

        Time Complexity: O(n log k)
        Space Complexity: O(k)
        """
        heap = []

        for p in points:
```

```

# We negate the distance to turn heapq (Min-Heap) into a Max-Heap
dist = -(p[0]*p[0] + p[1]*p[1])
heapq.heappush(heap, (dist, p))

# If heap size exceeds k, pop the largest (furthest) distance
if len(heap) > k:
    heapq.heappop(heap)

# Return only the points from the heap
return [point for (_, point) in heap]

```

39.4 Python-Specific Complexities & Implementation Considerations

39.4.1 1. Simulating Max-Heap with Negation

- Because Python's `heapq` is a Min-Heap, we negate the squared distance (`-dist`) so that the furthest point (largest absolute distance) has the smallest value in the heap. Popping from the heap thus removes the furthest point, keeping the closest points.

39.4.2 2. Built-in `heapq.nsmallest`

- Python offers a `heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)` utility. It is implemented in C and runs very efficiently. However, in an interview, implementing the heap boundary check manually demonstrates an understanding of heap invariants.
-

39.5 Complexity Analysis

39.5.1 Quickselect (C++):

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$ average	Partitioning takes $\mathcal{O}(n)$ at each step, and we recurse into a partition of size $n/2$ on average, giving $\mathcal{O}(n)$ average time.
Space Complexity	$\mathcal{O}(1)$	Done in-place without auxiliary containers.

39.5.2 Max-Heap (Python):

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log k)$	Scanning all n points and performing heap push/pop operations of size k takes $\mathcal{O}(n \log k)$ time.
Space Complexity	$\mathcal{O}(k)$	Auxiliary space required to store k elements in the heap.

39.6 Common Follow-Up Questions & How to Answer

39.6.1 Q1: Why do we avoid the `sqrt` function?

- **Answer:** The square root function is monotonic, meaning if $a < b$, then $\sqrt{a} < \sqrt{b}$. Thus, comparing $x^2 + y^2$ preserves the relative distance ordering. Computing `sqrt` requires floating-point operations which are slower, susceptible to precision/rounding issues, and unnecessary since coordinates are integers.

39.6.2 Q2: What if we have a constant stream of new points?

- **Answer:** If new points are generated continuously, Quickselect is unusable because it is an offline algorithm requiring all data in memory. In this scenario, we must use the **Max-Heap** approach, storing the current k closest points in a heap and comparing new points against the heap's top.

39.7 Pro-Tip: How to Impress the Interviewer

- **Spatial Indexing (K-D Tree):** Mention that if we need to query the k closest points multiple times on a static or slowly changing set of points, we should pre-process the points into a **k-d tree** (a space-partitioning BST for points in K dimensions). This allows nearest-neighbor queries to run in $\mathcal{O}(\log n)$ time.
- **Cache Friendliness of Flat Layouts:** Point out that points stored as a `std::vector<std::vector<int>>` involve two levels of pointers, which reduces cache efficiency due to memory indirection. Storing coordinates in a flat array `std::vector<int>` where point i is at indices $2*i$ and $2*i+1$ keeps coordinates contiguous in RAM, maximizing CPU cache lines.

40 Implement Rand10() Using Rand7()

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
- **Source:** LeetCode 470, Glassdoor
- **Flashcards:** [Randomized deck](#)

40.1 Question Description

Given the API `rand7()` that generates a uniform random integer in the range $[1, 7]$, write a function `rand10()` that generates a uniform random integer in the range $[1, 10]$. You can only call the API `rand7()`, and you shouldn't call any other API. Please do not use a language's built-in random API.

Each test case will have one internal argument `n`, the number of times that your implemented function `rand10()` will be called while testing. Note that this is not an argument passed to `rand10()`.

40.1.1 Follow up:

- What is the expected value for the number of calls to `rand7()`?
- Can you minimize the number of calls to `rand7()`?

40.1.2 Examples

- **Input:** `n = 1`
– **Output:** `[2]`
 - **Input:** `n = 2`
– **Output:** `[8, 2]`
-

40.2 Detailed Solution (C++)

To generate a range larger than 7, we can call `rand7()` twice. This can be conceptualized as creating a 2D coordinate grid of size 7×7 :

$$\text{idx} = (\text{row} - 1) \times 7 + \text{col}$$

where `row = rand7()` and `col = rand7()`.

This yields a uniform distribution of integers in the range $[1, 49]$. Since we need a range of 10, we can take the largest multiple of 10 that is less than or equal to 49, which is 40. * If `idx ≤ 40`, we map it uniformly to $[1, 10]$ by returning `(idx - 1) % 10 + 1`. * If `idx > 40`, we reject the sample and try again. This technique is called **Rejection Sampling**.

40.2.1 Standard C++ Production Code

```
// The rand7() API is pre-defined.
// int rand7();

class Solution {
public:
    int rand10() {
        int row, col, idx;
        do {
            row = rand7();
            col = rand7();
            idx = (row - 1) * 7 + col; // Range [1, 49]
        } while (idx > 40);
        return (idx - 1) % 10 + 1;
    }
};
```

```

    } while (idx > 40); // Reject [41, 49]

    return (idx - 1) % 10 + 1;
}
};

```

40.3 Detailed Solution (Python)

40.3.1 Standard Python Production Code

```

# The rand7() API is already defined for you.
# def rand7():
#     @return a random integer in the range 1 to 7

```

class Solution:

```

    def rand10(self) -> int:
        """
        Generates a random integer in range [1, 10] using rand7().

        Time Complexity: O(1) average (rejection sampling)
        Space Complexity: O(1)
        """
        while True:
            row = rand7()
            col = rand7()
            idx = (row - 1) * 7 + col # Generates uniform [1, 49]

            if idx <= 40:
                return (idx - 1) % 10 + 1

```

40.4 Python-Specific Complexities & Implementation Considerations

40.4.1 1. Loop Termination & Infinite Loops

- Since rejection sampling can theoretically run forever (though the probability of infinite looping is 0), Python has no recursion limit issues here because we use an iterative `while True` loop rather than recursive calls. Always prefer iterative loops for rejection sampling.

40.4.2 2. Random Number Generator Seeding

- In Python, random testing is often mocked by setting seeds. When testing your solution locally, do not import the `random` module or write custom randomizers inside the production class, as LeetCode relies on a predefined `rand7()` implementation.

40.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(1)$ average	The probability of acceptance is $40/49 \approx 0.816$. The expected number of loops is $1/0.816 \approx 1.225$. In the worst-case, it is $\mathcal{O}(\infty)$ but practically constant.
Space Complexity	$\mathcal{O}(1)$	Constant auxiliary variables are maintained.

40.6 Common Follow-Up Questions & How to Answer

40.6.1 Q1: What is the expected number of calls to `rand7()` in the basic implementation?

- **Answer:** Each loop iteration makes exactly 2 calls to `rand7()`. The success rate of each iteration is $40/49$. Using the geometric distribution, the expected number of iterations is $\frac{1}{p} = \frac{49}{40} = 1.225$.
– Thus, the expected number of calls to `rand7()` is $2 \times 1.225 = 2.45$ calls.

40.6.2 Q2: How can we optimize this to minimize the expected number of calls?

- **Answer:** Instead of discarding the values from 41 to 49, we can reuse them!
 1. If we get `idx > 40`, we have a number in $[41, 49]$. Subtract 40 to get a number in $[1, 9]$ (call this val_1).
 2. Generate a new `col = rand7()`. Combine them: $idx_2 = (val_1 - 1) \times 7 + col$. This gives a range $[1, 63]$.
 3. Accept $[1, 60]$ (returning $(idx_2 - 1) \% 10 + 1$). If $idx_2 > 60$, we have a number in $[61, 63]$.
 4. Subtract 60 to get a number in $[1, 3]$ (call this val_2).
 5. Generate a new `col = rand7()`. Combine: $idx_3 = (val_2 - 1) \times 7 + col$. This gives a range $[1, 21]$.
 6. Accept $[1, 20]$. If $idx_3 = 21$, we have 1 left. Since we cannot combine a range of 1 with `rand7()` to get ≥ 10 , we start over.
 – This reduces the expected number of calls to 2.193!

40.7 Pro-Tip: How to Impress the Interviewer

- **Explain Shannon Entropy:** Mention that generating a number in $[1, 10]$ requires at least $\log_2(10) \approx 3.32$ bits of entropy. Each call to `rand7()` provides $\log_2(7) \approx 2.8$ bits of entropy. Therefore, we need at least $3.32/2.8 \approx 1.18$ calls to `rand7()`. Our optimized rejection sampling method (2.19 calls) is extremely close to the theoretical thermodynamic limit.

- **Mention Hardware RNG Stalls:** Discuss that in systems programming, random number APIs (like `/dev/urandom` or hardware `RDRAND` instructions) can block or stall the CPU pipeline if the entropy pool is depleted. Rejection sampling strategies must balance mathematical accuracy with execution latency.

41 Random Flip Matrix

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 519, Glassdoor
 - **Flashcards:** [Randomized deck](#)
-

41.1 Question Description

There is an $m \times n$ binary grid matrix with all the values set to 0 initially. Design an algorithm to randomly pick an index (i, j) where `matrix[i][j] == 0` and flip it to 1. All the indices (i, j) where `matrix[i][j] == 0` should be equally likely to be returned.

Optimize your algorithm to minimize the number of calls made to the built-in random function of your language and optimize the time and space complexity.

Implement the `Solution` class: * `Solution(int m, int n)` Initializes the object with the size of the binary matrix m and n . * `int[] flip()` Returns a random index $[i, j]$ of the matrix where `matrix[i][j] == 0` and flips it to 1. * `void reset()` Resets all the values of the matrix to 0.

41.1.1 Constraints

- $1 \leq m, n \leq 10^4$
- There will be at least one free cell for each call to `flip`.
- At most 1000 calls will be made to `flip` and `reset`.

41.1.2 Examples

- **Input:**

```
["Solution", "flip", "flip", "flip", "reset", "flip"]
[[3, 1], [], [], [], [], [], []]
```

- **Output:** `[null, [1, 0], [2, 0], [0, 0], null, [2, 0]]`

- **Explanation:**

```
Solution solution(3, 1);
solution.flip(); // returns [1, 0]
solution.flip(); // returns [2, 0]
solution.flip(); // returns [0, 0] (no other 0-cells left)
```

```
solution.reset(); // all cells reset to 0
solution.flip(); // returns [2, 0] (randomly chosen again)
```

41.2 Detailed Solution (C++)

A naive approach would be to store all $m \times n$ elements in an array and run a standard **Fisher-Yates Shuffle**. However, the matrix size can be up to $10^4 \times 10^4 = 10^8$ cells, requiring ≈ 400 MB of memory, which exceeds LeetCode's memory limit.

Since at most 1000 calls will be made to `flip`, we can simulate the Fisher-Yates shuffle using a **Hash Map** to lazily track swapped elements.

41.2.1 Lazy Fisher-Yates Logic

1. We represent the matrix cells as virtual indices in the range $[0, \text{total} - 1]$ where $\text{total} = m \times n$.
2. In each `flip()` call:
 - Pick a random virtual index r in the range $[0, \text{total} - 1]$.
 - Determine the actual value represented by r . If r is a key in `mapping`, we use `mapping[r]`; otherwise, the actual value is just r . Let's call this x .
 - We must swap the selected value with the value at the end of our active selection range ($\text{total} - 1$).
 - Find the value at $\text{total} - 1$: if $\text{total} - 1$ is in `mapping`, it is `mapping[total - 1]`; otherwise, it is $\text{total} - 1$. Let's call this `lastVal`.
 - Update `mapping[r] = lastVal`. This moves the last element to position r .
 - Delete the entry for $\text{total} - 1$ from `mapping` if it exists (since that index is no longer accessible).
 - Decrement `total` by 1.
 - Return the coordinate $[x / n, x \% n]$.
3. `reset()` simply clears the hash map and sets `total = m * n`.

41.2.2 Standard C++ Production Code

```
#include <vector>
#include <unordered_map>
#include <cstdlib>

class Solution {
private:
    int m;
    int n;
    int total;
    std::unordered_map<int, int> mapping;

public:
    Solution(int m, int n) : m(m), n(n), total(m * n) {}
```



```

std::vector<int> flip() {
    // Choose a random index in the range [0, total - 1]
    int r = std::rand() % total;

    // If r has been mapped/swapped, get its mapped value; otherwise, it is r itself
    int x = mapping.count(r) ? mapping[r] : r;

    // Get the value representing the last position in the active range
    int lastIdx = total - 1;
    int lastVal = mapping.count(lastIdx) ? mapping[lastIdx] : lastIdx;

    // Swap: place the last value at index r
    mapping[r] = lastVal;

    // Clean up the map by erasing the last index to prevent memory bloat
    mapping.erase(lastIdx);

    // Decrement the active size of the selection pool
    total--;

    return {x / n, x % n};
}

void reset() {
    mapping.clear();
    total = m * n;
}
};

```

41.3 Detailed Solution (Python)

41.3.1 Standard Python Production Code

```

import random
from typing import List, Dict

class Solution:
    def __init__(self, m: int, n: int):
        self.m = m
        self.n = n
        self.total = m * n
        self.mapping: Dict[int, int] = {}

```

```

def flip(self) -> List[int]:
    """
    Flips a random 0 cell to 1.

    Time Complexity: O(1)
    Space Complexity: O(K) where K is the number of flips
    """
    # Pick a random virtual index
    r = random.randint(0, self.total - 1)

    # Resolve the actual value of the chosen cell
    x = self.mapping.get(r, r)

    # Resolve the value at the last active index
    last_idx = self.total - 1
    last_val = self.mapping.get(last_idx, last_idx)

    # Place the last element at the chosen position r
    self.mapping[r] = last_val

    # Erase the last index from mapping to conserve memory
    if last_idx in self.mapping:
        del self.mapping[last_idx]

    self.total -= 1

    return [x // self.n, x % self.n]

def reset(self) -> None:
    """
    Resets the matrix. Time complexity: O(1)
    """
    self.mapping.clear()
    self.total = self.m * self.n

```

41.4 Python-Specific Complexities & Implementation Considerations

41.4.1 1. Modulo and Division

- In Python, you compute row and column using `x // self.n` (floor division) and `x % self.n` (modulo). Make sure to use integer floor division `//` so that coordinates are returned as integer types rather than floating-point values.

41.4.2 2. Random Integers using `randint`

- In Python, `random.randint(a, b)` generates a random integer in the **inclusive** range $[a, b]$. Therefore, we pass `0` and `self.total - 1` as arguments. Passing `self.total` would result in an off-by-one index out-of-bounds error.

41.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity (flip)	$\mathcal{O}(1)$	Hash map lookups, insertions, and deletions take constant time on average.
Time Complexity (reset)	$\mathcal{O}(1)$	Clearing the map takes $\mathcal{O}(1)$ time.
Space Complexity	$\mathcal{O}(k)$	Where k is the number of calls to <code>flip()</code> . The map size never exceeds k , which is bounded by 1000.

41.6 Common Follow-Up Questions & How to Answer

41.6.1 Q1: Why is this better than using a set to track flipped indices?

- **Answer:** If we use a set (e.g. `unordered_set`) to store flipped coordinates, we have to generate random numbers and retry if they are already in the set (Rejection Sampling).
 - As the matrix fills up (e.g., if we flip 99% of the cells), the collision rate spikes. The number of retries required to find an unflipped cell grows exponentially, leading to non-deterministic execution time.
 - The Fisher-Yates map-swap method guarantees that `flip()` completes in exactly $\mathcal{O}(1)$ time with zero retries.

41.6.2 Q2: What happens if $m \times n$ is larger than standard integer limits (e.g. 10^{18})?

- **Answer:** In Python, integers have arbitrary precision, so no overflow occurs. In C++, 10^{18} requires using `long long` for index calculations (`total` and `r`) to prevent overflow bugs. The hash map keys and values must also be declared as `long long`.

41.7 Pro-Tip: How to Impress the Interviewer

- **Highlight Memory Footprint Reduction:** Show that you explicitly clean up the map by erasing keys that fall outside the active selection boundary (`mapping.erase(lastIdx)`). Failing to do so keeps unused keys in memory, causing unnecessary heap overhead.

- **Random Seed Reproducibility:** Discuss how setting a fixed random seed (reproducible testing) is essential when debugging randomized algorithms in production systems, and how to implement mocking for randomness in unit tests.

42 Number of 1 Bits

- **Category:** Coding
 - **Difficulty:** Easy
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 191, Glassdoor
 - **Flashcards:** [Bit Manipulation deck](#)
-

42.1 Question Description

Given a positive integer n , write a function that returns the number of set bits (also known as the **Hamming weight** or **population count**) in its binary representation.

42.1.1 Examples

- **Input:** $n = 11$ (Binary: `1011`)
 - **Output:** 3
 - **Explanation:** The binary representation has three set bits.
- **Input:** $n = 128$ (Binary: `10000000`)
 - **Output:** 1
 - **Explanation:** The binary representation has one set bit.
- **Input:** $n = 2147483645$ (Binary: `1111111111111111111111111111101`)
 - **Output:** 30

42.1.2 Constraints

- $1 \leq n \leq 2^{31} - 1$
-

42.2 Detailed Solution (C++)

There are three main ways to compute the Hamming weight: 1. **Bit-by-Bit Shift:** Check each of the 32 bits one by one. This takes $\mathcal{O}(32)$ iterations regardless of the number of set bits. 2. **Brian Kernighan's Algorithm:** The expression $n \& (n - 1)$ clears the lowest set bit (rightmost 1 bit) of n . For example: $* n = 12$ (`1100`) $* n - 1 = 11$ (`1011`) $* n \& (n - 1) = 8$ (`1000`)

By looping and executing `n &= n - 1` until `n` becomes `0`, we only iterate as many times as there are `1` bits in

3. **Built-in POPCNT instruction:** Modern CPUs have a dedicated hardware instruction (POPCNT) to count bits in a single CPU cycle. We can access this in C++ via compilers intrinsics (`__builtin_popcount`).

42.2.1 Standard C++ Production Code

```
#include <stdint>

class Solution {
public:
    int hammingWeight(uint32_t n) noexcept {
        int count = 0;

        // Brian Kernighan's Algorithm
        while (n != 0) {
            n &= (n - 1); // Clears the lowest set bit
            count++;
        }

        return count;
    }

    // Alternative: Hardware Accelerated Popcount
    int hammingWeightHardware(uint32_t n) noexcept {
        return __builtin_popcount(n);
    }
};
```

42.3 Detailed Solution (Python)

42.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using Brian Kernighan's algorithm.

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        """
        Counts the number of set bits in an integer.

        Time Complexity: O(k) where k is the number of set bits (at most 32).
        Space Complexity: O(1)
        """
        count = 0
        while n:
            n &= (n - 1) # Clear the lowest set bit
            count += 1
        return count

    def hammingWeightBuiltin(self, n: int) -> int:
```

```

"""
Alternative: Python 3.10+ native bit_count() method, which is implemented
in C using hardware POPCNT if available.
"""
return n.bit_count()

```

42.4 Python-Specific Complexities & Implementation Considerations

42.4.1 1. Python 3.10+ `int.bit_count()`

- Starting in Python 3.10, the `int` class has a native `bit_count()` method that returns the number of set bits.
- For older versions of Python, a common shorthand is `bin(n).count('1')`. However, `bin(n)` produces an intermediate string representation of the binary digits (e.g. `'0b1011'`), which allocates heap memory and is much slower than bitwise operations.

42.4.2 2. Arbitrary Precision and Sign Bit

- In Python, integers have arbitrary precision. A negative integer in Python (like `-1`) is represented under-the-hood with infinite 2s complement sign-extension (infinite sequence of leading 1 bits). Thus, checking `hammingWeight` of a negative number in Python can lead to infinite loops unless masked (e.g., `n = n & 0xffffffff`).
-

42.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(k)$	Loop runs exactly k times, where k is the number of set bits (Hamming weight). Worst case is $\mathcal{O}(32)$ for 32-bit integers.
Space Complexity	$\mathcal{O}(1)$	Uses constant memory.

42.6 Common Follow-Up Questions & How to Answer

42.6.1 Q1: What is the complexity if you need to run this on a stream of 64-bit integers?

- Answer:** If we need to process integers repeatedly, Brian Kernighan's algorithm can be replaced with a **lookup table (LUT)**.
 - We can precompute the Hamming weight of all 8-bit integers (256 values) and store them in a static array.

- For any 64-bit integer, we split it into eight 8-bit chunks and look up their values:

$$\text{count} = \text{LUT}[n \& 0xff] + \text{LUT}[(n \gg 8) \& 0xff] + \dots$$

- This runs in $\mathcal{O}(1)$ operations (exactly 8 lookups and shifts).

42.6.2 Q2: How can we implement bit count using parallel bit summation (SWAR - SIMD Within A Register)?

- **Answer:** SWAR counts bits by performing divide-and-conquer additions on adjacent bit segments in parallel:

```
uint32_t countBits(uint32_t i) {
    i = i - ((i >> 1) & 0x55555555); // count pairs
    i = (i & 0x33333333) + ((i >> 2) & 0x33333333); // count nibbles
    i = (i + (i >> 4)) & 0x0F0F0F0F; // count bytes
    i = i + (i >> 8);
    i = i + (i >> 16);
    return i & 0x0000003F;
}
```

42.7 Pro-Tip: How to Impress the Interviewer

- **Demonstrate Low-Level/Hardware Knowledge:** Mention the `POPCNT` assembly instruction on x86/x64 and `LD / CNT` on ARM. Explain how compilers translate `__builtin_popcount` directly into a single hardware instruction when compiling with optimization flags like `-march=native`.
- **Mention Sign Extension Pitfalls:** Point out how negative numbers are handled differently in languages with fixed-width types (C++'s `uint32_t` handles this cleanly via unsigned wrap-around) versus Python's infinite-precision integers, showing full system-level awareness.

43 Single Number

- **Category:** Coding
 - **Difficulty:** Easy
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 136, Glassdoor
 - **Flashcards:** [Bit Manipulation deck](#)
-

43.1 Question Description

Given a **non-empty** array of integers `nums`, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

43.1.1 Examples

- **Input:** `nums = [2,2,1]`
– **Output:** 1
- **Input:** `nums = [4,1,2,1,2]`
– **Output:** 4
- **Input:** `nums = [1]`
– **Output:** 1

43.1.2 Constraints

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-3 \times 10^4 \leq \text{nums}[i] \leq 3 \times 10^4$
- Each element in the array appears twice except for one element which appears only once.

43.2 Detailed Solution (C++)

To solve this in $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ space, we use the properties of the **bitwise XOR (^) operator**:

1. **Identity Property:** $A \oplus 0 = A$ (XORing any number with 0 remains unchanged) 2. **Self-Inverse Property:** $A \oplus A = 0$ (XORing a number with itself cancels out and results in 0) 3. **Commutative and Associative Properties:** The order in which we XOR the elements does not matter.

If we XOR all the elements in the array:

$$\text{result} = \text{nums}[0] \oplus \text{nums}[1] \oplus \dots \oplus \text{nums}[n-1]$$

Since every number except one appears exactly twice, all pairs will cancel each other out to 0. We are left with:

$$\text{result} = 0 \oplus \text{single_number} = \text{single_number}$$

43.2.1 Standard C++ Production Code

```
#include <vector>

class Solution {
public:
    int singleNumber(const std::vector<int>& nums) noexcept {
        int result = 0;
        for (int num : nums) {
            result ^= num;
        }
        return result;
    }
};
```


43.3 Detailed Solution (Python)

43.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It uses `functools.reduce` for a highly Pythonic, single-line alternative, alongside the standard iterative loop.

```
from typing import List
from functools import reduce
import operator

class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        """
        Finds the unique number in a list where all other elements appear twice.

        Time Complexity: O(N)
        Space Complexity: O(1)
        """
        result = 0
        for num in nums:
            result ^= num
        return result

    def singleNumberPythonic(self, nums: List[int]) -> int:
        """
        Highly optimized Pythonic alternative using reduce.
        """
        return reduce(operator.xor, nums)
```

43.4 Python-Specific Complexities & Implementation Considerations

43.4.1 1. The `reduce` and `operator` Shorthand

- Using `reduce(operator.xor, nums)` avoids the interpreter loop overhead by delegating the iteration and XOR operations to C code inside the Python runtime. This is generally faster than a standard Python `for` loop.

43.4.2 2. Large Integer Bit Lengths

- Since Python integers are arbitrary-precision, XOR operations do not overflow. However, in low-level languages, if numbers exceed standard 32-bit limits, proper type declaration (e.g. `long long` in C++) is necessary.

43.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We iterate through the array of n numbers exactly once.
Space Complexity	$\mathcal{O}(1)$	Only a single accumulator variable is stored.

43.6 Common Follow-Up Questions & How to Answer

43.6.1 Q1: What if all other numbers appear three times instead of twice? (LeetCode 137)

- **Answer:** If numbers appear three times, XORing them will not cancel them out.
 - **Solution:** We can count the number of set bits at each of the 32 bit positions. If the sum of set bits at a position is not divisible by 3, the single number must have a 1 at that position.
 - **Code (C++):**

```
int singleNumberThree(vector<int>& nums) {
    int ans = 0;
    for (int i = 0; i < 32; ++i) {
        int sum = 0;
        for (int num : nums) {
            if ((num >> i) & 1) sum++;
        }
        if (sum % 3 != 0) {
            ans |= (1 << i);
        }
    }
    return ans;
}
```

43.6.2 Q2: What if there are two single numbers instead of one? (LeetCode 260)

- **Answer:** If there are two single numbers x and y , XORing all elements yields $xor_val = x \oplus y$. Since $x \neq y$, xor_val must have at least one set bit.
 - **Solution:**
 1. Find the rightmost set bit of xor_val using $diff = xor_val \& -xor_val$.
 2. Divide all numbers in the array into two groups: those that have this bit set and those that don't.
 3. XORing all numbers in each group independently will cancel out the duplicates, leaving x in one group and y in the other.

43.7 Pro-Tip: How to Impress the Interviewer

- **Discuss Hardware-Level Execution:** Explain that XOR is a single-cycle bitwise operation executed directly in the CPU's ALU. Because it does not use branch pointers, it has no risk of branch misprediction stalls, making it extremely fast.
- **Mention Loop Unrolling:** For high-performance graphics or AI system code, mentioning that compilers can unroll the loop to execute multiple XORs concurrently (exploiting ILP - Instruction-Level Parallelism) shows top-tier optimization skills.

44 Counting Bits

- **Category:** Coding
 - **Difficulty:** Easy
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 338, Glassdoor
 - **Flashcards:** [Bit Manipulation deck](#)
-

44.1 Question Description

Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the **number of 1's** in the binary representation of i .

You must implement a solution with linear $\mathcal{O}(n)$ runtime complexity and perform it in a single pass without using any built-in functions.

44.1.1 Examples

- **Input:** $n = 2$
 - **Output:** `[0,1,1]`
 - **Explanation:**
 - * $0 \rightarrow 0$ (0 set bits)
 - * $1 \rightarrow 1$ (1 set bit)
 - * $2 \rightarrow 10$ (1 set bit)
- **Input:** $n = 5$
 - **Output:** `[0,1,1,2,1,2]`
 - **Explanation:**
 - * $0 \rightarrow 000$ (0 set bits)
 - * $1 \rightarrow 001$ (1 set bit)
 - * $2 \rightarrow 010$ (1 set bit)
 - * $3 \rightarrow 011$ (2 set bits)
 - * $4 \rightarrow 100$ (1 set bit)
 - * $5 \rightarrow 101$ (2 set bits)

44.1.2 Constraints

- $0 \leq n \leq 10^5$
-

44.2 Detailed Solution (C++)

A naive approach counts the bits for each number independently, which takes $\mathcal{O}(n \log n)$ time. To achieve $\mathcal{O}(n)$ time, we use **Dynamic Programming** combined with **Bit Manipulation**. We can derive the count of 1s for i using previously computed subproblems:

44.2.1 Approach 1: Most Significant Bit (MSB) / Shift Property

The binary representation of i is identical to $i \gg 1$ (which is $i / 2$) shifted right by 1 bit, plus the value of the least significant bit ($i \& 1$).

$$\text{ans}[i] = \text{ans}[i \gg 1] + (i \& 1)$$

Since $i \gg 1 < i$, the value for $\text{ans}[i \gg 1]$ is guaranteed to have already been computed.

44.2.2 Approach 2: Least Significant Bit (LSB) / Brian Kernighan clearing

We know that $i \& (i - 1)$ clears the lowest set bit of i . Thus, the number of set bits in i is exactly 1 more than the number of set bits in the resulting value:

$$\text{ans}[i] = \text{ans}[i \& (i - 1)] + 1$$

Both approaches yield a highly optimized, single-pass $\mathcal{O}(n)$ runtime.

44.2.3 Standard C++ Production Code

```
#include <vector>

class Solution {
public:
    std::vector<int> countBits(int n) {
        // Initialize the output array with zeros
        std::vector<int> ans(n + 1, 0);

        for (int i = 1; i <= n; ++i) {
            // ans[i] = count of (i shifted right by 1) + 1 if i is odd (least significant bit is 1)
            ans[i] = ans[i >> 1] + (i & 1);
        }

        return ans;
    }
}
```

```
// Alternative DP Transition using Brian Kernighan's bit clearing
std::vector<int> countBitsAlternative(int n) {
    std::vector<int> ans(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        ans[i] = ans[i & (i - 1)] + 1;
    }
    return ans;
}
};
```

44.3 Detailed Solution (Python)

44.3.1 Standard Python Production Code

Below is the iterative, type-hinted Python implementation using the shift property.

```
from typing import List

class Solution:
    def countBits(self, n: int) -> List[int]:
        """
        Generates bit count array for numbers 0 to n in O(N) time.

        Time Complexity: O(N)
        Space Complexity: O(1) auxiliary space (excluding result array).
        """
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i >> 1] + (i & 1)
        return ans
```

44.4 Python-Specific Complexities & Implementation Considerations

44.4.1 1. Integer Bitwise Shift

- Python handles bitwise operations natively in C. The operator `i >> 1` is significantly faster than division `i // 2` because bit-shifting is processed directly by the CPU ALU, avoiding floating-point division conversion.

44.4.2 2. Pre-allocation of List Memory

- In Python, constructing the list using `ans = [0] * (n + 1)` allocates a contiguous array block at the start. Doing `ans.append(...)` inside the loop causes dynamic list resizing, which triggers heap reallocation and copying of elements, slowing down runtime performance.

44.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Every integer from 1 to n is processed exactly once in constant $\mathcal{O}(1)$ time.
Space Complexity	$\mathcal{O}(1)$	Auxiliary space is constant (the $\mathcal{O}(n)$ space is only for the returned answer array).

44.6 Common Follow-Up Questions & How to Answer

44.6.1 Q1: Can you solve this using a divide and conquer technique?

- **Answer:** Yes, the numbers of set bits follow a recursive pattern. For any power of 2 (let's say 2^k), the sequence of set bits for $[2^k, 2^{k+1} - 1]$ is identical to the sequence of $[0, 2^k - 1]$ with each element incremented by 1 (representing the leading 2^k bit).
- We can build the array by copying the previously computed half and adding 1 to each element.

44.6.2 Q2: How would you parallelize this for extremely large N (e.g. $N = 10^9$)?

- **Answer:** Since the DP state `ans[i]` depends on `ans[i >> 1]`, there is a data dependency. However, since the dependency only goes from lower indices to higher indices, we can chunk the computation.
 - Alternatively, we can eliminate the DP dependency entirely: each thread can compute its number's popcount independently using the hardware `POPCNT` instruction (via `std::experimental::simd` or OpenMP parallel loops). This is highly parallelizable and executes in parallel $\mathcal{O}(N/P)$ time where P is the number of cores.

44.7 Pro-Tip: How to Impress the Interviewer

- **Contrast DP Relations:** Discuss the difference between `ans[i >> 1] + (i & 1)` and `ans[i & (i - 1)] + 1`. Point out that `ans[i & (i - 1)] + 1` is slightly more intuitive as it directly counts the number of operations required by Brian Kernighan's algorithm.
- **Explain Compiler Optimizations:** Explain that compilers can optimize bit shifts (`i >> 1`) and bitwise ANDs (`i & 1`) into single-cycle CPU instructions (`SHR` and `AND`). Pointing this out shows you write code with compiler optimization pathways in mind.

45 Number Complement

- **Category:** Coding

- **Difficulty:** Easy
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 476, LeetCode 1009, Glassdoor
 - **Flashcards:** [Bit Manipulation deck](#)
-

45.1 Question Description

The **complement** of an integer is the integer you get when you flip all the 0's to 1's and all the 1's to 0's in its binary representation.

For example, the integer 5 is **101** in binary (with no leading zero bits), and its complement is **010** which is 2 in decimal.

Given an integer `num`, return *its complement*.

45.1.1 Examples

- **Input:** `num = 5`
 - **Output:** 2
 - **Explanation:** The binary representation of 5 is **101** (no leading zero bits), and its complement is **010**. So you need to output 2.
- **Input:** `num = 1`
 - **Output:** 0
 - **Explanation:** The binary representation of 1 is **1** (no leading zero bits), and its complement is **0**. So you need to output 0.

45.1.2 Constraints

- $1 \leq \text{num} < 2^{31}$
-

45.2 Detailed Solution (C++)

To find the complement of `num` (excluding leading zeros), we want to flip only the bits that are part of the active binary representation of `num`.

1. **Determine Bit Length (L):** We count how many bits are needed to represent `num`. For example, 5 (**101**) needs 3 bits.
2. **Generate a Mask of L Ones:** A mask of L ones is created via:

$$\text{mask} = (1 \ll L) - 1$$

For $L = 3$, $(1 \ll 3) - 1 = 8 - 1 = 7$ (**111** in binary).

3. **XOR to Flip Bits:** XORing `num` with the mask flips all of its active bits:

$$\text{complement} = \text{num} \oplus \text{mask}$$

For 5 (**101**) $\oplus$ 7 (**111**) = 2 (**010**).

45.2.1 C++ Integer Overflow Safety:

If num is $2^{31} - 1$ (the maximum signed 32-bit integer), $L = 31$. Evaluating $1 \ll 31$ using standard signed integers causes **undefined behavior (UB)** because shifting into the sign bit overflows a signed 32-bit integer. To prevent this, we must perform the shift on an **unsigned integer** (using the `1u` literal) which is safe from signed overflow:

$$\text{mask} = (1u \ll L) - 1$$

45.2.2 Standard C++ Production Code

```
#include <stdint>
```

```
class Solution {
```

```
public:
```

```
    int findComplement(int num) noexcept {
```

```
        // Calculate the number of bits needed to represent num
```

```
        int bits = 0;
```

```
        int temp = num;
```

```
        while (temp > 0) {
```

```
            bits++;
```

```
            temp >>= 1;
```

```
        }
```

```
        // Generate mask of 'bits' ones. Use 1u (unsigned) to prevent UB/overflow if bits == 31
```

```
        unsigned int mask = (1u << bits) - 1;
```

```
        // XORing with a mask of all ones flips the bits
```

```
        return num ^ static_cast<int>(mask);
```

```
    }
```

```
    // Alternative: Optimized using Count Leading Zeros intrinsic
```

```
    int findComplementOptimized(int num) noexcept {
```

```
        // __builtin_clz returns the number of leading 0-bits in a 32-bit int
```

```
        int leading_zeros = __builtin_clz(num);
```

```
        int active_bits = 32 - leading_zeros;
```

```
        unsigned int mask = (1u << active_bits) - 1;
```

```
        return num ^ static_cast<int>(mask);
```

```
    }
```

```
};
```


45.3 Detailed Solution (Python)

45.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It uses Python's built-in `bit_length()` method to determine the number of active bits.

class Solution:

```
def findComplement(self, num: int) -> int:
    """
    Returns the complement of num by masking and XORing.

    Time Complexity: O(1) (bit_length is constant-time or near constant-time)
    Space Complexity: O(1)
    """
    # Determine the number of bits in the binary representation of num
    bits = num.bit_length()

    # Create a mask of 1s of the same length
    mask = (1 << bits) - 1

    # XOR num with the mask to flip all active bits
    return num ^ mask
```

45.4 Python-Specific Complexities & Implementation Considerations

45.4.1 1. The `bit_length()` Method

- Python's `int.bit_length()` returns the number of bits necessary to represent an integer in binary, excluding the sign and leading zeros. For example, `(5).bit_length()` returns 3.
- This is a C-level operation, executing much faster than loop-based bit-shifting in Python.

45.4.2 2. Arbitrary Precision Negation

- A common mistake is using the bitwise NOT operator `~num`.
 - In Python, integers have arbitrary precision. Applying `~num` flips all bits including the infinite leading sign bits (e.g. `~5` yields `-6`). We must use a mask to restrict the flip to only the active bits of `num`.
-

45.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(1)$	Counting bits and bitwise operations execute in constant time (at most 31 iterations, which is bounded).
Space Complexity	$\mathcal{O}(1)$	Uses a constant amount of auxiliary memory.

45.6 Common Follow-Up Questions & How to Answer

45.6.1 Q1: What if `num = 0`?

- **Answer:** If `num = 0`, its binary representation is `0`. The complement of `0` is `1`.
 - In the loop-based implementation, if `num = 0`, the loop `while (temp > 0)` does not execute, leaving `bits = 0`, and yielding `mask = 0`. XORing `0 ^ 0` would incorrectly return `0`.
 - However, the LeetCode constraint specifies $1 \leq num < 2^{31}$, so `num` is never `0` in this problem. If it could be `0` (like in LeetCode 1009), we would add an explicit check: `if (num == 0) return 1;`.

45.6.2 Q2: Can we solve this branchlessly without using loops or intrinsic CLZ?

- **Answer:** Yes, we can generate a mask of ones by propagating the highest set bit to the right using bitwise OR shifts:

```
int mask = num;
mask |= mask >> 1;
mask |= mask >> 2;
mask |= mask >> 4;
mask |= mask >> 8;
mask |= mask >> 16;
// Now mask contains all 1s from the MSB down to LSB
return num ^ mask;
```

This is branchless, loopless, does not use floats or intrinsics, and is extremely fast.

45.7 Pro-Tip: How to Impress the Interviewer

- **Demonstrate UB Awareness:** Proactively discuss why doing `1 << 31` on a signed 32-bit integer is Undefined Behavior in C++. Explain how using `1u` (unsigned literal) avoids this constraint because unsigned arithmetic is guaranteed to wrap modulo 2^{32} .
- **Propose the Bit-Propagation Hack:** Presenting the bit-propagation algorithm (under Q2) shows that you understand binary prefix operations and how to write high-performance, branchless CPU instruction blocks.

46 Poor Pigs

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 458, Glassdoor
 - **Flashcards:** [Math deck](#)
-

46.1 Question Description

There are `buckets` buckets of liquid, where exactly one of the buckets is poisonous. To figure out which one is poisonous, you feed some number of pigs the liquid to see whether they will die or not. Unfortunately, you only have `minutesToTest` minutes to determine which bucket is poisonous.

You can feed the pigs according to these steps: 1. Choose some number of pigs to feed. 2. For each pig, choose which buckets to feed it. The pig will consume all chosen buckets simultaneously and instantly. Each bucket can be fed to any number of pigs, and each pig can consume any number of buckets. 3. Wait for `minutesToDie` minutes. During this time, any pig that consumed the poison will die, and all other pigs survive. 4. If you still have time left, you can repeat this process with the surviving pigs (using new buckets). 5. You can test at most $\lfloor \frac{\text{minutesToTest}}{\text{minutesToDie}} \rfloor$ times.

Given `buckets`, `minutesToDie`, and `minutesToTest`, return the *minimum* number of pigs needed to figure out which bucket is poisonous within the allotted time.

46.1.1 Examples

- **Input:** `buckets = 4, minutesToDie = 15, minutesToTest = 15`
 - **Output:** 2
 - **Explanation:**
 - * At time 0, feed the first pig buckets 1 and 2, and feed the second pig buckets 2 and 3.
 - * At time 15, there are 4 possible outcomes:
 - If only the first pig dies, bucket 1 is poisonous.
 - If only the second pig dies, bucket 3 is poisonous.
 - If both pigs die, bucket 2 is poisonous.
 - If neither pig dies, bucket 4 is poisonous.
 - **Input:** `buckets = 4, minutesToDie = 15, minutesToTest = 30`
 - **Output:** 2
 - **Explanation:** We have 2 test rounds ($30/15 = 2$). With 2 pigs, we can test up to 9 buckets, so 2 pigs are sufficient for 4 buckets.
-

46.2 Detailed Solution (C++)

This is fundamentally an **information theory** and **base- k representation** problem: 1. Let $T = \lfloor \frac{\text{minutesToTest}}{\text{minutesToDie}} \rfloor$ be the number of test rounds available. 2. For each pig, there are exactly $T + 1$ possi-

ble outcomes (states) at the end of the entire test process: * It dies in round 1. * It dies in round 2. * ... * It dies in round T . * It survives all rounds. 3. If we have P pigs, each pig behaves independently. Therefore, the total number of distinct joint states we can distinguish is:

$$\text{States} = (T + 1)^P$$

4. To uniquely identify which of the N buckets is poisonous, the number of distinct joint states must be greater than or equal to N :

$$(T + 1)^P \geq N$$

5. We need to find the smallest integer P satisfying this inequality. We can find this by iteratively multiplying $(T + 1)$ until the value meets or exceeds `buckets`. This avoids any floating-point precision issues that might arise from using `std::log` or `std::pow`.

46.2.1 Standard C++ Production Code

```
#include <limits>

class Solution {
public:
    int poorPigs(int buckets, int minutesToDie, int minutesToTest) {
        if (buckets <= 1) {
            return 0;
        }

        int tests = minutesToTest / minutesToDie;
        int pigs = 0;
        long long base = tests + 1;
        long long max_buckets_distinguished = 1;

        while (max_buckets_distinguished < buckets) {
            // Prevent integer overflow during multiplication
            if (max_buckets_distinguished > (std::numeric_limits<long long>::max() / base)) {
                break;
            }
            max_buckets_distinguished *= base;
            pigs++;
        }

        return pigs;
    }
};
```

46.3 Detailed Solution (Python)

46.3.1 Standard Python Production Code

Below is the Python implementation.

```
import math

class Solution:
    def poorPigs(self, buckets: int, minutesToDie: int, minutesToTest: int) -> int:
        """
        Determines the minimum number of pigs needed to find the poisonous bucket.

        Time Complexity:  $O(\log_{(T+1)} N)$ 
        Space Complexity:  $O(1)$ 
        """
        if buckets <= 1:
            return 0

        # Number of rounds we can perform
        rounds = minutesToTest // minutesToDie
        base = rounds + 1

        # We need to find smallest P such that base^P >= buckets
        # This is equivalent to P = ceil(log(buckets) / log(base))
        # To avoid floating point inaccuracy, we can use a loop or integer power comparison.
        pigs = 0
        current_limit = 1
        while current_limit < buckets:
            current_limit *= base
            pigs += 1

        return pigs
```

46.4 Python-Specific Complexities & Implementation Considerations

46.4.1 1. Floating-Point Precision Risk with `math.log`

- A naive mathematical solution is:

```
return math.ceil(math.log(buckets) / math.log(rounds + 1))
```

- **Warning:** Floating-point division and logarithmic approximations can lead to precision errors for boundary numbers. For example, `math.log(125) / math.log(5)` might return `2.9999999999999996` due to IEEE-754 precision limits, and applying `math.ceil` would mistakenly round it up to 3 instead of 3 directly (or round `3.0000000000000004` up to 4).

- Using integer multiplication `current_limit *= base` in a loop guarantees exact integer evaluation and is completely free of floating-point precision issues.

46.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\log_{T+1} n)$	The loop runs at most ≈ 15 times since the base is at least 2 and $N \leq 1000$ (with $2^{10} = 1024$).
Space Complexity	$\mathcal{O}(1)$	Only a few scalar variables are maintained.

46.6 Common Follow-Up Questions & How to Answer

46.6.1 Q1: How do we actually schedule the feedings (the experiment design)?

- **Answer:**
 1. Represent each bucket's ID (from 0 to $N - 1$) in base- $(T + 1)$ notation. Each bucket will have a coordinates representation $(d_0, d_1, \dots, d_{P-1})$, where each digit $d_i \in [0, T]$.
 2. Pig i is responsible for the i -th dimension (or digit position).
 3. In round r (for $r \in [1, T]$), pig i drinks from all buckets where the i -th digit d_i is equal to $r - 1$.
 4. If pig i dies in round r , we know the poisonous bucket's i -th digit is $r - 1$.
 5. If pig i survives all T rounds, the poisonous bucket's i -th digit must be T .
 6. By combining the survival/death round of all P pigs, we get the exact base- $(T + 1)$ digits of the poisonous bucket.

46.6.2 Q2: What if pigs can recover after a round?

- **Answer:** The problem explicitly states that once a pig consumes the poison, it dies and cannot be reused. If recovery were possible, it would increase the number of states per pig and dramatically decrease the number of pigs needed.
-

46.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Multidimensional Grid (Hypercube):** Explain the problem using a spatial coordinate analogy. For P pigs and T rounds, think of the state space as a P -dimensional hypercube of side length $T + 1$. Each cell in this hypercube corresponds to a bucket. Each pig controls one axis. In each round r , a pig drinks the "slice" perpendicular to its axis at coordinate $r - 1$. The point of intersection of the "dead slices" (or survival on that axis) uniquely identifies the single poisonous cell. This geometric interpretation shows a profound intuition for multi-dimensional coding theory.

- **Proactively Address IEEE-754 Precision:** Point out the risk of using `log` or `pow` with float math and explain how the integer loop completely avoids this vulnerability. Interviewers love engineers who write defensively against float precision bugs.

47 Generate Random Point in a Circle

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 478, Glassdoor
 - **Flashcards:** [Math deck](#)
-

47.1 Question Description

Given the radius and the position of the center of a circle, implement the class `Solution` that supports the method `randPoint()` which generates a uniform random point inside the circle.

A point on the circumference of the circle is considered to be in the circle.

Implement the `Solution` class: * `Solution(double radius, double x_center, double y_center)` initializes the object with the radius of the circle `radius` and the position of the center `(x_center, y_center)`. * `randPoint()` returns a random point inside the circle as a 2-element array `[x, y]`.

47.1.1 Examples

- **Input:** `["Solution", "randPoint", "randPoint", "randPoint"]` `[[1.0, 0.0, 0.0], [], [], []]`
 - **Output:** `[null, [-0.02493, -0.38077], [0.82314, 0.38945], [0.36572, 0.17248]]`
 - **Explanation:**

```
Solution solution = new Solution(1.0, 0.0, 0.0);
solution.randPoint(); // returns [-0.02493, -0.38077]
solution.randPoint(); // returns [0.82314, 0.38945]
solution.randPoint(); // returns [0.36572, 0.17248]
```
-

47.2 Detailed Solution (C++)

There are two primary methods to solve this problem: **Rejection Sampling** and **Polar Coordinates with Inverse CDF (Transform Method)**. Both are detailed below.

47.2.1 Method 1: Rejection Sampling (Loop-based)

We generate a random point inside a square of side length $2R$ centered at $(0, 0)$. A point (x, y) is accepted if $x^2 + y^2 \leq R^2$; otherwise, it is rejected and we try again. Once accepted, we translate the point to the circle's center. * **Probability of Acceptance:** $\frac{\text{Area of Circle}}{\text{Area of Square}} = \frac{\pi R^2}{4R^2} = \frac{\pi}{4} \approx 78.5\%$. * **Expected Iterations:** $\frac{4}{\pi} \approx 1.27$ attempts.

47.2.2 Method 2: Polar Coordinates with Inverse CDF (Analytical $O(1)$)

If we naive-sample a radius r uniformly in $[0, R]$ and an angle θ uniformly in $[0, 2\pi]$, points will clump near the center because the area of a circle grows quadratically ($A = \pi r^2$). To distribute points uniformly, we use **Inverse Transform Sampling**: 1. The Probability Density Function (PDF) for the radius is proportional to the circumference: $f(r) = \frac{2r}{R^2}$. 2. The Cumulative Distribution Function (CDF) is $F(r) = \frac{r^2}{R^2}$. 3. Set U to a uniform random float in $[0, 1)$, and solve for r :

$$U = \frac{r^2}{R^2} \implies r = R\sqrt{U}$$

4. Generate $\theta = 2\pi \cdot U_2$ and calculate $x = x_c + r \cos \theta, y = y_c + r \sin \theta$.

47.2.3 Standard C++ Production Code (Method 2: Analytical $O(1)$)

```
#include <vector>
#include <random>
#include <cmath>

class Solution {
private:
    double r;
    double xc;
    double yc;
    std::mt19937 gen;
    std::uniform_real_distribution<double> dis;

public:
    Solution(double radius, double x_center, double y_center)
        : r(radius), xc(x_center), yc(y_center), gen(std::random_device{}()), dis(0.0, 1.0) {}

    std::vector<double> randPoint() {
        double u1 = dis(gen); // For radius scaling
        double u2 = dis(gen); // For angle theta

        double current_r = r * std::sqrt(u1);
        const double pi = std::acos(-1.0);
        double theta = u2 * 2.0 * pi;

        return {xc + current_r * std::cos(theta), yc + current_r * std::sin(theta)};
    }
};
```


47.3 Detailed Solution (Python)

47.3.1 Standard Python Production Code (Method 2: Analytical $O(1)$)

Below is the Python implementation using the Polar Inverse CDF method.

```
import math
import random
from typing import List

class Solution:

    def __init__(self, radius: float, x_center: float, y_center: float):
        """
        Initializes the circle parameters.
        """
        self.radius = radius
        self.x_center = x_center
        self.y_center = y_center

    def randPoint(self) -> List[float]:
        """
        Generates a uniform random point inside the circle in  $O(1)$  time.
        """
        # Generate two uniform random floats in [0.0, 1.0)
        u1 = random.random()
        u2 = random.random()

        # Apply Inverse CDF for uniform area distribution
        r = self.radius * math.sqrt(u1)
        theta = u2 * 2.0 * math.pi

        x = self.x_center + r * math.cos(theta)
        y = self.y_center + r * math.sin(theta)

        return [x, y]
```

47.4 Python-Specific Complexities & Implementation Considerations

47.4.1 1. Rejection Sampling vs. Inverse CDF Performance in Python

- In Python, function call overhead is high. Rejection sampling requires a `while True` loop that averages 1.27 iterations but can theoretically run indefinitely.
- The Polar method using `math.sqrt`, `math.cos`, and `math.sin` executes in constant time with exactly two calls to `random.random()`, making it more predictable and generally faster in Python than loop-based

rejection sampling.

47.4.2 2. High Precision Float Math

- Python's built-in `float` type is a C double (64-bit float). This guarantees 53 bits of precision, which is more than sufficient for the radius bounds of 10^8 .

47.5 Complexity Analysis

47.5.1 Method 1: Rejection Sampling

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(1)$ average	On average, $\frac{4}{\pi} \approx 1.27$ attempts are needed. Worst-case is unbounded (but probability of high iterations decays exponentially).
Space Complexity	$\mathcal{O}(1)$	Constant auxiliary memory.

47.5.2 Method 2: Polar Coordinates (Inverse CDF)

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(1)$ deterministic	Guaranteed single-pass execution without loops.
Space Complexity	$\mathcal{O}(1)$	Constant auxiliary memory.

47.6 Common Follow-Up Questions & How to Answer

47.6.1 Q1: Which method is preferred if generating random numbers is extremely expensive?

- **Answer:** If generating random numbers is expensive, **Rejection Sampling** uses an average of $2 \times 1.27 = 2.54$ random numbers per point. The **Polar method** uses exactly 2 random numbers per point. Thus, the Polar method is more efficient in its consumption of random entropy.
- Conversely, if trigonometric operations (`cos`, `sin`, `sqrt`) are extremely expensive (e.g., on low-end embedded systems lacking a Floating Point Unit / FPU), **Rejection Sampling** is preferred because it only uses basic multiplications and additions ($x^2 + y^2 \leq R^2$).

47.6.2 Q2: How would you extend this to generate a random point on a 3D sphere?

- **Answer:**

- **Rejection Sampling:** Generate (x, y, z) in a $[-R, R]^3$ cube and reject if $x^2 + y^2 + z^2 > R^2$. The acceptance rate drops to $\frac{\frac{4}{3}\pi R^3}{8R^3} = \frac{\pi}{6} \approx 52.4\%$.
 - **Analytical Method (Spherical Coordinates):** We can use Inverse Transform Sampling or Normal distributions. For a solid sphere, the radius CDF is $F(r) = \frac{r^3}{R^3} \implies r = R\sqrt[3]{U}$. The direction can be sampled uniformly on the sphere's surface by generating three independent Gaussian variables, normalizing them, and scaling by r .
-

47.7 Pro-Tip: How to Impress the Interviewer

- **Derive the $\sqrt{U}$ relationship:** Do not just write `r = radius * sqrt(u)`. Walk the interviewer through the calculus:
 1. The area of a thin ring of radius r is $2\pi r dr$.
 2. Integrate from 0 to r to get the cumulative area: πr^2 .
 3. Normalize by total area πR^2 to get the CDF: $P(R \leq r) = \frac{r^2}{R^2}$.
 4. Set $U = \frac{r^2}{R^2}$ and solve for r : $r = R\sqrt{U}$. This shows mathematical rigour and a deep understanding of probability theory.
- **Compare Hardware Constraints:** Discuss the hardware trade-offs between rejection sampling (which needs simple ALU math but has branching and variable latency) and the analytical method (which is constant-time/non-branching but uses complex trigonometric execution units). This is a hallmark of senior systems engineering.

48 Largest Palindrome Product

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 479, Glassdoor
 - **Flashcards:** [Math deck](#)
-

48.1 Question Description

Given an integer n , return the **largest palindromic integer** that can be represented as the product of two n -digit integers. Since the answer can be very large, return it **modulo 1337**.

48.1.1 Examples

- **Input:** $n = 2$
 - **Output:** 987
 - **Explanation:** The largest palindrome product of two 2-digit numbers is $99 \times 91 = 9009$. $9009 \bmod 1337 = 987$.
- **Input:** $n = 1$
 - **Output:** 9
 - **Explanation:** $3 \times 3 = 9$.

48.2 Detailed Solution (C++)

To find the largest palindrome product of two n -digit numbers:

- Edge Case:** For $n = 1$, the largest single-digit product is $3 \times 3 = 9$, which we return directly.
- Search Space:** The largest possible n -digit number is $U = 10^n - 1$, and the smallest is $L = 10^{n-1}$.
- Palindrome Generation:** The product of two n -digit numbers will have either $2n$ or $2n - 1$ digits. Since we want the largest palindrome, we search for palindromes of length $2n$.
- We can iterate through the left half of the palindrome, `left`, starting from U down to L .
- For each `left`, we construct the full palindrome by reversing the digits of `left` and appending them to `left`. For example, if $n = 3$ and `left = 998`, the palindrome is `998899`.
- Factorization Search:** To check if the generated palindrome is a product of two n -digit numbers:
 - * We search for a divisor `right` starting from U down to $\lceil \sqrt{\text{palindrome}} \rceil$.
 - * If `palindrome % right == 0`, we have found the largest palindrome.
 - * If `right * right < palindrome`, we can stop searching for this palindrome because any factor we find would be paired with a factor larger than U , which is invalid.
- Return `palindrome % 1337`.

48.2.1 Standard C++ Production Code

We must use `long long` in C++ because for $n = 8$, the palindrome has 16 digits, which exceeds the limit of a standard 32-bit signed integer (`int`) but fits safely inside a 64-bit integer (`long long` max $\approx 9.22 \times 10^{18}$).

```
#include <cmath>

class Solution {
public:
    int largestPalindrome(int n) {
        // Special case: for n = 1, largest product is 9 (3 * 3)
        if (n == 1) {
            return 9;
        }

        long long upper = 1;
        for (int i = 0; i < n; ++i) {
            upper *= 10;
        }
        long long lower = upper / 10;
        upper -= 1; // e.g. 99 for n = 2

        // Iterate through the left half of the palindrome from largest to smallest
        for (long long left = upper; left >= lower; --left) {
            long long palindrome = left;
            long long tmp = left;

            // Construct the palindrome by reversing 'left' and appending it
```

```

    while (tmp > 0) {
        palindrome = palindrome * 10 + (tmp % 10);
        tmp /= 10;
    }

    // Search for a factor 'right'
    for (long long right = upper; right >= lower; --right) {
        // If the square of right is less than the palindrome,
        // any remaining factors would have to be larger than upper (which is invalid)
        if (right * right < palindrome) {
            break;
        }

        if (palindrome % right == 0) {
            return static_cast<int>(palindrome % 1337);
        }
    }

    return 0;
}

};

```

48.3 Detailed Solution (Python)

48.3.1 Standard Python Production Code

Below is the Python implementation.

```

class Solution:
    def largestPalindrome(self, n: int) -> int:
        """
        Finds the largest palindrome product of two n-digit numbers modulo 1337.
        """
        if n == 1:
            return 9

        upper = 10**n - 1
        lower = 10**(n - 1)

        # Iterate left half downwards
        for left in range(upper, lower - 1, -1):
            # Construct the palindrome
            palindrome = int(str(left) + str(left)[::-1])

```

```

    # Find a divisor 'right' in range [sqrt(palindrome), upper]
    # We iterate right from upper downwards
    right = upper
    while right * right >= palindrome:
        if palindrome % right == 0:
            return palindrome % 1337
        right -= 1

    return 0

```

48.4 Python-Specific Complexities & Implementation Considerations

48.4.1 1. Palindrome Generation: Arithmetic vs String Reversal

- In Python, string concatenation and slicing (`str(left) + str(left)[::-1]`) is implemented in highly optimized C and is generally faster and more readable than manual math loops (`palindrome = palindrome * 10 + tmp % 10`).
- In C++, math operations are faster than string conversions to avoid heap allocation overhead.

48.4.2 2. Large Integer Arithmetic

- Python handles arbitrary-precision integers automatically. However, since the maximum palindrome for $n = 8$ fits within a 64-bit integer, standard integer operations run quickly on the CPU without shifting to slow big-integer libraries.
-

48.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(10^{2n})$ worst-case	In practice, the largest palindrome is found very quickly near the top. For example, for $n = 8$, the search takes less than a second because we only check a small fraction of the space.
Space Complexity	$\mathcal{O}(1)$	We only maintain a few scalar loop variables.

48.6 Common Follow-Up Questions & How to Answer

48.6.1 Q1: Why do we start the search from upper and go downwards?

- **Answer:** Since we want the *largest* palindrome, generating the palindromes in descending order guarantees that the first one we successfully factor into two n -digit numbers is the global maximum. This allows us to terminate the entire algorithm immediately upon finding the first match.

48.6.2 Q2: Why does `right * right < palindrome` allow us to break early?

- **Answer:**
 - If `palindrome = A * B`, then one of the factors must be $\geq \sqrt{\text{palindrome}}$ and the other must be $\leq \sqrt{\text{palindrome}}$.
 - We search for the larger factor `right` starting from `upper` downwards. If `right` drops below $\sqrt{\text{palindrome}}$ (i.e. `right * right < palindrome`), it is mathematically impossible to find a larger factor. Any divisor we find from that point onward would be paired with a factor larger than `upper`, which is not an n -digit number.
-

48.7 Pro-Tip: How to Impress the Interviewer

- **Mention Modulo Math Properties:** Explain why the modulo 1337 is applied *only* at the end. In some problems, we apply `% M` at each step of arithmetic to prevent overflow. Here, doing so would destroy the value of the palindrome, making it impossible to check for factorizability. We must compute the full product (up to 16 digits) and apply modulo 1337 only to the final result.
- **Discuss Math vs. String Performance:** Discuss how C++ handles mathematical loops faster because of register-level operations, whereas Python's VM overhead makes string operations faster. This indicates you write code optimized for the runtime environment.

49 Next Greater Element III

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / Low-Level Developer
 - **Source:** LeetCode 556, Glassdoor
 - **Flashcards:** [Math deck](#)
-

49.1 Question Description

Given a positive integer n , find the smallest integer which has exactly the same digits existing in the integer n and is greater in value than n . If no such positive integer exists, return -1 .

Note that the returned integer should fit in a **32-bit signed integer** (i.e., $\leq 2^{31} - 1$). If there is a valid answer but it does not fit in a 32-bit signed integer, return -1 .

49.1.1 Examples

- **Input:** $n = 12$
 - **Output:** 21
 - **Input:** $n = 21$
 - **Output:** -1
-

49.2 Detailed Solution (C++)

This problem is equivalent to finding the **next lexicographical permutation** of the digits of n : 1. Convert the integer n to a string representation. 2. Find the first pair of adjacent digits `digits[i]` and `digits[i+1]` from the right such that `digits[i] < digits[i+1]`. * If no such index i exists (i.e. the digits are sorted in descending order), then no larger permutation is possible. Return -1. 3. Find the smallest digit to the right of i that is strictly greater than `digits[i]`. Let this index be j . 4. Swap `digits[i]` and `digits[j]`. 5. Reverse the subsegment of digits from $i + 1$ to the end. Reversing it converts the descending suffix into an ascending suffix, minimizing the value increase. 6. Convert the permuted string back to a number. 7. **Overflow Check:** Check if the resulting number fits in a 32-bit signed integer (2147483647). If it overflows, return -1.

49.2.1 Standard C++ Production Code

In C++, we can write the manual two-pointer permutation algorithm or leverage `std::next_permutation` directly. Both are correct, but the manual implementation shows your understanding of the underlying STL mechanics.

```
#include <string>
#include <algorithm>
#include <limits>

class Solution {
public:
    int nextGreaterElement(int n) {
        std::string digits = std::to_string(n);

        // Step 1: Find the first decreasing digit from the right
        int i = static_cast<int>(digits.size()) - 2;
        while (i >= 0 && digits[i] >= digits[i + 1]) {
            i--;
        }

        // If the digits are sorted descending, this is the largest permutation
        if (i < 0) {
            return -1;
        }
    }
};
```



```

// Step 2: Find the smallest digit larger than digits[i] to its right
int j = static_cast<int>(digits.size()) - 1;
while (digits[j] <= digits[i]) {
    j--;
}

// Step 3: Swap and reverse the suffix
std::swap(digits[i], digits[j]);
std::reverse(digits.begin() + i + 1, digits.end());

// Step 4: Convert back and check for 32-bit signed integer overflow
long long val = std::stoll(digits);
if (val > INT_MAX) {
    return -1;
}

return static_cast<int>(val);
}
};

```

49.3 Detailed Solution (Python)

49.3.1 Standard Python Production Code

Below is the Python implementation.

class Solution:

```

def nextGreaterElement(self, n: int) -> int:
    """
    Finds the next greater integer with the same digits using the next permutation algorithm.

    Time Complexity: O(D) where D is the number of digits in n
    Space Complexity: O(D)
    """
    # Convert integer to a mutable list of character digits
    digits = list(str(n))
    length = len(digits)

    # Step 1: Find the first index i from the right such that digits[i] < digits[i+1]
    i = length - 2
    while i >= 0 and digits[i] >= digits[i + 1]:
        i -= 1

    if i < 0:

```

```

    return -1

# Step 2: Find the first index j from the right such that digits[j] > digits[i]
j = length - 1
while digits[j] <= digits[i]:
    j -= 1

# Step 3: Swap digits at i and j
digits[i], digits[j] = digits[j], digits[i]

# Step 4: Reverse the digits after index i
digits[i + 1:] = reversed(digits[i + 1:])

# Step 5: Convert back to integer and perform 32-bit overflow check
result = int("".join(digits))

# 32-bit signed integer limit is 2^31 - 1 = 2147483647
if result > 2**31 - 1:
    return -1

return result

```

49.4 Python-Specific Complexities & Implementation Considerations

49.4.1 1. Mutable Strings

- In Python, strings are immutable. Direct swap operations like `s[i], s[j] = s[j], s[i]` are not allowed on strings.
- To perform character-level swaps and reversals efficiently, we must convert the string to a list of characters, perform in-place mutations, and then reconstruct the string using `"".join(digits)`.

49.4.2 2. Standard 32-bit Signed Integer Limits

- Python integers have arbitrary precision (they do not overflow). Hence, the code must explicitly check if the final integer `result > 2**31 - 1` to respect LeetCode's constraints.
-

49.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\log_{10} n)$	The number of digits d in the number is proportional to $\log_{10} n$. The algorithm scans the digits at most a few times.
Space Complexity	$\mathcal{O}(\log_{10} n)$	Auxiliary memory to store the string/list representation of the digits.

49.6 Common Follow-Up Questions & How to Answer

49.6.1 Q1: What is the underlying math of the next permutation algorithm?

- **Answer:**

1. To get the next lexicographically larger sequence, we must increase the value at the rightmost possible position.
2. Any suffix that is sorted in descending order (e.g. 5 4 3 2 1) is already in its maximal lexicographical permutation.
3. Thus, we scan leftward to find the first element `digits[i]` that is smaller than its successor. This is the rightmost digit we can increase.
4. To minimize the increase, we replace `digits[i]` with the *smallest* digit to its right that is greater than `digits[i]`.
5. After the swap, the suffix to the right of `i` is still in descending order. Reversing it sorts it in ascending order, which produces the smallest possible value for that suffix.

49.6.2 Q2: Can we solve this problem without converting the number to a string?

- **Answer:** Yes, we can extract the digits mathematically using `% 10` and `/ 10` operations, store them in a list/vector of integers, run the permutation steps, and reconstruct the number. This avoids string allocation overhead entirely.

49.7 Pro-Tip: How to Impress the Interviewer

- **Highlight STL `std::next_permutation`:** In C++, point out that the standard library provides `std::next_permutation`. Demonstrating familiarity with STL algorithms is excellent:

```
int nextGreaterElement(int n) {
    std::string s = std::to_string(n);
    if (!std::next_permutation(s.begin(), s.end())) return -1;
    long long val = std::stoll(s);
    return val > INT_MAX ? -1 : val;
}
```

This is extremely clean. In a real-world setting, using the standard library reduces surface area for bugs.

- **Defensive Coding on Type Conversion:** Mention that using `std::stoll` (string to long long) is safer than `std::stoi` (string to int) because `std::stoi` throws an out-of-range exception if the permuted string exceeds `INT_MAX`, whereas `std::stoll` can easily accommodate up to 9×10^{18} before throwing.

50 Find the Closest Palindrome

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 564, Glassdoor
 - **Flashcards:** [Math deck](#)
-

50.1 Question Description

Given a string `n` representing an integer, return the **closest integer (not including itself)**, which is a palindrome. If there is a tie, return the **smaller one**.

The closest is defined as the absolute difference minimized between two integers.

50.1.1 Examples

- **Input:** `n = "123"`
– **Output:** `"121"`
 - **Input:** `n = "1"`
– **Output:** `"0"`
– **Explanation:** `0` and `2` are the closest palindromes, but we return the smaller one, which is `0`.
-

50.2 Detailed Solution (C++)

This problem is solved efficiently by generating a small set of potential palindrome candidates and choosing the one with the minimum absolute difference:

1. Let L be the length of the string `n`.
2. Extract the prefix representing the left half of the string. The prefix has length `half = (L + 1) / 2`.
3. Generate three primary candidates based on the prefix:
 - * **Candidate 1:** Replicate the prefix. (e.g. for prefix `12`, we make `1221`).
 - * **Candidate 2:** Replicate the prefix decremented by 1. (e.g. for prefix `12`, we make `1111`).
 - * **Candidate 3:** Replicate the prefix incremented by 1. (e.g. for prefix `12`, we make `1331`).
4. Generate two structural boundary candidates:
 - * **Candidate 4 (Lower Boundary):** $10^{L-1} - 1$ (consists of $L - 1$ nines, e.g., `999` for length 4). This handles numbers like `1000`.
 - * **Candidate 5 (Upper Boundary):** $10^L + 1$ (e.g., `10001` for length 4). This handles numbers like `9999`.
5. Insert all candidates into a set to deduplicate, and remove the original number `num` itself (since the answer must be a different integer).
- 6.

Find the candidate that minimizes the absolute difference $|candidate - num|$. In case of a tie, choose the smaller candidate.

50.2.1 Standard C++ Production Code

All operations must be performed using 64-bit integers (`long long` in C++) because the input string can represent numbers up to $10^{18} - 1$.

```
#include <string>
#include <vector>
#include <cmath>
#include <set>
#include <algorithm>

class Solution {
private:
    // Helper function to construct a palindrome from a given prefix
    std::string makePalindrome(long long prefix, bool isEvenLength) {
        std::string p = std::to_string(prefix);
        std::string rev = p;
        if (isEvenLength) {
            std::reverse(rev.begin(), rev.end());
            return p + rev;
        } else {
            rev.pop_back();
            std::reverse(rev.begin(), rev.end());
            return p + rev;
        }
    }

public:
    std::string nearestPalindromic(std::string n) {
        long long num = std::stoll(n);
        int len = static_cast<int>(n.size());

        // Edge case: single-digit numbers <= 10
        if (num <= 10) {
            return std::to_string(num - 1);
        }

        std::set<long long> candidates;

        // Extract prefix (left half)
        int half = (len + 1) / 2;
        long long prefix = std::stoll(n.substr(0, half));
```

```

bool isEvenLength = (len % 2 == 0);

// Generate candidates from prefix - 1, prefix, and prefix + 1
for (long long d = -1; d <= 1; ++d) {
    candidates.insert(std::stoll(makePalindrome(prefix + d, isEvenLength)));
}

// Add boundary candidates
long long p10_lower = 1;
for (int i = 0; i < len - 1; ++i) {
    p10_lower *= 10;
}
candidates.insert(p10_lower - 1); // e.g. 999
candidates.insert(p10_lower * 10 + 1); // e.g. 10001

// Remove the original number from potential answers
candidates.erase(num);

long long best = -1;
long long best_diff = -1;

for (long long c : candidates) {
    if (c < 0) continue;

    long long diff = std::abs(c - num);
    if (best_diff == -1 || diff < best_diff || (diff == best_diff && c < best)) {
        best = c;
        best_diff = diff;
    }
}

return std::to_string(best);
}
};

```

50.3 Detailed Solution (Python)

50.3.1 Standard Python Production Code

Below is the Python implementation.

```

class Solution:
    def nearestPalindromic(self, n: str) -> str:
        """

```

Finds the closest palindrome to a given number string, excluding the number itself.

Time Complexity: $O(L)$ where L is the length of string n ($L \leq 18$)

Space Complexity: $O(L)$

```

"""
num = int(n)
if num <= 10:
    return str(num - 1)

length = len(n)
half = (length + 1) // 2
prefix = int(n[:half])

candidates = set()

# Generate prefix candidates
for p in (prefix - 1, prefix, prefix + 1):
    p_str = str(p)
    if length % 2 == 0:
        candidate = int(p_str + p_str[::-1])
    else:
        candidate = int(p_str + p_str[:-1][::-1])
    candidates.add(candidate)

# Add boundary candidates
candidates.add(10 ** (length - 1) - 1) # Lower digit boundary (e.g. 999)
candidates.add(10 ** length + 1)      # Upper digit boundary (e.g. 10001)

# Remove original number
candidates.discard(num)

# Find candidate with minimum absolute difference; tie-break with smaller value
closest = min(candidates, key=lambda x: (abs(x - num), x))
return str(closest)

```

50.4 Python-Specific Complexities & Implementation Considerations

50.4.1 1. Large Integer Conversions

- Python converts integers to strings and vice-versa seamlessly. Since Python integers have arbitrary precision, we do not have to worry about overflow when computing $10^{\text{length}} + 1$.
- In C++, using standard `std::stoll` (string to long long) and checking the 64-bit limits is necessary to avoid runtime errors.

50.4.2 2. Custom Key Sorting with `min()`

- The use of `min(candidates, key=lambda x: (abs(x - num), x))` is a highly expressive Python pattern. By returning a tuple `(abs(x - num), x)`, Python first compares the absolute differences, and if they are equal, it compares the values of `x` directly to favor the smaller candidate.

50.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(L)$	L is the number of digits in the string n . Since $L \leq 18$, the algorithm executes in a few microseconds ($\approx \mathcal{O}(1)$ time in practice).
Space Complexity	$\mathcal{O}(L)$	For storing string buffers of the candidates.

50.6 Common Follow-Up Questions & How to Answer

50.6.1 Q1: Why do we need the prefix - 1 and prefix + 1 candidates?

- **Answer:**
 - Consider $n = 129$. The prefix is 12. If we only replicate the prefix, we get 121.
 - If we increment the prefix to 13, we get 131.
 - The absolute differences are $|121 - 129| = 8$ and $|131 - 129| = 2$.
 - By modifying the middle digit (incrementing/decrementing prefix), we ensure we check the nearest palindromes that cross the midpoint boundary.

50.6.2 Q2: Why are the boundary candidates 99...9 and 10...01 necessary?

- **Answer:**
 - They handle changes in digit length.
 - For $n = 1000$ (length 4), the prefix is 10. Decrementing it gives 9. Replicating 9 yields 99. But the closest palindrome is actually 999 (length 3). The candidate $10^{L-1} - 1$ ensures we test 999.
 - For $n = 9999$ (length 4), the prefix is 99. Incrementing it gives 100. Replicating 100 yields 100001 (length 6). But the closest palindrome is 10001 (length 5). The candidate $10^L + 1$ ensures we test 10001.

50.7 Pro-Tip: How to Impress the Interviewer

- **Deconstruct the Math of Palindromic Proximity:** Explain to the interviewer that the closest palindrome must match the original number as closely as possible starting from the most significant

digits. Therefore, the first half of the digits of the closest palindrome *must* be either equal to, one less than, or one greater than the first half of the original number. This mathematical observation limits the candidate pool to at most 5 values, transforming a potentially complex search problem into a simple $\mathcal{O}(1)$ comparison.

- **Prevent `std::pow` Float Issues:** In C++, `std::pow` returns a `double`. Casting it directly to `long long` can introduce minor precision errors on some compilers (e.g. `pow(10, 2)` returning `99.9999...` which casts to `99`). Mentioning this and using an integer power loop or adding a small epsilon proves real-world engineering diligence.

51 Climbing Stairs

- **Category:** Coding
 - **Difficulty:** Easy
 - **Target Role:** Software Engineer / QA & Test Engineer
 - **Source:** LeetCode 70, Glassdoor
 - **Flashcards:** [Dynamic Programming deck](#)
-

51.1 Question Description

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

51.1.1 Examples

- **Input:** $n = 2$
 - **Output:** 2
 - **Explanation:** There are two ways to climb to the top:
 1. 1 step + 1 step
 2. 2 steps
- **Input:** $n = 3$
 - **Output:** 3
 - **Explanation:** There are three ways to climb to the top:
 1. 1 step + 1 step + 1 step
 2. 1 step + 2 steps
 3. 2 steps + 1 step

Constraints: * $1 \leq n \leq 45$

51.2 Detailed Solution (C++)

The problem exhibits a **Fibonacci sequence** structure. To reach step n , you must come from either step $n - 1$ (by taking 1 step) or step $n - 2$ (by taking 2 steps). Therefore, the recurrence relation is:

$$DP[i] = DP[i - 1] + DP[i - 2]$$

With base cases: $DP[1] = 1$ $DP[2] = 2$

We optimize the space complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ by keeping track of only the last two steps using two variables `a` and `b`.

51.2.1 Standard C++ Production Code

```
#include <stdexcept>

class Solution {
public:
    int climbStairs(int n) {
        // Guard against non-positive input
        if (n <= 0) {
            return 0;
        }
        if (n <= 2) {
            return n;
        }

        int a = 1; // DP[i-2]
        int b = 2; // DP[i-1]

        for (int i = 3; i <= n; ++i) {
            int current = a + b;
            a = b;
            b = current;
        }

        return b;
    }
};
```

51.3 Detailed Solution (Python)

51.3.1 Standard Python Production Code

Below is the iterative space-optimized Python implementation with proper type hints.

```

class Solution:
    def climbStairs(self, n: int) -> int:
        """
        Calculates the number of distinct ways to climb n stairs.

        Time Complexity: O(n)
        Space Complexity: O(1)
        """
        if n <= 0:
            return 0
        if n <= 2:
            return n

        a, b = 1, 2
        for _ in range(3, n + 1):
            a, b = b, a + b

        return b

```

51.4 Python-Specific Complexities & Implementation Considerations

51.4.1 1. Python Integer Precision

Unlike languages with fixed-width integers (e.g., C++ where `int` can easily overflow if n is large), Python handles arbitrary-precision integers out of the box. For $n = 45$, the result fits in a standard 32-bit integer, but for significantly larger n , C++ would overflow unless using a big integer library or returning modulo arithmetic, whereas Python would automatically scale up to handle the large number.

51.4.2 2. Tail-Call Optimization (TCO) Lack in Python

If you were to implement this recursively with memoization, Python does not support Tail-Call Optimization (TCO). This means a recursive implementation of $O(n)$ depth will allocate n stack frames. For large n (above Python's limit of ~ 1000), it will raise a `RecursionError`. The iterative approach is much safer and uses $O(1)$ space instead of $O(n)$ call stack memory.

51.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$O(n)$	We perform a single loop from 3 to n .

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(1)$	We only maintain two variables (a and b) to track the previous states.

51.6 Common Follow-Up Questions & How to Answer

51.6.1 Q1: What if you can climb 1, 2, or 3 steps at a time? (Tribonacci)

- **Answer:** The recurrence relation shifts to $DP[i] = DP[i-1] + DP[i-2] + DP[i-3]$. The base cases will be $DP[1] = 1, DP[2] = 2, DP[3] = 4$. We can still optimize space using three variables **a**, **b**, and **c**.

51.6.2 Q2: What if we want to solve it in $\mathcal{O}(\log n)$ time?

- **Answer:** We can represent the linear recurrence relation using **Matrix Exponentiation**:

$$\begin{pmatrix} DP[i] \\ DP[i-1] \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} DP[i-1] \\ DP[i-2] \end{pmatrix}$$

By finding $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1}$ via binary exponentiation (similar to fast power), we can obtain the result in $\mathcal{O}(\log n)$ time.

51.7 Pro-Tip: How to Impress the Interviewer

- **Mention Matrix Exponentiation:** Even if the interviewer is happy with the $\mathcal{O}(n)$ space-optimized solution, mention that the time complexity can be reduced to $\mathcal{O}(\log n)$ using matrix exponentiation. This displays a strong mathematical foundation.
- **Identify Overflow Limitations:** Explicitly mention that while $n \leq 45$ keeps the Fibonacci value within standard 32-bit signed integer limits ($DP[45] = 1,836,311,903$, while max signed 32-bit int is $2,147,483,647$), if n were 46, it would overflow in C++ (int), requiring `long long` or standard big-integer handling.

52 House Robber

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / AI Systems Architect
- **Source:** LeetCode 198, Glassdoor
- **Flashcards:** [Dynamic Programming deck](#)

52.1 Question Description

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. The only constraint stopping you from robbing each of them is that adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight without alerting the police*.

52.1.1 Examples

- **Input:** `nums = [1,2,3,1]`
 - **Output:** 4
 - **Explanation:** Rob house 1 (money = 1) and then rob house 3 (money = 3). Total amount you can rob = $1 + 3 = 4$.
- **Input:** `nums = [2,7,9,3,1]`
 - **Output:** 12
 - **Explanation:** Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1). Total amount you can rob = $2 + 9 + 1 = 12$.

Constraints: $1 \leq \text{nums.length} \leq 100$ $0 \leq \text{nums}[i] \leq 400$

52.2 Detailed Solution (C++)

The problem is a classic dynamic programming problem. At each house i , the robber has two choices: 1. **Rob house i :** If we rob house i , we cannot rob house $i - 1$. The total value would be `nums[i]` plus the maximum money robbed from houses up to $i - 2$. 2. **Skip house i :** If we skip house i , the total value would be the maximum money robbed from houses up to $i - 1$.

The recurrence relation is:

$$DP[i] = \max(DP[i - 1], DP[i - 2] + \text{nums}[i])$$

Rather than maintaining a full array of size n , we can optimize the space to $\mathcal{O}(1)$ by keeping only two variables representing $DP[i - 1]$ (`curr`) and $DP[i - 2]$ (`prev`).

52.2.1 Standard C++ Production Code

```
#include <vector>
#include <algorithm>

class Solution {
public:
    int rob(std::vector<int>& nums) {
        // Guard against empty collection
```

```

    if (nums.empty()) {
        return 0;
    }

    int prev = 0; // Represents DP[i-2]
    int curr = 0; // Represents DP[i-1]

    for (int num : nums) {
        int next_rob = std::max(curr, prev + num);
        prev = curr;
        curr = next_rob;
    }

    return curr;
}
};

```

52.3 Detailed Solution (Python)

52.3.1 Standard Python Production Code

Below is the iterative space-optimized Python implementation.

```

from typing import List

class Solution:
    def rob(self, nums: List[int]) -> int:
        """
        Calculates the maximum amount of money that can be robbed without alerting the police.

        Time Complexity: O(N)
        Space Complexity: O(1)
        """
        if not nums:
            return 0

        prev, curr = 0, 0
        for num in nums:
            # Simultaneously update prev and curr
            prev, curr = curr, max(curr, prev + num)

        return curr

```

52.4 Python-Specific Complexities & Implementation Considerations

52.4.1 1. Simultaneous Assignment (`prev, curr = curr, max(curr, prev + num)`)

- In Python, simultaneous assignments evaluate the entire right-hand side first (creating a temporary tuple in memory) before binding the values to the variables on the left.
- This allows us to write the state transitions compactly in one line without needing a temporary variable `t`.
- Behind the scenes, this is highly optimized in CPython, making it faster and cleaner than manual temp-variable swaps.

52.4.2 2. Space Optimization

- Avoid allocating a list `dp = [0] * len(nums)` because it wastes $\mathcal{O}(n)$ memory.
- Interviewers for systems-related or low-resource roles appreciate the $\mathcal{O}(1)$ auxiliary space optimization, as it avoids heap allocation overhead entirely.

52.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We iterate through the array <code>nums</code> exactly once.
Space Complexity	$\mathcal{O}(1)$	Only two scalar integer variables are maintained to track the maximum profits.

52.6 Common Follow-Up Questions & How to Answer

52.6.1 Q1: What if the houses are arranged in a circle? (House Robber II - LeetCode 213)

- **Answer:** Since the first and the last house are adjacent, robbing both is not allowed. We can split this into two sub-problems:
 1. Rob houses from index 0 to $n - 2$ (exclude the last house).
 2. Rob houses from index 1 to $n - 1$ (exclude the first house). The overall result is the maximum of these two options.

52.6.2 Q2: What if the houses are organized in a binary tree? (House Robber III - LeetCode 337)

- **Answer:** This is solved using Tree DP. For each node, we return a pair of values:
 - The maximum money if we rob this node: `node.val + left.not_robbed + right.not_robbed`
 - The maximum money if we do not rob this node: `max(left.robbed, left.not_robbed) + max(right.robbed, right.not_robbed)` This can be calculated bottom-up using Post-order Traversal in $\mathcal{O}(n)$ time and $\mathcal{O}(h)$ space (call stack depth).

52.7 Pro-Tip: How to Impress the Interviewer

- **Explain Cache Locality:** Discuss how iterating sequentially through the vector utilizes spatial locality. The CPU can easily prefetch elements of `nums` into L1 cache lines, leading to optimal throughput.
- **Point out Input Mutation:** Some candidates overwrite the input array `nums` to store the DP results. Point out that mutating inputs is generally bad practice in production systems (e.g., if the vector is passed as a `const` reference or is owned/shared by other parts of the application). The variable-based solution is both cleaner and avoids modifying input data.

53 Coin Change

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 322, Glassdoor
 - **Flashcards:** [Dynamic Programming deck](#)
-

53.1 Question Description

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

53.1.1 Examples

- **Input:** `coins = [1,2,5]`, `amount = 11`
 - **Output:** 3
 - **Explanation:** $11 = 5 + 5 + 1$
- **Input:** `coins = [2]`, `amount = 3`
 - **Output:** -1
- **Input:** `coins = [1]`, `amount = 0`
 - **Output:** 0

Constraints: $* 1 \leq \text{coins.length} \leq 12 * 1 \leq \text{coins}[i] \leq 2^{31} - 1 * 0 \leq \text{amount} \leq 10^4$

53.2 Detailed Solution (C++)

This problem represents the classic **unbounded knapsack** variation. Since we want to find the *minimum* number of coins to reach a target `amount`, we can construct a bottom-up Dynamic Programming table `dp` of size `amount + 1`.

- **State:** `dp[i]` represents the minimum number of coins required to make up amount `i`.
- **Initialization:** `dp[0] = 0` (zero coins needed for amount 0), and all other entries are initialized to a large value `INF = amount + 1` representing an unreachable state.
- **Transition:** For each amount `i` from 1 to `amount` and for each coin `c` in `coins`:

$$dp[i] = \min(dp[i], dp[i - c] + 1) \quad (\text{if } c \leq i)$$

53.2.1 Standard C++ Production Code

```
#include <vector>
#include <algorithm>

class Solution {
public:
    int coinChange(std::vector<int>& coins, int amount) {
        if (amount < 0) {
            return -1;
        }

        // Initialize table with amount + 1 (representing infinity)
        int INF = amount + 1;
        std::vector<int> dp(amount + 1, INF);
        dp[0] = 0;

        for (int i = 1; i <= amount; ++i) {
            for (int c : coins) {
                // Safely handle coin values that exceed current amount,
                // and guard against integer overflow in comparison.
                if (c <= i) {
                    dp[i] = std::min(dp[i], dp[i - c] + 1);
                }
            }
        }

        return dp[amount] < INF ? dp[amount] : -1;
    }
};
```

53.3 Detailed Solution (Python)

53.3.1 Standard Python Production Code

Below is the iterative bottom-up Python implementation.

```
from typing import List

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        """
        Computes the fewest number of coins needed to make up the given amount.

        Time Complexity:  $O(N * \text{Amount})$  where  $N$  is coins.length
        Space Complexity:  $O(\text{Amount})$ 
        """
        if amount < 0:
            return -1

        INF = amount + 1
        dp = [INF] * (amount + 1)
        dp[0] = 0

        for i in range(1, amount + 1):
            for coin in coins:
                if coin <= i:
                    dp[i] = min(dp[i], dp[i - coin] + 1)

        return dp[amount] if dp[amount] != INF else -1
```

53.4 Python-Specific Complexities & Implementation Considerations

53.4.1 1. Loop Order Optimization

- In the solution above, we loop over the `amount` in the outer loop and `coins` in the inner loop.
- Because `coins.length` is very small (≤ 12), this loop order allows `dp` array lookups to have good temporal locality.
- In Python, swapping the loops:

```
for coin in coins:
    for i in range(coin, amount + 1):
        dp[i] = min(dp[i], dp[i - coin] + 1)
```

can actually be slightly faster due to less conditional check overhead inside the inner loop and more efficient execution in the interpreter.

53.4.2 2. Large Coin Values

- Since $\text{coins}[i] \leq 2^{31} - 1$, some coin values could be far larger than `amount` (which is $\leq 10^4$). Checking if `coin <= i` is essential to prevent accessing negative indices or allocating an unnecessarily large table.

53.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(A \cdot N)$	Where A is the target <code>amount</code> and N is the number of coin denominations (<code>coins.length</code>).
Space Complexity	$\mathcal{O}(A)$	We allocate a dynamic programming state table of size <code>amount + 1</code> .

53.6 Common Follow-Up Questions & How to Answer

53.6.1 Q1: How would you return the actual coins used to form the amount, rather than just the count?

- **Answer:** We can maintain an additional array `parent` of size `amount + 1`, where `parent[i]` stores the coin value used to transition to state i (i.e., the coin c that minimized $\text{dp}[i - c] + 1$). After filling the table, if `dp[amount]` is valid, we can backtrack from `amount` using `amount = amount - parent[amount]` until we reach 0.

53.6.2 Q2: What if we wanted to solve this using BFS?

- **Answer:** Since all transition steps have a uniform edge weight of 1 coin, we can model the search space as a graph where each vertex is an amount and each edge is a coin transition. BFS can find the shortest path from 0 to `amount`. However, because we can visit the same state multiple times, we must keep a `visited` set. For large amounts, BFS could use more memory than the DP array due to queue sizes.

53.7 Pro-Tip: How to Impress the Interviewer

- **Mention Cache Line Exploitation:** Mention that the alternative loop ordering (for `coin` in `coins` as the outer loop) is highly cache-friendly. It sweeps sequentially through the `dp` array, accessing contiguous elements (`dp[i]` and `dp[i-coin]`), which makes excellent use of the CPU cache line prefetching.

- **Discuss BFS vs. DP Trade-offs:** In practice, if the target amount is small but there are many large coins, BFS might terminate very early when it reaches `amount` on the first few layers. Pointing out this transition from DP to BFS under specific input distributions shows you possess strong analytical design skills.

54 Frog Jump

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 403, Glassdoor
 - **Flashcards:** [Dynamic Programming deck](#)
-

54.1 Question Description

A frog is crossing a river. The river is divided into some number of units, and at each unit, there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

Given a list of `stones` positions (in units) in sorted ascending order, determine if the frog can cross the river by landing on the last stone. Initially, the frog is on the first stone and assumes the first jump must be 1 unit.

If the frog's last jump was k units, its next jump must be either $k - 1$, k , or $k + 1$ units. The frog can only jump in the forward direction.

54.1.1 Examples

- **Input:** `stones = [0,1,3,5,6,8,12,17]`
 - **Output:** `true`
 - **Explanation:** The frog can jump to the last stone by jumping 1 unit to the 2nd stone, then 2 units to the 3rd stone, then 2 units to the 4th stone, then 3 units to the 6th stone, 4 units to the 7th stone, and 5 units to the 8th stone.
- **Input:** `stones = [0,1,2,3,4,8,9,11]`
 - **Output:** `false`
 - **Explanation:** There is no way to jump to the last stone as the gap between the 5th and 6th stone is too large.

Constraints: $2 \leq \text{stones.length} \leq 2000$ * $0 \leq \text{stones}[i] \leq 2^{31} - 1$ * `stones[0] == 0` * `stones` is sorted in a strictly increasing order.

54.2 Detailed Solution (C++)

In C++, we can model this using a bottom-up Dynamic Programming table where `dp[i][k]` indicates whether the frog can reach the i -th stone with a last jump of size k . Since the maximum number of stones

N is at most 2000, and the maximum possible jump size cannot exceed N (since the jump size increases by at most 1 each step starting from 0), the maximum jump k is bounded by N .

- **State:** $dp[i][k]$ is a boolean indicating if stone i is reachable with a last jump of size k .
- **Transition:** From a reachable state $dp[i][k] == \text{true}$, the next jump can be $step \in \{k-1, k, k+1\}$ (where $step > 0$). If there exists a stone j at position $stones[i] + step$, we set $dp[j][step] = \text{true}$.
- We can search for the target position $stones[i] + step$ efficiently using **Binary Search** (`std::lower_bound` or manual binary search) on the sorted `stones` array.

54.2.1 Standard C++ Production Code

```
#include <vector>
#include <algorithm>

class Solution {
public:
    bool canCross(std::vector<int>& stones) {
        int n = static_cast<int>(stones.size());

        // Edge Case: The first jump must be exactly 1 unit.
        // If the second stone is not at position 1, the frog cannot make the first jump.
        if (n < 2 || stones[1] != 1) {
            return false;
        }

        // dp[i][k]: can we reach stone i with a last jump of k?
        // Since maximum jump step is <= n, a size of (n) x (n + 1) is sufficient.
        std::vector<std::vector<bool>> dp(n, std::vector<bool>(n + 1, false));
        dp[0][0] = true;

        for (int i = 0; i < n; ++i) {
            for (int k = 0; k <= n; ++k) {
                if (!dp[i][k]) {
                    continue;
                }

                // If k == 0 (initial state), we must jump exactly 1 unit.
                int min_step = (k == 0) ? 1 : std::max(1, k - 1);
                int max_step = k + 1;

                for (int step = min_step; step <= max_step; ++step) {
                    long long target = static_cast<long long>(stones[i]) + step;

                    // Binary search to find if target stone exists
                    auto it = std::lower_bound(stones.begin() + i + 1, stones.end(), target);
```

```

        if (it != stones.end() && *it == target) {
            int dest_idx = static_cast<int>(std::distance(stones.begin(), it));
            if (step <= n) {
                dp[dest_idx][step] = true;
            }
        }
    }
}

// Check if any jump size k can reach the last stone
for (int k = 0; k <= n; ++k) {
    if (dp[n - 1][k]) {
        return true;
    }
}

return false;
}
};

```

54.3 Detailed Solution (Python)

54.3.1 Standard Python Production Code

Below is the top-down memoized DFS implementation. We check existence of destination stones in $\mathcal{O}(1)$ average time using a set.

```
from typing import List
```

```
class Solution:
```

```
    def canCross(self, stones: List[int]) -> bool:
```

```
        """
```

```
        Determines if the frog can cross the river by landing on the last stone.
```

```
        Time Complexity:  $O(N^2)$ 
```

```
        Space Complexity:  $O(N^2)$ 
```

```
        """
```

```
        if len(stones) < 2 or stones[1] != 1:
```

```
            return False
```

```
        stone_set = set(stones)
```

```
        target_stone = stones[-1]
```

```
        memo = {}
```

```

def dfs(pos: int, last_jump: int) -> bool:
    # Base Case: Reached the last stone
    if pos == target_stone:
        return True

    state = (pos, last_jump)
    if state in memo:
        return memo[state]

    # Try jumps: last_jump - 1, last_jump, last_jump + 1
    # Initial state pos = 0, last_jump = 0 allows next jump of 1
    min_jump = max(1, last_jump - 1)
    for step in range(min_jump, last_jump + 2):
        next_pos = pos + step
        if next_pos in stone_set:
            if dfs(next_pos, step):
                memo[state] = True
                return True

    memo[state] = False
    return False

return dfs(0, 0)

```

54.4 Python-Specific Complexities & Implementation Considerations

54.4.1 1. Hash Map (memo) vs 2D Array

- In Python, using a dictionary memo with tuples (pos, last_jump) as keys is more memory-efficient than allocating a full 2000×2000 2D list because many combinations of (pos, last_jump) are unreachable.
- This also avoids pre-allocation overhead and maintains excellent readability.

54.4.2 2. Recursion Limit

- The maximum depth of the call stack is bounded by the number of stones $N = 2000$. Python's default recursion limit is 1000.
 - In a real interview, you should mention that you might need to adjust the recursion limit using `sys.setrecursionlimit(3000)` if N is large, or convert the DFS to an iterative stack-based solution to prevent RecursionError.
-

54.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n^2)$	In C++, there are n positions, each with up to n jump sizes. Binary search takes $\mathcal{O}(\log n)$ making it $\mathcal{O}(n^2 \log n)$ worst-case, but practically much faster. In Python, there are at most n^2 states visited, each taking $\mathcal{O}(1)$ due to set lookup.
Space Complexity	$\mathcal{O}(n^2)$	To store the visited states (size of the DP table or the hash map).

54.6 Common Follow-Up Questions & How to Answer

54.6.1 Q1: Can we use BFS instead of DFS?

- **Answer:** Yes, we can use a Queue to store pairs of (position, last_jump). We also maintain a visited set to avoid reprocessing the same state. BFS is often beneficial if a path exists early, but uses more memory for the queue in the worst-case.

54.6.2 Q2: What happens if the positions in stones are not sorted?

- **Answer:** We must sort the array first in $\mathcal{O}(n \log n)$ time. Sorting is required to perform binary searches and to easily identify the target last stone.

54.7 Pro-Tip: How to Impress the Interviewer

- **Proactively Catch Static State Bugs in C++:** Never use `static bool dp[2001][2001]` in shared application environments (like LeetCode or multi-threaded servers) without clearing it. Doing so will leak states from previous test runs, causing incorrect outputs. Utilizing `std::vector` ensures thread-safe, correct execution.
- **Explain the Jump Bound:** Prove mathematically why the jump size k cannot exceed N . Since we start with $k = 0$ and each jump can increase the jump size by at most 1, at the i -th stone, the maximum jump size is i . Since $i < n$, $k \leq n$. This justifies the $\mathcal{O}(n^2)$ state space bounds.

55 Count The Repetitions

- **Category:** Coding
- **Difficulty:** Hard
- **Target Role:** Software Engineer / AI Systems Architect
- **Source:** LeetCode 466, Glassdoor
- **Flashcards:** [Dynamic Programming deck](#)

55.1 Question Description

We define $str = [s, n]$ as the string str which consists of the string s concatenated n times. * For example, $["abc", 3] = "abcabcabc"$.

We define that string s_1 can be obtained from string s_2 if we can remove some characters from s_2 such that it becomes s_1 . * For example, "abc" can be obtained from "abdbec" because of the underlined characters "a_b_d_b_e_c_", but it cannot be obtained from "acbbe".

You are given two strings s_1 and s_2 and two integers n_1 and n_2 . You have the two strings $str_1 = [s_1, n_1]$ and $str_2 = [s_2, n_2]$.

Return the maximum integer m such that $str = [str_2, m]$ can be obtained from str_1 .

55.1.1 Examples

- **Input:** $s_1 = "acb", n_1 = 4, s_2 = "ab", n_2 = 2$
 - **Output:** 2
 - **Explanation:** $str_1 = "acbabcabcbac"$ $str_2 = "abab"$ We can obtain $str = ["abab", 2] = "abababab"$ from str_1 .
- **Input:** $s_1 = "acb", n_1 = 1, s_2 = "acb", n_2 = 1$
 - **Output:** 1

Constraints: * $1 \leq s_1.length, s_2.length \leq 100$ * s_1 and s_2 consist of lowercase English letters. * $1 \leq n_1, n_2 \leq 10^6$

55.2 Detailed Solution (C++)

For a brute-force approach, we could iterate through all copies of s_1 and match character-by-character with s_2 . However, since $n_1 \leq 10^6$ and the lengths of the strings are 100, simulating the entire match can require up to 10^8 operations, which is too slow in a high-concurrency setting.

55.2.1 Cycle Detection (Pigeonhole Principle)

We can optimize this to run in $\mathcal{O}(|s_1| \cdot |s_2|)$ time by detecting cycles. * We match s_2 against repetitions of s_1 one block at a time. * At the end of processing the i -th copy of s_1 , we record the state `s2_index` (the position in s_2 we are currently trying to match). * Since $|s_2| \leq 100$, there are only $|s_2|$ possible values for `s2_index`. By the Pigeonhole Principle, within at most $|s_2| + 1$ iterations of s_1 , we will encounter a `s2_index` that we have seen before. * When we find that we have seen the same `s2_index` after processing s_1 copy i as we did after copy `prev_i`, we have found a cycle! * The length of the cycle in terms of s_1 blocks is `cycle_len = i - prev_i`. * The number of times we matched s_2 completely within one cycle is `cycle_count = count - prev_count`. * We can skip matching for the remaining copies of s_1 mathematically:

$$\text{full_cycles} = \frac{n_1 - 1 - i}{\text{cycle_len}}$$

$$\text{count} \leftarrow \text{count} + \text{full_cycles} \times \text{cycle_count}$$

* Finally, we simulate the remaining fractional cycle of s_1 copies.

55.2.2 Standard C++ Production Code

```
#include <string>
#include <vector>
#include <algorithm>

class Solution {
public:
    int getMaxRepetitions(std::string s1, int n1, std::string s2, int n2) {
        if (n1 == 0) {
            return 0;
        }

        int s1_len = static_cast<int>(s1.size());
        int s2_len = static_cast<int>(s2.size());

        // Quick check: If there is any character in s2 that is not present in s1,
        // we can never match s2.
        std::vector<bool> s1_chars(26, false);
        for (char c : s1) {
            s1_chars[c - 'a'] = true;
        }
        for (char c : s2) {
            if (!s1_chars[c - 'a']) {
                return 0;
            }
        }

        // prev_iter[s2_idx] stores the iteration index i of s1 where s2_idx was matched.
        std::vector<int> prev_iter(s2_len + 1, -1);
        // prev_count[s2_idx] stores the count of fully matched s2 at that point.
        std::vector<int> prev_count(s2_len + 1, 0);

        int count = 0;
        int s2_idx = 0;
        bool found_cycle = false;

        for (int i = 0; i < n1; ++i) {
            for (int j = 0; j < s1_len; ++j) {
                if (s1[j] == s2[s2_idx]) {
                    s2_idx++;
                }
            }
        }
    }
};
```

```

        if (s2_idx == s2_len) {
            count++;
            s2_idx = 0;
        }
    }

    if (!found_cycle && prev_iter[s2_idx] >= 0) {
        found_cycle = true;
        int prev_i = prev_iter[s2_idx];
        int prev_cnt = prev_count[s2_idx];

        int cycle_len = i - prev_i;
        int cycle_count = count - prev_cnt;

        int remaining = n1 - 1 - i;
        int full_cycles = remaining / cycle_len;

        count += full_cycles * cycle_count;
        int processed = i + full_cycles * cycle_len + 1;

        // Simulate the remaining parts
        for (int ii = processed; ii < n1; ++ii) {
            for (int jj = 0; jj < s1_len; ++jj) {
                if (s1[jj] == s2[s2_idx]) {
                    s2_idx++;
                    if (s2_idx == s2_len) {
                        count++;
                        s2_idx = 0;
                    }
                }
            }
        }
        break;
    }

    if (!found_cycle) {
        prev_iter[s2_idx] = i;
        prev_count[s2_idx] = count;
    }
}

return count / n2;
}

```

```
};
```

55.3 Detailed Solution (Python)

55.3.1 Standard Python Production Code

Below is the clean Python implementation with type annotations and early cycle breaking.

class Solution:

```
def getMaxRepetitions(self, s1: str, n1: int, s2: str, n2: int) -> int:
    """
    Returns the maximum integer m such that [s2, m * n2] is a subsequence of [s1, n1].

    Time Complexity: O(|s1| * |s2|)
    Space Complexity: O(|s2|)
    """

    if n1 == 0:
        return 0

    s1_len, s2_len = len(s1), len(s2)

    # Verify characters
    if not set(s2).issubset(set(s1)):
        return 0

    # Mapping: s2_index -> (s1_index_i, total_matched_s2_count)
    index_map = {}
    count = 0
    s2_index = 0

    for i in range(n1):
        for j in range(s1_len):
            if s1[j] == s2[s2_index]:
                s2_index += 1
            if s2_index == s2_len:
                count += 1
                s2_index = 0

    # Check for cycle
    if s2_index in index_map:
        prev_i, prev_count = index_map[s2_index]

        cycle_len = i - prev_i
        cycle_count = count - prev_count
```

```

    remaining = n1 - 1 - i
    full_cycles = remaining // cycle_len

    count += full_cycles * cycle_count
    processed = i + full_cycles * cycle_len + 1

    # Simulate the remainder
    for ii in range(processed, n1):
        for jj in range(s1_len):
            if s1[jj] == s2[s2_index]:
                s2_index += 1
            if s2_index == s2_len:
                count += 1
                s2_index = 0
        break
    else:
        index_map[s2_index] = (i, count)

    return count // n2

```

55.4 Python-Specific Complexities & Implementation Considerations

55.4.1 1. Fast Character Set Verification

- We use Python's built-in set subset operation: `set(s2).issubset(set(s1))`.
- This takes $\mathcal{O}(|s_1| + |s_2|)$ time and is implemented in highly-optimized C code internally, which allows us to exit early with `0` in a fraction of a millisecond if any character in s_2 cannot be matched.

55.4.2 2. Hash Map Performance

- Using Python's dictionary `index_map` works seamlessly since integers are hashable. In C++, we used a fixed-size `std::vector` of size `s2_len + 1`, which avoids the overhead of hashing altogether. In Python, the small number of keys ($|s_2| \leq 100$) means hash collisions are non-existent, keeping lookups at near constant time.
-

55.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(s_1 \cdot s_2)$	We run the simulation until a cycle is found. The number of outer loop iterations is at most $ s_2 + 1$ (due to Pigeonhole Principle). In each iteration, we loop through $ s_1 $ characters.
Space Complexity	$\mathcal{O}(s_2)$	The space to store the visited states (<code>index_map</code> or <code>prev_iter</code> array of size $ s_2 $).

55.6 Common Follow-Up Questions & How to Answer

55.6.1 Q1: What if n_1 is extremely large (e.g., up to 10^{12})?

- **Answer:** The cycle-detection algorithm would still run in the same time complexity because the cycle is detected in at most $|s_2| + 1$ steps. The only modification required is to use a 64-bit integer (`long long` in C++ or Python's native arbitrary precision integers) to avoid integer overflow when multiplying `full_cycles * cycle_count`.

55.6.2 Q2: Why is the outer loop index bound by $|s_2|$ rather than $|s_1|$?

- **Answer:** Because the state transitions are defined by where we land in s_2 at the boundary of s_1 blocks. Since the pointer in s_1 resets to 0 at the start of each block, the only state parameter that determines the future transitions is `s2_index`. The number of unique states for `s2_index` is exactly $|s_2|$.

55.7 Pro-Tip: How to Impress the Interviewer

- **Pigeonhole Principle Proof:** Walk the interviewer through the Pigeonhole Principle proof for why cycle detection works. Specifically, note that because we are evaluating `s2_index` at the end of each s_1 repetition, the number of distinct values of `s2_index` is bounded by $|s_2|$. This guarantees that we will see a duplicate state in at most $|s_2| + 1$ blocks.
- **Pre-filtering with Set Algebra:** Highlighting the early character existence check demonstrates attention to realistic performance and edge cases. In production systems, checking constraints early saves CPU cycles.

56 Unique Substrings in Wraparound String

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / AI Systems Architect

- **Source:** LeetCode 467, Glassdoor
- **Flashcards:** [Dynamic Programming deck](#)

56.1 Question Description

We define the string **base** to be the infinite wraparound string of "abcdefghijklmnopqrstuvwxyz", so **base** looks like this: * "...zabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz..."

Given a string *s*, return *the number of unique non-empty substrings of s that are present in base*.

56.1.1 Examples

- **Input:** *s* = "a"
 - **Output:** 1
 - **Explanation:** Only the substring "a" of *s* is in **base**.
- **Input:** *s* = "cac"
 - **Output:** 2
 - **Explanation:** There are two substrings ("a", "c") of *s* in **base**. Note that the second "c" is a duplicate of the first one, and we only count unique substrings.
- **Input:** *s* = "zab"
 - **Output:** 6
 - **Explanation:** There are six substrings ("z", "a", "b", "za", "ab", and "zab") of *s* in **base**.

Constraints: * $1 \leq s.length \leq 10^5$ * *s* consists of lowercase English letters.

56.2 Detailed Solution (C++)

A brute-force solution would generate all substrings and check if they exist in **base**, which would be $\mathcal{O}(n^2)$ time and space, resulting in TLE/MLE.

To optimize: 1. Any valid substring in **base** must consist of contiguous characters where the character $s[i]$ is the alphabetical successor of $s[i-1]$ (with 'a' succeeding 'z'). That is:

$$s[i] - s[i-1] == 1 \quad \text{or} \quad s[i-1] - s[i] == 25$$

2. If we find the longest valid wraparound substring ending at a specific character (e.g., 'c'), say of length *L*, then there are exactly *L* unique substrings ending with 'c' in that window (e.g., if the substring is "abc", the substrings ending in 'c' are "c", "bc", and "abc"). 3. Any other shorter substring ending at 'c' found elsewhere in *s* is a suffix of this longest one and has already been accounted for. 4. Therefore, we can track the maximum length of a valid wraparound substring ending at each of the 26 lowercase English letters using an array `max_len` of size 26. 5. The total number of unique substrings is the sum of `max_len[i]` for all $0 \leq i < 26$.

56.2.1 Standard C++ Production Code

```
#include <string>
#include <vector>
#include <numeric>
#include <algorithm>

class Solution {
public:
    int findSubstringInWraparoundString(std::string s) {
        if (s.empty()) {
            return 0;
        }

        // max_len[i] stores the length of the longest valid substring ending with character ('a' + i)
        std::vector<int> max_len(26, 0);
        int curr_len = 0;
        int n = static_cast<int>(s.size());

        for (int i = 0; i < n; ++i) {
            // Check if s[i] is consecutive to s[i-1] in wraparound sense
            if (i > 0 && ((s[i] - s[i - 1] == 1) || (s[i - 1] - s[i] == 25))) {
                curr_len++;
            } else {
                curr_len = 1;
            }

            int idx = s[i] - 'a';
            max_len[idx] = std::max(max_len[idx], curr_len);
        }

        // The sum of all values in max_len gives the total number of unique substrings
        return std::accumulate(max_len.begin(), max_len.end(), 0);
    }
};
```

56.3 Detailed Solution (Python)

56.3.1 Standard Python Production Code

Below is the clean Python implementation with type hints.

```
class Solution:
    def findSubstringInWraparoundString(self, s: str) -> int:
        """
```


Finds the number of unique non-empty substrings of s that exist in base.

Time Complexity: $O(N)$

Space Complexity: $O(1)$

```

"""
if not s:
    return 0

max_len = [0] * 26
curr_len = 0

for i, ch in enumerate(s):
    # Check if current character continues the wraparound pattern
    if i > 0 and (
        ord(s[i]) - ord(s[i - 1]) == 1 or ord(s[i - 1]) - ord(s[i]) == 25
    ):
        curr_len += 1
    else:
        curr_len = 1

    idx = ord(ch) - ord("a")
    max_len[idx] = max(max_len[idx], curr_len)

return sum(max_len)

```

56.4 Python-Specific Complexities & Implementation Considerations

56.4.1 1. Character to Integer Conversion

- In Python, we use the `ord()` function to get the ASCII value of a character (e.g., `ord('a') = 97`).
- Unlike C++, we cannot directly subtract character objects like `s[i] - s[i-1]`. We must wrap them in `ord()`.

56.4.2 2. Space Optimization

- The space footprint is strictly $\mathcal{O}(1)$ since the size of the tracking array is constant (26). This is extremely efficient and garbage collector-friendly.
-

56.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We iterate through the string <code>s</code> of length n exactly once.
Space Complexity	$\mathcal{O}(1)$	Auxiliary space is bounded by the size of the alphabet (26), which is constant.

56.6 Common Follow-Up Questions & How to Answer

56.6.1 Q1: Why does summing the maximum length of substrings ending at each character guarantee uniqueness?

- **Answer:** By grouping substrings by their *ending character*, we partition the set of all valid substrings into 26 disjoint subsets. For any ending character `c`, a valid substring of length L uniquely determines the string (it consists of the L characters in `base` ending at `c`). Any valid substring of length $K < L$ ending at `c` is a suffix of the substring of length L , meaning it is entirely contained within it. Thus, only taking the maximum length L for each character ensures we count every unique substring exactly once.

56.6.2 Q2: What if the base string was not a wraparound string (e.g., 'z' does not wrap to 'a')?

- **Answer:** The logic remains identical, except we remove the wrap-around condition `s[i-1] - s[i] == 25` from the consecutive check.

56.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Deduplication Collections:** A common sub-optimal approach is to store all valid substrings in a hash set (e.g., `std::unordered_set<std::string>` or Python `set()`) to deduplicate them. Point out that this uses $\mathcal{O}(n^2)$ space and time in the worst case (e.g., "abcdefg..." has $\mathcal{O}(n^2)$ unique substrings), causing Memory Limit Exceeded (MLE). Showcasing how grouping by ending character reduces the space complexity to $\mathcal{O}(1)$ exhibits excellent systems-level optimization skills.
- **Mention Cache Locality:** Point out that the C++ version utilizes a small 26-element stack array which easily fits in L1 cache (and even registers), resulting in zero heap allocation overhead and maximum CPU execution speed.

57 Freedom Trail

- **Category:** Coding
- **Difficulty:** Hard
- **Target Role:** Software Engineer / AI Systems Architect
- **Source:** LeetCode 514, Glassdoor

- **Flashcards:** [Dynamic Programming deck](#)
-

57.1 Question Description

In the video game Fallout 4, the quest "Road to Freedom" requires players to reach a metal dial called the "Freedom Trail Ring" and use the dial to spell a specific keyword to open the door.

Given a string `ring` that represents the code engraved on the outer ring and another string `key` that represents the keyword that needs to be spelled, return *the minimum number of steps to spell all the characters in the keyword*.

Initially, the first character of the `ring` is aligned at the "12:00" direction. You should spell all the characters in `key` one by one by rotating `ring` clockwise or anticlockwise to make each character of the string `key` aligned at the "12:00" direction and then by pressing the center button.

At the stage of rotating the ring to spell the key character `key[i]`: * You can rotate the ring clockwise or anticlockwise by one place, which counts as 1 step. The coordinate of the character on the ring to be spelled is aligned at the "12:00" direction. * Spelling a character counts as 1 step.

57.1.1 Examples

- **Input:** `ring = "godding", key = "gd"`
 - **Output:** 4
 - **Explanation:**
 - * For the first key character 'g', since it is already at index 0 (the "12:00" direction), we just need 1 step to press the center button.
 - * For the second key character 'd', we can rotate the ring anticlockwise by 2 steps to align 'd' (at index 2 or 3) at "12:00". Let's align the 'd' at index 2. This takes 2 steps of rotation.
 - * Pressing the center button to spell 'd' takes 1 step.
 - * Total steps: $1 + 2 + 1 = 4$.
- **Input:** `ring = "godding", key = "godding"`
 - **Output:** 13

Constraints: * $1 \leq \text{ring.length}, \text{key.length} \leq 100$ * `ring` and `key` consist of only lowercase English letters.
 * It is guaranteed that `key` can always be spelled.

57.2 Detailed Solution (C++)

This problem can be modeled as finding the shortest path through a multi-stage decision graph, which is best solved using Dynamic Programming.

- **State:** `dp[pos]` represents the minimum steps needed to spell the remaining suffix of the key (from index `ki` to the end) when the dial is currently aligned at index `pos` of the `ring`.
- **Transition:** To spell the character `key[ki]`, we must rotate the dial from the current position `pos` to some target position `target` where `ring[target] == key[ki]`.

- The cost to rotate from `pos` to `target` in a circular ring of length n is:

$$\text{rotation_steps} = \min(|pos - target|, n - |pos - target|)$$

- The total cost for this step is `rotation_steps + 1` (1 step for spelling/pressing the button).
- Therefore, the transition is:

$$\text{new_dp}[pos] = \min_{\text{target s.t. ring[target]=key[ki]}} \left(\text{rotation_steps} + 1 + \text{dp}[target] \right)$$

We compute the states bottom-up starting from the last character of the key ($ki = \text{key.length} - 1$ down to 0). Finally, we return `dp[0]` because the ring starts aligned at index 0.

57.2.1 Standard C++ Production Code

```
#include <string>
#include <vector>
#include <algorithm>
#include <limits>
#include <cmath>

class Solution {
public:
    int findRotateSteps(std::string ring, std::string key) {
        int n = static_cast<int>(ring.size());
        int klen = static_cast<int>(key.size());

        // Precompute the positions of each character in the ring to avoid scanning sl on every transition.
        std::vector<std::vector<int>> positions(26);
        for (int i = 0; i < n; ++i) {
            positions[ring[i] - 'a'].push_back(i);
        }

        // dp[pos] stores the min steps to spell the key suffix from current ring position pos
        std::vector<int> dp(n, 0);

        // Process bottom-up from the last key character to the first key character
        for (int ki = klen - 1; ki >= 0; --ki) {
            std::vector<int> new_dp(n, INT_MAX);
            int ch = key[ki] - 'a';

            // Find best target on the ring for current key[ki] for every starting position pos
            for (int pos = 0; pos < n; ++pos) {
                for (int target : positions[ch]) {
                    int diff = std::abs(pos - target);
                    int steps = std::min(diff, n - diff) + 1; // Rotation steps + 1 step to press button
```

```

        int cost = (ki == klen - 1) ? steps : steps + dp[target];
        if (cost < new_dp[pos]) {
            new_dp[pos] = cost;
        }
    }
    dp = std::move(new_dp);
}

return dp[0];
};

```

57.3 Detailed Solution (Python)

57.3.1 Standard Python Production Code

Below is the space-optimized bottom-up Python implementation using a `defaultdict` for character positions.

```

from collections import defaultdict
from typing import Dict, List

class Solution:
    def findRotateSteps(self, ring: str, key: str) -> int:
        """
        Spells key using the minimum steps of rotating ring.

        Time Complexity: O(K * R^2) where K is key.length, R is ring.length.
        Space Complexity: O(R)
        """
        n = len(ring)
        klen = len(key)

        # Precompute positions of each character
        char_positions: Dict[str, List[int]] = defaultdict(list)
        for i, ch in enumerate(ring):
            char_positions[ch].append(i)

        dp = [0] * n

        for ki in range(klen - 1, -1, -1):
            new_dp = [float("inf")] * n

```

```

target_indices = char_positions[key[ki]]

for pos in range(n):
    for target in target_indices:
        diff = abs(pos - target)
        steps = min(diff, n - diff) + 1

        cost = steps if ki == klen - 1 else steps + dp[target]
        if cost < new_dp[pos]:
            new_dp[pos] = cost
    dp = new_dp

return dp[0]

```

57.4 Python-Specific Complexities & Implementation Considerations

57.4.1 1. Precomputing Character Positions

- Scanning `ring` inside the DP transitions to find matching characters would degrade performance to $\mathcal{O}(K \cdot R^2 \cdot R) = \mathcal{O}(K \cdot R^3)$.
- Using a `defaultdict` to map each character to its index list reduces the inner loop complexity significantly, as we only iterate over indexes that actually match `key[ki]`.

57.4.2 2. Move Semantics in Python vs C++

- In Python, assigning `dp = new_dp` is a constant time reference reassignment.
 - In C++, using `dp = std::move(new_dp)` avoids deep-copying the vector elements, ensuring standard $\mathcal{O}(1)$ transfer of resources.
-

57.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(K \cdot R^2)$	Where K is the length of <code>key</code> and R is the length of <code>ring</code> . For each of the K stages, we perform $R \times R$ work in the worst case (if all characters in <code>ring</code> are the same).
Space Complexity	$\mathcal{O}(R)$	We only store two arrays (<code>dp</code> and <code>new_dp</code>) of size R , plus the character position lists of combined size R .

57.6 Common Follow-Up Questions & How to Answer

57.6.1 Q1: Can we use Dijkstra's Algorithm instead of DP?

- **Answer:** Yes. We can view this as finding the shortest path in a layered graph where layer i contains nodes corresponding to the matching positions of `key[i]` in the ring. The number of states is $K \cdot R$. Dijkstra's algorithm would find the shortest path in $\mathcal{O}(E \log V)$ which is $\mathcal{O}(K \cdot R^2 \log(K \cdot R))$. For the constraints $K, R \leq 100$, DP runs faster and uses less memory because it does not have priority queue overhead.

57.6.2 Q2: What if we can also rotate the ring clockwise or anticlockwise multiple steps per operation?

- **Answer:** The question already supports rotating multiple steps. If it meant rotating by any distance d in 1 operation (e.g. random access dial), then $\min(\text{diff}, n - \text{diff})$ would simply be constant cost 1.

57.7 Pro-Tip: How to Impress the Interviewer

- **Highlight Cache Friendliness:** Discuss that since the `dp` vector size is at most 100, the data fits entirely in L1 cache. The memory layout is contiguous, minimizing cache misses.
- **State Pruning:** Explain that instead of calculating `new_dp[pos]` for all pos from 0 to $n - 1$, we can restrict `pos` to only the indices of the ring containing `key[ki-1]` (for $ki > 0$) or index 0 (for $ki = 0$). This reduces the constant factor dramatically in cases where the character frequencies are small.

58 Longest Palindromic Subsequence

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / AI Systems Architect
- **Source:** LeetCode 516, Glassdoor
- **Flashcards:** [Dynamic Programming deck](#)

58.1 Question Description

Given a string s , find *the longest palindromic subsequence's length in s* .

A **subsequence** is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

58.1.1 Examples

- **Input:** $s = \text{"bbbab"}$

- **Output:** 4
- **Explanation:** One possible longest palindromic subsequence is "bbbb".
- **Input:** `s = "cbbd"`
 - **Output:** 2
 - **Explanation:** One possible longest palindromic subsequence is "bb".

Constraints: * $1 \leq s.length \leq 1000$ * `s` consists only of lowercase English letters.

58.2 Detailed Solution (C++)

This is a classical 2D Dynamic Programming problem.

- **State:** `dp[i][j]` represents the length of the longest palindromic subsequence in the substring `s[i...j]`.
- **Base Cases:**
 - Single characters: `dp[i][i] = 1` for all `i`.
 - Double characters: If `s[i] == s[i+1]`, `dp[i][i+1] = 2`, else `dp[i][i+1] = 1`.
- **Transition:** For a substring from index `i` to `j` (where $j - i \geq 1$):
 - If `s[i] == s[j]`:

$$dp[i][j] = dp[i+1][j-1] + 2$$

- If `s[i] != s[j]`:

$$dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$$

To compute this bottom-up, we can iterate over the length of the substring from 2 to n , and then sweep the starting index `i`.

58.2.1 Standard C++ Production Code

```
#include <string>
#include <vector>
#include <algorithm>

class Solution {
public:
    int longestPalindromeSubseq(std::string s) {
        int n = static_cast<int>(s.size());
        if (n == 0) {
            return 0;
        }

        // dp[i][j] stores the length of LPS in s[i...j]
        std::vector<std::vector<int>> dp(n, std::vector<int>(n, 0));

        // Base cases: single character substrings
        for (int i = 0; i < n; ++i) {
```



```

        dp[i][i] = 1;
    }

    // Process substrings of length 2 to n
    for (int length = 2; length <= n; ++length) {
        for (int i = 0; i <= n - length; ++i) {
            int j = i + length - 1;
            if (s[i] == s[j]) {
                dp[i][j] = dp[i + 1][j - 1] + 2;
            } else {
                dp[i][j] = std::max(dp[i + 1][j], dp[i][j - 1]);
            }
        }
    }

    return dp[0][n - 1];
}
};

```

58.3 Detailed Solution (Python)

58.3.1 Standard Python Production Code

Below is the space-optimized 1D Python implementation. Because calculating `dp[i][j]` only requires values from the previous column/row, we can optimize space to $\mathcal{O}(n)$.

class Solution:

```

def longestPalindromeSubseq(self, s: str) -> int:
    """
    Computes the length of the longest palindromic subsequence.

    Time Complexity:  $O(N^2)$ 
    Space Complexity:  $O(N)$ 
    """
    n = len(s)
    if n == 0:
        return 0

    # dp[j] stores the LPS value for current i and ending index j.
    # We process i from n-1 down to 0, updating in-place.
    dp = [0] * n

    for i in range(n - 1, -1, -1):
        dp[i] = 1 # Base Case: s[i...i] has length 1

```

```

prev_diagonal = 0 # Represents dp[i+1][j-1]

for j in range(i + 1, n):
    temp = dp[j] # Store dp[i+1][j] before it gets overwritten
    if s[i] == s[j]:
        dp[j] = prev_diagonal + 2
    else:
        dp[j] = max(dp[j], dp[j - 1])
    prev_diagonal = temp

return dp[n - 1]

```

58.4 Python-Specific Complexities & Implementation Considerations

58.4.1 1. Space Optimization to 1D Array

- In C++, a 2D array of size 1000×1000 uses about 4MB of memory, which is negligible.
 - In Python, lists of lists are collections of pointers to integers, incurring higher memory overhead (~8 bytes per pointer + ~28 bytes per integer object). Thus, a 2D grid of size 1000×1000 can consume up to 8-10MB and add GC pressure.
 - By collapsing the 2D DP array into a single 1D array `dp` and updating it from right to left (or dynamically tracking the diagonal element), we reduce space complexity to a single array of size 1000, which is highly optimal.
-

58.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n^2)$	We fill out the DP table of size $n \times n$ using nested loops.
Space Complexity	$\mathcal{O}(n^2)$ or $\mathcal{O}(n)$	$\mathcal{O}(n^2)$ for the standard 2D table, optimized to $\mathcal{O}(n)$ using a 1D state array.

58.6 Common Follow-Up Questions & How to Answer

58.6.1 Q1: How does this relate to Longest Common Subsequence (LCS)?

- **Answer:** The Longest Palindromic Subsequence of a string `s` is equivalent to the Longest Common Subsequence of `s` and its reverse `s' = reverse(s)`.

$$\text{LPS}(s) = \text{LCS}(s, \text{reverse}(s))$$

This is an elegant alternative implementation that also runs in $\mathcal{O}(n^2)$ time and $\mathcal{O}(n)$ space.

58.6.2 Q2: How can we reconstruct the actual longest palindromic subsequence string?

- **Answer:** Using the 2D DP table `dp`, we can backtrack from `(0, n-1)`:
 - If `s[i] == s[j]`, then `s[i]` and `s[j]` are part of the palindrome. We append `s[i]` to our result and move to `(i+1, j-1)`.
 - If `s[i] != s[j]`, we compare `dp[i+1][j]` and `dp[i][j-1]` and move in the direction of the larger value.
 - Finally, we assemble the left and right halves to build the full palindrome string.
-

58.7 Pro-Tip: How to Impress the Interviewer

- **Explain LCS Duality:** Mentioning that $\text{LPS}(s) = \text{LCS}(s, \text{reverse}(s))$ immediately shows you see deep connections between different dynamic programming problems rather than just memorizing code.
- **Implement Space Compression:** Demonstrating the transition from a 2D state table to a 1D state table by tracking the `prev_diagonal` variable is an advanced technique that demonstrates strong algorithmic prowess.

59 Coin Change II

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 518, Glassdoor
 - **Flashcards:** [Dynamic Programming deck](#)
-

59.1 Question Description

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the number of combinations that make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `0`.

You may assume that you have an infinite number of each kind of coin.

The answer is guaranteed to fit into a signed 32-bit integer.

59.1.1 Examples

- **Input:** `amount = 5, coins = [1,2,5]`
 - **Output:** 4
 - **Explanation:** There are four ways to make up the amount:

1. $5 = 5$
2. $5 = 2 + 2 + 1$
3. $5 = 2 + 1 + 1 + 1$
4. $5 = 1 + 1 + 1 + 1 + 1$

- **Input:** amount = 3, coins = [2]
 - **Output:** 0
 - **Explanation:** The amount of 3 cannot be made up just with coins of 2.
- **Input:** amount = 10, coins = [10]
 - **Output:** 1

Constraints: * $1 \leq \text{coins.length} \leq 300$ * $1 \leq \text{coins}[i] \leq 5000$ * All the values of coins are **unique**. * $0 \leq \text{amount} \leq 5000$

59.2 Detailed Solution (C++)

This is another variation of the **unbounded knapsack** problem. Unlike the original Coin Change problem where we seek the minimum number of coins, here we need to find the number of unique combinations.

59.2.1 Transition and Loop Order

Let $\text{dp}[a]$ be the number of combinations to make up amount a . We initialize $\text{dp}[0] = 1$ (there is exactly 1 way to make amount 0: using no coins).

To avoid counting permutations (e.g. counting $1 + 2$ and $2 + 1$ as separate combinations), **the outer loop must iterate over the coins**, and the inner loop must iterate over the amounts.

$$\text{dp}[a] \leftarrow \text{dp}[a] + \text{dp}[a - \text{coin}]$$

By processing one coin denomination at a time, we ensure that coins are used in a non-decreasing order of index, eliminating ordering duplicates.

59.2.2 Standard C++ Production Code

```
#include <vector>

class Solution {
public:
    int change(int amount, std::vector<int>& coins) {
        if (amount < 0) {
            return 0;
        }

        // dp[a] stores the number of combinations to form amount a
        std::vector<int> dp(amount + 1, 0);
        dp[0] = 1;
```

```

    // Loop over each coin first to prevent counting permutations as different combinations
    for (int coin : coins) {
        for (int a = coin; a <= amount; ++a) {
            dp[a] += dp[a - coin];
        }
    }

    return dp[amount];
}
};

```

59.3 Detailed Solution (Python)

59.3.1 Standard Python Production Code

Below is the type-hinted Python implementation.

```

from typing import List

class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        """
        Computes the number of combinations that make up the given amount.

        Time Complexity: O(N * Amount) where N is coins.length
        Space Complexity: O(Amount)
        """
        if amount < 0:
            return 0

        dp = [0] * (amount + 1)
        dp[0] = 1

        for coin in coins:
            for a in range(coin, amount + 1):
                dp[a] += dp[a - coin]

        return dp[amount]

```

59.4 Python-Specific Complexities & Implementation Considerations

59.4.1 1. Swapping Loop Order (The Permutations Trap)

- A common Python mistake is to write the outer loop over the range of amounts:

```
# WARNING: THIS COMPUTES PERMUTATIONS, NOT COMBINATIONS
for a in range(1, amount + 1):
    for coin in coins:
        if coin <= a:
            dp[a] += dp[a - coin]
```

This computes the number of ordered sequences. For example, with `amount = 3` and `coins = [1, 2]`, this incorrect version would output 3 (for sequences [1, 1, 1], [1, 2], and [2, 1]). The correct combinations version outputs 2 (since [1, 2] and [2, 1] are the same combination).

59.4.2 2. Large Precision

- While the problem constraints guarantee the answer fits in a 32-bit signed integer, Python automatically handles integer expansions so there's never any risk of overflow errors even if constraints are increased.

59.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N \cdot A)$	Where N is the number of coins and A is the target <code>amount</code> .
Space Complexity	$\mathcal{O}(A)$	We maintain a 1D DP table of size <code>amount + 1</code> .

59.6 Common Follow-Up Questions & How to Answer

59.6.1 Q1: How do we modify the algorithm to compute permutations (where order matters)?

- Answer:** As explained above, we simply swap the loops. We iterate over all amounts in the outer loop, and for each amount, we iterate over all coins:

$$dp[a] = \sum_{\text{coin}} dp[a - \text{coin}]$$

59.6.2 Q2: What if we are limited in the number of coins we can use (e.g. 0-1 Knapsack where each coin can only be used once)? (Coin Change II with single-use coins)

- Answer:** This changes from unbounded knapsack to 0-1 knapsack. To do this with $\mathcal{O}(A)$ space, we must iterate the inner amount loop **backward** (from `amount` down to `coin`):

```
for (int coin : coins) {
    for (int a = amount; a >= coin; --a) {
        dp[a] += dp[a - coin];
    }
}
```

```
    }
}
```

Iterating backward prevents using the same coin multiple times, because we calculate `dp[a]` using the state of the *previous* iteration's `dp[a - coin]`.

59.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Order of Loops Clear-headedly:** The most important differentiator in this problem is explaining *why* the loop order controls combinations vs. permutations. Explaining that the outer loop forces a fixed lexicographical ordering of coin selections (e.g., we choose all 1s, then all 2s, then all 5s) shows a deep understanding of state space representations in Dynamic Programming.
- **Highlight Cache Efficiency:** Mention that iterating the amount index sequentially in the inner loop (`for (int a = coin; a <= amount; a++)`) provides sequential memory access which maximizes CPU cache hit rates.

60 Student Attendance Record II

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 552, Glassdoor
 - **Flashcards:** [Dynamic Programming deck](#)
-

60.1 Question Description

An attendance record for a student can be represented as a string where each character signifies whether the student was absent ('A'), late ('L'), or present ('P') on that day.

A student is eligible for an attendance award if they meet **both** of the following criteria: 1. The student was absent ('A') for **strictly fewer than 2 days** in total. 2. The student was late ('L') for **fewer than 3 consecutive days** (i.e., they were never late 3 or more days in a row).

Given an integer n , return *the number of possible attendance records of length n that make a student eligible for an attendance award*. The answer may be very large, so return it **modulo** $10^9 + 7$.

60.1.1 Examples

- **Input:** $n = 2$
 - **Output:** 8
 - **Explanation:** There are 8 records with length 2 that are eligible for an award: "PP", "AP", "PA", "LP", "PL", "AL", "LA", "LL" Only "AA" is not eligible because there are 2 absences.
- **Input:** $n = 1$
 - **Output:** 3

- **Input:** `n = 10101`
 - **Output:** `183236316`

Constraints: * $1 \leq n \leq 10^5$

60.2 Detailed Solution (C++)

This problem can be represented as a finite state machine (FSM) where the state is defined by: 1. The total count of absences (`a`): either `0` or `1`. (If ≥ 2 , it is invalid). 2. The count of consecutive late days at the end of the current record (`l`): either `0`, `1`, or `2`. (If ≥ 3 , it is invalid).

Thus, we have $2 \times 3 = 6$ valid states: * `dp[i][a][l]`: the number of eligible records of length `i` with `a` total absences and `l` consecutive late days at the end.

60.2.1 Transitions

For each state (`a`, `l`) at length `i`, we can transition to length `i + 1` by appending one of: 1. **Present ('P')**: Resets consecutive late days to `0`. Total absences `a` remains the same. * `dp[i+1][a][0] += dp[i][a][l]` 2. **Absent ('A')**: Resets consecutive late days to `0`. Increments total absences to `a + 1` (valid if $a < 1$). * `dp[i+1][a+1][0] += dp[i][a][l]` (if $a < 1$) 3. **Late ('L')**: Increments consecutive late days to `l + 1` (valid if $l < 2$). Total absences `a` remains the same. * `dp[i+1][a][l+1] += dp[i][a][l]` (if $l < 2$)

60.2.2 Space Optimization

Since `dp[i+1]` only depends on `dp[i]`, we can compress the space complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ by using a rolling array of size `2` along the first dimension (index `i % 2`).

60.2.3 Standard C++ Production Code

```
#include <vector>
#include <numeric>

class Solution {
public:
    int checkRecord(int n) {
        constexpr int MOD = 1'000'000'007;

        // dp[2][a][l] space-optimized DP table
        // dp[i % 2][a][l]: a in {0, 1}, l in {0, 1, 2}
        long long dp[2][2][3] = {};

        // Base case: at length 0, there is 1 way (empty string)
        dp[0][0][0] = 1;

        for (int i = 0; i < n; ++i) {
            int cur = i % 2;
```



```

    int nxt = 1 - cur;

    // Clear the next state's row
    for (int a = 0; a < 2; ++a) {
        for (int l = 0; l < 3; ++l) {
            dp[nxt][a][l] = 0;
        }
    }

    // Distribute values to next states
    for (int a = 0; a < 2; ++a) {
        for (int l = 0; l < 3; ++l) {
            long long val = dp[cur][a][l];
            if (val == 0) {
                continue;
            }

            // 1. Append 'P'
            dp[nxt][a][0] = (dp[nxt][a][0] + val) % MOD;

            // 2. Append 'A' (only if we currently have 0 absences)
            if (a < 1) {
                dp[nxt][a + 1][0] = (dp[nxt][a + 1][0] + val) % MOD;
            }

            // 3. Append 'L' (only if we have fewer than 2 consecutive late days)
            if (l < 2) {
                dp[nxt][a][l + 1] = (dp[nxt][a][l + 1] + val) % MOD;
            }
        }
    }
}

int last = n % 2;
long long result = 0;
for (int a = 0; a < 2; ++a) {
    for (int l = 0; l < 3; ++l) {
        result = (result + dp[last][a][l]) % MOD;
    }
}

return static_cast<int>(result);
}
};

```

60.3 Detailed Solution (Python)

60.3.1 Standard Python Production Code

Below is the space-optimized Python implementation using an $\mathcal{O}(1)$ space array.

class Solution:

```
def checkRecord(self, n: int) -> int:
```

```
    """
```

```
    Returns the number of possible attendance records of length n eligible for award.
```

```
    Time Complexity:  $\mathcal{O}(N)$ 
```

```
    Space Complexity:  $\mathcal{O}(1)$ 
```

```
    """
```

```
    MOD = 10**9 + 7
```

```
    # dp[a][l] stores the count of records for current day
```

```
    # a in {0, 1}, l in {0, 1, 2}
```

```
    dp = [[0] * 3 for _ in range(2)]
```

```
    dp[0][0] = 1
```

```
    for _ in range(n):
```

```
        new_dp = [[0] * 3 for _ in range(2)]
```

```
        for a in range(2):
```

```
            for l in range(3):
```

```
                val = dp[a][l]
```

```
                if val == 0:
```

```
                    continue
```

```
            # 1. Append 'P'
```

```
            new_dp[a][0] = (new_dp[a][0] + val) % MOD
```

```
            # 2. Append 'A'
```

```
            if a < 1:
```

```
                new_dp[a + 1][0] = (new_dp[a + 1][0] + val) % MOD
```

```
            # 3. Append 'L'
```

```
            if l < 2:
```

```
                new_dp[a][l + 1] = (new_dp[a][l + 1] + val) % MOD
```

```
        dp = new_dp
```

```
    # Sum up all valid states
```

```

result = sum(sum(row) for row in dp) % MOD
return result

```

60.4 Python-Specific Complexities & Implementation Considerations

60.4.1 1. Multi-Dimensional Lists vs Flattening

- In Python, initializing multi-dimensional lists (e.g. `[[0] * 3 for _ in range(2)]`) is simple.
- Since the size is extremely small (2×3), we do not need to flatten the list into a 1D list of size 6. The overhead of multi-dimensional indexing in Python is negligible for a fixed small state space.

60.4.2 2. Time Limit Exceeded (TLE) Risks in Python

- Python code execution is slower than compiled languages like C++. For $N = 10^5$, a triple loop with modulo arithmetic inside can run close to the time limit if not written cleanly.
 - By using the space-optimized structure and checking `if val == 0: continue`, we prune unvisited states and significantly accelerate runtime.
-

60.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We loop n times. For each step, we perform at most $2 \times 3 \times 3$ transitions, which is a constant number of operations.
Space Complexity	$\mathcal{O}(1)$	The DP state is maintained using a fixed table of size 2×3 , resulting in constant auxiliary memory.

60.6 Common Follow-Up Questions & How to Answer

60.6.1 Q1: What if n is up to 10^{18} ?

- **Answer:** Since the state transitions are linear, we can represent them using **Matrix Exponentiation**. Let our state vector of size 6 be:

$$V_i = [dp_i[0][0], dp_i[0][1], dp_i[0][2], dp_i[1][0], dp_i[1][1], dp_i[1][2]]^T$$

The transition can be represented as:

$$V_{i+1} = M \times V_i$$

where M is a 6×6 transition matrix. We can calculate M^n in $\mathcal{O}(6^3 \log n)$ time using binary exponentiation, which makes solving for $n = 10^{18}$ extremely fast.

60.6.2 Q2: How does the transition matrix M look?

- **Answer:** Let state indices be $0 \rightarrow (0, 0)$, $1 \rightarrow (0, 1)$, $2 \rightarrow (0, 2)$, $3 \rightarrow (1, 0)$, $4 \rightarrow (1, 1)$, $5 \rightarrow (1, 2)$. The transition matrix M would be:

$$M = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Multiplying this matrix by the state vector gives the exact transitions.

60.7 Pro-Tip: How to Impress the Interviewer

- **Mention Matrix Exponentiation:** Proactively bringing up matrix exponentiation for large n (like $n = 10^{15}$) immediately sets you apart as a candidate with deep algebraic computer science knowledge.
- **Prune Zero-Value States:** Explain that checking `if (val == 0) continue;` is a standard micro-optimization that avoids executing expensive modulo operations for unreachable states, which is critical in competitive programming and high-performance computing.

61 Optimal Division

- **Category:** Coding
 - **Difficulty:** Medium
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 553, Glassdoor
 - **Flashcards:** [Dynamic Programming deck](#)
-

61.1 Question Description

You are given an integer array `nums`. The adjacent integers in `nums` will perform float division. * For example, if `nums = [2, 3, 4]`, we evaluate the expression `2 / 3 / 4`.

However, you can add any number of parentheses at any position to change the priority of operations. You want to add these parentheses such that the value of the expression after the evaluation is **maximum**.

Return *the corresponding expression that has the maximum value in string format*.

Note: Your expression should not contain redundant parentheses.

61.1.1 Examples

- **Input:** `nums = [1000,100,10,2]`
 - **Output:** `"1000/(100/10/2)"`
 - **Explanation:** $1000/(100/10/2) = 1000/((100/10)/2) = 1000/(10/2) = 1000/5 = 200$ The parenthesized version `"1000/((100/10)/2)"` contains redundant outer parentheses in the denominator, so we return `"1000/(100/10/2)"`. Other groupings yield smaller results:
 - * $1000 / 100 / 10 / 2 = 0.5$
 - * $1000 / (100 / 10) / 2 = 50$
 - * $1000 / (100 / (10 / 2)) = 50$
 - * $1000 / 100 / (10 / 2) = 2$
- **Input:** `nums = [2,3,4]`
 - **Output:** `"2/(3/4)"`
 - **Explanation:** $2 / (3 / 4) = 8 / 3 = 2.667$.

Constraints: * $1 \leq \text{nums.length} \leq 10$ * $2 \leq \text{nums}[i] \leq 1000$ * There is only one optimal division for the given input.

61.2 Detailed Solution (C++)

While this problem is categorized under **Dynamic Programming**, a mathematical analysis reveals an optimal $\mathcal{O}(n)$ greedy solution.

61.2.1 Mathematical Proof of Greedy Optimality

For any expression of the form:

$$X = \text{nums}[0] / \text{nums}[1] / \text{nums}[2] / \dots / \text{nums}[n-1]$$

1. Since all $\text{nums}[i] \geq 2$, multiplying by any number increases the value, and dividing by any number decreases it. 2. Regardless of how parentheses are placed, the first number $\text{nums}[0]$ will always remain in the **numerator** (since no operator precedes it). 3. The second number $\text{nums}[1]$ will always remain in the **denominator** (since it is directly divided by $\text{nums}[0]$, and any parentheses group starting at index 1 is still placed in the denominator). 4. For any subsequent number $\text{nums}[i]$ ($i \geq 2$), it can either be in the numerator or denominator. To maximize the overall expression, we should place all subsequent numbers in the **numerator** if possible. 5. By grouping the entire denominator together:

$$\text{nums}[0] / (\text{nums}[1] / \text{nums}[2] / \dots / \text{nums}[n-1])$$

We mathematically evaluate this as:

$$\frac{\text{nums}[0]}{\frac{\text{nums}[1]}{\text{nums}[2] \times \dots \times \text{nums}[n-1]}} = \frac{\text{nums}[0] \times \text{nums}[2] \times \dots \times \text{nums}[n-1]}{\text{nums}[1]}$$

This maximizes the numerator and minimizes the denominator. Thus, this single parenthesization is mathematically guaranteed to be the unique optimal division.

61.2.2 Standard C++ Production Code (Greedy $\mathcal{O}(n)$)

```
#include <string>
#include <vector>

class Solution {
public:
    std::string optimalDivision(std::vector<int>& nums) {
        int n = static_cast<int>(nums.size());
        if (n == 0) {
            return "";
        }
        if (n == 1) {
            return std::to_string(nums[0]);
        }
        if (n == 2) {
            return std::to_string(nums[0]) + "/" + std::to_string(nums[1]);
        }

        // For n > 2: nums[0] / (nums[1] / nums[2] / ... / nums[n-1])
        std::string result = std::to_string(nums[0]) + "/" + std::to_string(nums[1]);
        for (int i = 2; i < n; ++i) {
            result += "/" + std::to_string(nums[i]);
        }
        result += ")";

        return result;
    }
};
```

61.3 Detailed Solution (Python)

61.3.1 Standard Python Production Code

Below is the concise, Pythonic implementation using `join()`.

```
from typing import List

class Solution:
    def optimalDivision(self, nums: List[int]) -> str:
        """
        Formats the optimal division to maximize the evaluated expression value.

        Time Complexity:  $O(N)$ 
        Space Complexity:  $O(N)$ 
        """
```

```

"""
n = len(nums)
if n == 0:
    return ""
if n == 1:
    return str(nums[0])
if n == 2:
    return f"{nums[0]}/{nums[1]}"

# Join the numbers from index 1 to end with '/', wrap in parentheses,
# and prepend the first number.
denominator = "/".join(map(str, nums[1:]))
return f"{nums[0]}/{denominator}"

```

61.4 Dynamic Programming Alternative (Conceptual)

If constraints were different (e.g. numbers could be fractions or negative, where greedy logic fails), we would use Dynamic Programming similar to Matrix Chain Multiplication: * **State**: For subarray `nums[i...j]`, store both `max_val` (with its string representation) and `min_val` (with its string representation). * **Transition**:

$$\text{max_val}(i, j) = \max_{i \leq k < j} \left(\frac{\text{max_val}(i, k)}{\text{min_val}(k + 1, j)} \right)$$

$$\text{min_val}(i, j) = \min_{i \leq k < j} \left(\frac{\text{min_val}(i, k)}{\text{max_val}(k + 1, j)} \right)$$

* Time complexity of this DP approach would be $\mathcal{O}(n^3)$. Given $N \leq 10$, this DP is highly feasible but mathematically redundant here.

61.5 Python-Specific Complexities & Implementation Considerations

61.5.1 1. Fast String Interpolation with f-Strings

- In Python, using f-strings (e.g., `f"{nums[0]}/{denominator}"`) is faster and cleaner than concatenation (`str(nums[0]) + "/" + ...`) because f-strings are evaluated at runtime as optimized bytecode.

61.5.2 2. Standard `map` and `join`

- `"/".join(map(str, nums[1:]))` is highly optimized in CPython, running the loop and conversion in native C, which is faster than manual string accumulation loops.
-

61.6 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We iterate through the array once to convert integers to strings and join them.
Space Complexity	$\mathcal{O}(n)$	Auxiliary memory is used to store the output string.

61.7 Common Follow-Up Questions & How to Answer

61.7.1 Q1: What if the elements in `nums` could be float values between 0 and 1?

- **Answer:** The greedy solution fails because dividing by a value between 0 and 1 increases the result (e.g., $X/0.5 = 2X$). We would have to fall back to the $\mathcal{O}(n^3)$ Dynamic Programming approach (outlined in the DP Alternative section) to find the exact combinations of max/min values for all intervals.

61.7.2 Q2: How do you prove that `nums[1]` must always be in the denominator?

- **Answer:** The expression is evaluated from left to right by default. The division operator immediately following `nums[0]` is `/`. Since it acts on `nums[0]` and whatever follows it (either `nums[1]` alone or a parenthesized expression starting with `nums[1]`), `nums[1]` will always act as a divisor to `nums[0]`. There is no way to place parentheses such that `nums[1]` is multiplied by `nums[0]`.

61.8 Pro-Tip: How to Impress the Interviewer

- **Explain the DP fallback:** Even if you know the greedy $\mathcal{O}(n)$ math shortcut, explain the general $\mathcal{O}(n^3)$ Dynamic Programming solution first to show that you recognize the problem structure. Then, present the greedy mathematical optimization as a special-case optimization enabled by the constraint `nums[i] ≥ 2`. This demonstrates superb domain expertise and flexibility.
- **Prevent String Reallocation Overhead:** In languages like C++, appending to strings in a loop causes frequent reallocations. Mention that using `std::string::reserve()` with an estimated size based on digits and slashes is a good system-level hygiene optimization.

62 Out of Boundary Paths

- **Category:** Coding
- **Difficulty:** Medium
- **Target Role:** Software Engineer / AI Systems Architect
- **Source:** LeetCode 576, Glassdoor
- **Flashcards:** [Dynamic Programming deck](#)

62.1 Question Description

There is an $m \times n$ grid with a ball. The ball is initially at the position `[startRow, startColumn]`. You are allowed to move the ball to one of the four adjacent cells in the grid (possibly out of the grid crossing the grid boundary). You can apply at most `maxMove` moves to the ball.

Given the five integers m , n , `maxMove`, `startRow`, `startColumn`, return *the number of paths to move the ball out of the grid boundary*. Since the answer can be very large, return it **modulo** $10^9 + 7$.

62.1.1 Examples

- **Input:** $m = 2, n = 2, \text{maxMove} = 2, \text{startRow} = 0, \text{startColumn} = 0$
– **Output:** 6
- **Input:** $m = 1, n = 3, \text{maxMove} = 3, \text{startRow} = 0, \text{startColumn} = 1$
– **Output:** 12

Constraints: $1 \leq m, n \leq 50$ $0 \leq \text{maxMove} \leq 50$ $0 \leq \text{startRow} < m$ $0 \leq \text{startColumn} < n$

62.2 Detailed Solution (C++)

This problem can be modeled as finding the total number of paths on a 2D grid that escape the boundaries within `maxMove` steps.

- **State:** `dp[r][c]` represents the number of paths that can reach cell (r, c) at the current move number.
- **Transition:** In each step, a ball at cell (r, c) can move to 4 adjacent directions (nr, nc) .
 - If the adjacent cell is **inside** the grid boundaries, the paths at `dp[r][c]` propagate to `next[nr][nc]`.
 - If the adjacent cell is **outside** the grid boundaries, the ball escapes. The paths at `dp[r][c]` are added to our total escape path count `result`.
- **Space Optimization:** Since the states for `move + 1` only depend on states from `move`, we can use two 2D grids (or flattened 1D arrays of size $m \times n$) and swap them after each move. This reduces space complexity from $\mathcal{O}(\text{maxMove} \cdot m \cdot n)$ to $\mathcal{O}(m \cdot n)$.

62.2.1 Standard C++ Production Code

```
#include <vector>
#include <algorithm>

class Solution {
public:
    int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
        if (maxMove <= 0) {
            return 0;
        }

        constexpr int MOD = 1'000'000'007;
```

```

// Flattened 1D representation for 2D DP to optimize memory footprint and CPU cache usage
std::vector<int> dp(m * n, 0);
std::vector<int> nxt(m * n, 0);

// Start position has 1 path initially
dp[startRow * n + startColumn] = 1;

int dirs[4][2] = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
long long result = 0;

for (int move = 0; move < maxMove; ++move) {
    std::fill(nxt.begin(), nxt.end(), 0);

    for (int r = 0; r < m; ++r) {
        for (int c = 0; c < n; ++c) {
            int val = dp[r * n + c];
            if (val == 0) {
                continue; // Skip unreachable states
            }

            for (int d = 0; d < 4; ++d) {
                int nr = r + dirs[d][0];
                int nc = c + dirs[d][1];

                if (nr >= 0 && nr < m && nc >= 0 && nc < n) {
                    nxt[nr * n + nc] = (nxt[nr * n + nc] + val) % MOD;
                } else {
                    result = (result + val) % MOD;
                }
            }
        }
    }

    std::swap(dp, nxt);
}

return static_cast<int>(result);
}
};

```

62.3 Detailed Solution (Python)

62.3.1 Standard Python Production Code

Below is the space-optimized Python implementation using two 1D lists of size $m \times n$.

class Solution:

```
def findPaths(self, m: int, n: int, maxMove: int, startRow: int, startColumn: int) -> int:
    """
    Calculates the number of paths to move the ball out of boundary in at most maxMove moves.

    Time Complexity: O(maxMove * m * n)
    Space Complexity: O(m * n)
    """

    if maxMove <= 0:
        return 0

    MOD = 10**9 + 7

    # Flattened representation of m x n grid
    dp = [0] * (m * n)
    dp[startRow * n + startColumn] = 1

    dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    result = 0

    for _ in range(maxMove):
        nxt = [0] * (m * n)
        for r in range(m):
            for c in range(n):
                val = dp[r * n + c]
                if val == 0:
                    continue

                for dr, dc in dirs:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < m and 0 <= nc < n:
                        nxt[nr * n + nc] = (nxt[nr * n + nc] + val) % MOD
                    else:
                        result = (result + val) % MOD

        dp = nxt

    return result
```

62.4 Python-Specific Complexities & Implementation Considerations

62.4.1 1. Flattening the 2D Grid

- Instead of using lists of lists (e.g. `dp[r][c]`), we flatten the 2D grid of size $m \times n$ into a single 1D list of size $m \cdot n$.
- Accessing `dp[r * n + c]` is much faster in CPython because it avoids double-indirection pointer lookups (which require searching the heap for two separate list objects). This micro-optimization reduces Python execution time by 20–30%.

62.4.2 2. Iterative Space-swapping vs DFS Recursion

- A recursive solution with `@lru_cache` (top-down) is simple to write but requires $\mathcal{O}(\text{maxMove} \cdot m \cdot n)$ stack frames. For large moves, this risks hitting the Python recursion depth limit and has high call overhead.
- The iterative bottom-up layout with state-swapping keeps auxiliary memory to $\mathcal{O}(m \cdot n)$ and executes extremely fast.

62.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N \cdot m \cdot n)$	Where N is <code>maxMove</code> , m is grid height, and n is grid width. We iterate N times over $m \times n$ states, doing 4 lookups per cell.
Space Complexity	$\mathcal{O}(m \cdot n)$	Due to space-swapping, we only maintain two grids of size $m \times n$ instead of a 3D table of size $N \times m \times n$.

62.6 Common Follow-Up Questions & How to Answer

62.6.1 Q1: What if `maxMove` is very large but the grid is small?

- **Answer:** Similar to other FSM and grid-walk problems, if `maxMove` is up to 10^{18} and $m \times n \leq 100$, we can model the transition using **Matrix Exponentiation** over a transition matrix of size $(m \cdot n + 1) \times (m \cdot n + 1)$ (where the $+1$ represents the sink escape state). However, matrix exponentiation is slow if $m \cdot n$ is large because multiplying matrices takes $\mathcal{O}((m \cdot n)^3)$ operations.

62.6.2 Q2: How can we optimize this if m and n are large but `maxMove` is very small?

- **Answer:** If `maxMove` is small (e.g. 5) but the grid is $10^9 \times 10^9$, we can use a **hash map** to store only the reachable states rather than allocating a full grid. Starting with `{ (startRow, startColumn): 1 }`,

we transition to a new hash map on each step. Since the number of reachable states grows by at most $O(\text{maxMove}^2)$, the time and space complexity becomes $\mathcal{O}(\text{maxMove}^2)$ instead of $\mathcal{O}(\text{maxMove} \cdot m \cdot n)$, which is an enormous speedup.

62.7 Pro-Tip: How to Impress the Interviewer

- **Propose Sparse Map DP:** Bringing up the hash map approach for extremely large grids (e.g., $m, n = 10^9$, $\text{maxMove} = 10$) shows you don't just write static arrays but think about sparsity and how to optimize algorithms based on input data scales.
- **Highlight Cache Locality:** Discuss how flattening the 2D array into a 1D vector (`dp[r * n + c]`) places contiguous row values next to each other in physical memory. When the processor loads a value, it loads the adjacent columns into L1 cache, preventing memory stalls.

63 Super Egg Drop

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / AI Systems Architect
 - **Source:** LeetCode 887, Glassdoor
 - **Flashcards:** [Dynamic Programming deck](#)
-

63.1 Question Description

You are given k identical eggs and you have access to a building with n floors numbered from 1 to n .

There exists a critical floor f where $0 \leq f \leq n$. Any egg dropped from a floor higher than f will break, and any egg dropped from floor f or below will survive.

Return *the minimum number of moves you need to determine f with certainty*.

In each move, you may take an unbroken egg and drop it from any floor x ($1 \leq x \leq n$). If the egg breaks, you cannot drop it again. You can only determine f if you know with certainty that $f == x$ after some number of drops.

63.1.1 Examples

- **Input:** $k = 1, n = 2$
 - **Output:** 2
 - **Explanation:**
 - * Drop the egg from floor 1. If it breaks, $f = 0$.
 - * If it does not break, drop it from floor 2. If it breaks, $f = 1$. Otherwise, $f = 2$.
 - * Thus, we need at most 2 moves.
- **Input:** $k = 3, n = 14$
 - **Output:** 4

- **Input:** $k = 2, n = 100$
 – **Output:** 14

Constraints: $* 1 \leq k \leq 100 * 1 \leq n \leq 10^4$

63.2 Detailed Solution (C++)

A traditional dynamic programming approach defines $dp[i][j]$ as the minimum drops to test j floors with i eggs. This leads to the recurrence:

$$dp[i][j] = 1 + \min_{1 \leq x \leq j} \left(\max(dp[i-1][x-1], dp[i][j-x]) \right)$$

This takes $\mathcal{O}(k \cdot n^2)$ or $\mathcal{O}(k \cdot n \log n)$ time, which is too slow for very large n .

63.2.1 Mathematical Re-framing (Dual DP)

Instead of asking "what is the minimum moves to test n floors with k eggs?", we ask: **"Given m moves and k eggs, what is the maximum number of floors we can test?"**

Let $dp[m][k]$ be the maximum number of floors we can test with m moves and k eggs. Suppose we drop an egg from an optimal floor X .

1. **The egg breaks:** We have $k - 1$ eggs and $m - 1$ moves left. The maximum floors we can check below X is $dp[m-1][k-1]$.
2. **The egg survives:** We have k eggs and $m - 1$ moves left. The maximum floors we can check above X is $dp[m-1][k]$.

Including floor X itself, the maximum number of floors we can cover is:

$$dp[m][k] = dp[m-1][k-1] + dp[m-1][k] + 1$$

We want to find the smallest number of moves m such that $dp[m][k] \geq n$. Since the state $dp[m][k]$ only depends on the previous move $m-1$, we can optimize space to $\mathcal{O}(k)$ by using a 1D array updated in-place from right to left.

63.2.2 Standard C++ Production Code

```
#include <vector>

class Solution {
public:
    int superEggDrop(int k, int n) {
        // dp[i] represents the maximum floors we can test with currently 'moves' moves and i eggs.
        std::vector<int> dp(k + 1, 0);
        int moves = 0;

        // Loop until the maximum testable floors with k eggs exceeds or equals n
        while (dp[k] < n) {
            moves++;
        }
    }
};
```

```

        // Update backward to use state from moves - 1
        for (int i = k; i >= 1; --i) {
            dp[i] = dp[i] + dp[i - 1] + 1;
        }
    }

    return moves;
}
};

```

63.3 Detailed Solution (Python)

63.3.1 Standard Python Production Code

Below is the space-optimized Python implementation with type hints.

```

class Solution:
    def superEggDrop(self, k: int, n: int) -> int:
        """
        Calculates the minimum number of moves to find the critical floor.

        Time Complexity: O(K * Moves) where Moves <= N
        Space Complexity: O(K)
        """
        # dp[i] is the maximum floors we can test with current moves and i eggs
        dp = [0] * (k + 1)
        moves = 0

        while dp[k] < n:
            moves += 1
            # Update backward to prevent overwriting values needed for the transition
            for i in range(k, 0, -1):
                dp[i] = dp[i] + dp[i - 1] + 1

        return moves

```

63.4 Python-Specific Complexities & Implementation Considerations

63.4.1 1. In-place Update Order

- We must iterate the inner loop backward (`range(k, 0, -1)`).
- If we updated forward, `dp[i - 1]` would already represent the value for `moves` instead of `moves - 1`. This is identical to the space-compression mechanism in the 0-1 Knapsack problem.

63.4.2 2. Time-Limit Safety

- When $k = 1$, the number of moves is n , so the outer loop runs n times. With $n = 10^4$ and $k = 1$, the inner loop is small, and the total operations are around 10^4 , which is well within Python's safety margin.

63.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(k \cdot m)$	Where m is the number of moves returned. Since $m \leq \log_2 n$ for large k and $m = n$ for $k = 1$, worst-case time complexity is $\mathcal{O}(k \cdot \min(n, 2^{\text{something}}))$. For practical constraints, it is extremely fast.
Space Complexity	$\mathcal{O}(k)$	We only maintain a 1D DP array of size $k + 1$.

63.6 Common Follow-Up Questions & How to Answer

63.6.1 Q1: What is the worst-case number of moves when $k = 1$?

- **Answer:** When we have only 1 egg, we cannot risk breaking it because we would have 0 eggs left to test the remaining floors. Thus, we must test floors sequentially from 1 to n . The worst-case number of moves is exactly n .

63.6.2 Q2: What is the optimal number of moves when we have an infinite number of eggs ($k \geq \log_2(n) + 1$)?

- **Answer:** If we have sufficient eggs, we can perform a standard **binary search** on the floors. Each drop halves the search space. The maximum number of moves required is $\lceil \log_2(n + 1) \rceil$.

63.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Dual State Representation:** Most candidates only memorize the standard $\mathcal{O}(k \cdot n \log n)$ DP with binary search. Explaining the dual state formulation (re-framing the problem to maximize floors for given moves) demonstrates a higher-order capacity for math and problem-solving.
- **Connect to Pascal's Triangle:** Point out that the recurrence relation $\text{dp}[m][k] = \text{dp}[m - 1][k - 1] + \text{dp}[m - 1][k]$

$1] + dp[m - 1][k] + 1$ resembles the recurrence of binomial coefficients (Pascal's Triangle):

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

In fact, $dp[m][k] = \sum_{i=1}^k \binom{m}{i}$. Discussing this closed-form mathematical representation shows deep structural intuition.

64 LFU Cache

- **Category:** Coding
 - **Difficulty:** Hard
 - **Target Role:** Software Engineer / Low-Level Developer / AI Systems Architect
 - **Source:** LeetCode 460, Glassdoor
 - **Flashcards:** [Design deck](#)
-

64.1 Question Description

Design and implement a data structure for a **Least Frequently Used (LFU)** cache.

Implement the `LFUCache` class:

- `LFUCache(int capacity)` Initializes the object with the **capacity** of the data structure.
- `int get(int key)` Gets the value of the **key** if the **key** exists in the cache. Otherwise, returns **-1**.
- `void put(int key, int value)` Updates the value of the **key** if present, or inserts the **key** if not already present. When the cache reaches its **capacity**, it should invalidate and remove the **least frequently used** key before inserting a new item. For this problem, when there is a **tie** (i.e., two or more keys with the same frequency), the key that has been **least recently used** should be invalidated.

To determine the least frequently used key, a **use counter** is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to **1** (due to the `put` operation). The use counter for a key in the cache is incremented on either a `get` or `put` operation.

The functions `get` and `put` must each run in $\mathcal{O}(1)$ average time complexity.

64.1.1 Examples

- **Input:** `["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]`
`[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]`
- **Output:** `[null, null, null, 1, null, -1, 3, null, -1, 3, 4]`
- **Explanation:**

```
# cnt(x) = use counter for key x
# cache=[] displays LFU/LRU ordering (leftmost is least recently used)
lfu = LFUCache(2)
```

```

lfu.put(1, 1)  # cache=[1], cnt(1)=1
lfu.put(2, 2)  # cache=[2, 1], cnt(2)=1, cnt(1)=1
lfu.get(1)     # returns 1; cache=[1, 2], cnt(1)=2, cnt(2)=1
lfu.put(3, 3)  # 2 is evicted (cnt(2)=1 is lowest); cache=[3, 1], cnt(3)=1, cnt(1)=2
lfu.get(2)     # returns -1 (not found)
lfu.get(3)     # returns 3; cache=[3, 1], cnt(3)=2, cnt(1)=2
lfu.put(4, 4)  # 1 is evicted (tie break: cnt(1)=cnt(3)=2, but 1 is LRU); cache=[4, 3], cnt(4)=1, cnt(3)=2
lfu.get(1)     # returns -1 (not found)
lfu.get(3)     # returns 3; cache=[3, 4], cnt(3)=3, cnt(4)=1
lfu.get(4)     # returns 4; cache=[4, 3], cnt(4)=2, cnt(3)=3

```

64.1.2 Constraints

- $1 \leq \text{capacity} \leq 10^4$
- $0 \leq \text{key} \leq 10^5$
- $0 \leq \text{value} \leq 10^9$
- At most 2×10^5 calls will be made to get and put.

64.2 Detailed Solution (C++)

To implement LFU Cache in $\mathcal{O}(1)$ average time complexity: 1. **Cache Map:** `std::unordered_map<int, Node>` maps key to its details: `val`, current frequency `freq`, and an iterator to its position in the frequency lists to allow $\mathcal{O}(1)$ deletion. 2. **Frequency Lists:** `std::unordered_map<int, std::list<int>>` `freqMap` maps each frequency count to a doubly-linked list of keys sharing that frequency. The most recently accessed key is pushed to the front of the list, so the LRU key for a frequency is always at the back of the list. 3. **Global Minimum Frequency:** `minFreq` tracks the lowest active frequency across the cache. When eviction is required, we remove the key from the back of `freqMap[minFreq]`.

64.2.1 Update State (touch helper):

When a key is accessed: 1. Remove the key from its current frequency list in `freqMap[oldFreq]` in $\mathcal{O}(1)$ using the saved iterator. 2. Increment the key's frequency: `freq++`. 3. Push the key to the front of the new frequency list `freqMap[newFreq]`, and update the saved list iterator in the node. 4. If the old frequency list is now empty and `oldFreq == minFreq`, increment `minFreq`.

64.2.2 Standard C++ Production Code

```

#include <unordered_map>
#include <list>
#include <iterator>

```

```

class LFUCache {
private:
    struct Node {
        int key;

```

```

    int val;
    int freq;
    std::list<int>::iterator it; // Iterator pointing to its location in the frequency list
};

int cap;
int minFreq;
std::unordered_map<int, Node> cache;
std::unordered_map<int, std::list<int>> freqMap; // Maps frequency -> list of keys (front is MRU, back is LRU)

void touch(int key) {
    auto& node = cache[key];
    int oldFreq = node.freq;
    node.freq++;

    // Remove from the old frequency list
    freqMap[oldFreq].erase(node.it);

    // Add to the new frequency list
    freqMap[node.freq].push_front(key);
    node.it = freqMap[node.freq].begin();

    // If the old frequency list is empty and was the minFreq, increment minFreq
    if (freqMap[oldFreq].empty() && oldFreq == minFreq) {
        minFreq++;
    }
}

public:
    LFUCache(int capacity) : cap(capacity), minFreq(0) {}

    int get(int key) {
        if (cap == 0 || !cache.count(key)) {
            return -1;
        }
        touch(key);
        return cache[key].val;
    }

    void put(int key, int value) {
        if (cap == 0) {
            return;
        }

```

```

    // If key already exists, update value and touch frequency
    if (cache.count(key)) {
        cache[key].val = value;
        touch(key);
        return;
    }

    // Evict if cache is at capacity
    if (cache.size() >= static_cast<size_t>(cap)) {
        // Retrieve the LRU key with the minimum frequency
        int evict_key = freqMap[minFreq].back();
        freqMap[minFreq].pop_back();
        cache.erase(evict_key);
    }

    // Insert new element with frequency 1
    minFreq = 1;
    freqMap[1].push_front(key);
    cache[key] = Node{key, value, 1, freqMap[1].begin()};
}
};

```

64.3 Detailed Solution (Python)

To achieve true $\mathcal{O}(1)$ average operations in Python without writing a custom doubly linked list from scratch, we use `collections.OrderedDict`.

In Python, an `OrderedDict` maintains insertion order. The oldest inserted item can be removed in $\mathcal{O}(1)$ using `popitem(last=False)`.

64.3.1 Standard Python Production Code

```

from collections import OrderedDict, defaultdict

class LFUCache:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.min_freq = 0
        self.cache = {} # maps key -> (val, freq)
        self.freq_map = defaultdict(OrderedDict) # maps freq -> OrderedDict of {key: True} (MRU is right, LRU is left)

    def _touch(self, key: int) -> None:
        val, freq = self.cache[key]

```

```
# Remove from old frequency OrderedDict
del self.freq_map[freq][key]
if not self.freq_map[freq]:
    del self.freq_map[freq]
    if freq == self.min_freq:
        self.min_freq += 1

# Update frequency and insert into new frequency OrderedDict
new_freq = freq + 1
self.cache[key] = (val, new_freq)
self.freq_map[new_freq][key] = True

def get(self, key: int) -> int:
    if self.cap == 0 or key not in self.cache:
        return -1
    self._touch(key)
    return self.cache[key][0]

def put(self, key: int, value: int) -> None:
    if self.cap == 0:
        return

    if key in self.cache:
        _, freq = self.cache[key]
        self.cache[key] = (value, freq)
        self._touch(key)
        return

    # Evict LFU (LRU as tiebreaker) if cache is full
    if len(self.cache) >= self.cap:
        # popitem(last=False) pops the oldest/first inserted key (LRU) in O(1)
        evict_key, _ = self.freq_map[self.min_freq].popitem(last=False)
        del self.cache[evict_key]
        if not self.freq_map[self.min_freq]:
            del self.freq_map[self.min_freq]

    # Insert new key
    self.min_freq = 1
    self.cache[key] = (value, 1)
    self.freq_map[1][key] = True
```

64.4 Python-Specific Complexities & Implementation Considerations

64.4.1 1. OrderedDict vs Standard Set/Dict

- In older Python versions, standard dictionaries did not guarantee order, and standard `set` objects do not support $\mathcal{O}(1)$ popping of the *oldest* element.
- While Python 3.7+ dictionaries maintain insertion order, `collections.OrderedDict` provides explicit methods like `popitem(last=False)` (which pops the first-inserted key in $\mathcal{O}(1)$), making it the most readable and correct tool for simulating doubly-linked list nodes.

64.4.2 2. Guarding against Capacity 0

- Always check for `capacity == 0` during initialization and `put` operations. Failing to check this can lead to evicting elements on an empty cache or infinite loops if elements are inserted into a zero-capacity cache.

64.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity (get)	$\mathcal{O}(1)$	Dictionary lookup and <code>OrderedDict</code> / <code>list</code> iterator deletion/insertion take constant average time.
Time Complexity (put)	$\mathcal{O}(1)$	Insertion, eviction, and pointer manipulation run in constant average time.
Space Complexity	$\mathcal{O}(\text{capacity})$	We store at most <code>capacity</code> number of nodes in the hash maps and lists.

64.6 Common Follow-Up Questions & How to Answer

64.6.1 Q1: What is the difference between LRU (Least Recently Used) and LFU (Least Frequently Used)?

- **Answer:**
 - **LRU** only considers the *recency* of access. If a key was accessed long ago, it is evicted, even if it had been accessed 100 times before.
 - **LFU** considers *frequency* first, using recency only as a tiebreaker. If a key is accessed 100 times, it will not be evicted in favor of a key accessed once, even if the latter was accessed more recently. LFU is better for static popular content patterns, but LRU adapts better to shifting hot-spots.

64.6.2 Q2: How can we prevent "frequency starvation" in LFU?

- **Answer:** In a basic LFU cache, items that get a huge burst of traffic early on will accumulate high frequency counts (e.g., 1000 accesses) and will never be evicted, even if they are never accessed again (frequency starvation).
 - **Solution:** Introduce **frequency decay**. Periodically (e.g. after a certain number of operations or time interval), divide all use counters by 2, or subtract a constant, allowing older items' weights to decay over time.
-

64.7 Pro-Tip: How to Impress the Interviewer

- **Use List Iterators Correctly:** In C++, point out that saving `std::list<int>::iterator` inside the `Node` avoids searching the list when deleting elements. Searching a linked list takes $\mathcal{O}(N)$ time; using the iterator directly reduces list deletion to a true $\mathcal{O}(1)$ pointer update.
- **Explain Memory Overhead:** Discuss the memory cost of LFU. Because we store list pointers, frequency tables, and cache values, LFU has a high memory overhead per cache item compared to a simple array or hashtable. Pointing this out highlights cost-conscious systems design thinking.